OpenglEs之EGL环境搭建

来源：微信公众号 · 求索君Fly > 发布时间：2022-08-26 08:09

> 原文链接：https://mp.weixin.qq.com/s/5QBYGxdmQbXhWo6iqGbHbw

前言

前面我们发布了一系列的入门教程，例如C++系列的指针扫盲、多线程的使用等，JNI入门系列，ffmpeg入门系列等，有感兴趣的童鞋们可以关注往回自行查阅。

今天我们的主题依然是音视频开发的范畴，做过音视频开发的都知道Opengl也是音视频开发中的一项重要技能，特别是涉及到视频录制、特效处理、画质渲染细分功能。因此后续笔者打算再出一系列的Opengl ES的学习笔记，
希望能与大家共同温故知新。

因为前面介绍了一些NDK和C++的教程，所以为了巩固，后续的一些demo多以NDK的形式呈现给大家，使用Opengl ES3的版本。

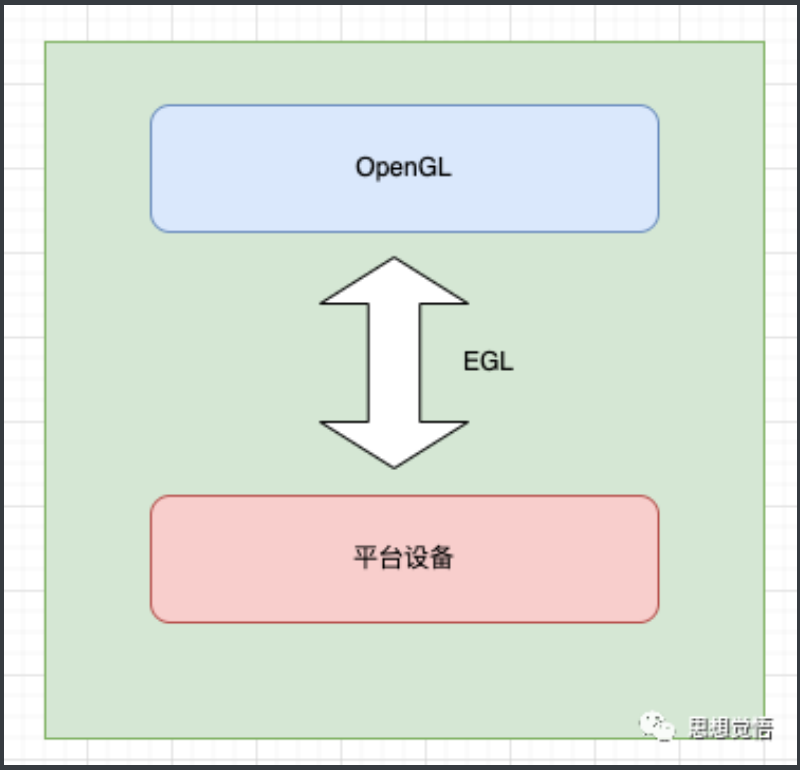
今天我们的主题是Opengl ES的第一篇-->EGL

EGL是什么

众所周知，Opengl是跨平台的，那么面对各种平台的差异性，Opengl是如何抹平而做到跨平台的呢？这也许就是EGL的功劳吧，简单地说EGL就是Opengl和平台各平台之间的一个适配器，是一系列的接口，具体实现是由具体的设备厂商实现的。

EGL 是渲染 API（如 OpenGL ES）和原生窗口系统之间的接口.通常来说，OpenGL 是一个操作 GPU 的 API，它通过驱动向 GPU 发送相关指令，控制图形渲染管线状态机的运行状态，但是当涉及到与本地窗口系统进行交互时，就需要这么一个中间层，且它最好是与平台无关的，

因此 EGL 被设计出来，作为 OpenGL 和原生窗口系统之间的桥梁。

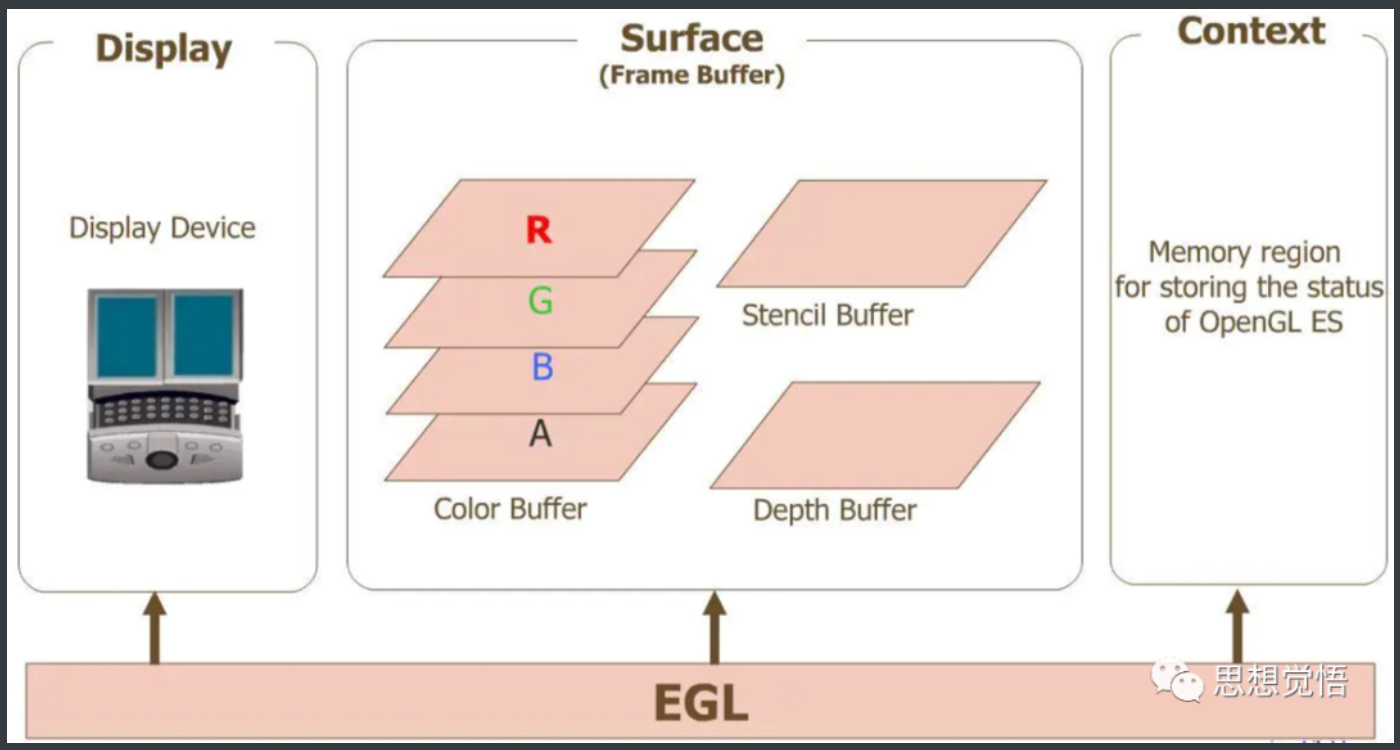


> Opengl与EGL的关系图

EGL API 是独立于 OpenGL 各版本标准的独立的一套 API，其主要作用是为 OpenGL 指令 创建上下文 Context 、绘制目标 Surface 、配置 FrameBuffer 属性、Swap 提交绘制结果等。EGL提供如下机制：

- 与设备的原生窗口系统通信
- 查询绘图表面的可用类型和配置
- 创建绘图表面
- 在 OpenGL ES 和其他图形渲染 API 之间同步渲染
- 管理纹理贴图等渲染资源

下面这张图可简要看出EGL的接口能力：

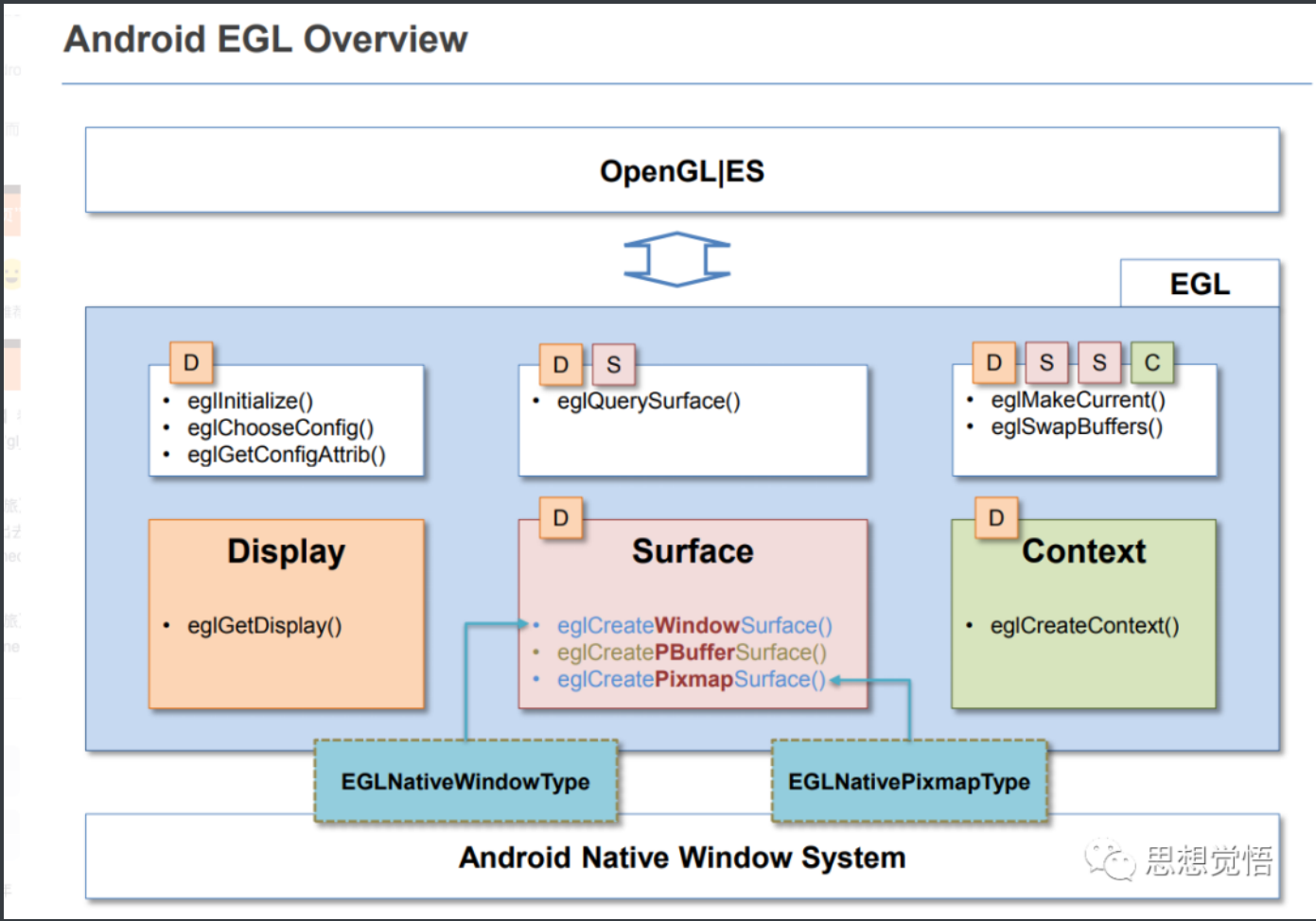


EGL创建流程

想要在安卓上使用Opengl ES我们可以直接使用GLSurfaceView来进行Opengl的渲染，因为GLSurfaceView内部给我们封装好了EGL环境和渲染线程。如果我们想要更高的拓展性，我们也使用SurfaceView，然后参考SurfaceView中的EGL环境搭建、线程模型来
自行搭建Opengl ES的渲染环境。

本着学习探索的目的，我们尝试在NDK搭建EGL环境。

下面这张图展示的是安卓系统上EGL的主要使用API：

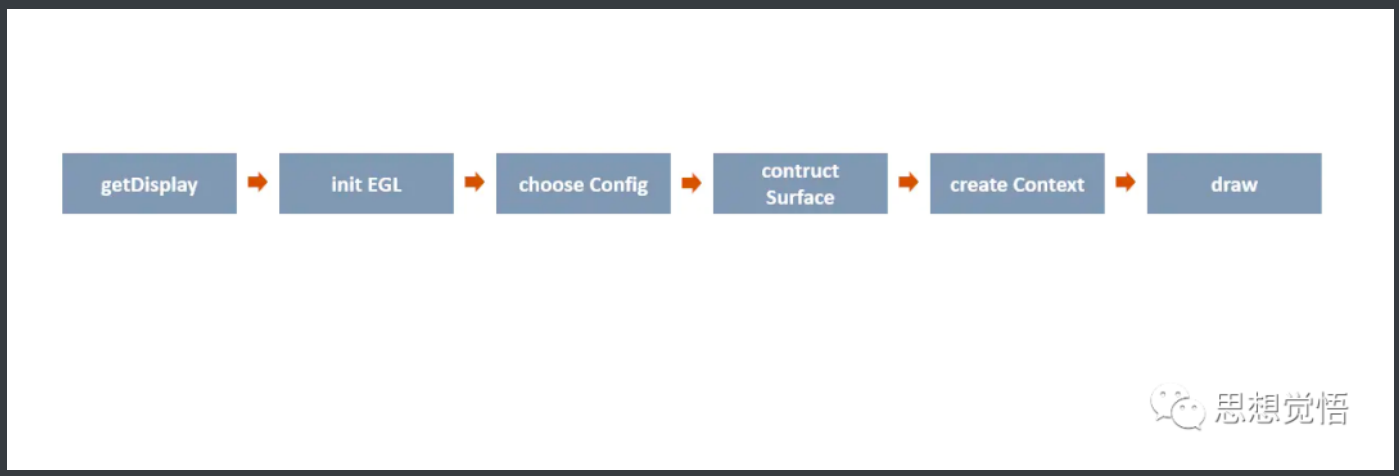


需要说明的一点是EGL是单线程模型的，也就说说EGL环境的创建、渲染操作、EGL环境的销毁都必须在同一个线程内完成，否则是无效的。当然我们可以通过共享EGL上下文来做多多线程渲染，但这些都是后话了...

任何OpenGL ES应用程序都必须在开始渲染之前使用EGL执行如下任务：

1. 查询并初始化设备商可用的显示器。
2. 创建渲染表面。
   EGL中创建的表面可以分类为屏幕上的表面或者屏幕外的表面。屏幕上的表面连接到原生窗口系统，而屏幕外的表面是不显示但是可以用作渲染表面的像素缓冲区。这些表面可以用来渲染纹理，并可以在多个Khronos API之间共享。
3. 创建渲染上下文。
   EGL是创建OpenGL ES渲染上下文所必需的。这个上下文必须连接到合适的表面才能开始渲染。

下面是EGL环境创建的主要流程：



说完烦躁的基础理论，那就放码过来吧!!!

使用Android Studio创建一个Native工程，然后配置好 `CMakeLists.txt` 引入相关库：

```
cmake_minimum_required(VERSION 3.18.1)

project("learn")

#找到包含所有的cpp文件
file(GLOB allCpp *.cpp **/**.cpp **/**/**.cpp  **/**/**/**.cpp
**/**/**/**/**.cpp)

add_library( # Sets the name of the library.
        # 库名称
        learn

        SHARED

        ${allCpp})

target_link_libraries(
        learn
        # 引入egl
        egl
        # 引入gles 3
        GLESv3
```

```
        # 安卓相关库
        android
        # 安卓log
        log)
```

下面我们创建一个与Native映射的EGLHelper类：

```
package com.fly.opengles.learn.egl;

import android.view.Surface;

public class EGLHelper {
    protected long nativePtr;

    public void surfaceCreated(Surface surface) {
        nativePtrInit();
        n_surfaceCreated(nativePtr,surface);
    }

    public void surfaceChanged(int width, int height) {
        nativePtrInit();
        n_surfaceChanged(nativePtr,width,height);
    }

    public void surfaceDestroyed() {
        if(nativePtr != 0){
            n_surfaceDestroyed(nativePtr);
            nativePtr = 0;
        }
    }

    private void nativePtrInit(){
        if(nativePtr == 0){
            nativePtr = n_nativePtrInit();
        }
    }
```

```
    private native long n_nativePtrInit();
    private native void n_surfaceCreated(long nativePtr,Surface
surface);
    private native void n_surfaceChanged(long nativePtr,int width, int
height);
    private native void n_surfaceDestroyed(long nativePtr);
}
```

然后自定义一个MySurfaceView继承于SurfaceView，在它的Callback回调方法中对EGL进行操作：

```
public class MySurfaceView extends SurfaceView implements
SurfaceHolder.Callback {

    private EGLHelper eglHelper;

    public MySurfaceView(Context context) {
        this(context,null);
    }

    public MySurfaceView(Context context, AttributeSet attrs) {
        super(context, attrs);
        eglHelper = new EGLHelper();
        getHolder().addCallback(this);
    }

    @Override
    public void surfaceCreated(@NonNull SurfaceHolder surfaceHolder) {
        eglHelper.surfaceCreated(surfaceHolder.getSurface());
    }

    @Override
    public void surfaceChanged(@NonNull SurfaceHolder surfaceHolder,
int i, int w, int h) {
        eglHelper.surfaceChanged(w,h);
```

```
    }

    @Override
    public void surfaceDestroyed(@NonNull SurfaceHolder surfaceHolder)
{
        eglHelper.surfaceDestroyed();
    }
}
```

测试效果时，我们在布局中使用我们自定义好MySurfaceView即可，自此java层代码编写完毕，在NDK层我们将EGL环境创建完毕后即可通过MySurfaceView看到渲染结果。

为了方便调试和debug，我们定义 `Log.h` 日志工具：

```
#ifndef NDK_OPENGLES_LEARN_LOG_H
#define NDK_OPENGLES_LEARN_LOG_H

#include "android/log.h"

#define LOGD(FORMAT, ...) __android_log_print(ANDROID_LOG_DEBUG,
"fly_learn_opengl", FORMAT, ##__VA_ARGS__);
#define LOGE(FORMAT, ...) __android_log_print(ANDROID_LOG_ERROR,
"fly_learn_opengl", FORMAT, ##__VA_ARGS__);

#endif //NDK_OPENGLES_LEARN_LOG_H
```

将EGL的相关操作封装在类C++的类EglHelper中：

EglHelper.h

```
#ifndef NDK_OPENGLES_LEARN_EGLHELPER_H
#define NDK_OPENGLES_LEARN_EGLHELPER_H

#include "EGL/egl.h"
```

```
class EglHelper {

public:
    EGLDisplay  mEglDisplay;
    EGLSurface  mEglSurface;
    EGLConfig  mEglConfig;
    EGLContext mEglContext;

public:
    EglHelper();
    ~EglHelper();
    int initEgl(EGLNativeWindowType win);
    int swapBuffers();
    void destroyEgl();
};


#endif
```

EglHelper.cpp主要实现如下，EGL的主要创建过程在函数 `initEgl` 中，具体看注释：

```
#include "EglHelper.h"
#include "../utils/Log.h"

EglHelper::EglHelper() {

    mEglDisplay = EGL_NO_DISPLAY;
    mEglSurface = EGL_NO_SURFACE;
    mEglContext = EGL_NO_CONTEXT;
    mEglConfig = NULL;
}

EglHelper::~EglHelper() {
    destroyEgl();
}
```

```
int EglHelper::initEgl(EGLNativeWindowType window) {

    //1、获取显示设备
    mEglDisplay = eglGetDisplay(EGL_DEFAULT_DISPLAY);
    if(mEglDisplay == EGL_NO_DISPLAY)
    {
        LOGE("eglGetDisplay error");
        return -1;
    }
    // 2、 EGL初始化
    EGLint *version = new EGLint[2];
    if(!eglInitialize(mEglDisplay, &version[0], &version[1]))
    {
        LOGE("eglInitialize error");
        return -1;
    }

    //3、 资源配置，例如颜色位数等
    const EGLint attribs[] = {
            EGL_RED_SIZE, 8,
            EGL_GREEN_SIZE, 8,
            EGL_BLUE_SIZE, 8,
            EGL_ALPHA_SIZE, 8,
            EGL_DEPTH_SIZE, 8,
            EGL_STENCIL_SIZE, 8,
            EGL_RENDERABLE_TYPE, EGL_OPENGL_ES2_BIT,
            EGL_NONE
    };

    EGLint num_config;
    if(!eglChooseConfig(mEglDisplay, attribs, NULL, 1, &num_config))
    {
        LOGE("eglChooseConfig  error 1");
        return -1;
    }
```

```
    //4、ChooseConfig
    if(!eglChooseConfig(mEglDisplay, attribs, &mEglConfig, num_config,
&num_config))
    {
        LOGE("eglChooseConfig  error 2");
        return -1;
    }

    // 5、创建上下文
    int attrib_list[] = {
            EGL_CONTEXT_CLIENT_VERSION, 2,
            EGL_NONE
    };

    mEglContext = eglCreateContext(mEglDisplay, mEglConfig,
EGL_NO_CONTEXT, attrib_list);

    if(mEglContext == EGL_NO_CONTEXT)
    {
        LOGE("eglCreateContext  error");
        return -1;
    }

    //6、创建渲染的Surface
    mEglSurface = eglCreateWindowSurface(mEglDisplay, mEglConfig,
window, NULL);
    if(mEglSurface == EGL_NO_SURFACE)
    {
        LOGE("eglCreateWindowSurface  error");
        return -1;
    }

    // 7、使用
    if(!eglMakeCurrent(mEglDisplay, mEglSurface, mEglSurface,
mEglContext))
    {
```

```
        LOGE("eglMakeCurrent  error");
        return -1;
    }
    LOGD("egl init success! ");
    return 0;
}

int EglHelper::swapBuffers() {

    if(mEglDisplay != EGL_NO_DISPLAY && mEglSurface != EGL_NO_SURFACE)
    {
        if(eglSwapBuffers(mEglDisplay, mEglSurface))
        {
            return 0;
        }
    }
    return -1;
}

void EglHelper::destroyEgl() {

    if(mEglDisplay != EGL_NO_DISPLAY)
    {
        eglMakeCurrent(mEglDisplay, EGL_NO_SURFACE, EGL_NO_SURFACE,
EGL_NO_CONTEXT);
    }
    if(mEglDisplay != EGL_NO_DISPLAY && mEglSurface != EGL_NO_SURFACE)
    {
        eglDestroySurface(mEglDisplay, mEglSurface);
        mEglSurface = EGL_NO_SURFACE;
    }
    if(mEglDisplay != EGL_NO_DISPLAY && mEglContext != EGL_NO_CONTEXT)
{
        eglDestroyContext(mEglDisplay, mEglContext);
        mEglContext = EGL_NO_CONTEXT;
    }
```

```
        if(mEglDisplay != EGL_NO_DISPLAY)
        {
            eglTerminate(mEglDisplay);
            mEglDisplay = EGL_NO_DISPLAY;
        }
    }
```

自己EGL环境创建完毕，我们通过JNI调用起来看看效果，native-lib.cpp：

```
    #include <jni.h>
    #include <string>
    #include "eglhelper/EglHelper.h"
    #include <cstdint>
    #include "android/native_window.h"
    #include "android/native_window_jni.h"
    #include "GLES3/gl3.h"

    jlong eglHelperNativePtrInit(JNIEnv *env, jobject thiz) {
        EglHelper *eglHelper = new EglHelper();
        return reinterpret_cast<uintptr_t>(eglHelper);
    }

    void eglSurfaceCreated(JNIEnv *env, jobject thiz,jlong native_ptr,
    jobject surface) {
        if(native_ptr != 0){
            EglHelper *eglHelper = reinterpret_cast<EglHelper *>
    (native_ptr);
            ANativeWindow *nativeWindow = ANativeWindow_fromSurface(env,
    surface);
            eglHelper->initEgl(nativeWindow);
        }
    }

    void eglSurfaceChanged(JNIEnv *env, jobject thiz,jlong native_ptr,
    jint width,jint height) {
```

```
    if(native_ptr != 0){
        //设置视口大小
        glViewport(0, 0, width, height);
        // 绿色清屏
//        glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
        // 蓝色清屏
        glClearColor(0.0f, 0.0f, 1.0f, 1.0f);
        glClear(GL_COLOR_BUFFER_BIT);
        EglHelper *eglHelper = reinterpret_cast<EglHelper *>
(native_ptr);
        eglHelper->swapBuffers();
    }
}

void eglSurfaceDestroyed(JNIEnv *env, jobject thiz,jlong native_ptr) {
    if(native_ptr != 0){
        EglHelper *eglHelper = reinterpret_cast<EglHelper *>
(native_ptr);
        delete eglHelper;
    }
}

static JNINativeMethod nativeMethod_EGLHelper[] = {
        // Java中的函数名
        {"n_nativePtrInit",
                // 函数签名信息
         "()J",
                // native的函数指针
         (jlong *) (eglHelperNativePtrInit)},

        {"n_surfaceCreated",
                // 函数签名信息
         "(JLandroid/view/Surface;)V",
                // native的函数指针
         (void *) (eglSurfaceCreated)},
```

```
        {"n_surfaceChanged",
                // 函数签名信息
         "(JII)V",
                // native的函数指针
         (void *) (eglSurfaceChanged)},

        {"n_surfaceDestroyed",
                // 函数签名信息
         "(J)V",
                // native的函数指针
         (void *) (eglSurfaceDestroyed)},
};

static int RegisterNativeMethods(JNIEnv *env, const char *className,
JNINativeMethod *methods, int methodNum)
{
    jclass clazz = env->FindClass(className);
    if (clazz == NULL)
    {
        return JNI_FALSE;
    }
    if (env->RegisterNatives(clazz, methods, methodNum) < 0)
    {
        return JNI_FALSE;
    }
    return JNI_TRUE;
}

// 类库加载时自动调用
JNIEXPORT jint JNICALL JNI_OnLoad(JavaVM *vm, void *reversed)
{
    JNIEnv *env = NULL;
    // 初始化JNIEnv
    if(vm->GetEnv(reinterpret_cast<void **>(&env), JNI_VERSION_1_6) !=
JNI_OK){
        return JNI_FALSE;
```

```
    }
    // 动态注册

 RegisterNativeMethods(env,"com/fly/opengles/learn/egl/EGLHelper",nati
veMethod_EGLHelper,sizeof(nativeMethod_EGLHelper) /
sizeof(JNINativeMethod) );
    // 返回JNI使用的版本
    return JNI_VERSION_1_6;
}
```

上述 native-lib.cpp 涉及到了之前介绍过的JNI函数签名、动态注册等相关知识点，忘记了的童鞋可往回看之前的记录。

如无意外，运行看到的是一个蓝屏画面则说明EGL环境搭建成功了，后续开启你的Opengl炫酷之旅吧!!!

OpenglEs之着色器

扯谈

如果守时也是一种诚信，诚信就能上教科书，那么我天天上班不迟到算守时吧？能上教科书吗？

前言

在前面我们介绍了 OpenglEs之EGL环境搭建 ，在后面的例子中，我们将无可避免地需要使用到着色器。而着色器才是Opengl的灵魂所在，有了着色器才有了Opengl天马行空的世界。

图形渲染管线

要想理解什么是着色器以及着色器的作用就必须先了解下图形渲染管线。

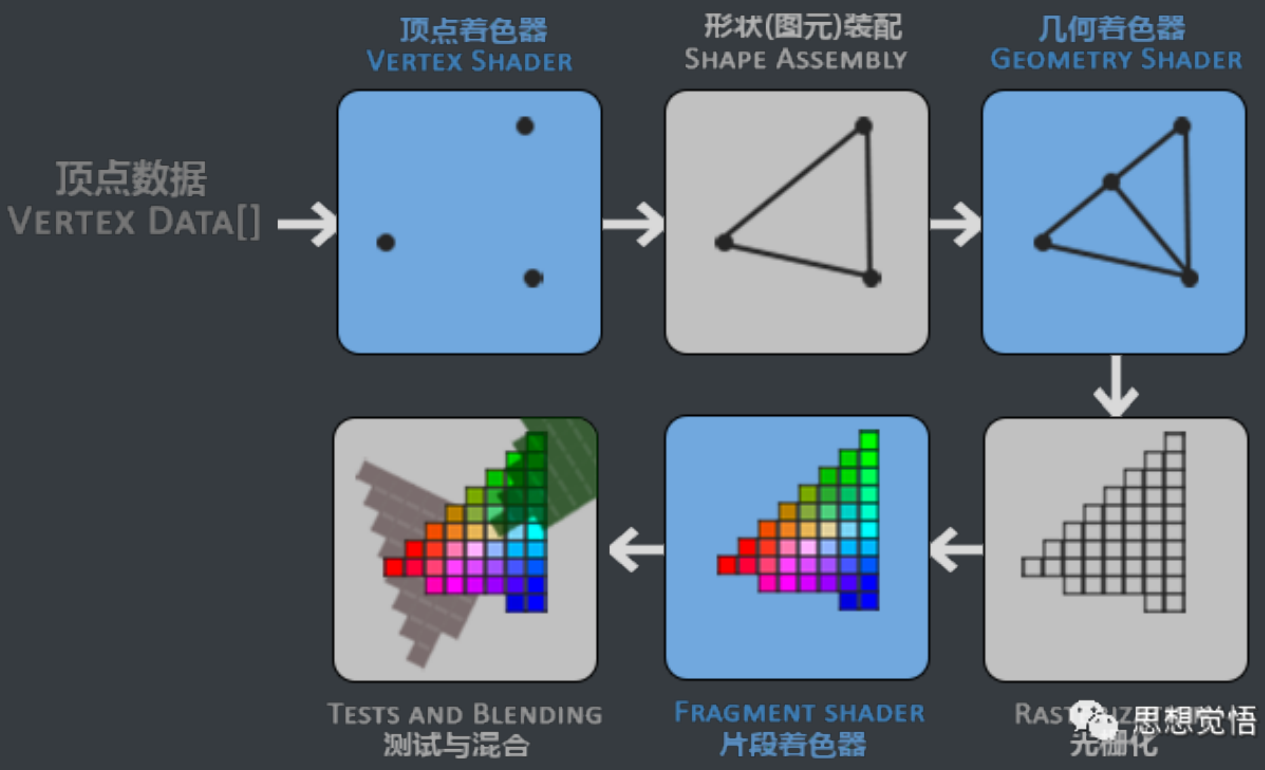
在OpenGL中，任何事物都在3D空间中，而屏幕和窗口却是2D像素数组，这导致OpenGL的大部分工作都是关于把3D坐标转变为适应你屏幕的2D像素。将OpenGL中的3D世界转换到屏幕窗口中的2D世界中展示出来的过程称为图形渲染管线。也可以说将一堆原始图形数据途经一个输送管道，期间经过各种变化处理最终出现在屏幕的过程就是图形渲染管线。图形渲染管线可以被划分为两个主要部分：第一部分把你的3D坐标转换为2D坐标，第二部分是把2D坐标转变为实际的有颜色的像素。

图形渲染管线可以被划分为几个阶段，每个阶段将会把前一个阶段的输出作为输入。所有这些阶段都是高度专门化的（它们都有一个特定的函数），并且很容易并行执行。正是由于它们具有并行执行的特性，当今大多数显卡都有成千上万的小处理核心，它们在GPU上为每一个（渲染管线）阶段运行各自的小程序，从而在图形渲染管线中快速处理你的数据。这些小程序叫做着色器(Shader)。

着色器

着色器是进行三维图形学编程的先进方法，从某种意义上来说 Shader 的出现是图形学中的一种”退步”，因为在这之前所有的功能都直接由固定管线提供，而开发人员只需要为其指定参数（如光照属性、旋转角度等），但是由于 Shader 的出现这些功能现在都需要开发者自己通过 Shader 实现。尽管如此，这种可编程性能够提供给开发者更多的灵活性和创造性。

下图是Opengl图形渲染管线的抽象过程：



蓝色部分代表的是我们可以注入自定义的着色器的部分。也就说我们通过自定义顶点着色器、几何着色器和片段着色器就可以实现我们想要各种效果，同时几何着色器又是可选的，一般使用默认的即可，因此我们只需将重点放在顶点着色器和片段着色器即可。

下面参照上图，简单介绍一下各个阶段的主要功能职责：

- 顶点着色器
  图形渲染管线的第一个部分是顶点着色器(Vertex Shader)，它把一个单独的顶点作为输入。顶点着色器主要的目的是把3D坐标转为另一种3D坐标，同时顶点着色器允许我们对顶点属性进行一些基本处理，例如矩阵变换等。
- 图元装配
  图元装配阶段将顶点着色器输出的所有顶点作为输入，并所有的点装配成指定图元的形状。例如绘制正方形时，就是通过两个三角形组装成正方形，绘制多面体时，就是将n多个三角形组装成多面体的过程。

- 几何着色器
  图元装配阶段的输出会传递给几何着色器。几何着色器把图元形式的一系列顶点的集合作为输入，它可以通过产生新顶点构造出新的（或是其它的）图元来生成其他形状。
- 光栅化
  几何着色器的输出会被传入光栅化阶段，这里它会把图元映射为最终屏幕上相应的像素，生成供片段着色器(Fragment Shader)使用的片段(Fragment)。在片段着色器运行之前会执行裁切(Clipping)。裁切会丢弃超出你的视图以外的所有像素，用来提升执行效率。
- 片段着色器
  片段着色器的主要目的是计算一个像素的最终颜色，这也是所有OpenGL高级效果产生的地方。通常，片段着色器包含3D场景的数据（比如光照、阴影、光的颜色等等），这些数据可以被用来计算最终像素的颜色。
- 测试与混合
  这个阶段检测片段的对应的深度（和模板(Stencil)）值，用它们来判断这个像素是其它物体的前面还是后面，决定是否应该丢弃。这个阶段也会检查alpha值（alpha值定义了一个物体的透明度）并对物体进行混合(Blend)。所以，即使在片段着色器中计算出来了一个像素输出的颜色，在渲染多个三角形的时候最后的像素颜色也可能完全不同。

GLSL

OpenGL着色器是用OpenGL着色器语言(OpenGL Shading Language, GLSL)写成的。

在OpenGL中，我们必须定义至少一个顶点着色器和一个片段着色器，因为GPU中没有默认的顶点/片段着色器。

我们来看一个最简单的顶点着色器的例子：

```
#version 300 es
in vec4 aPos;
void main()
{
    gl_Position = aPos;
}
```

GLSL的语法与C语言很类似，相信有过编程经验的伙伴们大概都能看懂以上这个着色器。

顶点着色器的第一行 `#version 300 es` 首先声明了着色器版本号，300的意思是代表着色器语言需要是3.0之后的版本。

第二行 `in vec4 aPos;` 声明了一个名为aPos的4个分量的输入变量。其中int代表的是定义输入变量，也就是说可以利用这个变量通过CPU向GPU传递数据，
与之对应的是使用out关键字声明的输出变量。注意，int和out是Opengl 3.0之后所做的修改，在Opengl 2.0中与之对应的关键字是attribute和varying，也就是说在Opengl 3中attribute改成了in而varying改成了out。

后面定义的就是一个main函数，这个没什么好解析的，就像所有语言程序一样，作为程序的一个入口，而gl_Position是Opengl中顶点着色器的一个内建输出变量，代表的是最终经过处理后的顶点坐标。

下面我们再来看一个简单的片段着色器的例子：

```
#version 300 es
precision mediaump float;
out vec4 FragColor;
void main()
{
    FragColor = vec4(1.0f, 0.5f, 0.2f, 1.0f);
}
```

正如顶点着色器那样，片段着色器的第一行也是声明着色器的版本。第二行 `precision mediaump float;` 则是声明着色器中浮点变量的默认精度。

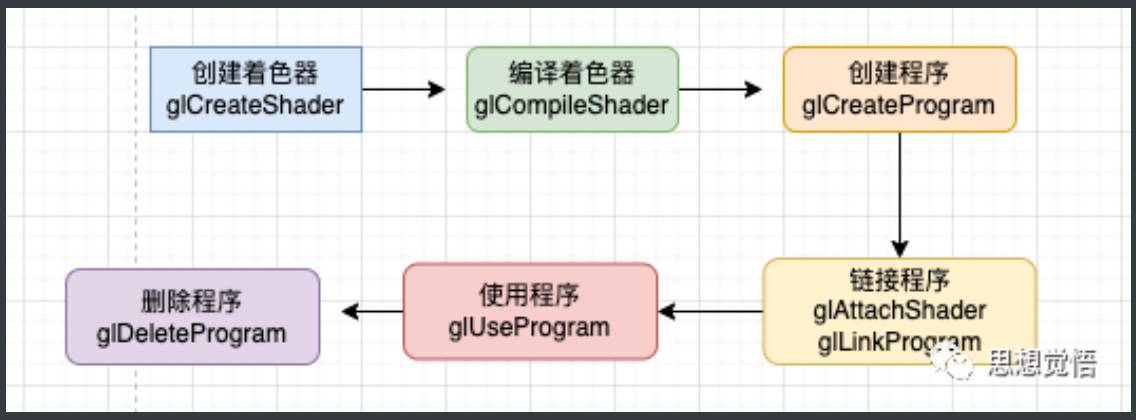
在Opengl ES 3的版本中我们需要手动地使用out关键字声明输出着色器的颜色变量，例如上述例子中的第三行，而在Opengl ES2中这不是必须的，因为在Opengl ES2中直接使用内置的变量gl_FragColor即可。至于为什么这样做，笔者也不得而知...



这里只是通过两个简单的例子让大家对着色器有一个初步的了解，更多关于着色器GLSL的编写规范大家可以参考相关的资料或者书籍进行自我学习，知识繁多，不是三言两语可描述清楚，这里就不再累赘。。。

着色器链接与使用

上面我们对着色器有了一个初步的认识，那么这些着色器小程序如何使用起来呢?



如上图所示，着色器小程序需要通过创建、编译、链接、使用这些重要步骤。这些步骤也没什么好解析的，就像一套规则，大家遵循就是了。

以下是笔者写的一个加载使用着色器的一个工具ShaderUtils.h:

```
#ifndef NDK_OPENGLES_LEARN_SHADERUTILS_H
#define NDK_OPENGLES_LEARN_SHADERUTILS_H

#include "GLES3/gl3.h"
```

```
#include "Log.h"

static int loadShaders(int shaderType, const char *code)
{
    // 按照类型，创建着色器
    int shader = glCreateShader(shaderType);
    // 加载着色器代码
    glShaderSource(shader, 1, &code, 0);
    // 编译
    glCompileShader(shader);
    // 检测编译状态
    int result = GL_FALSE;
    glGetShaderiv(shader,GL_COMPILE_STATUS,&result);
    if(result != GL_TRUE){
        GLint infoLen = 0;
        glGetShaderiv(shader,GL_INFO_LOG_LENGTH,&infoLen);
        char error[infoLen + 1];
        // 获取编译错误
        glGetShaderInfoLog(shader, sizeof(error) /
sizeof(error[0]),&infoLen,error);
        LOGE("着色器编译失败:%s,%s",error,code);
    }
    return shader;
}

static int createProgram(const char *vertex , const char * fragment)
{
    int vertexShader = loadShaders(GL_VERTEX_SHADER, vertex);
    int fragmentShader = loadShaders(GL_FRAGMENT_SHADER, fragment);
    int program = glCreateProgram();
    glAttachShader(program, vertexShader);
    glAttachShader(program, fragmentShader);
    glLinkProgram(program);
    // 获取一下是否有错误
    GLint status = GL_FALSE;
    glGetProgramiv(program,GL_LINK_STATUS,&status);
```

```
        if(status != GL_TRUE){
            GLint infoLen = 0;
            glGetProgramiv(program,GL_INFO_LOG_LENGTH,&infoLen);
            char error[infoLen + 1];
            glGetProgramInfoLog(program,sizeof(error) / sizeof(error[0])
    ,&infoLen,error);
            LOGE("着色器程序链接失败:",error);
        }
        // 更严谨一点可通过glValidateProgram()验证着色器程序
        // 删除着色器
        glDeleteShader(vertexShader);
        glDeleteShader(fragmentShader);
        return program;
    }

    #endif //NDK_OPENGLES_LEARN_SHADERUTILS_H
```

关于着色器的介绍就到这里，要想用好着色器，需要在弄懂GLSL语法之后多加实践...

参考资料

《LearnOpenGL CN》
《OPENGL ES 3.0编程指南》

OpenglEs之三角形绘制

在前面我们已经在NDK层搭建好了EGL环境，也介绍了一些着色器相关的理论知识，那么这次我们就使用已经搭配的EGL绘制一个三角形吧。

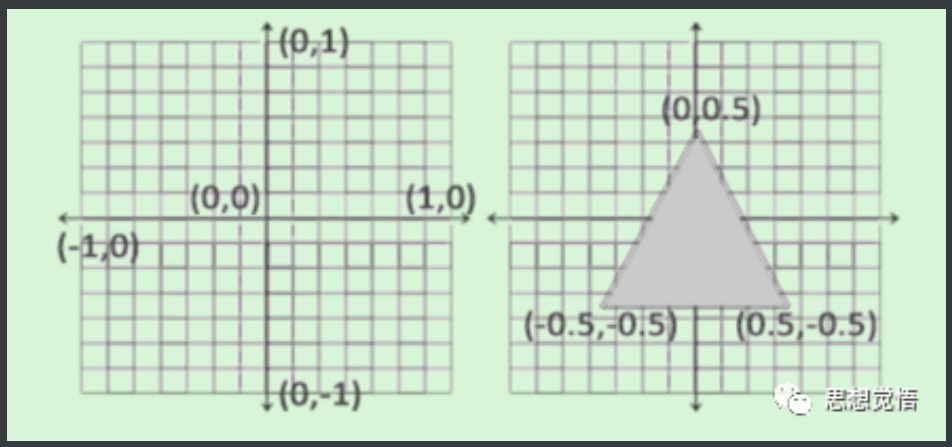
在Opengl ES的世界中，无论多复杂的形状都是由点、线或三角形组成的。因此三角形的绘制在Opengl ES中相当重要，犹比武林高手的内功心法...

坐标系

在Opengl ES中有很多坐标系，今天我们首先了解一些标准化的设备坐标。

标准化设备坐标(Normalized Device Coordinates, NDC)，一旦你的顶点坐标已经在顶点着色器中处理过，它们就是标准化设备坐标了，
标准化设备坐标是一个x、y和z的值都在-1.0到1.0的之间，任何落在-1和1范围外的坐标都会被丢弃/裁剪，不会显示在你的屏幕上。

如下图，在标准化设备坐标中，假设有一个正方形的屏幕，那么屏幕中心就是坐标原点，左上角就是坐标(-1,1)，右下角则是坐标(1,-1)。



上代码

这里需要说明亮点：

1. **在后续的实战例子中，经常会复用到前面介绍的demo的代码，因此如果是复用之前的代码逻辑，为了节省篇幅，笔者就不重复贴了。**
2. **在demo中为了简洁，并没有开启子线程作为GL线程，很明显这是不对，实际开发中都应该开启子线程对Opengl进行操作。**

首先为了后续方便使用，我们在Java层和C++分别创建一个BaseOpengl的基类：

BaseOpengl.java

```
public class BaseOpengl {

    // 三角形
    public static final int DRAW_TYPE_TRIANGLE = 0;

    public long glNativePtr;
    protected EGLHelper eglHelper;
    protected int drawType;

    public BaseOpengl(int drawType) {
        this.drawType = drawType;
        this.eglHelper = new EGLHelper();
    }

    public void surfaceCreated(Surface surface) {
        eglHelper.surfaceCreated(surface);
    }

    public void surfaceChanged(int width, int height) {
        eglHelper.surfaceChanged(width,height);
    }

    public void surfaceDestroyed() {
        eglHelper.surfaceDestroyed();
    }

    public void release(){
```

```
            if(glNativePtr != 0){
                n_free(glNativePtr,drawType);
                glNativePtr = 0;
            }
        }

        public void onGlDraw(){
            if(glNativePtr == 0){
                glNativePtr =
    n_gl_nativeInit(eglHelper.nativePtr,drawType);
            }
            if(glNativePtr != 0){
                n_onGlDraw(glNativePtr,drawType);
            }
        }

        // 绘制
        private native void n_onGlDraw(long ptr,int drawType);
        protected native long n_gl_nativeInit(long eglPtr,int drawType);
        private native void n_free(long ptr,int drawType);

    }
```

下面是C++的BaseOpengl：

BaseOpengl.h

```
    #ifndef NDK_OPENGLES_LEARN_BASEOPENGL_H
    #define NDK_OPENGLES_LEARN_BASEOPENGL_H
    #include "../eglhelper/EglHelper.h"
    #include "GLES3/gl3.h"
    #include <string>

    class BaseOpengl {
    public:
        EglHelper *eglHelper;
```

```
    GLint program{0};

public:
    BaseOpengl();
    // 析构函数必须是虚函数
    virtual ~BaseOpengl();
    // 加载着色器并链接成程序
    void initGlProgram(std::string ver,std::string fragment);
    // 绘制
    virtual void onDraw() = 0;
};


#endif //NDK_OPENGLES_LEARN_BASEOPENGL_H
```

注意基类的析构函数一定要是虚函数，为什么？如果不是虚函数的话则会导致无法完全析构，具体原因请大家面向搜索引擎编程。

BaseOpengl.cpp

```
#include "BaseOpengl.h"
#include "../utils/ShaderUtils.h"

BaseOpengl::BaseOpengl() {

}

void BaseOpengl::initGlProgram(std::string ver, std::string fragment)
{
    program = createProgram(ver.c_str(),fragment.c_str());
}

BaseOpengl::~BaseOpengl(){
    eglHelper = nullptr;
    if(program != 0){
        glDeleteProgram(program);
```

```
        }
    }
```

然后使用BaseOpengl自定义一个SurfaceView，为MyGLSurfaceView：

```
public class MyGLSurfaceView extends SurfaceView implements
SurfaceHolder.Callback {

    public BaseOpengl baseOpengl;
    private OnDrawListener onDrawListener;

    public MyGLSurfaceView(Context context) {
        this(context,null);
    }

    public MyGLSurfaceView(Context context, AttributeSet attrs) {
        super(context, attrs);
        getHolder().addCallback(this);
    }

    public void setBaseOpengl(BaseOpengl baseOpengl) {
        this.baseOpengl = baseOpengl;
    }

    public void setOnDrawListener(OnDrawListener onDrawListener) {
        this.onDrawListener = onDrawListener;
    }

    @Override
    public void surfaceCreated(@NonNull SurfaceHolder surfaceHolder) {
        if(null != baseOpengl){
            baseOpengl.surfaceCreated(surfaceHolder.getSurface());
        }
    }

    @Override
```

```
    public void surfaceChanged(@NonNull SurfaceHolder surfaceHolder,
int i, int w, int h) {
        if(null != baseOpengl){
            baseOpengl.surfaceChanged(w,h);
        }
        if(null != onDrawListener){
            onDrawListener.onDrawFrame();
        }
    }

    @Override
    public void surfaceDestroyed(@NonNull SurfaceHolder surfaceHolder)
{
        if(null != baseOpengl){
            baseOpengl.surfaceDestroyed();
        }
    }

    public interface OnDrawListener{
        void onDrawFrame();
    }
}
```

有了以上基类，既然我们的目标是绘制一个三角形，那么我们在Java层和C++层再新建一个TriangleOpengl的类吧，他们都继承TriangleOpengl：

TriangleOpengl.java

```
public class TriangleOpengl extends BaseOpengl{

    public TriangleOpengl() {
        super(BaseOpengl.DRAW_TYPE_TRIANGLE);
    }

}
```

C++ TriangleOpengl类，TriangleOpengl.h：

```
#ifndef NDK_OPENGLES_LEARN_TRIANGLEOPENGL_H
#define NDK_OPENGLES_LEARN_TRIANGLEOPENGL_H
#include "BaseOpengl.h"

class TriangleOpengl: public BaseOpengl{
public:
    TriangleOpengl();
    virtual ~TriangleOpengl();
    virtual void onDraw();

private:
    GLint positionHandle{-1};
    GLint colorHandle{-1};
};


#endif //NDK_OPENGLES_LEARN_TRIANGLEOPENGL_H
```

TriangleOpengl.cpp:

```
#include "TriangleOpengl.h"
#include "../utils/Log.h"

// 定点着色器
static const char *ver = "#version 300 es\n"
                         "in vec4 aColor;\n"
                         "in vec4 aPosition;\n"
                         "out vec4 vColor;\n"
                         "void main() {\n"
                         "    vColor = aColor;\n"
                         "    gl_Position = aPosition;\n"
                         "}";
```

```
// 片元着色器
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
                              "in vec4 vColor;\n"
                              "out vec4 fragColor;\n"
                              "void main() {\n"
                              "    fragColor = vColor;\n"
                              "}";

// 三角形三个顶点
const static GLfloat VERTICES[] = {
        0.0f,0.5f,
        -0.5f,-0.5f,
        0.5f,-0.5f
};

// rgba
const static GLfloat COLOR_ICES[] = {
        0.0f,0.0f,1.0f,1.0f
};

TriangleOpengl::TriangleOpengl():BaseOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
    colorHandle = glGetAttribLocation(program,"aColor");
    LOGD("program:%d",program);
    LOGD("positionHandle:%d",positionHandle);
    LOGD("colorHandle:%d",colorHandle);
}

TriangleOpengl::~TriangleOpengl() noexcept {

}

void TriangleOpengl::onDraw() {
    LOGD("TriangleOpengl onDraw");
```

```
        glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
        glClear(GL_COLOR_BUFFER_BIT);
        glUseProgram(program);
        /**
         * size 几个数字表示一个点，显示是两个数字表示一个点
         * normalized 是否需要归一化，不用，这里已经归一化了
         * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
         */

    glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);
        // 启用顶点数据
        glEnableVertexAttribArray(positionHandle);

        // 这个不需要glEnableVertexAttribArray
        glVertexAttrib4fv(colorHandle, COLOR_ICES);

        glDrawArrays(GL_TRIANGLES,0,3);

        glUseProgram(0);

        // 禁用顶点
        glDisableVertexAttribArray(positionHandle);
        if(nullptr != eglHelper){
            eglHelper->swapBuffers();
        }
        LOGD("TriangleOpengl onDraw--end");
    }
```

在前面的章节中我们介绍了着色器的创建、编译、链接等，但是缺少了具体使用方式，这里我们补充说明一下。

着色器的使用只要搞懂如何传递数据给着色器中变量。首先我们需要获取到着色器程序中的变量，然后赋值。

我们看上面的TriangleOpengl.cpp的构造函数：

```
TriangleOpengl::TriangleOpengl():BaseOpengl() {
    initGlProgram(ver,fragment);
    // 获取aPosition变量
    positionHandle = glGetAttribLocation(program,"aPosition");
    // 获取aColor
    colorHandle = glGetAttribLocation(program,"aColor");
    LOGD("program:%d",program);
    LOGD("positionHandle:%d",positionHandle);
    LOGD("colorHandle:%d",colorHandle);
}
```

由上，我们通过函数 `glGetAttribLocation` 获取了变量aPosition和aColor的句柄，这里我们定义的aPosition和aColor是向量变量，如果我们定义的是uniform统一变量的话，则需要使用函数 `glGetUniformLocation` 获取统一变量句柄。有了这些变量句柄，我们就可以通过这些变量句柄传递函数给着色器程序了，具体可参考TriangleOpengl.cpp的onDraw函数。

此外如果变量是一个统一变量(uniform)的话，则通过一系列的 `glUniform...` 函数传递参数。

这里说明一下函数 `glVertexAttribPointer` 的stride参数，一般情况下不会用到，传递0即可，但是如果需要提高性能，例如将顶点坐标和纹理/颜色坐标等放在同一个数组中传递，则需要使用到这个stride参数了，目前顶点坐标数组和其他数组是分离的，暂时可以不管。

在Activity中调用一下测试结果：

```
public class DrawTriangleActivity extends AppCompatActivity {

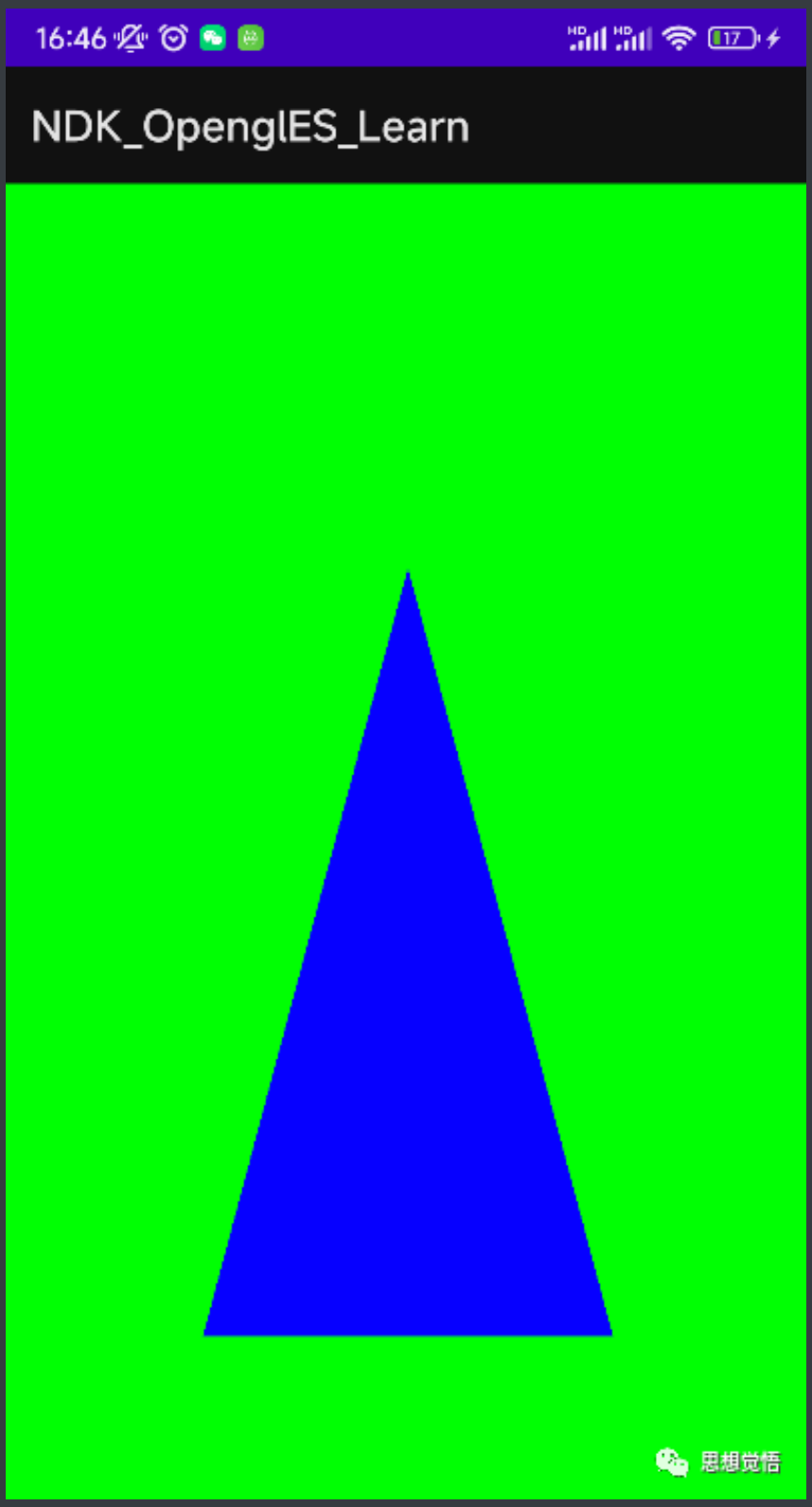
    private TriangleOpengl mTriangleOpengl;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_draw_triangle);
        MyGLSurfaceView glSurfaceView =
findViewById(R.id.my_gl_surface_view);
```

```
            mTriangleOpengl = new TriangleOpengl();
            glSurfaceView.setBaseOpengl(mTriangleOpengl);
            glSurfaceView.setOnDrawListener(new
    MyGLSurfaceView.OnDrawListener() {
                @Override
                public void onDrawFrame() {
                    mTriangleOpengl.onGlDraw();
                }
            });
        }

        @Override
        protected void onDestroy() {
            if(null != mTriangleOpengl){
                mTriangleOpengl.release();
            }
            super.onDestroy();
        }
    }
```

如果运行起来，看到一个蓝色的三角形，则说明三角形绘制成功啦!



源码

想来还是不贴源码链接了，纸上得来终觉浅，绝知此事要躬行。很多时候就是这样，你看着觉得很简单，实际如何还得动手敲，只有在敲的过程中出了问题，然后你解决了，只是才算是你的。

在这个系列完毕后再贴出整个项目demo的代码吧。。。s

Opengl ES之四边形绘制

四边形的绘制在Opengl ES是很重要的一项技巧，比如做视频播放器时视频的渲染就需要使用到Opengl ES绘制四边形的相关知识。然而在Opengl ES却没有直接提供
绘制四边形的相关函数，那么如何实现四边形的绘制呢?

在Opengl ES的世界中所有的元素都是由点、线和三角形组成的。众所周知，四边形可以拆分成两个三角形，因此四边形的绘制在Opengl ES的世界中可以分解成绘制两个三角形的任务。

关于三角形的绘制请参考之前的文章：OpenglEs之三角形绘制

代码准备

为了使得项目代码更好地展现我们的demo效果，我们先来将之前的代码抽象起来，做了个简单的封装，统一一个BaseGlActivity：

```
public class BaseGlActivity extends AppCompatActivity {

    protected BaseOpengl baseOpengl;
    private MyGLSurfaceView myGLSurfaceView;

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_base_gl);
        baseOpengl = createOpengl();
        myGLSurfaceView = findViewById(R.id.my_gl_surface_view);
        if(null != baseOpengl){
            myGLSurfaceView.setBaseOpengl(baseOpengl);
            myGLSurfaceView.setOnDrawListener(new
MyGLSurfaceView.OnDrawListener() {
```

```
                @Override
                public void onDrawFrame() {
                    baseOpengl.onGlDraw();
                }
            });
        }
    }

    @Override
    protected void onDestroy() {
        if(null != baseOpengl){
            baseOpengl.release();
        }
        super.onDestroy();
    }

   // 子类返回具体的Opelg实体即可
    public BaseOpengl createOpengl() {
        return null;
    }

}
```

其他代码，例如BaseOpengl和MyGLSurfaceView和以前的一样。

然后在Java层和C++都新建一个RectOpengl类，用来表示绘制四边形。

RectOpengl.java

```
public class RectOpengl extends BaseOpengl{

    public RectOpengl() {
        super(BaseOpengl.DRAW_TYPE_RECT);
    }
}
```

RectOpengl.h

```
#ifndef NDK_OPENGLES_LEARN_RECTOPENGL_H
#define NDK_OPENGLES_LEARN_RECTOPENGL_H
#include "BaseOpengl.h"

class RectOpengl:public BaseOpengl {
public:
    RectOpengl();
    virtual ~RectOpengl();
    virtual void onDraw();
private:
    GLint positionHandle{-1};
    GLint colorHandle{-1};
};

#endif //NDK_OPENGLES_LEARN_RECTOPENGL_H
```

六点顶点绘制四边形

这是所有人都能想到的最简单的方式了，每个三角形需要三个顶点，绘制两个三角形则需要六个顶点，通过调整顶点，将两个三角形紧挨着就组成了四边形。
因此，我们的顶点着色器和片段着色器都不需要修改，直接使用之前绘制三角形的那些着色器即可。

例如使用下面六个顶点绘制两个三角形：

```
// 使用绘制两个三角形组成一个矩形的形式  GL_TRIANGLES
//const static GLfloat VERTICES[] = {
//        // 第一个三角形
//        0.5f,0.5f,
//        -0.5f,0.5f,
//        -0.5f,-0.5f,
//        // 第二个三角形
//        -0.5f,-0.5f,
```

```
//        0.5f,-0.5f,
//        0.5f,0.5f
//};

// 6个顶点绘制两个三角形组成矩形  GL_TRIANGLES
// glDrawArrays(GL_TRIANGLES,0,6);
```

这个理解起来比较简单，笔者就不画说明图了。

四个顶点绘制四边形

使用四个顶点绘制两个三角形主要是复用了部分顶点，然后在 `glDrawArrays` 函数中使用绘制三角形带的方式进行绘制。

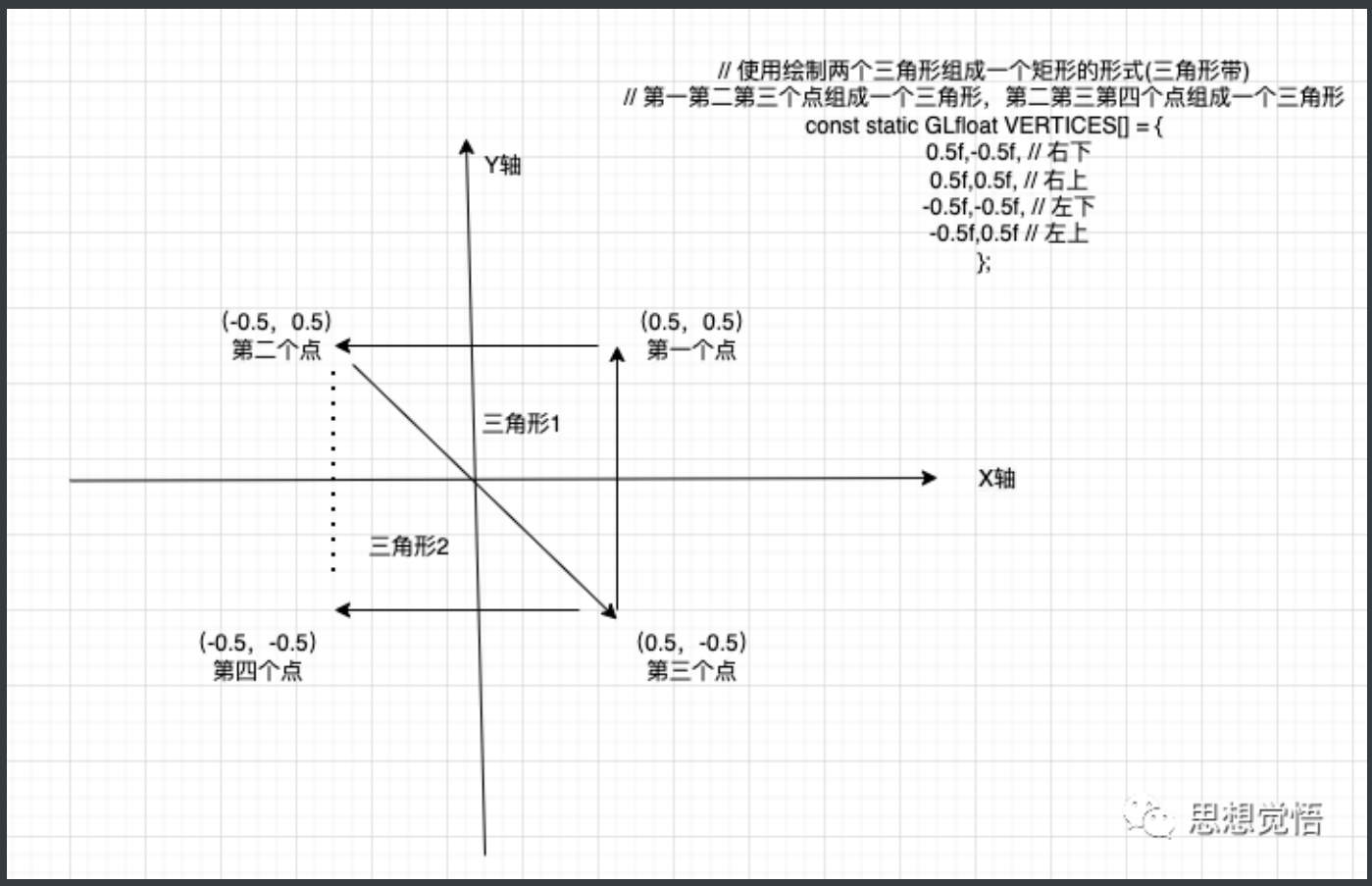
```
// 4个顶点绘制两个三角形组成矩形
glDrawArrays(GL_TRIANGLE_STRIP,0,4);
```

GL_TRIANGLE_STRIP会复用前面的顶点，因为顶点复用，因此这种方式可以绘制出 `n-2` 个三角形（n是顶点的个数）。
例如使用三角形带的方式搭配以下的顶点坐标即可绘制出一个矩形：

```
const static GLfloat VERTICES[] = {
        0.5f,-0.5f, // 右下
        0.5f,0.5f, // 右上
        -0.5f,-0.5f, // 左下
        -0.5f,0.5f // 左上
};
```

假如有1、2、3、4这四个顶点坐标，那么GL_TRIANGLE_STRIP绘制三角形的的方式是顶点1、2、3组成一个三角形，顶点2、3、4组成一个三角形。

具体看图：



RectOpengl完整代码：

```
#include "RectOpengl.h"
#include "../utils/Log.h"


// 顶点着色器
static const char *ver = "#version 300 es\n"
                         "in vec4 aColor;\n"
                         "in vec4 aPosition;\n"
                         "out vec4 vColor;\n"
                         "void main() {\n"
                         "    vColor = aColor;\n"
                         "    gl_Position = aPosition;\n"
                         "}";

// 片元着色器
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
```

```
                                "in vec4 vColor;\n"
                                "out vec4 fragColor;\n"
                                "void main() {\n"
                                "    fragColor = vColor;\n"
                                "}";

// 使用绘制两个三角形组成一个矩形的形式  GL_TRIANGLES
//const static GLfloat VERTICES[] = {
//        // 第一个三角形
//        0.5f,0.5f,
//        -0.5f,0.5f,
//        -0.5f,-0.5f,
//        // 第二个三角形
//        -0.5f,-0.5f,
//        0.5f,-0.5f,
//        0.5f,0.5f
//};

// 使用绘制两个三角形组成一个矩形的形式(三角形带)
// 第一第二第三个点组成一个三角形，第二第三第四个点组成一个三角形
const static GLfloat VERTICES[] = {
        0.5f,-0.5f, // 右下
        0.5f,0.5f, // 右上
        -0.5f,-0.5f, // 左下
        -0.5f,0.5f // 左上
};


// rgba
const static GLfloat COLOR_ICES[] = {
        0.0f,0.0f,1.0f,1.0f
};

RectOpengl::RectOpengl():BaseOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
```

```
    colorHandle = glGetAttribLocation(program,"aColor");
    LOGD("program:%d",program);
    LOGD("positionHandle:%d",positionHandle);
    LOGD("colorHandle:%d",colorHandle);
}

void RectOpengl::onDraw() {
    // 清屏
    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);
    /**
     * size 几个数字表示一个点，显示是两个数字表示一个点
     * normalized 是否需要归一化，不用，这里已经归一化了
     * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
     */

 glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);

    // 这个不需要glEnableVertexAttribArray
    glVertexAttrib4fv(colorHandle, COLOR_ICES);

    // 6个顶点绘制两个三角形组成矩形
    // glDrawArrays(GL_TRIANGLES,0,6);

    // 4个顶点绘制两个三角形组成矩形
    glDrawArrays(GL_TRIANGLE_STRIP,0,4);

    glUseProgram(0);

    // 禁用顶点
    glDisableVertexAttribArray(positionHandle);
    if(nullptr != eglHelper){
        eglHelper->swapBuffers();
```

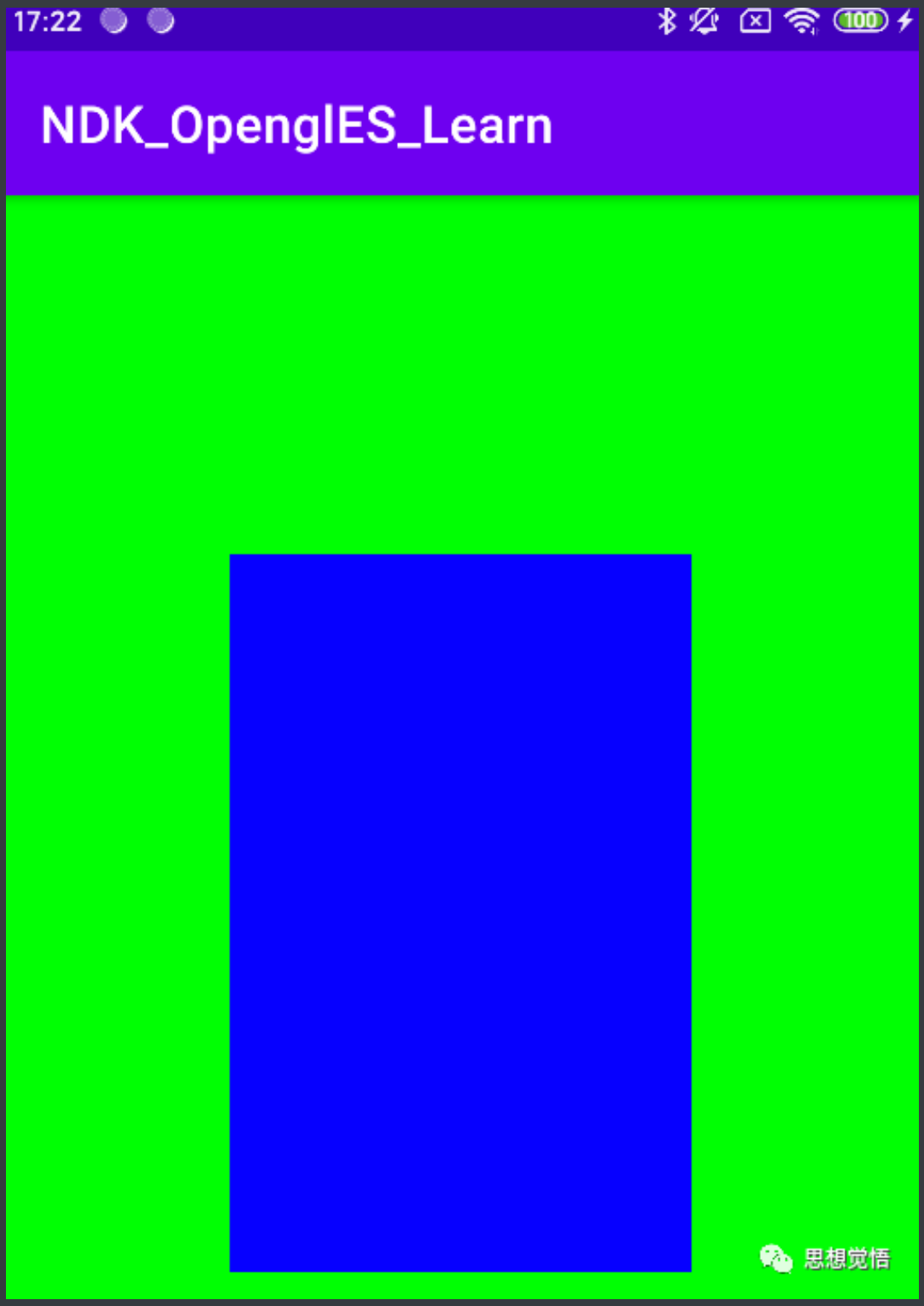
```
    }
}

RectOpengl::~RectOpengl() noexcept {

}
```

DrawRectActivity.java

```
public class DrawRectActivity extends BaseGlActivity{
    @Override
    public BaseOpengl createOpengl() {
        return new RectOpengl();
    }
}
```



运行结果

Opengl ES之纹理贴图

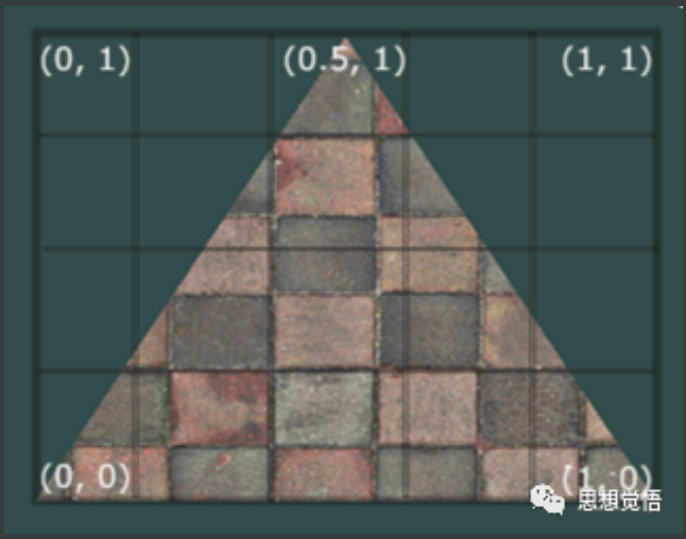
纹理可以理解为一个二维数组，它可以存储大量的数据，这些数据可以发送到着色器上。一般情况下我们所说的纹理是表示一副2D图，此时纹理存储的数据就是这个图的像素数据。

所谓的纹理贴图，就是使用Opengl将这个纹理数据渲染出来，这个过程有点像装修工人给墙体贴瓷砖，而瓷砖好比作纹理。

纹理坐标

如果为了将一副纹理图贴到Opengl绘制的一个矩形上，那么就需要解决一个问题，如何知道矩形的具体某个点对应纹理图的某个点呢？为了解决这个问题就引出了纹理坐标，
通过矩形的顶点坐标与纹理坐标关联，这样就明确了每个顶点应该显示纹理图的那部分像素数据。

纹理坐标在x和y轴上，范围为0到1之间。使用纹理坐标获取纹理颜色叫做采样(Sampling)。纹理坐标起始于(0, 0)，也就是纹理图片的左下角，终始于(1, 1)，即纹理图片的右上角，如下图所示：



纹理环绕

纹理坐标的值介于0到1之间，如果我们把纹理坐标设置成大于1那么会发生什么呢？OpenGL默认的行为是重复这个纹理图像，那么利用这个默认的特性我们能做些什么呢？那么比较火的抖音四分屏、九分屏滤镜不就是可以用这个特性巧妙地实现吗。

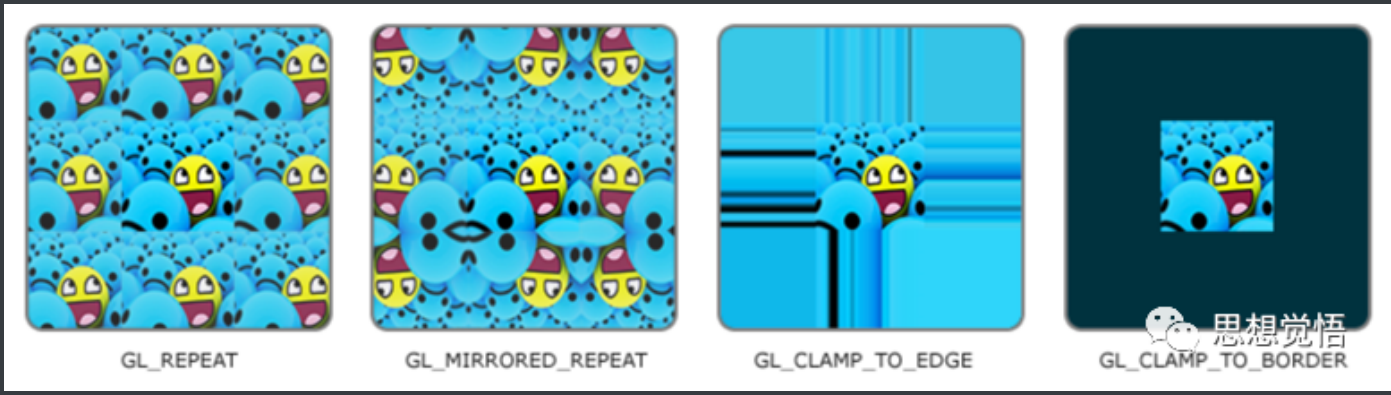
以下是通过改变纹理坐标实现四分屏和九分屏的一个小技巧：

```
// 4分屏
const static GLfloat TEXTURE_COORD[] = {
        2.0f,2.0f, // 右下
        2.0f,0.0f, // 右上
        0.0f,2.0f, // 左下
        0.0f,0.0f // 左上
};

// 九分屏
const static GLfloat TEXTURE_COORD[] = {
        3.0f,3.0f, // 右下
        3.0f,0.0f, // 右上
        0.0f,3.0f, // 左下
        0.0f,0.0f // 左上
};
```

当然，当纹理坐标超过1这个范围时，Opengl也提供了其他的选择，例如：

```
GL_REPEAT // 对纹理的默认行为。重复纹理图像。
GL_MIRRORED_REPEAT //和GL_REPEAT一样，但每次重复图片是镜像放置的。
GL_CLAMP_TO_EDGE //纹理坐标会被约束在0到1之间，超出的部分会重复纹理坐标的边缘，产生一种边缘被拉伸的效果。
GL_CLAMP_TO_BORDER //超出的坐标为用户指定的边缘颜色。
```



> 纹理环绕效果图

以上特性可以通过函数 `glTexParameteri` 设置：

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_MIRRORED_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_MIRRORED_REPEAT);
```

纹理过滤

纹理过滤实际就是纹理在放大缩小的过程中像素的处理方式。其中在Opengl ES常用的两种纹理过滤方式是GL_NEAREST（邻近过滤)和GL_LINEAR（也叫线性过滤)。

- GL_NEAREST是OpenGL默认的纹理过滤方式。当设置为GL_NEAREST的时候，OpenGL会选择中心点最接近纹理坐标的那个像素。
- GL_LINEAR（也叫线性过滤，(Bi)linear Filtering）它会基于纹理坐标附近的纹理像素，计算出一个插值，近似出这些纹理像素之间的颜色。一个纹理像素的中心距离纹理坐标越近，那么这个纹理像素的颜色对最终的样本颜色的贡献越大。

GL_NEAREST产生了颗粒状的图案，我们能够清晰看到组成纹理的像素，而GL_LINEAR能够产生更平滑的图案，很难看出单个的纹理像素。

同理，纹理过滤特性也是通过函数 `glTexParameteri` 设置：

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
```

纹理单元

纹理单元的主要目的是让我们在着色器中可以使用多于一个的纹理。通过把纹理单元赋值给采样器，我们可以一次绑定多个纹理，只要我们首先激活对应的纹理单元。
例如使用Opengl ES对视频解码后的YUV进行渲染就需要用到纹理单元的相关知识点。

Opengl中纹理的使用

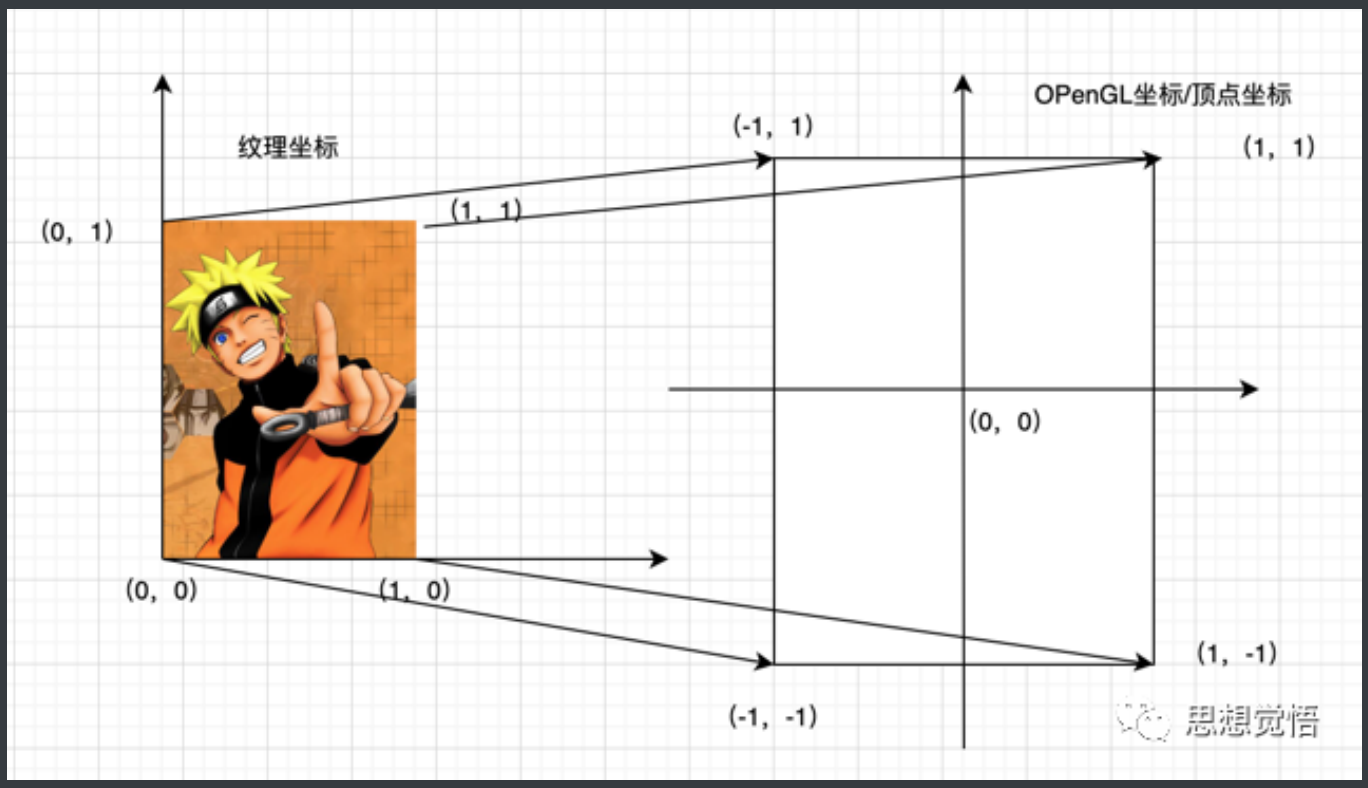
在Opengl中使用纹理主要有以下几个步骤：

- 创建纹理 `glGenTextures`
- 激活纹理 `glActiveTexture`
- 绑定纹理 `glBindTexture`，传递特定的纹理id进行绑定
- 上传纹理数据 `glTexImage2D`
- 解除纹理绑定， `glBindTexture`，传递0进行解除绑定

纹理坐标映射关系

在了解纹理贴图之前我们先回顾一下三个坐标系统，分别是纹理坐标系统、手机屏幕坐标系统、Opengl坐标系统。这三个坐标系统的的原点各不相同，纹理坐标系统我们上面已经介绍过了，这里不再重复。而手机屏幕坐标系统则是原点位于左上角，X轴向右为正，Y轴向下为正的坐标系统。
而Opengl坐标系统则是原点位于中心，X轴向右为正，Y轴向下为正，其值介于-1到1之间的一套坐标系统。

既然纹理贴图就像装修工人贴瓷砖一样，那么直接将纹理坐标和Opengl的顶点坐标一一对应起来即可，也就是如下图：

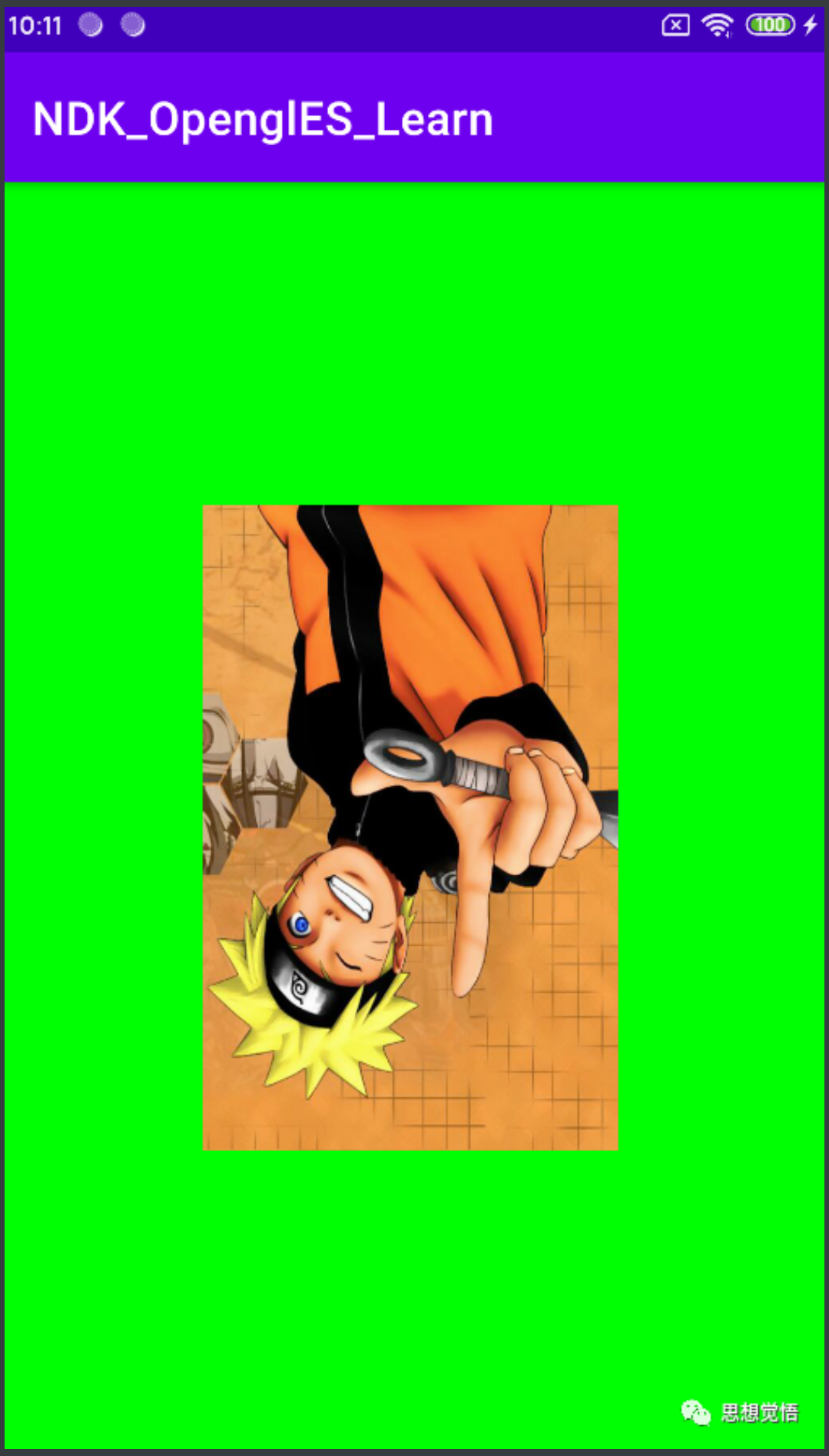


我们按照这个映射关系建立贴图：

```
// 顶点坐标，使用绘制两个三角形组成一个矩形的形式(三角形带)
// 第一第二第三个点组成一个三角形，第二第三第四个点组成一个三角形
const static GLfloat VERTICES[] = {
        0.5f,-0.5f, // 右下
        0.5f,0.5f, // 右上
        -0.5f,-0.5f, // 左下
        -0.5f,0.5f // 左上
};

// 纹理坐标（原点在左下角，这样贴图看到的会是倒置的
const static GLfloat TEXTURE_COORD[] = {
        1.0f,0.0f, // 右下
        1.0f,1.0f, // 右上
        0.0f,0.0f, // 左下
        0.0f,1.0f // 左上
};
```

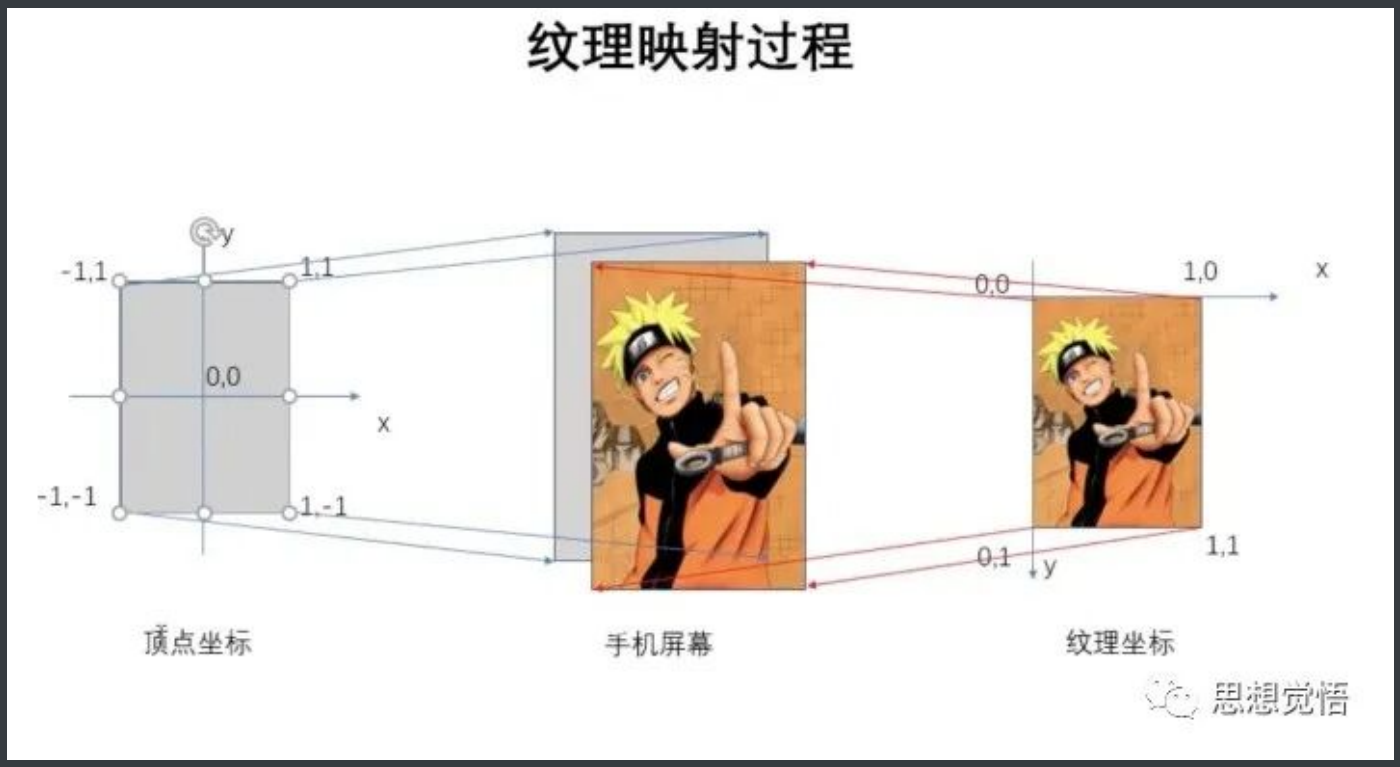
运行发现图是贴上去了，但是看到的贴图却是倒置的，如下：



这是为什么呢?

因为纹理的生成是由图片像素来生成的，而图像的存储是从左上角开始的，但是纹理坐标原点却是在左下角的(笔者也不知道为什么要这么奇葩)，所以就产生了倒置现象，因此正确的映射关系应该是以图片的左上角为原点做映射才对，而这也刚好与手机屏幕坐标系统匹配。

也就说正确的映射关系是需要先将以左下角为原点的纹理坐标进行倒置，然后再建立映射关系，这也是为什么有些博客说纹理坐标的原点是在左上角的原因(其实这是不对的，纹理坐标就是在图片的左下角，说在左上角的就是一个技巧)，那么纹理坐标倒置后再映射如图：



废话少说，放码过来...

```
#include "TextureMapOpengl.h"

#include "../utils/Log.h"

// 顶点着色器
static const char *ver = "#version 300 es\n"
                         "in vec4 aPosition;\n"
                         "in vec2 aTexCoord;\n"
                         "out vec2 TexCoord;\n"
                         "void main() {\n"
                         "  TexCoord = aTexCoord;\n"
```

```
                            "  gl_Position = aPosition;\n"
                            "}";

// 片元着色器
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
                              "out vec4 FragColor;\n"
                              "in vec2 TexCoord;\n"
                              "uniform sampler2D ourTexture;\n"
                              "void main()\n"
                              "{\n"
                              "    FragColor = texture(ourTexture,
TexCoord);\n"
                              "}";


// 使用绘制两个三角形组成一个矩形的形式(三角形带)
// 第一第二第三个点组成一个三角形，第二第三第四个点组成一个三角形
const static GLfloat VERTICES[] = {
        0.5f,-0.5f, // 右下
        0.5f,0.5f, // 右上
        -0.5f,-0.5f, // 左下
        -0.5f,0.5f // 左上
};

// 纹理坐标（原点在左下角，这样贴图看到的会是倒置的
//const static GLfloat TEXTURE_COORD[] = {
//        1.0f,0.0f, // 右下
//        1.0f,1.0f, // 右上
//        0.0f,0.0f, // 左下
//        0.0f,1.0f // 左上
//};

// 贴图纹理坐标（参考手机屏幕坐标系统，原点在左上角）
```

```
//由于对一个OpenGL纹理来说，它没有内在的方向性，因此我们可以使用不同的坐标把它定向到任何我们喜欢的方向上，然而大多数计算机图像都有一个默认的方向，它们通常被规定为y轴向下，X轴向右
const static GLfloat TEXTURE_COORD[] = {
        1.0f,1.0f, // 右下
        1.0f,0.0f, // 右上
        0.0f,1.0f, // 左下
        0.0f,0.0f // 左上
};

// 四分屏  GL_REPEAT环绕方式
//const static GLfloat TEXTURE_COORD[] = {
//        2.0f,2.0f, // 右下
//        2.0f,0.0f, // 右上
//        0.0f,2.0f, // 左下
//        0.0f,0.0f // 左上
//};

// 九分屏 GL_REPEAT环绕方式
//const static GLfloat TEXTURE_COORD[] = {
//        3.0f,3.0f, // 右下
//        3.0f,0.0f, // 右上
//        0.0f,3.0f, // 左下
//        0.0f,0.0f // 左上
//};


TextureMapOpengl::TextureMapOpengl():BaseOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
    textureHandle = glGetAttribLocation(program,"aTexCoord");
    textureSampler = glGetUniformLocation(program,"ourTexture");
    LOGD("program:%d",program);
    LOGD("positionHandle:%d",positionHandle);
    LOGD("textureHandle:%d",textureHandle);
    LOGD("textureSample:%d",textureSampler);
```

```
}

void TextureMapOpengl::setPixel(void *data, int width, int height, int length) {
    LOGD("texture setPixel");
    glGenTextures(1, &textureId);

    // 激活纹理，注意以下这个两句是搭配的，glActiveTexture激活的是那个纹理，就设置的sampler2D是那个
    // 默认是0，如果不是0的话，需要在onDraw的时候重新激活一下?
//    glActiveTexture(GL_TEXTURE0);
//    glUniform1i(textureSampler, 0);

// 例如，一样的
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA, GL_UNSIGNED_BYTE, data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);

    glBindTexture(GL_TEXTURE_2D, textureId);

    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}
```

```
void TextureMapOpengl::onDraw() {
    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);

    // 激活纹理
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId);

    /**
     * size 几个数字表示一个点，显示是两个数字表示一个点
     * normalized 是否需要归一化，不用，这里已经归一化了
     * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
     */
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);

 glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);

    // 纹理坐标
    glEnableVertexAttribArray(textureHandle);

 glVertexAttribPointer(textureHandle,2,GL_FLOAT,GL_FALSE,0,TEXTURE_COO
RD);

    // 4个顶点绘制两个三角形组成矩形
     glDrawArrays(GL_TRIANGLE_STRIP,0,4);

    glUseProgram(0);

    // 禁用顶点
    glDisableVertexAttribArray(positionHandle);
    if(nullptr != eglHelper){
```

```
            eglHelper->swapBuffers();
        }

        glBindTexture(GL_TEXTURE_2D, 0);
    }

    TextureMapOpengl::~TextureMapOpengl() {
        LOGD("TextureMapOpengl析构函数");
    }
```

仔细看注释多理解...



纹理贴图运行结果

Opengl ES之VBO和VAO

前言

本文主要介绍了什么是VBO/VAO，为什么需要使用VBO/VAO以及如何使用VBO和VAO。

VBO

什么是VBO

VBO(vertex Buffer Object)：顶点缓冲对象。是在显卡存储空间中开辟的一块区域，在显卡存储空间中开辟一块区域，用于存放顶点的各类属性信息。如顶点坐标、纹理坐标、顶点颜色等数据。
在渲染时直接从显VBO去取数据而不必与CPU进行数据交换。

为什么需要使用VBO

将顶点数据保存在内存中，在调用glDrawArrays或者glDrawElements等绘制方法前需要调用相应的方法将数据送入显存，I/O开销大，性能不够好。
若采用顶点缓冲区对象存放顶点数据，则不需要在每次绘制前都将顶点数据复制进显存，而是在初始化顶点缓冲区对象时一次性将顶点数据送入显存，
每次绘制时直接使用显存中的数据，可以大大提高渲染性能。

如何使用VBO

- 使用函数 `glGenBuffers` 和一个缓冲ID生成一个VBO对象：

```
unsigned int VBO;
glGenBuffers(1, &VBO);
```

- 使用函数 `glBindBuffer` 绑定顶点缓冲对象的缓冲类型是GL_ARRAY_BUFFER

OpenGL有很多缓冲对象类型，顶点缓冲对象的缓冲类型是GL_ARRAY_BUFFER。

```
glBindBuffer(GL_ARRAY_BUFFER, VBO);
```

- 使用函数 `glBufferData` 把定义好的顶点数据复制到缓冲的内存中：

```
// vertices表示顶点数组
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);
```

`glBufferData` 是一个专门用来把用户定义的数据复制到当前绑定缓冲的函数。它的第一个参数是目标缓冲的类型，其中VBO代表是的GL_ARRAY_BUFFER。第二个参数指定传输数据的大小(以字节为单位)，用一个简单的sizeof计算出顶点数据大小就行。
第三个参数是我们希望发送的实际数据。第四个参数指定了我们希望显卡如何管理给定的数据，它有三种形式：

> GL_STATIC_DRAW ： 数据不会或几乎不会改变。
> GL_DYNAMIC_DRAW：数据会被改变很多。
> GL_STREAM_DRAW ： 数据每次绘制时都会改变。

一般情况下位置数据不会改变，每次渲染调用时都保持原样，所以它的使用类型一般是GL_STATIC_DRAW。如果一个缓冲中的数据将频繁被改变，那么使用的类型就是GL_DYNAMIC_DRAW或GL_STREAM_DRAW，这样就能确保显卡把数据放在能够高速写入的内存部分。

- 用完后使用函数 `glDeleteBuffers` 删除缓冲区

今天我们以之前绘制四边形的实践为例子，使用VBO的方式来实现四边形的绘制：Opengl ES之四边形绘制

我们的目标是灵活使用Opengl绘制一个蓝色的四边形...

普通常规的绘制这里就不多说了，后续可以看代码，或者回顾之前的四边形绘制的文章，这里主要介绍搭配VBO的两种绘制方式：

首先它们使用的顶点着色器和片段着色器都是一样的，都是：

```
// 顶点着色器
static const char *ver = "#version 300 es\n"
                         "in vec4 aColor;\n"
                         "in vec4 aPosition;\n"
                         "out vec4 vColor;\n"
                         "void main() {\n"
                         "    vColor = aColor;\n"
                         "    gl_Position = aPosition;\n"
                         "}";

// 片元着色器
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
                              "in vec4 vColor;\n"
                              "out vec4 fragColor;\n"
                              "void main() {\n"
                              "    fragColor = vColor;\n"
                              "}";
```

1. 顶点坐标与颜色值坐标分离的方式(数组结构)

先看顶点数据与颜色数据：

```
// 使用绘制两个三角形组成一个矩形的形式(三角形带)
// 第一第二第三个点组成一个三角形，第二第三第四个点组成一个三角形
const static GLfloat VERTICES[] = {
        0.5f,-0.5f, // 右下
        0.5f,0.5f, // 右上
        -0.5f,-0.5f, // 左下
        -0.5f,0.5f, // 左上
};

// vbo颜色
```

```
const static GLfloat COLOR_ICES[] = {
        0.0f,0.0f,1.0f,1.0f,
        0.0f,0.0f,1.0f,1.0f,
        0.0f,0.0f,1.0f,1.0f,
        0.0f,0.0f,1.0f,1.0f,
};
```

VBO数据与绑定：

```
  // vbo
    glGenBuffers(3,vbo);
    glBindBuffer(GL_ARRAY_BUFFER,vbo[0]);

 glBufferData(GL_ARRAY_BUFFER,sizeof(VERTICES),VERTICES,GL_STATIC_DRAW
);

    // 颜色
    glBindBuffer(GL_ARRAY_BUFFER,vbo[1]);

 glBufferData(GL_ARRAY_BUFFER,sizeof(COLOR_ICES),COLOR_ICES,GL_STATIC_
DRAW);
```

主要绘制代码：

```
        // 使用VBO的方式绘制，顶点与颜色分开
    //    glBindBuffer(GL_ARRAY_BUFFER,vbo[0]);
    //    glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,(void
    *)0);
    //    // 启用顶点数据
    //    glEnableVertexAttribArray(positionHandle);
    //
    //    glBindBuffer(GL_ARRAY_BUFFER,vbo[1]);
    //    glVertexAttribPointer(colorHandle,4,GL_FLOAT,GL_FALSE,0,(void
    *)0);
    //    // 启用颜色顶点数据
    //    glEnableVertexAttribArray(colorHandle);
    //    // 解除绑定
    //    glBindBuffer(GL_ARRAY_BUFFER,0);
```

2. 顶点坐标与颜色值坐标结合的方式(结构数组)

顶点数据与颜色数据混合打乱:

```
    const static GLfloat VERTICES_AND_COLOR[] = {
            0.5f,-0.5f, // 右下
            // 颜色
            0.0f,0.0f,1.0f,1.0f,
            0.5f,0.5f, // 右上
            // 颜色
            0.0f,0.0f,1.0f,1.0f,
            -0.5f,-0.5f, // 左下
            // 颜色
            0.0f,0.0f,1.0f,1.0f,
            -0.5f,0.5f, // 左上
            // 颜色
            0.0f,0.0f,1.0f,1.0f,
    };
```

VBO数据绑定:

```
    // vbo
    glGenBuffers(3,vbo);
    glBindBuffer(GL_ARRAY_BUFFER,vbo[0]);

 glBufferData(GL_ARRAY_BUFFER,sizeof(VERTICES),VERTICES,GL_STATIC_DRAW
);

    // 颜色
    glBindBuffer(GL_ARRAY_BUFFER,vbo[1]);

 glBufferData(GL_ARRAY_BUFFER,sizeof(COLOR_ICES),COLOR_ICES,GL_STATIC_
DRAW);

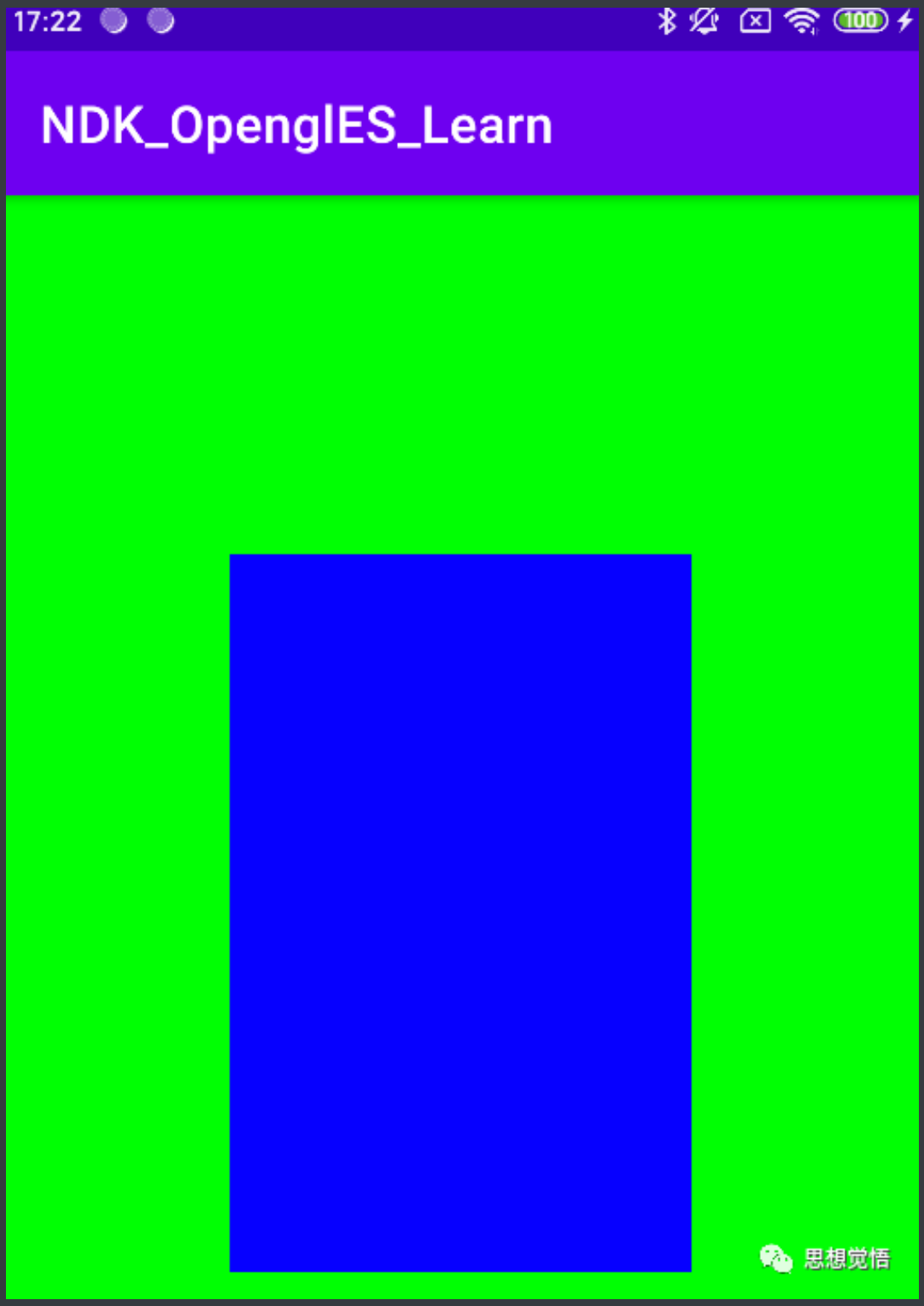
    // 顶点与颜色混合，先顶点坐标，再颜色坐标
    glBindBuffer(GL_ARRAY_BUFFER,vbo[2]);

 glBufferData(GL_ARRAY_BUFFER,sizeof(VERTICES_AND_COLOR),VERTICES_AND_
COLOR
                ,GL_STATIC_DRAW);
```

主要绘制代码：

```
    // VBO绘制 顶点坐标与颜色坐标一起
//    glBindBuffer(GL_ARRAY_BUFFER,vbo[2]);
//    // stride 步长 每个顶点坐标之间相隔6个数据点，数据类型是float
//    glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,6 *
sizeof(float ),(void *)0);
//    // 启用顶点数据
//    glEnableVertexAttribArray(positionHandle);
//    // stride 步长 每个颜色坐标之间相隔6个数据点，数据类型是float，颜色坐标索
引从2开始
//    glVertexAttribPointer(colorHandle,4,GL_FLOAT,GL_FALSE,6 *
sizeof(float ) ,(void *)(2 * sizeof(float)) );
//    // 启用颜色顶点数据
//    glEnableVertexAttribArray(colorHandle);
//    // 解除绑定
//    glBindBuffer(GL_ARRAY_BUFFER,0);
```

它们的运行结果都是成功绘制了一个蓝色的四边形：



性能

在上面我们使用了多种不同的组合方式进行了四边形的绘制，其中有将顶点坐标和颜色坐标分离写在两个不同的数组的方式，也有将顶点坐标和颜色坐标组合写在同一个数组，然后使用步长(stride)和偏移量(*pointer)参数控制的方式进行绘制，那么这两种方式那种性能更佳呢?

其中将各种顶点坐标分离在不同数组的写法又成为数组结构，而将各种顶点坐标合并成一个数组的写法又称为结构数组，在《OPENGL ES 3.0编程指南》一书中作者指出，结构数组的写法性能更好。

> 在大部分情况下，答案是结构数组。
> 原因是，每个顶点的属性数据可以顺序方式读取，这最有可能造成高效的内存访问模式。

VAO

什么是VAO

VAO(vertex Array Object)：顶点数组对象。

注意： VAO是OpenGL ES 3.0之后才推出的新特性， 所以在使用VAO前需要确定OpenGL ES的版本是否是3.0之后的版本。

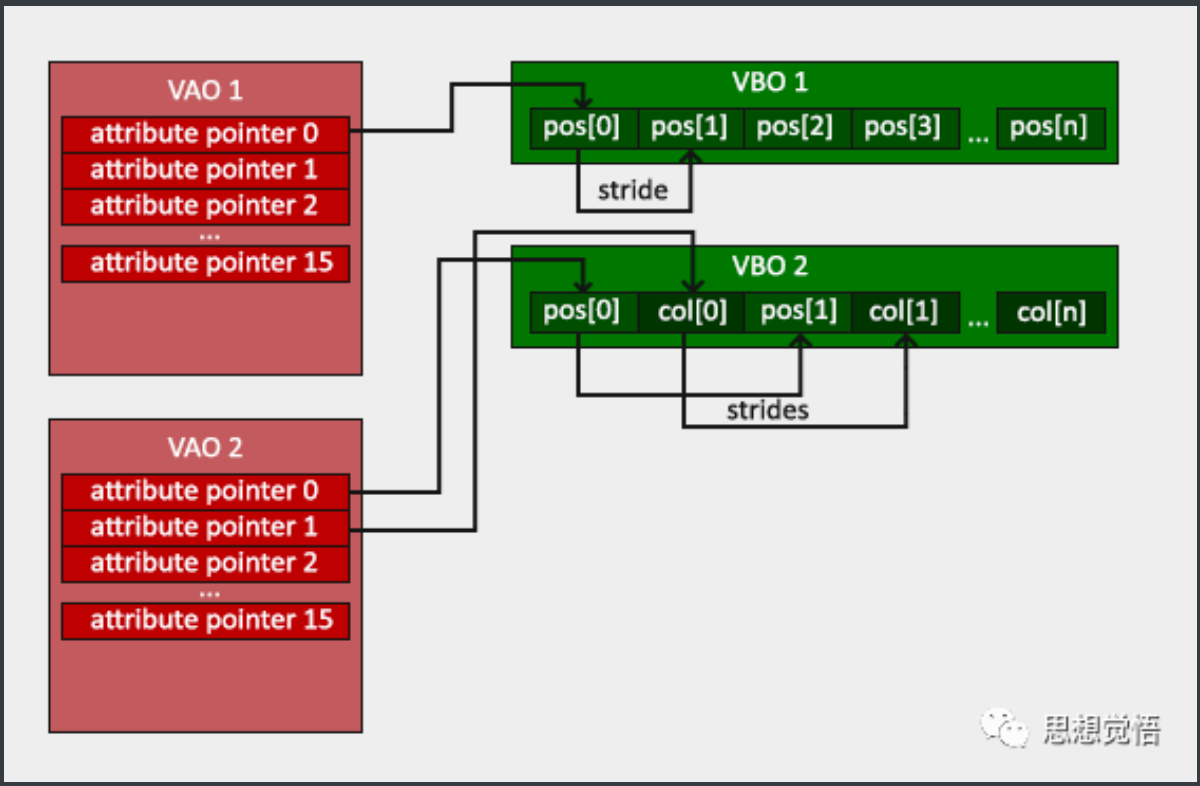
顶点数组对象可以像顶点缓冲对象那样被绑定，任何随后的顶点属性调用都会储存在这个VAO中。这样的好处就是，当配置顶点属性指针时，你只需要将那些调用执行一次，之后再绘制物体的时候只需要绑定相应的VAO就行了。这使在不同顶点数据和属性配置之间切换变得非常简单，只需要绑定不同的VAO就行了。刚刚设置的所有状态都将存储在VAO中。

一个顶点数组对象会储存以下这些内容：

- glEnableVertexAttribArray和glDisableVertexAttribArray的调用。
- 通过glVertexAttribPointer设置的顶点属性配置。
- 通过glVertexAttribPointer调用与顶点属性关联的顶点缓冲对象。

在上面VBO的介绍中我们知道每次在绘制的时候都需要频繁地绑定与解绑VBO，每次绘制还需要取出VBO中的数据进行赋值之后才能进行绘制渲染。当数据量大的时候，重复这些操作就会变得很繁琐。通过VAO就可以简化这个过程，因此VAO可以简单理解成VBO的管理者，避免在帧绘制时再去手动操纵VBO，VAO不能单独使用，
需要搭配VBO使用。

对于GPU来说VBO就是一堆数据，但是这堆数据怎么解析使用，需要`glEnableVertexAttribArray`等相关函数在每次绘制的时候告诉GPU，那么VAO的作用就是简化这个过程的，只需要在初始化的时候将这些解析逻辑与VAO绑定一次即可，
然后每次在绘制的时候只需绑定对应的VAO，而不必每次再绑定VBO，然后去告诉GPU如何解析相关数据了，可以说是一个性能的优化了。



> VAO与VBO经典图

如何使用VAO

- 首先调用函数`glGenVertexArrays`生成VAO

```
unsigned int VAO;
glGenVertexArrays(1, &VAO);
```

- 调用函数`glBindVertexArray`绑定VAO

```
glBindVertexArray(VAO);
// 管理VBO，让VAO记住VBO的数据如何解析使用
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);
// 3. 设置顶点属性指针
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float),
(void*)0);
glEnableVertexAttribArray(0);
glBindVertexArray(0);
```

然后在绘制的时候使用函数 `glBindVertexArray(VAO)` 使用即可。

- 退出时通过函数 `glDeleteVertexArrays` 删除VAO

```
glDeleteVertexArrays
```

主要绑定代码：

```
    // VAO
    glGenVertexArrays(1,&vao);
    glBindVertexArray(vao);
    // VAO与VBO关联
    glBindBuffer(GL_ARRAY_BUFFER,vbo[2]);
    // stride 步长 每个顶点坐标之间相隔6个数据点，数据类型是float
    glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,6 *
sizeof(float ),(void *)0);
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);
    // stride 步长 每个颜色坐标之间相隔6个数据点，数据类型是float，颜色坐标索引
从2开始
    glVertexAttribPointer(colorHandle,4,GL_FLOAT,GL_FALSE,6 *
sizeof(float ) ,(void *)(2 * sizeof(float)) );
    // 启用颜色顶点数据
    glEnableVertexAttribArray(colorHandle);
```

```
        // 解除绑定
        glBindBuffer(GL_ARRAY_BUFFER,0);

        glBindVertexArray(0);
```

主要使用绘制代码：

```
       // VBO与VAO配合绘制
        // 使用vao
        glBindVertexArray(vao);
        // 4个顶点绘制两个三角形组成矩形
        glDrawArrays(GL_TRIANGLE_STRIP,0,4);
        glUseProgram(0);
        // vao解除绑定
        glBindVertexArray(vao);
```

下面是完整代码：

VBOVAOOpengl.h

```
    static const int NUM_VBO = 3;

    class VBOVAOOpengl:public BaseOpengl{

    public:
        VBOVAOOpengl();
        virtual ~VBOVAOOpengl();
        virtual void onDraw();
    private:
        GLint positionHandle{-1};
        GLint colorHandle{-1};
        GLuint vbo[NUM_VBO];
        GLuint vao{0};

    };
```

VBOVAOOpengl.cpp

```
    // 顶点着色器
    static const char *ver = "#version 300 es\n"
                             "in vec4 aColor;\n"
                             "in vec4 aPosition;\n"
                             "out vec4 vColor;\n"
                             "void main() {\n"
                             "    vColor = aColor;\n"
                             "    gl_Position = aPosition;\n"
                             "}";

    // 片元着色器
    static const char *fragment = "#version 300 es\n"
                                  "precision mediump float;\n"
                                  "in vec4 vColor;\n"
                                  "out vec4 fragColor;\n"
                                  "void main() {\n"
                                  "    fragColor = vColor;\n"
                                  "}";

    // 使用绘制两个三角形组成一个矩形的形式(三角形带)
    // 第一第二第三个点组成一个三角形，第二第三第四个点组成一个三角形
    const static GLfloat VERTICES[] = {
            0.5f,-0.5f, // 右下
            0.5f,0.5f, // 右上
            -0.5f,-0.5f, // 左下
            -0.5f,0.5f, // 左上
    };

    const static GLfloat VERTICES_AND_COLOR[] = {
            0.5f,-0.5f, // 右下
            // 颜色
            0.0f,0.0f,1.0f,1.0f,
            0.5f,0.5f, // 右上
            // 颜色
```

```
        0.0f,0.0f,1.0f,1.0f,
        -0.5f,-0.5f, // 左下
        // 颜色
        0.0f,0.0f,1.0f,1.0f,
        -0.5f,0.5f, // 左上
        // 颜色
        0.0f,0.0f,1.0f,1.0f,
};

// rgba
//const static GLfloat COLOR_ICES[] = {
//        0.0f,0.0f,1.0f,1.0f
//};

// vbo颜色
const static GLfloat COLOR_ICES[] = {
        0.0f,0.0f,1.0f,1.0f,
        0.0f,0.0f,1.0f,1.0f,
        0.0f,0.0f,1.0f,1.0f,
        0.0f,0.0f,1.0f,1.0f,
};

VBOVAOOpengl::VBOVAOOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
    colorHandle = glGetAttribLocation(program,"aColor");
    // vbo
    glGenBuffers(3,vbo);
    glBindBuffer(GL_ARRAY_BUFFER,vbo[0]);

 glBufferData(GL_ARRAY_BUFFER,sizeof(VERTICES),VERTICES,GL_STATIC_DRAW
);

    // 颜色
    glBindBuffer(GL_ARRAY_BUFFER,vbo[1]);
```

```
 glBufferData(GL_ARRAY_BUFFER,sizeof(COLOR_ICES),COLOR_ICES,GL_STATIC_
DRAW);

    // 顶点与颜色混合，先顶点坐标，再颜色坐标
    glBindBuffer(GL_ARRAY_BUFFER,vbo[2]);

 glBufferData(GL_ARRAY_BUFFER,sizeof(VERTICES_AND_COLOR),VERTICES_AND_
COLOR
                 ,GL_STATIC_DRAW);

    glBindBuffer(GL_ARRAY_BUFFER,0);

    // VAO
    glGenVertexArrays(1,&vao);
    glBindVertexArray(vao);
    // VAO与VBO关联
    glBindBuffer(GL_ARRAY_BUFFER,vbo[2]);
    // stride 步长 每个顶点坐标之间相隔6个数据点，数据类型是float
    glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,6 *
sizeof(float ),(void *)0);
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);
    // stride 步长 每个颜色坐标之间相隔6个数据点，数据类型是float，颜色坐标索引
从2开始
    glVertexAttribPointer(colorHandle,4,GL_FLOAT,GL_FALSE,6 *
sizeof(float ) ,(void *)(2 * sizeof(float)) );
    // 启用颜色顶点数据
    glEnableVertexAttribArray(colorHandle);
    // 解除绑定
    glBindBuffer(GL_ARRAY_BUFFER,0);

    glBindVertexArray(0);

    LOGD("program:%d",program);
    LOGD("positionHandle:%d",positionHandle);
```

```
    LOGD("colorHandle:%d",colorHandle);
}

void VBOVAOOpengl::onDraw() {
    // 清屏
    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);

    // 普通方式绘制
//    /**
//     * size 几个数字表示一个点，显示是两个数字表示一个点
//     * normalized 是否需要归一化，不用，这里已经归一化了
//     * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
//     */
//
glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);
//    // 启用顶点数据
//    glEnableVertexAttribArray(positionHandle);
//
//    // 这个不需要glEnableVertexAttribArray
//    glVertexAttrib4fv(colorHandle, COLOR_ICES);

// ############################################## 分割线
##################################
     // 使用VBO的方式绘制，顶点与颜色分开
    //    /**
//    glBindBuffer(GL_ARRAY_BUFFER,vbo[0]);
//    glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,(void
*)0);
//    // 启用顶点数据
//    glEnableVertexAttribArray(positionHandle);
//
//    glBindBuffer(GL_ARRAY_BUFFER,vbo[1]);
//    glVertexAttribPointer(colorHandle,4,GL_FLOAT,GL_FALSE,0,(void
*)0);
```

```
//    // 启用颜色顶点数据
//    glEnableVertexAttribArray(colorHandle);
//    // 解除绑定
//    glBindBuffer(GL_ARRAY_BUFFER,0);

// ########################################### 分割线
################################
    // VBO绘制 顶点坐标与颜色坐标一起
//    glBindBuffer(GL_ARRAY_BUFFER,vbo[2]);
//    // stride 步长 每个顶点坐标之间相隔6个数据点，数据类型是float
//    glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,6 *
sizeof(float ),(void *)0);
//    // 启用顶点数据
//    glEnableVertexAttribArray(positionHandle);
//    // stride 步长 每个颜色坐标之间相隔6个数据点，数据类型是float，颜色坐标索
引从2开始
//    glVertexAttribPointer(colorHandle,4,GL_FLOAT,GL_FALSE,6 *
sizeof(float ) ,(void *)(2 * sizeof(float)) );
//    // 启用颜色顶点数据
//    glEnableVertexAttribArray(colorHandle);
//    // 解除绑定
//    glBindBuffer(GL_ARRAY_BUFFER,0);

// ########################################### 分割线
################################

    // VBO与VAO配合绘制
    // 使用vao
    glBindVertexArray(vao);
    // 4个顶点绘制两个三角形组成矩形
    glDrawArrays(GL_TRIANGLE_STRIP,0,4);
    glUseProgram(0);
    // vao解除绑定
    glBindVertexArray(vao);

    // 禁用顶点
```

```
        glDisableVertexAttribArray(positionHandle);
        if(nullptr != eglHelper){
            eglHelper->swapBuffers();
        }
    }

    VBOVAOOpengl::~VBOVAOOpengl() noexcept {
        glDeleteBuffers(NUM_VBO,vbo);
        glDeleteVertexArrays(1,&vao);
    }
```

s

Opengl ES之EBO

前面我们介绍了VBO与VAO等缓冲对象，今天我们来介绍一下EBO。

对于VBO或VAO可以查看之前的文章：Opengl ES之VBO和VAO

EBO是个啥

EBO(Element Buffer Object，也叫IBO:Index Buffer Object)索引缓冲区对象，这个缓冲区主要用来存储顶点的索引信息，索引的意义在于减少重复数据，主要是在函数 `glDrawElements` 中使用。

在之前的文章中，我们绘制三角形、四边形等，因为顶点数据或者公用的顶点坐标不多，都是调用函数 `glDrawArrays` 进行绘制的，试想一下，如果需要绘制一个顶点数量比较多，而且多个顶点之间需要经过
不同的组合进行顶点复用的，那么函数 `glDrawArrays` 及很难满足绘制需求了，此时就需要使用EBO搭配函数 `glDrawElements` 进行绘制了，例如使用8个顶点绘制一个正方体的例子。

在OpenGL通过索引缓冲对象(EBO)来对顶点进行复用，做到重复的顶点只需要分配一次内存，再绘图的时候后通过EBO告诉GPU顶点的索引。

如果使用了EBO，那么绘图函数 `glDrawElements` 通过索引到相应的顶点缓冲区去拿数据，如果绑定了VAO就到VAO里拿数据。

EBO的使用

EBO的使用和前面介绍的VBO的使用差不多：

- 函数 `glGenBuffers` 创建对象ID

```
unsigned int EBO;
glGenBuffers(1, &EBO);
```

- 函数 `glBindBuffer` 绑定EBO

与VBO类似，只不过EBO的类型变成了GL_ELEMENT_ARRAY_BUFFER

```
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
```

- 函数 `glBufferData` 绑定数据

```
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices,
GL_STATIC_DRAW);
```

对于GL_STATIC_DRAW等几个参数的类型在VBO的文章中已经做了介绍，这里就不多说了，童鞋们可以看之前的文章。

当以上步骤配置完毕就可以使用EBO了，在绘制函数 `glDrawElements` 调用前调用函数 `glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);` 进行绑定即可使用。

- 在析构函数中调用函数 `glDeleteBuffers` 销毁EBO

```
    glDeleteBuffers(1,&EBO);
```

注意：笔者在结合使用VAO、VBO和EBO的时候发现，解除绑定时需要先解除VAO的绑定，再解除其他VBO及EBO的绑定，否则可能会导致绘制不生效。

实例Demo

我们沿用之前VBO/VAO绘制四边形的例子，在上面拓展一下使用四个顶点坐标结合EBO绘制一个四边形。

主要代码EBOOpengl.cpp:

```
```

```
#include "EBOOpengl.h"

#include "../utils/Log.h"


// 顶点着色器
static const char *ver = "#version 300 es\n"
                         "in vec4 aColor;\n"
                         "in vec4 aPosition;\n"
                         "out vec4 vColor;\n"
                         "void main() {\n"
                         "    vColor = aColor;\n"
                         "    gl_Position = aPosition;\n"
                         "}";

// 片元着色器
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
                              "in vec4 vColor;\n"
                              "out vec4 fragColor;\n"
                              "void main() {\n"
                              "    fragColor = vColor;\n"
                              "}";

const static GLfloat VERTICES_AND_COLOR[] = {
        0.5f, -0.5f, // 右下
        // 颜色
        0.0f, 0.0f, 1.0f, 1.0f,
        0.5f, 0.5f, // 右上
        // 颜色
        0.0f, 0.0f, 1.0f, 1.0f,
        -0.5f, -0.5f, // 左下
        // 颜色
        0.0f, 0.0f, 1.0f, 1.0f,
        -0.5f, 0.5f, // 左上
        // 颜色
```

```
        0.0f, 0.0f, 1.0f, 1.0f,
};

//const static unsigned int indices[] = {
//        // 注意索引从0开始!
//        // 此例的索引(0,1,2,3)就是顶点数组vertices的下标,
//        // 这样可以由下标代表顶点组合成矩形
//        0, 1, 2, // 第一个三角形
//        1, 2, 3  // 第二个三角形
//};

//// 使用shor类型
//const static unsigned short indices[] = {
//        // 注意索引从0开始!
//        // 此例的索引(0,1,2,3)就是顶点数组vertices的下标,
//        // 这样可以由下标代表顶点组合成矩形
//        0, 1, 2, // 第一个三角形
//        1, 2, 3  // 第二个三角形
//};

// 使用byte类型
const static uint8_t indices[] = {
        // 注意索引从0开始!
        // 此例的索引(0,1,2,3)就是顶点数组vertices的下标,
        // 这样可以由下标代表顶点组合成矩形
        0, 1, 2, // 第一个三角形
        1, 2, 3  // 第二个三角形
};

EBOOpengl::EBOOpengl() {
    initGlProgram(ver, fragment);
    positionHandle = glGetAttribLocation(program, "aPosition");
    colorHandle = glGetAttribLocation(program, "aColor");

    // VAO
    glGenVertexArrays(1, &vao);
```

```
    glBindVertexArray(vao);

    // vbo
    glGenBuffers(1, &vbo);
    glBindBuffer(GL_ARRAY_BUFFER, vbo);
    glBufferData(GL_ARRAY_BUFFER, sizeof(VERTICES_AND_COLOR),
VERTICES_AND_COLOR, GL_STATIC_DRAW);

    // stride 步长 每个顶点坐标之间相隔6个数据点，数据类型是float
    glVertexAttribPointer(positionHandle, 2, GL_FLOAT, GL_FALSE, 6 *
sizeof(float), (void *) 0);
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);
    // stride 步长 每个颜色坐标之间相隔6个数据点，数据类型是float，颜色坐标索引
从2开始
    glVertexAttribPointer(colorHandle, 4, GL_FLOAT, GL_FALSE, 6 *
sizeof(float),
                          (void *) (2 * sizeof(float)));
    // 启用颜色顶点数据
    glEnableVertexAttribArray(colorHandle);

    // EBO
    glGenBuffers(1,&ebo);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER,ebo);

 glBufferData(GL_ELEMENT_ARRAY_BUFFER,sizeof(indices),indices,GL_STATI
C_DRAW);

    // 这个顺序不能乱啊，先解除vao，再解除其他的，不然在绘制的时候可能会不起作用，
需要重新glBindBuffer才生效
    // vao解除
    glBindVertexArray(0);
    // 解除绑定
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    // 解除绑定
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER,0);
```

```
    LOGD("program:%d", program);
    LOGD("positionHandle:%d", positionHandle);
    LOGD("colorHandle:%d", colorHandle);
}

void EBOOpengl::onDraw() {
    // 清屏
    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);

    // VBO与VAO配合绘制
    // 使用vao
    glBindVertexArray(vao);
    // 使用EBO
//    glDrawElements(GL_TRIANGLES,6,GL_UNSIGNED_INT,(void *)0);
// 使用short类型节省内存
//    glDrawElements(GL_TRIANGLES,6,GL_UNSIGNED_SHORT,(void *)0);
// 使用byte类型节省内存
    glDrawElements(GL_TRIANGLES,6,GL_UNSIGNED_BYTE,(void *)0);
    glUseProgram(0);
    // vao解除绑定
    glBindVertexArray(0);

    // 禁用顶点
    glDisableVertexAttribArray(positionHandle);
    if (nullptr != eglHelper) {
        eglHelper->swapBuffers();
    }
}

EBOOpengl::~EBOOpengl() noexcept {
    glDeleteBuffers(1,&ebo);
    glDeleteBuffers(1,&vbo);
    glDeleteVertexArrays(1,&vao);
```

```
    }
```

技巧：在确保数据不会发生溢出的情况下尽量使用占有内存小的数据类型以节约内存，例如使用short比int要节省，使用byte比short要节省。

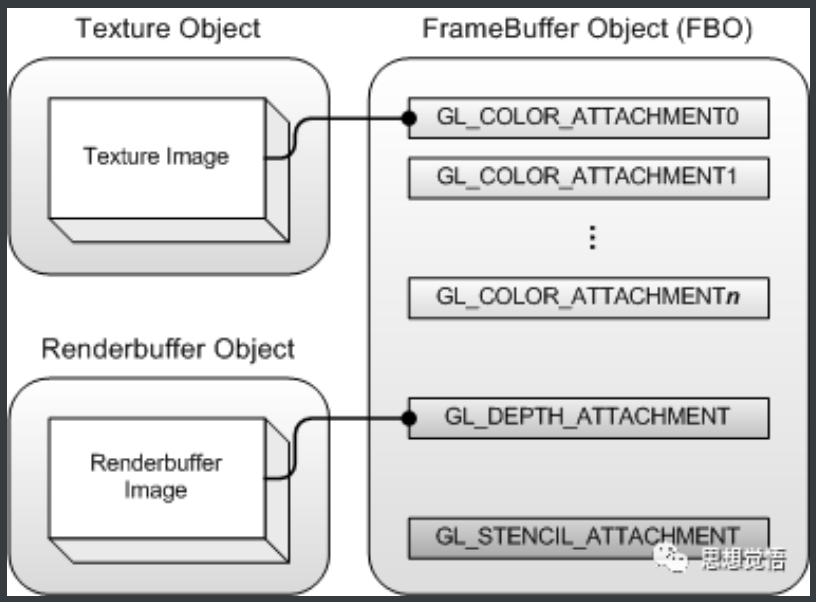
Opengl ES之FBO

FBO介绍

FBO帧缓冲对象，它的主要作用一般就是用作离屏渲染，例如做Camera相机图像采集进行后期处理时就可能会用到FBO。假如相机出图的是OES纹理，为了方便后期处理，
一般先将OES纹理通过FBO转换成普通的2D纹理，然后再通过FBO等增加美颜等其他各种特效滤镜，最后将FBO一路流送进编码器进行编码，另外一路渲染到屏幕上进行预览显示。

FBO总结起来就是可以暂时将未处理完的帧不直接渲染到屏幕上，而是渲染到离屏Buffer中缓存起来，在恰当的时机再取出来渲染到屏幕。

FBO（Frame Buffer Object）帧缓冲对象提供了与颜色缓冲区（color buffer）、深度缓冲区（depth buffer）和模版缓冲区（stencil buffer），但并不会直接为这些缓冲区分配空间，而只是为这些缓冲区提供一个或多个挂接点。我们需要分别为各个缓冲区创建对象，申请空间，然后挂接到相应的挂接点上。



> FBO提供的挂接点如图所示

从上图可以看出FBO中包含了：

1. 多个颜色附着点（GL_COLOR_ATTACHMENT0、GL_COLOR_ATTACHMENT1...）
2. 一个深度附着点（GL_DEPTH_ATTACHMENT）
3. 一个模板附着点（GL_STENCIL_ATTACHMENT）

所谓的颜色附着(纹理附着)就是用于将颜色渲染到纹理中去的意思。后面我们主要介绍FBO的颜色附着。

如何使用FBO

1. 使用函数 `glGenFramebuffers` 生成一个FBO对象，保存对象ID。
2. 使用函数 `glBindFramebuffer` 绑定FBO。
3. 使用函数 `glFramebufferTexture2D` 关联纹理和FBO，并执行渲染步骤。后续如果需要使用FBO的效果时只需要操作与FBO绑定的纹理即可。
4. 使用函数 `glBindFramebuffer` 解绑FBO，一般在Opengl中ID参数传递0就是解绑。
5. 使用函数 `glDeleteFramebuffers` 删除FBO。

当挂接完成之后，我们在执行FBO下面的操作之前，可以检查一下FBO的状态，使用函数 `GLenum glCheckFramebufferStatus(GLenum target)` 检查。

本着学以致用的原则，我们将结合之前的文章，例如纹理贴图、VBO/VAO、EBO等相关知识点，使用这些知识点结合FBO绘制做一个实践的例子：首先将纹理渲染到FBO上去，然后再将FBO的纹理渲染到屏幕上。

插个话。。。总有人盗用不贴原文链接，看看是谁。。。

首先上代码，然后我们挑重要的稍微解读一下：
FBOOpengl.h

```
class FBOOpengl:public BaseOpengl{

public:
    FBOOpengl();
    void onFboDraw();
    virtual ~FBOOpengl();
```

```
    // override要么就都写，要么就都不写，不要一个虚函数写override，而另外一个虚函数不写override，不然可能编译不过
    virtual void onDraw() override;
    virtual void setPixel(void *data, int width, int height, int length) override;
private:
    void fboPrepare();
    GLint positionHandle{-1};
    GLint textureHandle{-1};
    GLuint vbo{0};
    GLuint vao{0};
    GLuint ebo{0};
    // 本身图像纹理id
    GLuint imageTextureId{0};
    // fbo纹理id
    GLuint fboTextureId{0};
    GLint textureSampler{-1};
    GLuint fboId{0};
    // 用于fbo的vbo和vao  也可以用数组的形式，这里为了方便理解先独立开来
    GLuint fboVbo{0};
    GLuint fboVao{0};
    int imageWidth{0};
    int imageHeight{0};
};
```

注意：override作为现代C++的一个关键字，使用的时候需要注意一点，要么就整个类的虚函数都用，要么整个类的虚函数都不用，不要一个虚函数用override修饰，另外一个虚函数又不用override关键字修饰，不然很有可能会编译不过的。

在FBOOpengl中为了区分屏幕渲染和FBO离屏渲染，我们声明了两套VAO和VBO。

FBOOpengl.cpp

```
#include "FBOOpengl.h"
#include "../utils/Log.h"
```

```
// 顶点着色器
static const char *ver = "#version 300 es\n"
                         "in vec4 aPosition;\n"
                         "in vec2 aTexCoord;\n"
                         "out vec2 TexCoord;\n"
                         "void main() {\n"
                         "  TexCoord = aTexCoord;\n"
                         "  gl_Position = aPosition;\n"
                         "}";

// 片元着色器
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
                              "out vec4 FragColor;\n"
                              "in vec2 TexCoord;\n"
                              "uniform sampler2D ourTexture;\n"
                              "void main()\n"
                              "{\n"
                              "    FragColor = texture(ourTexture,
TexCoord);\n"
                              "}";

const static GLfloat VERTICES_AND_TEXTURE[] = {
        0.5f, -0.5f, // 右下
        // 纹理坐标
        1.0f,1.0f,
        0.5f, 0.5f, // 右上
        // 纹理坐标
        1.0f,0.0f,
        -0.5f, -0.5f, // 左下
        // 纹理坐标
        0.0f,1.0f,
        -0.5f, 0.5f, // 左上
        // 纹理坐标
        0.0f,0.0f
};
```

```
// 纹理坐标原点在图片的左上角     又是倒置的? 什么鬼? 疑惑吧?
//const static GLfloat FBO_VERTICES_AND_TEXTURE[] = {
//        1.0f, -1.0f, // 右下
//        // 纹理坐标
//        1.0f,1.0f,
//        1.0f, 1.0f, // 右上
//        // 纹理坐标
//        1.0f,0.0f,
//        -1.0f, -1.0f, // 左下
//        // 纹理坐标
//        0.0f,1.0f,
//        -1.0f, 1.0f, // 左上
//        // 纹理坐标
//        0.0f,0.0f
//};

// 真正的纹理坐标在图片的左下角
const static GLfloat FBO_VERTICES_AND_TEXTURE[] = {
        1.0f, -1.0f, // 右下
        // 纹理坐标
        1.0f,0.0f,
        1.0f, 1.0f, // 右上
        // 纹理坐标
        1.0f,1.0f,
        -1.0f, -1.0f, // 左下
        // 纹理坐标
        0.0f,0.0f,
        -1.0f, 1.0f, // 左上
        // 纹理坐标
        0.0f,1.0f
};

// 使用byte类型比使用short或者int类型节约内存
const static uint8_t indices[] = {
        // 注意索引从0开始!
```

```
        // 此例的索引(0,1,2,3)就是顶点数组vertices的下标，
        // 这样可以由下标代表顶点组合成矩形
        0, 1, 2, // 第一个三角形
        1, 2, 3  // 第二个三角形
};

FBOOpengl::FBOOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
    textureHandle = glGetAttribLocation(program,"aTexCoord");
    textureSampler = glGetUniformLocation(program,"ourTexture");
    LOGD("program:%d",program);
    LOGD("positionHandle:%d",positionHandle);
    LOGD("textureHandle:%d",textureHandle);
    LOGD("textureSample:%d",textureSampler);
    // VAO
    glGenVertexArrays(1, &vao);
    glBindVertexArray(vao);

    // vbo
    glGenBuffers(1, &vbo);
    glBindBuffer(GL_ARRAY_BUFFER, vbo);
    glBufferData(GL_ARRAY_BUFFER, sizeof(VERTICES_AND_TEXTURE),
VERTICES_AND_TEXTURE, GL_STATIC_DRAW);

    // stride 步长 每个顶点坐标之间相隔4个数据点，数据类型是float
    glVertexAttribPointer(positionHandle, 2, GL_FLOAT, GL_FALSE, 4 *
sizeof(float), (void *) 0);
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);
    // stride 步长 每个颜色坐标之间相隔4个数据点，数据类型是float，颜色坐标索引
从2开始
    glVertexAttribPointer(textureHandle, 2, GL_FLOAT, GL_FALSE, 4 *
sizeof(float),
                          (void *) (2 * sizeof(float)));
    // 启用纹理坐标数组
```

```
    glEnableVertexAttribArray(textureHandle);

    // EBO
    glGenBuffers(1,&ebo);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER,ebo);

 glBufferData(GL_ELEMENT_ARRAY_BUFFER,sizeof(indices),indices,GL_STATI
C_DRAW);

    // 这个顺序不能乱啊，先解除vao，再解除其他的，不然在绘制的时候可能会不起作用，
需要重新glBindBuffer才生效
    // vao解除
    glBindVertexArray(0);
    // 解除绑定
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    // 解除绑定
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER,0);

    LOGD("program:%d", program);
    LOGD("positionHandle:%d", positionHandle);
    LOGD("colorHandle:%d", textureHandle);
}

void FBOOpengl::setPixel(void *data, int width, int height, int
length) {
    LOGD("texture setPixel");
    imageWidth = width;
    imageHeight = height;
    glGenTextures(1, &imageTextureId);

    // 激活纹理，注意以下这个两句是搭配的，glActiveTexture激活的是那个纹理，就设
置的sampler2D是那个
    // 默认是0，如果不是0的话，需要在onDraw的时候重新激活一下?
//    glActiveTexture(GL_TEXTURE0);
//    glUniform1i(textureSampler, 0);
```

```
// 例如，一样的
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, imageTextureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA,
GL_UNSIGNED_BYTE, data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);

    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}

void FBOOpengl::fboPrepare(){

    // VAO
    glGenVertexArrays(1, &fboVao);
    glBindVertexArray(fboVao);

    // vbo
    glGenBuffers(1, &fboVbo);
    glBindBuffer(GL_ARRAY_BUFFER, fboVbo);
    glBufferData(GL_ARRAY_BUFFER, sizeof(FBO_VERTICES_AND_TEXTURE),
FBO_VERTICES_AND_TEXTURE, GL_STATIC_DRAW);

    // stride 步长 每个顶点坐标之间相隔4个数据点，数据类型是float
    glVertexAttribPointer(positionHandle, 2, GL_FLOAT, GL_FALSE, 4 *
sizeof(float), (void *) 0);
```

```
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);
    // stride 步长 每个颜色坐标之间相隔4个数据点，数据类型是float，颜色坐标索引从2开始
    glVertexAttribPointer(textureHandle, 2, GL_FLOAT, GL_FALSE, 4 * sizeof(float),
                          (void *) (2 * sizeof(float)));
    // 启用纹理坐标数组
    glEnableVertexAttribArray(textureHandle);

    // EBO
    glGenBuffers(1,&ebo);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER,ebo);

 glBufferData(GL_ELEMENT_ARRAY_BUFFER,sizeof(indices),indices,GL_STATIC_DRAW);

    // 这个顺序不能乱啊，先解除vao，再解除其他的，不然在绘制的时候可能会不起作用，需要重新glBindBuffer才生效
    // vao解除
    glBindVertexArray(0);
    // 解除绑定
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    // 解除绑定
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER,0);

    glGenTextures(1, &fboTextureId);
    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, fboTextureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    glBindTexture(GL_TEXTURE_2D, GL_NONE);
```

```
    glGenFramebuffers(1,&fboId);
    glBindFramebuffer(GL_FRAMEBUFFER,fboId);
    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D,fboTextureId);
    glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0,
GL_TEXTURE_2D, fboTextureId, 0);
    // 这个纹理是多大的?
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, imageWidth, imageHeight,
0, GL_RGBA, GL_UNSIGNED_BYTE, nullptr);
    // 检查FBO状态
    if (glCheckFramebufferStatus(GL_FRAMEBUFFER)!=
GL_FRAMEBUFFER_COMPLETE) {
        LOGE("FBOSample::CreateFrameBufferObj glCheckFramebufferStatus
status != GL_FRAMEBUFFER_COMPLETE");
    }
    // 解绑
    glBindTexture(GL_TEXTURE_2D, GL_NONE);
    glBindFramebuffer(GL_FRAMEBUFFER, GL_NONE);
}

void FBOOpengl::onFboDraw() {
    fboPrepare();

    glBindFramebuffer(GL_FRAMEBUFFER, fboId);

    // 主要这个的大小要与FBO绑定时的纹理的glTexImage2D 设置的大小一致呀
    glViewport(0,0,imageWidth,imageHeight);

     // FBO绘制
    // 清屏
    glClearColor(0.0f, 0.0f, 1.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);

    // 激活纹理
    glActiveTexture(GL_TEXTURE1);
```

```
    glUniform1i(textureSampler, 1);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, imageTextureId);

    // VBO与VAO配合绘制
    // 使用vao
    glBindVertexArray(fboVao);
    // 使用EBO
// 使用byte类型节省内存
    glDrawElements(GL_TRIANGLES,6,GL_UNSIGNED_BYTE,(void *)0);
    glUseProgram(0);
    // vao解除绑定
    glBindVertexArray(0);

    if (nullptr != eglHelper) {
        eglHelper->swapBuffers();
    }
    glBindTexture(GL_TEXTURE_2D, 0);
    glBindFramebuffer(GL_FRAMEBUFFER, 0);
}

void FBOOpengl::onDraw() {

    // 先在FBO离屏渲染
    onFboDraw();

    // 恢复绘制屏幕宽高
    glViewport(0,0,eglHelper->viewWidth,eglHelper->viewHeight);

    // 绘制到屏幕
    // 清屏
    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);
```

```
    // 激活纹理
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, fboTextureId);

    // VBO与VAO配合绘制
    // 使用vao
    glBindVertexArray(vao);
    // 使用EBO
// 使用byte类型节省内存
    glDrawElements(GL_TRIANGLES,6,GL_UNSIGNED_BYTE,(void *)0);
    glUseProgram(0);
    // vao解除绑定
    glBindVertexArray(0);

    // 禁用顶点
    glDisableVertexAttribArray(positionHandle);
    if (nullptr != eglHelper) {
        eglHelper->swapBuffers();
    }

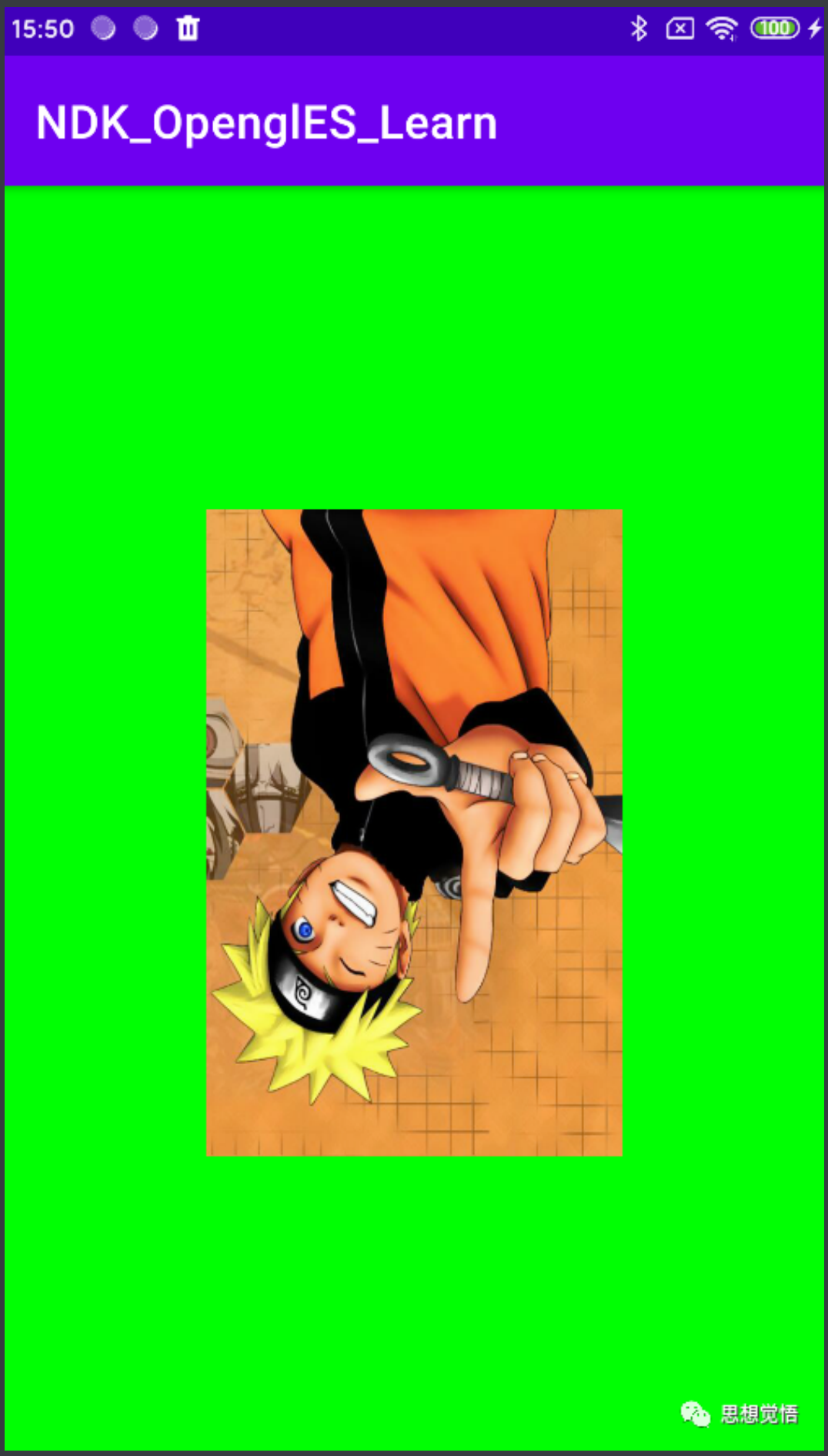
    glBindTexture(GL_TEXTURE_2D, 0);
}


FBOOpengl::~FBOOpengl() noexcept {
    glDeleteBuffers(1,&ebo);
    glDeleteBuffers(1,&vbo);
    glDeleteVertexArrays(1,&vao);
    // ... 删除其他，例如fbo等
}
```

按照之前Opengl ES之纹理贴图 一文所说的，在Opengl ES中进行纹理贴图时直接以图片的左上角为(0,0)原点进行贴图，以纠正纹理贴图倒置的问题，那么这次在绑定FBO之后之后我们就这么干，
使用以下的顶点坐标和纹理坐标：

```
// 纹理坐标原点在图片的左上角      又是倒置的？什么鬼？疑惑吧？
const static GLfloat FBO_VERTICES_AND_TEXTURE[] = {
        1.0f, -1.0f, // 右下
        // 纹理坐标
        1.0f,1.0f,
        1.0f, 1.0f, // 右上
        // 纹理坐标
        1.0f,0.0f,
        -1.0f, -1.0f, // 左下
        // 纹理坐标
        0.0f,1.0f,
        -1.0f, 1.0f, // 左上
        // 纹理坐标
        0.0f,0.0f
};
```

一运行，我们惊喜地发现，实际情况居然和 Opengl ES之纹理贴图 一文所说的不一样了，经过FBO后的贴图再渲染到屏幕时，居然图片是倒置的，如下图：



FBO纹理贴图倒置

这是什么为什么呢?

默认情况下，OpenGL ES 通过绘制到窗口系统提供的帧缓冲区，也就是屏幕本身就是一个默认的FBO，而使用FBO进行纹理贴图的时候需要以真正的纹理坐标(原点0，0在图片的左下角)为基准进行贴图。因此如果直接使用屏幕进行纹理贴图，其实是应该细分成两个过程的，先以左下角为纹理坐标原点进行贴图，然后将贴图后的屏幕默认FBO旋转绕X轴旋转180度与屏幕坐标(左上角是坐标原点)重合，但是这两个细分的过程可以做个取巧就是直接以左上角为纹理坐标原点进行贴图，得到的结果是一样的。

但是我们在单独使用FBO时，仍应该遵循以左下角为纹理坐标原点的原则进行纹理贴图。因此我们只需修改一下顶点坐标和纹理坐标，以左下角为纹理坐标作为原点进行FBO贴图，然后再将FBO旋绕到屏幕上即可:

```
// 真正的纹理坐标在图片的左下角
const static GLfloat FBO_VERTICES_AND_TEXTURE[] = {
        1.0f, -1.0f, // 右下
        // 纹理坐标
        1.0f,0.0f,
        1.0f, 1.0f, // 右上
        // 纹理坐标
        1.0f,1.0f,
        -1.0f, -1.0f, // 左下
        // 纹理坐标
        0.0f,0.0f,
        -1.0f, 1.0f, // 左上
        // 纹理坐标
        0.0f,1.0f
};
```

运行结果如图:


15:49
NDK_OpenglES_Learn
思想觉悟

Opengl ES之PBO

叨叨一句几句

关于Opengl的系列已经有较长的一段时间没有更新了，然而这个系列还远没有到完毕地步，后续至少还有关于Opengl矩阵变换、YUV与RGB互转、Opengl水印贴图、Opengl转场动画等主题文章。

断更的主要原因如果给自己找个借口的话可以说是工作比价忙，如果说的比较现实一点就是自己懒且没啥动力，毕竟写技术博客文章是一件时间成本投入很大，而收益产出极小的一件事情...

进入正题...

了解过Opengl的童鞋们都知道，在Opengl中存在这个各种O，例如VAO、VBO、FBO等，而出现各种各样的O一般都是因为考虑到性能的原因。

今天我们要介绍的主角PBO，它和之前我们介绍VBO很像，了解完PBO之后童鞋们可以对比一下PBO与VBO的差异点。

下面从两个方面介绍PBO，什么是PBO以及如何使用PBO。

> 本文首发于微信公总号号：思想觉悟

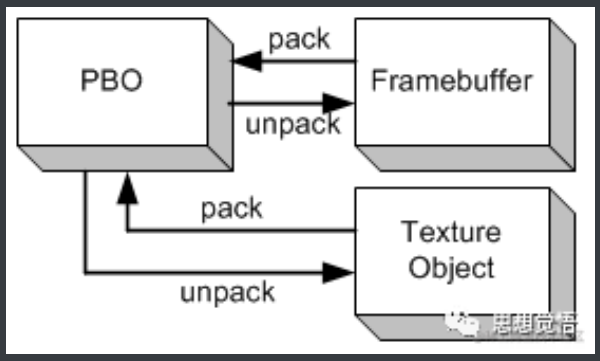
> 更多关于音视频、FFmpeg、Opengl、C++的原创文章请关注微信公众号：思想觉悟

什么是PBO

PBO(Pixel Buffer Object像素缓冲对象)。在了解什么是PBO之前，我们先来了解一下为什么会出现PBO这么一个东西?

所谓存在即合理，用发展的眼光看问题，PBO的出现肯定是为了替代某种东西，或者是为了解决某个问题。

在使用Opengl的时候经常需要在GPU和CPU之间传递数据，例如在使用Opengl将YUV数据转换成RGB数据时就需要先将YUV数据上传到GPU，一般使用函数 `glTexImage2D` ,处理完毕后再将RGB结果数据读取到CPU,
这时使用函数 `glReadPixels` 即可将数据取回。但是这两个函数都是比较缓慢的，特别是在数据量比较大的时候。PBO就是为了解决这个访问慢的问题而产生的。

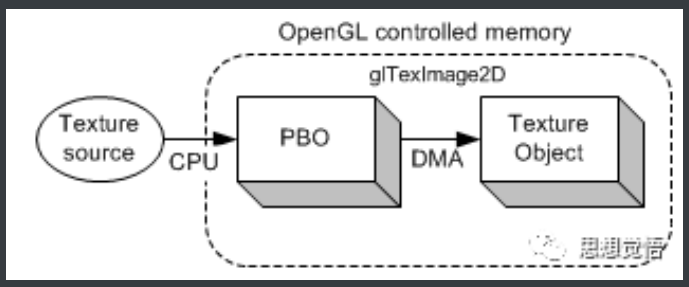


> 使用PBO交换数据图

PBO可以让我们通过一个内存指针，直接访问显存(GPU)的数据，我们将这块内存指针称作缓冲区。我们可以通过函数 `glMapBuffer` 得到它的内存指针，然后就对这块缓冲区的数据可以为所欲为了。

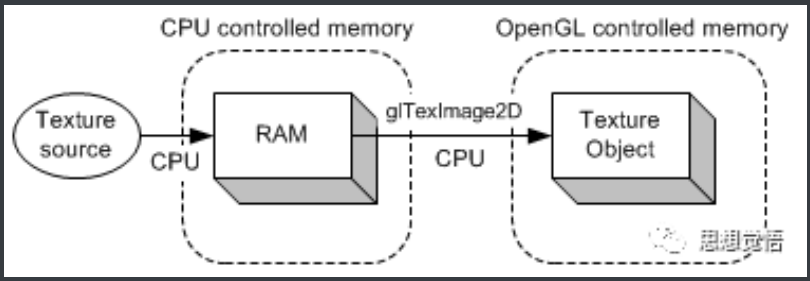
例如使用函数 `glReadPixels` 原本是要传一个内存指针进去的，但是有了缓冲区，它就可以把数据直接复制到缓冲区中去，而不是复制到内存条中去，这样就大大提高了数据的传递效率。

PBO的主要优点是通过直接内存访问的方式与显存进行快速像素数据传输，而无需占用CPU周期。



> PBO与DMA

可能看到这张图你没什么感觉，但是对比看看下面这张CPU与GPU直接传递数据的图你就会有所发现了：



> cpu与gpu常规数据交换

注意：PBO是OpenGL ES 3.0开始提供的一种方式，主要应用于从内存快速复制纹理到显存，或从显存复制像素数据到内存。

PBO的使用方式

既然PBO这么有效率，那么我们在什么情况下可能会用到PBO呢？有个常见的例子，例如我们在安卓上开发Camera应用录制视频时，如果需要用到x264进行软编码的话可能就会用到PBO，
首先我们将相机纹理图像送到Surface渲染显示，然后将Surface数据使用PBO的方式读取处理送到X264编码器中进行编码，当然在安卓上你也可以使用ImageReader...

下面我们来介绍下PBO的使用方式。

PBO的创建和初始化类似于VBO，但是在使用的时候需要用到 `GL_PIXEL_UNPACK_BUFFER` 和 `GL_PIXEL_PACK_BUFFER` 这两个令牌，其中 `GL_PIXEL_UNPACK_BUFFER` 绑定表示该PBO用于将像素数据从程序(CPU)传送到OpenGL中；绑定为 `GL_PIXEL_PACK_BUFFER` 表示该PBO用于从OpenGL中读回像素数据。

1. Pbo创建

先上代码，跟着注释看

```
void PBOOpengl::initPbo() {

    int imgByteSize = imageWidth * imageHeight * 4; // RGBA
```

```
    glGenBuffers(1, &uploadPboId);
    // 绑定pbo
    glBindBuffer(GL_PIXEL_UNPACK_BUFFER, uploadPboId);
    // 设置pbo内存大小
    // 这一步十分重要，第2个参数指定了这个缓冲区的大小，单位是字节，一定要注意
    //  然后第3个参数是初始化用的数据，如果你传个内存指针进去，这个函数就会把你的
    //  数据复制到缓冲区里，我们这里一开始并不需要什么数据，所以传个nullptr就行了
    glBufferData(GL_PIXEL_UNPACK_BUFFER, imgByteSize, nullptr,
GL_STREAM_DRAW);
    // 解除绑定
    glBindBuffer(GL_PIXEL_UNPACK_BUFFER, 0);

    glGenBuffers(1, &downloadPboId);
    glBindBuffer(GL_PIXEL_PACK_BUFFER, downloadPboId);
    glBufferData(GL_PIXEL_PACK_BUFFER, imgByteSize, nullptr,
GL_STREAM_DRAW);
    // 解除绑定
    glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
    LOGD("uploadPboId:%d---downloadPboId:%d---imgByteSize: %d",
uploadPboId, downloadPboId,
         imgByteSize);
}
```

上面的代码创建了两个PBO，其中 `uploadPboId` 用于纹理上传， `downloadPboId` 用于纹理下载。创建好PBO之后然后使用两个PBO专用的令牌进行绑定，之后就调用 `glBufferData` 给PBO分配缓冲区，当然，你也可以在使用的时候先进行绑定，然后重新调用 `glBufferData` 分配新的缓冲区。

2. Pbo上传纹理

所谓上传纹理是值将纹理数据从CPU传递到OpenGL，使用Pbo上传纹理时需要先使用令牌 `GL_PIXEL_UNPACK_BUFFER` 绑定对应的PBO，然后才行使用PBO的缓冲区：

```
// 单个PBO测试
```

```
void PBOOpengl::setPixel(void *data, int width, int height, int
length) {
    LOGD("texture setPixel");
    imageWidth = width;
    imageHeight = height;
    // Pbo初始化
    initPbo();

    glGenTextures(1, &imageTextureId);

    // 激活纹理，注意以下这个两句是搭配的，glActiveTexture激活的是那个纹理，就设
置的sampler2D是那个
    // 默认是0，如果不是0的话，需要在onDraw的时候重新激活一下?
//    glActiveTexture(GL_TEXTURE0);
//    glUniform1i(textureSampler, 0);

// 例如，一样的
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler, 2);

    // 本文首发于微信公总号号：思想觉悟
    // 更多关于音视频、FFmpeg、Opengl、C++的原创文章请关注微信公众号：思想觉悟

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, imageTextureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    // pixels参数传递空，后面会通过pbo更新纹理数据
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA,
GL_UNSIGNED_BYTE, nullptr);

    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);
```

```
    int dataSize = width * height * 4;
    // 使用Pbo
    glBindBuffer(GL_PIXEL_UNPACK_BUFFER, uploadPboId);
    // 将纹理数据拷贝进入缓冲区
    GLubyte *bufPtr = (GLubyte *)
glMapBufferRange(GL_PIXEL_UNPACK_BUFFER, 0,
                                                    dataSize,
                                                    GL_MAP_WRITE_BIT);
    if (bufPtr) {
        memcpy(bufPtr, data, static_cast<size_t>(dataSize));
        glUnmapBuffer(GL_PIXEL_UNPACK_BUFFER);
    }
    // 将pbo缓冲区中的数据拷贝到纹理，调用 glTexSubImage2D 后立即返回，不影响
CPU 时钟周期
    // 这个函数会判断 GL_PIXEL_UNPACK_BUFFER 这个地方有没有绑定一个缓冲区
    //   如果有，就从这个缓冲区读取数据，而不是data参数指定的那个内存
    // 这样glTexSubImage2D就会从我们的缓冲区中读取数据了
    // 这里为什么要用glTexSubImage2D呢，因为如果用glTexImage2D，
glTexImage2D会销毁纹理内存重新申请，glTexSubImage2D就仅仅只是更新纹理中的数据，
这就提高了速度，并且优化了显存的利用率
    glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, width, height, GL_RGBA,
GL_UNSIGNED_BYTE, nullptr);
    // Pbo解除
    glBindBuffer(GL_PIXEL_UNPACK_BUFFER, 0);
    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}
```

注释已经很详细了，就不多解析了，还是看不懂的私聊交流呗...

2. Pbo下载纹理

所谓上传纹理是值将纹理数据从OpenGL中读取回CPU，与上传纹理一样，下载纹理也是需要先使用令牌绑定PBO才能使用，下载纹理使用的令牌是 `GL_PIXEL_PACK_BUFFER` 。
下面的代码作用是将Opengl的渲染结果使用PBO读取出来：

```
// 单PBO读取测试
void PBOOpengl::readPixel(uint8_t **data,int *width,int *height) {

    *width = eglHelper->viewWidth;
    *height = eglHelper->viewHeight;
    int dataSize = eglHelper->viewWidth * eglHelper->viewHeight * 4;
    *data = new uint8_t[dataSize];
    // 方法一 正常的glReadPixels读取 最简单
//    glReadPixels(0, 0, eglHelper->viewWidth, eglHelper->viewHeight,
//                 GL_RGBA, GL_UNSIGNED_BYTE, *data);

    // 方法二 pbo读取
    // 首先我们要把缓冲区绑定到 GL_PIXEL_PACK_BUFFER 这个地方
    glBindBuffer(GL_PIXEL_PACK_BUFFER, downloadPboId);
    // 重新分配一下空间 如果必要，这里就懒得判断了
    glBufferData(GL_PIXEL_PACK_BUFFER, dataSize, nullptr,
GL_STREAM_DRAW);
    // 这个函数会判断 GL_PIXEL_PACK_BUFFER 这个地方有没有绑定一个缓冲区，如果有，那就把数据写入到这个缓冲区里
    // 前4个参数就是要读取的屏幕区域，不多解释
    //  格式是RGB，类型是BYTE，每个像素1字节
    // 如果GL_PIXEL_PACK_BUFFER有绑定缓冲区，最后一个参数就作为偏移值来使用，传nullptr就行
    glReadPixels(0, 0, *width, *height, GL_RGBA, GL_UNSIGNED_BYTE, 0);
    // 好了我们已经成功把屏幕的像素数据复制到了缓冲区里

    // 这时候，你可以用 glMapBuffer 得到缓冲区的内存指针，来读取里面的像素数据，保存到图片文件
    // 注意glMapBuffer的第1个参数不一定要是GL_PIXEL_PACK_BUFFER，你可以把缓冲区绑定到比如上面init函数的GL_ARRAY_BUFFER
    //  然后这里也传GL_ARRAY_BUFFER，由于懒得再绑定一次，就接着用上面绑定的GL_PIXEL_PACK_BUFFER吧
    GLubyte *mapPtr = static_cast<GLubyte *>
(glMapBufferRange(GL_PIXEL_PACK_BUFFER, 0, dataSize,

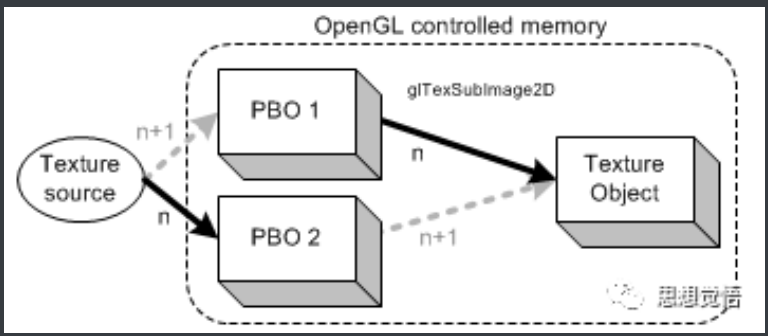
 GL_MAP_READ_BIT));
```

```
        if (nullptr != mapPtr)
        {
            LOGD("readPixel 数据拷贝");
            // 拷贝数据
            memcpy(*data,mapPtr,dataSize);
            // 解除Map
            glUnmapBuffer(GL_PIXEL_PACK_BUFFER);
        } else{
            LOGD("readPixel glMapBufferRange null");
        }
        // 完事了把GL_PIXEL_PACK_BUFFER这个地方的缓冲区解绑掉，以免别的函数误操作
        glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
    }
```

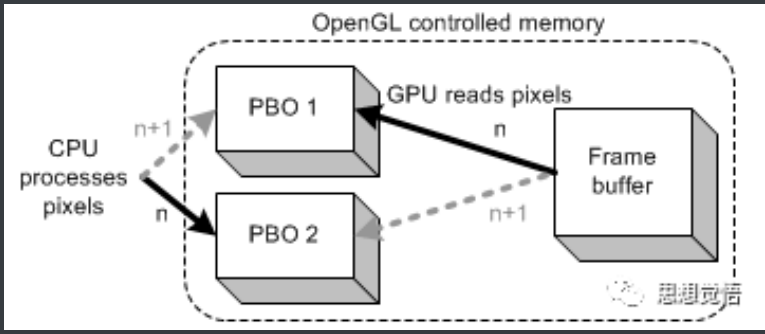
其实无论有没有使用PBO都是使用的函数 `glReadPixels` 进行读取，但是他们的参数却是不一样的，你发现细节了吗？如果是在PBO模式下使用 `glReadPixels` ，则会自动将结果发送到PBO的缓冲区，
并不会像单独使用 `glReadPixels` 那样需要阻塞进行等待返回，因此效率就高了...

双PBO

PBO的另一个优势是它还具备异步DMA(Direct Memory Access)传输，也正因为这个特性，使得在使用单个PBO的情况下，在一些机型上可能性能提升并不明显，所以通常需要两个PBO配合使用。



> 双PBO上传纹理数据



> 双PBO获取纹理数据

通过上面这两张图我们可以看到利用交替使用双PBO的方式可以将PBO的快速特性更进一步。

下面的代码展示了如何通过双PBO上传纹理和下载渲染结果：

使用双PBO上传纹理数据

```
// 双PBO测试
void PBOOpengl::setPixel(void *data, int width, int height, int
length) {
    LOGD("texture setPixel  uploadPboIndex:%d",uploadPboIndex);
    imageWidth = width;
    imageHeight = height;
    // Pbo初始化
    initPboArray();

    if(imageTextureId == 0){

        glGenTextures(1, &imageTextureId);

        // 激活纹理，注意以下这个两句是搭配的，glActiveTexture激活的是那个纹
理，就设置的sampler2D是那个
        // 默认是0，如果不是0的话，需要在onDraw的时候重新激活一下?
//    glActiveTexture(GL_TEXTURE0);
//    glUniform1i(textureSampler, 0);
```

```
// 例如，一样的
        glActiveTexture(GL_TEXTURE2);
        glUniform1i(textureSampler, 2);

        // 绑定纹理
        glBindTexture(GL_TEXTURE_2D, imageTextureId);
        // 为当前绑定的纹理对象设置环绕、过滤方式
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);
        // pixels参数传递空，后面会通过pbo更新纹理数据
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0,
GL_RGBA, GL_UNSIGNED_BYTE, nullptr);

        // 生成mip贴图
        glGenerateMipmap(GL_TEXTURE_2D);
    }

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, imageTextureId);

    int dataSize = width * height * 4;
    // 使用Pbo
    // 指针运算你要会一点呀
    glBindBuffer(GL_PIXEL_UNPACK_BUFFER, *(uploadPboIds +
uploadPboIndex % NUM_PBO ));
    // 将纹理数据拷贝进入缓冲区
    GLubyte *bufPtr = (GLubyte *)
glMapBufferRange(GL_PIXEL_UNPACK_BUFFER, 0,
                                                dataSize,
                                                GL_MAP_WRITE_BIT);
    if (bufPtr) {
        memcpy(bufPtr, data, static_cast<size_t>(dataSize));
```

```
            glUnmapBuffer(GL_PIXEL_UNPACK_BUFFER);
        }
        // 将pbo缓冲区中的数据拷贝到纹理，调用 glTexSubImage2D 后立即返回，不影响
    CPU 时钟周期
        // 这个函数会判断 GL_PIXEL_UNPACK_BUFFER 这个地方有没有绑定一个缓冲区
        //    如果有，就从这个缓冲区读取数据，而不是data参数指定的那个内存
        // 这样glTexSubImage2D就会从我们的缓冲区中读取数据了
        // 这里为什么要用glTexSubImage2D呢，因为如果用glTexImage2D，
    glTexImage2D会销毁纹理内存重新申请，glTexSubImage2D就仅仅只是更新纹理中的数据，
    这就提高了速度，并且优化了显存的利用率
        glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, width, height, GL_RGBA,
    GL_UNSIGNED_BYTE, nullptr);
        // Pbo解除
        glBindBuffer(GL_PIXEL_UNPACK_BUFFER, 0);
        // 解绑定
        glBindTexture(GL_TEXTURE_2D, 0);
        // 索引自加
        uploadPboIndex++;
    }
```

使用双PBO下载渲染结果：

```
    // 双PBO读取测试
    void PBOOpengl::readPixel(uint8_t **data,int *width,int *height) {

        *width = eglHelper->viewWidth;
        *height = eglHelper->viewHeight;
        int dataSize = eglHelper->viewWidth * eglHelper->viewHeight * 4;
        *data = new uint8_t[dataSize];

        // 首先我们要把缓冲区绑定到 GL_PIXEL_PACK_BUFFER 这个地方
        // 指针运算你要会一点呀
        glBindBuffer(GL_PIXEL_PACK_BUFFER, *(downloadPboIds +
    downloadPboIndex % NUM_PBO));
        // 重新分配一下空间
```

```
    glBufferData(GL_PIXEL_PACK_BUFFER, dataSize, nullptr,
GL_STREAM_DRAW);
    // 这个函数会判断 GL_PIXEL_PACK_BUFFER 这个地方有没有绑定一个缓冲区，如果
有，那就把数据写入到这个缓冲区里
    // 前4个参数就是要读取的屏幕区域，不多解释
    //  格式是RGB，类型是BYTE，每个像素1字节
    // 如果GL_PIXEL_PACK_BUFFER有绑定缓冲区，最后一个参数就作为偏移值来使用，传
nullptr就行
    glReadPixels(0, 0, *width, *height, GL_RGBA, GL_UNSIGNED_BYTE, 0);
    // 好了我们已经成功把屏幕的像素数据复制到了缓冲区里

    // 这时候，你可以用 glMapBuffer 得到缓冲区的内存指针，来读取里面的像素数据，
保存到图片文件
    // 注意glMapBuffer的第1个参数不一定要是GL_PIXEL_PACK_BUFFER，你可以把缓
冲区绑定到比如上面init函数的GL_ARRAY_BUFFER
    //  然后这里也传GL_ARRAY_BUFFER，由于懒得再绑定一次，就接着用上面绑定的
GL_PIXEL_PACK_BUFFER吧
    GLubyte *mapPtr = static_cast<GLubyte *>
(glMapBufferRange(GL_PIXEL_PACK_BUFFER, 0, dataSize,

 GL_MAP_READ_BIT));
    if (nullptr != mapPtr)
    {
        LOGD("readPixel 数据拷贝");
        // 拷贝数据
        memcpy(*data,mapPtr,dataSize);
        // 解除Map
        glUnmapBuffer(GL_PIXEL_PACK_BUFFER);
    } else{
        LOGD("readPixel glMapBufferRange null");
    }
    // 完事了把GL_PIXEL_PACK_BUFFER这个地方的缓冲区解绑掉，以免别的函数误操作
    glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
    // Pbo索引自加
    downloadPboIndex++;
}
```

下面我们使用双PBO上传纹理和双PBO下载实现一个纹理动态切换和下载渲染结果的小demo，完整代码如下：

PBOOpengl.h

```
#ifndef NDK_OPENGLES_LEARN_PBOOPENGL_H
#define NDK_OPENGLES_LEARN_PBOOPENGL_H

#include "BaseOpengl.h"

static const int NUM_PBO = 2;

class PBOOpengl: public BaseOpengl{

public:
    PBOOpengl();
    virtual ~PBOOpengl();
    // override要么就都写，要么就都不写，不要一个虚函数写override，而另外一个虚函数不写override，不然可能编译不过
    virtual void onDraw() override;
    virtual void setPixel(void *data, int width, int height, int length) override;
    virtual void readPixel(uint8_t **data,int *width,int *height) override;
private:
    void initPbo();
    void initPboArray();
    GLint positionHandle{-1};
    GLint textureHandle{-1};
    GLuint vbo{0};
    GLuint vao{0};
    GLuint ebo{0};
    // 本身图像纹理id
    GLuint imageTextureId{0};
    GLint textureSampler{-1};
    int imageWidth{0};
```

```
        int imageHeight{0};
        // 上传纹理的pbo
        GLuint uploadPboId{0};
        // cpu下载纹理的pbo
        GLuint downloadPboId{0};

        // 上传纹理的pbo
        GLuint *uploadPboIds{nullptr};
        // cpu下载纹理的pbo
        GLuint *downloadPboIds{nullptr};

        // Pbo 的索引，用于双PBO时
        int uploadPboIndex{0};
        int downloadPboIndex{0};

      // 本文首发于微信公总号号：思想觉悟
        // 更多关于音视频、FFmpeg、Opengl、C++的原创文章请关注微信公众号：思想觉悟
    };


    #endif //NDK_OPENGLES_LEARN_PBOOPENGL_H
```

PBOOpengl.cpp:

```
    #include "PBOOpengl.h"

    #include "../utils/Log.h"

    // 顶点着色器
    static const char *ver = "#version 300 es\n"
                             "in vec4 aPosition;\n"
                             "in vec2 aTexCoord;\n"
                             "out vec2 TexCoord;\n"
                             "void main() {\n"
                             "  TexCoord = aTexCoord;\n"
                             "  gl_Position = aPosition;\n"
```

```
                            "}";

// 片元着色器
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
                              "out vec4 FragColor;\n"
                              "in vec2 TexCoord;\n"
                              "uniform sampler2D ourTexture;\n"
                              "void main()\n"
                              "{\n"
                              "    FragColor = texture(ourTexture,
TexCoord);\n"
                              "}";

const static GLfloat VERTICES_AND_TEXTURE[] = {
        0.5f, -0.5f, // 右下
        // 纹理坐标
        1.0f, 1.0f,
        0.5f, 0.5f, // 右上
        // 纹理坐标
        1.0f, 0.0f,
        -0.5f, -0.5f, // 左下
        // 纹理坐标
        0.0f, 1.0f,
        -0.5f, 0.5f, // 左上
        // 纹理坐标
        0.0f, 0.0f
};

// 真正的纹理坐标在图片的左下角
const static GLfloat FBO_VERTICES_AND_TEXTURE[] = {
        1.0f, -1.0f, // 右下
        // 纹理坐标
        1.0f, 0.0f,
        1.0f, 1.0f, // 右上
        // 纹理坐标
```

```
        1.0f, 1.0f,
        -1.0f, -1.0f, // 左下
        // 纹理坐标
        0.0f, 0.0f,
        -1.0f, 1.0f, // 左上
        // 纹理坐标
        0.0f, 1.0f
};

// 使用byte类型比使用short或者int类型节约内存
const static uint8_t indices[] = {
        // 注意索引从0开始!
        // 此例的索引(0,1,2,3)就是顶点数组vertices的下标,
        // 这样可以由下标代表顶点组合成矩形
        0, 1, 2, // 第一个三角形
        1, 2, 3  // 第二个三角形
};

PBOOpengl::PBOOpengl():uploadPboIds(nullptr),downloadPboIds(nullptr) {
    initGlProgram(ver, fragment);
    positionHandle = glGetAttribLocation(program, "aPosition");
    textureHandle = glGetAttribLocation(program, "aTexCoord");
    textureSampler = glGetUniformLocation(program, "ourTexture");
    LOGD("program:%d", program);
    LOGD("positionHandle:%d", positionHandle);
    LOGD("textureHandle:%d", textureHandle);
    LOGD("textureSample:%d", textureSampler);
    // VAO
    glGenVertexArrays(1, &vao);
    glBindVertexArray(vao);

    // vbo
    glGenBuffers(1, &vbo);
    glBindBuffer(GL_ARRAY_BUFFER, vbo);
    glBufferData(GL_ARRAY_BUFFER, sizeof(VERTICES_AND_TEXTURE),
VERTICES_AND_TEXTURE,
```

```
                 GL_STATIC_DRAW);

    // stride 步长 每个顶点坐标之间相隔4个数据点，数据类型是float
    glVertexAttribPointer(positionHandle, 2, GL_FLOAT, GL_FALSE, 4 *
sizeof(float), (void *) 0);
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);
    // stride 步长 每个颜色坐标之间相隔4个数据点，数据类型是float，颜色坐标索引
从2开始
    glVertexAttribPointer(textureHandle, 2, GL_FLOAT, GL_FALSE, 4 *
sizeof(float),
                          (void *) (2 * sizeof(float)));
    // 启用纹理坐标数组
    glEnableVertexAttribArray(textureHandle);

    // EBO
    glGenBuffers(1, &ebo);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, ebo);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices,
GL_STATIC_DRAW);

    // 这个顺序不能乱啊，先解除vao，再解除其他的，不然在绘制的时候可能会不起作用，
需要重新glBindBuffer才生效
    // vao解除
    glBindVertexArray(0);
    // 解除绑定
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    // 解除绑定
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);

    LOGD("program:%d", program);
    LOGD("positionHandle:%d", positionHandle);
    LOGD("colorHandle:%d", textureHandle);
}

void PBOOpengl::initPbo() {
```

```
    int imgByteSize = imageWidth * imageHeight * 4; // RGBA

    glGenBuffers(1, &uploadPboId);
    // 绑定pbo
    glBindBuffer(GL_PIXEL_UNPACK_BUFFER, uploadPboId);
    // 设置pbo内存大小
    // 这一步十分重要，第2个参数指定了这个缓冲区的大小，单位是字节，一定要注意
    //  然后第3个参数是初始化用的数据，如果你传个内存指针进去，这个函数就会把你的
    //  数据复制到缓冲区里，我们这里一开始并不需要什么数据，所以传个nullptr就行了
    glBufferData(GL_PIXEL_UNPACK_BUFFER, imgByteSize, nullptr,
GL_STREAM_DRAW);
    glBindBuffer(GL_PIXEL_UNPACK_BUFFER, 0);
  // 本文首发于微信公总号号：思想觉悟
    // 更多关于音视频、FFmpeg、Opengl、C++的原创文章请关注微信公众号：思想觉悟
    glGenBuffers(1, &downloadPboId);
    glBindBuffer(GL_PIXEL_PACK_BUFFER, downloadPboId);
    glBufferData(GL_PIXEL_PACK_BUFFER, imgByteSize, nullptr,
GL_STREAM_DRAW);
    // 解除绑定
    glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
    LOGD("uploadPboId:%d---downloadPboId:%d---imgByteSize：%d",
uploadPboId, downloadPboId,
         imgByteSize);
}

void PBOOpengl::initPboArray() {
    int imgByteSize = imageWidth * imageHeight * 4; // RGBA
    if(nullptr == uploadPboIds){
        uploadPboIds = new GLuint[NUM_PBO];
        glGenBuffers(NUM_PBO, uploadPboIds);
        for (int i = 0; i < NUM_PBO; ++i) {
            // 绑定pbo
            glBindBuffer(GL_PIXEL_UNPACK_BUFFER, *(uploadPboIds + i));
            // 设置pbo内存大小
```

```
            // 这一步十分重要，第2个参数指定了这个缓冲区的大小，单位是字节，一定
要注意
            //  然后第3个参数是初始化用的数据，如果你传个内存指针进去，这个函数就
会把你的
            //  数据复制到缓冲区里，我们这里一开始并不需要什么数据，所以传个
nullptr就行了
            glBufferData(GL_PIXEL_UNPACK_BUFFER, imgByteSize, nullptr,
GL_STREAM_DRAW);
            glBindBuffer(GL_PIXEL_UNPACK_BUFFER, 0);
        }
    }

    if(nullptr == downloadPboIds){
        downloadPboIds = new GLuint[NUM_PBO];
        glGenBuffers(NUM_PBO, downloadPboIds);
        for (int i = 0; i < NUM_PBO; ++i) {
            // 绑定pbo
            glBindBuffer(GL_PIXEL_PACK_BUFFER, *(downloadPboIds + i));
            glBufferData(GL_PIXEL_PACK_BUFFER, imgByteSize, nullptr,
GL_STREAM_DRAW);
            glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
        }
    }

}

PBOOpengl::~PBOOpengl() {
    glDeleteBuffers(1, &ebo);
    glDeleteBuffers(1, &vbo);
    glDeleteVertexArrays(1, &vao);
    if(nullptr != uploadPboIds){
        glDeleteBuffers(NUM_PBO,uploadPboIds);
        delete [] uploadPboIds;
        uploadPboIds = nullptr;
    }
  // 本文首发于微信公总号号：思想觉悟
```

```
    // 更多关于音视频、FFmpeg、Opengl、C++的原创文章请关注微信公众号：思想觉悟
    if(nullptr != downloadPboIds){
        glDeleteBuffers(NUM_PBO,downloadPboIds);
        delete [] downloadPboIds;
        downloadPboIds = nullptr;
    }
}

void PBOOpengl::onDraw() {

    // 绘制屏幕宽高
    glViewport(0, 0, eglHelper->viewWidth, eglHelper->viewHeight);

    // 绘制到屏幕
    // 清屏
    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);

    // 激活纹理
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, imageTextureId);

    // VBO与VAO配合绘制
    // 使用vao
    glBindVertexArray(vao);
    // 使用EBO
// 使用byte类型节省内存
    glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_BYTE, (void *) 0);
    glUseProgram(0);
    // vao解除绑定
    glBindVertexArray(0);
```

```
    // 禁用顶点
    glDisableVertexAttribArray(positionHandle);
    if (nullptr != eglHelper) {
        eglHelper->swapBuffers();
    }

    glBindTexture(GL_TEXTURE_2D, 0);
}
//
//// 单个PBO测试
//void PBOOpengl::setPixel(void *data, int width, int height, int
length) {
//    LOGD("texture setPixel");
//    imageWidth = width;
//    imageHeight = height;
//    // Pbo初始化
//    initPbo();
//
//    glGenTextures(1, &imageTextureId);
//
//    // 激活纹理，注意以下这个两句是搭配的，glActiveTexture激活的是那个纹理，
就设置的sampler2D是那个
//    // 默认是0，如果不是0的话，需要在onDraw的时候重新激活一下?
////    glActiveTexture(GL_TEXTURE0);
////    glUniform1i(textureSampler, 0);
//
//// 例如，一样的
//    glActiveTexture(GL_TEXTURE2);
//    glUniform1i(textureSampler, 2);
//
//    // 绑定纹理
//    glBindTexture(GL_TEXTURE_2D, imageTextureId);
//    // 为当前绑定的纹理对象设置环绕、过滤方式
//    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
//    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
```

```
//    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR);
//    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);
//    // pixels参数传递空，后面会通过pbo更新纹理数据
//    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0,
GL_RGBA, GL_UNSIGNED_BYTE, nullptr);
//
//    // 生成mip贴图
//    glGenerateMipmap(GL_TEXTURE_2D);
//
//    int dataSize = width * height * 4;
//    // 使用Pbo
//    glBindBuffer(GL_PIXEL_UNPACK_BUFFER, uploadPboId);
//    // 将纹理数据拷贝进入缓冲区
//    GLubyte *bufPtr = (GLubyte *)
glMapBufferRange(GL_PIXEL_UNPACK_BUFFER, 0,
//                                                    dataSize,
//
GL_MAP_WRITE_BIT);
//    if (bufPtr) {
//        memcpy(bufPtr, data, static_cast<size_t>(dataSize));
//        glUnmapBuffer(GL_PIXEL_UNPACK_BUFFER);
//    }
//    // 将pbo缓冲区中的数据拷贝到纹理，调用 glTexSubImage2D 后立即返回，不影
响 CPU 时钟周期
//    // 这个函数会判断 GL_PIXEL_UNPACK_BUFFER 这个地方有没有绑定一个缓冲区
//    //   如果有，就从这个缓冲区读取数据，而不是data参数指定的那个内存
//    // 这样glTexSubImage2D就会从我们的缓冲区中读取数据了
//    // 这里为什么要用glTexSubImage2D呢，因为如果用glTexImage2D,
glTexImage2D会销毁纹理内存重新申请，glTexSubImage2D就仅仅只是更新纹理中的数据,
这就提高了速度，并且优化了显存的利用率
//    glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, width, height, GL_RGBA,
GL_UNSIGNED_BYTE, nullptr);
//    // Pbo解除
//    glBindBuffer(GL_PIXEL_UNPACK_BUFFER, 0);
```

```
//    // 解绑定
//    glBindTexture(GL_TEXTURE_2D, 0);
//}



// 双PBO测试
void PBOOpengl::setPixel(void *data, int width, int height, int
length) {
    LOGD("texture setPixel  uploadPboIndex:%d",uploadPboIndex);
    imageWidth = width;
    imageHeight = height;
    // Pbo初始化
    initPboArray();

    if(imageTextureId == 0){

        glGenTextures(1, &imageTextureId);

        // 激活纹理，注意以下这个两句是搭配的，glActiveTexture激活的是那个纹
理，就设置的sampler2D是那个
        // 默认是0，如果不是0的话，需要在onDraw的时候重新激活一下?
//    glActiveTexture(GL_TEXTURE0);
//    glUniform1i(textureSampler, 0);

// 例如，一样的
        glActiveTexture(GL_TEXTURE2);
        glUniform1i(textureSampler, 2);

        // 绑定纹理
        glBindTexture(GL_TEXTURE_2D, imageTextureId);
        // 为当前绑定的纹理对象设置环绕、过滤方式
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR);
```

```
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);
        // pixels参数传递空，后面会通过pbo更新纹理数据
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0,
GL_RGBA, GL_UNSIGNED_BYTE, nullptr);

        // 生成mip贴图
        glGenerateMipmap(GL_TEXTURE_2D);
    }

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, imageTextureId);

    int dataSize = width * height * 4;
    // 使用Pbo
    // 指针运算你要会一点呀
    glBindBuffer(GL_PIXEL_UNPACK_BUFFER, *(uploadPboIds +
uploadPboIndex % NUM_PBO ));
    // 将纹理数据拷贝进入缓冲区
    GLubyte *bufPtr = (GLubyte *)
glMapBufferRange(GL_PIXEL_UNPACK_BUFFER, 0,
                                                dataSize,
                                                GL_MAP_WRITE_BIT);
    if (bufPtr) {
        memcpy(bufPtr, data, static_cast<size_t>(dataSize));
        glUnmapBuffer(GL_PIXEL_UNPACK_BUFFER);
    }
    // 将pbo缓冲区中的数据拷贝到纹理，调用 glTexSubImage2D 后立即返回，不影响
CPU 时钟周期
    // 这个函数会判断 GL_PIXEL_UNPACK_BUFFER 这个地方有没有绑定一个缓冲区
    //   如果有，就从这个缓冲区读取数据，而不是data参数指定的那个内存
    // 这样glTexSubImage2D就会从我们的缓冲区中读取数据了
    // 这里为什么要用glTexSubImage2D呢，因为如果用glTexImage2D,
glTexImage2D会销毁纹理内存重新申请，glTexSubImage2D就仅仅只是更新纹理中的数据，
这就提高了速度，并且优化了显存的利用率
```

```
    glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, width, height, GL_RGBA,
GL_UNSIGNED_BYTE, nullptr);
    // Pbo解除
    glBindBuffer(GL_PIXEL_UNPACK_BUFFER, 0);
    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
    // 索引自加
    uploadPboIndex++;
}

//// 单PBO读取测试
//void PBOOpengl::readPixel(uint8_t **data,int *width,int *height) {
//
//    *width = eglHelper->viewWidth;
//    *height = eglHelper->viewHeight;
//    int dataSize = eglHelper->viewWidth * eglHelper->viewHeight * 4;
//    *data = new uint8_t[dataSize];
//    // 方法一 正常的glReadPixels读取 最简单
////    glReadPixels(0, 0, eglHelper->viewWidth, eglHelper-
>viewHeight,
////                 GL_RGBA, GL_UNSIGNED_BYTE, *data);
//
//    // 方法二 pbo读取
//    // 首先我们要把缓冲区绑定到 GL_PIXEL_PACK_BUFFER 这个地方
//    glBindBuffer(GL_PIXEL_PACK_BUFFER, downloadPboId);
//    // 重新分配一下空间
//    glBufferData(GL_PIXEL_PACK_BUFFER, dataSize, nullptr,
GL_STREAM_DRAW);
//    // 这个函数会判断 GL_PIXEL_PACK_BUFFER 这个地方有没有绑定一个缓冲区，如
果有，那就把数据写入到这个缓冲区里
//    // 前4个参数就是要读取的屏幕区域，不多解释
//    //  格式是RGB，类型是BYTE，每个像素1字节
//    // 如果GL_PIXEL_PACK_BUFFER有绑定缓冲区，最后一个参数就作为偏移值来使
用，传nullptr就行
//    glReadPixels(0, 0, *width, *height, GL_RGBA, GL_UNSIGNED_BYTE,
0);
```

```
//    // 好了我们已经成功把屏幕的像素数据复制到了缓冲区里
//
//    // 这时候，你可以用 glMapBuffer 得到缓冲区的内存指针，来读取里面的像素数据，保存到图片文件
//    // 注意glMapBuffer的第1个参数不一定要是GL_PIXEL_PACK_BUFFER，你可以把缓冲区绑定到比如上面init函数的GL_ARRAY_BUFFER
//    //  然后这里也传GL_ARRAY_BUFFER，由于懒得再绑定一次，就接着用上面绑定的GL_PIXEL_PACK_BUFFER吧
//    GLubyte *mapPtr = static_cast<GLubyte *>(glMapBufferRange(GL_PIXEL_PACK_BUFFER, 0, dataSize,
//                                                              GL_MAP_READ_BIT));
//    if (nullptr != mapPtr)
//    {
//        LOGD("readPixel 数据拷贝");
//        // 拷贝数据
//        memcpy(*data,mapPtr,dataSize);
//        // 解除Map
//        glUnmapBuffer(GL_PIXEL_PACK_BUFFER);
//    } else{
//        LOGD("readPixel glMapBufferRange null");
//    }
//    // 完事了把GL_PIXEL_PACK_BUFFER这个地方的缓冲区解绑掉，以免别的函数误操作
//    glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
//}

// 双PBO读取测试
void PBOOpengl::readPixel(uint8_t **data,int *width,int *height) {

    *width = eglHelper->viewWidth;
    *height = eglHelper->viewHeight;
    int dataSize = eglHelper->viewWidth * eglHelper->viewHeight * 4;
    *data = new uint8_t[dataSize];

    // 首先我们要把缓冲区绑定到 GL_PIXEL_PACK_BUFFER 这个地方
```

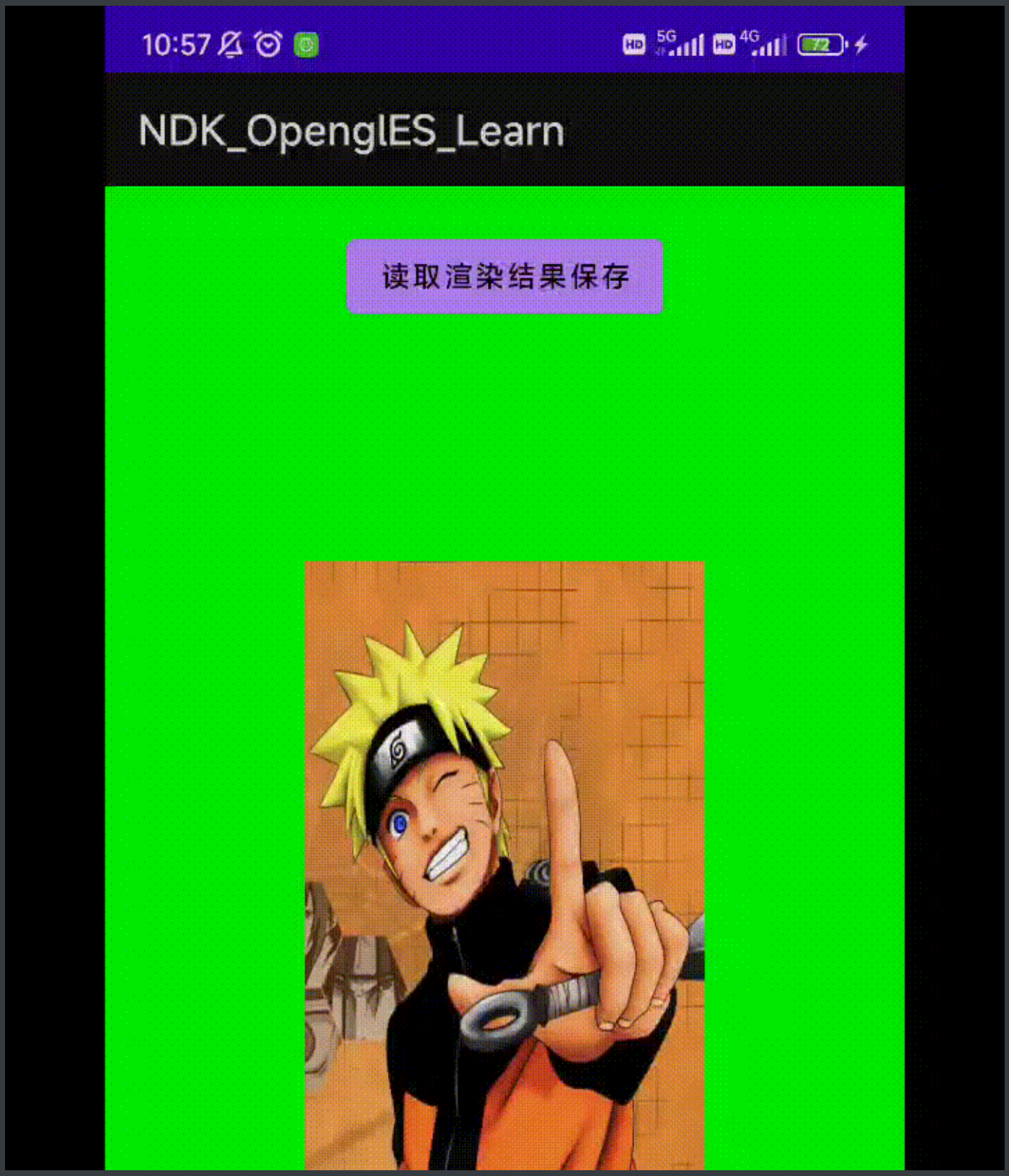
```
    // 指针运算你要会一点呀
    glBindBuffer(GL_PIXEL_PACK_BUFFER, *(downloadPboIds +
downloadPboIndex % NUM_PBO));
    // 重新分配一下空间
    glBufferData(GL_PIXEL_PACK_BUFFER, dataSize, nullptr,
GL_STREAM_DRAW);
    // 这个函数会判断 GL_PIXEL_PACK_BUFFER 这个地方有没有绑定一个缓冲区，如果
有，那就把数据写入到这个缓冲区里
    // 前4个参数就是要读取的屏幕区域，不多解释
    //  格式是RGB，类型是BYTE，每个像素1字节
    // 如果GL_PIXEL_PACK_BUFFER有绑定缓冲区，最后一个参数就作为偏移值来使用，传
nullptr就行
    glReadPixels(0, 0, *width, *height, GL_RGBA, GL_UNSIGNED_BYTE, 0);
    // 好了我们已经成功把屏幕的像素数据复制到了缓冲区里

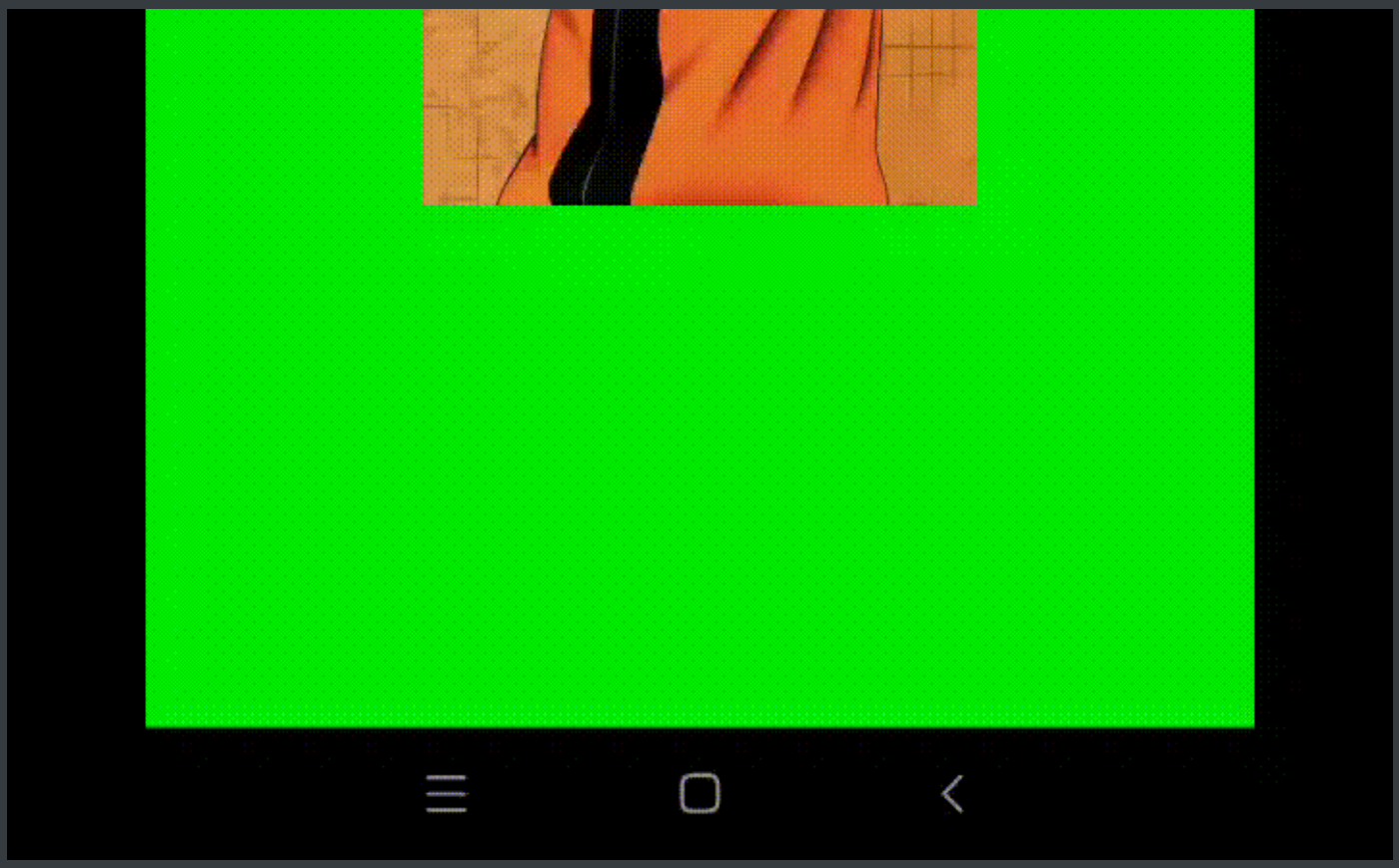
    // 这时候，你可以用 glMapBuffer 得到缓冲区的内存指针，来读取里面的像素数据，
保存到图片文件
    // 注意glMapBuffer的第1个参数不一定要是GL_PIXEL_PACK_BUFFER，你可以把缓
冲区绑定到比如上面init函数的GL_ARRAY_BUFFER
    //  然后这里也传GL_ARRAY_BUFFER，由于懒得再绑定一次，就接着用上面绑定的
GL_PIXEL_PACK_BUFFER吧
    GLubyte *mapPtr = static_cast<GLubyte *>
(glMapBufferRange(GL_PIXEL_PACK_BUFFER, 0, dataSize,

 GL_MAP_READ_BIT));
    if (nullptr != mapPtr)
    {
        LOGD("readPixel 数据拷贝");
        // 拷贝数据
        memcpy(*data,mapPtr,dataSize);
        // 解除Map
        glUnmapBuffer(GL_PIXEL_PACK_BUFFER);
    } else{
        LOGD("readPixel glMapBufferRange null");
    }
    // 完事了把GL_PIXEL_PACK_BUFFER这个地方的缓冲区解绑掉，以免别的函数误操作
```

```
        glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
        // Pbo索引自加
        downloadPboIndex++;
    }
```

运行结果：





通过点击按钮可以实现使用双PBO的方式下载纹理数据。

关于PBO的坑

以下是笔者在网上看到某博主总结的关于PBO的坑，由于设备有限，笔者也没有进行过严格的对比测试，这里引用原文仅供记录参考：

> 然而PBO还有一个非常坑的地方，经测试表明，在部分硬件上glMapBufferRange映射出来的Buffer拷贝极为耗时，可以高达30+ms，这对于音视频处理显然是不能接受的。通常，映射出来的是一个DirectByteBuffer，也是一个堆外内存（C内存），这部分内存本身只能通过Buffer.get(byte[])拷贝来拿到数据，但正常情况下只需要2-3ms。出现这种问题估计是硬件上留下的坑。
> 所以，在Android上使用PBO是有比较多的兼容性问题的，包括上面说的。正确使用PBO的方式是，首先判断是否支持PBO，如果支持，则还是先使用glReadPixels进行读取测试，记录平均耗时，然后再使用PBO进行读取测试，记录平均耗时，最后对比两个方式的耗时，选择最快的一个。这样动态处理是比较复杂的，然而在这种情况下你不得不这样做。那么有没有一种既简单又高效的方式呢?

参考

http://www.songho.ca/opengl/gl_pbo.html

Opengl ES之YUV数据渲染

YUV回顾

记得在音视频基础知识介绍中，笔者专门介绍过YUV的相关知识，可以参考：《音视频基础知识-YUV图像》

YUV数据量相比RGB较小，因此YUV适用于传输，但是YUV图不能直接用于显示，需要转换为RGB格式才能显示，因而YUV数据渲染实际上就是使用Opengl ES将YUV数据转换程RGB数据，然后显示出来的过程。

也就是说Opengl ES之所以能渲染YUV数据其实就是使用了Opengl强大的并行计算能力，快速地将YUV数据转换程了RGB数据。

> 本文首发于微信公总号号：思想觉悟

> 更多关于音视频、FFmpeg、Opengl、C++的原创文章请关注微信公众号：思想觉悟

YUV的格式比较多，我们今天就以YUV420SP为例，而YUV420SP又分为 `NV12` 和 `NV21` 两种，因此今天我们的主题就是如何使用Opengl ES对 `NV12` 和 `NV21` 数据进行渲染显示。

在着色器中使用 `texture2D` 对YUV数据进行归一化处理后Y数据的映射范围是0到1，而U和V的数据映射范围是-0.5到0.5。

因为YUV格式图像 UV 分量的默认值分别是 127 ， Y 分量默认值是 0 ， 8 个 bit 位的取值范围是 0 ~ 255，由于在 shader 中纹理采样值需要进行归一化，所以 UV 分量的采样值需要分别减去 0.5 ， 确保 YUV 到 RGB 正确转换。

YUV数据准备

首先我们可以使用 `ffmpeg` 命令行将一张png图片转换成YUV格式的图片：

```
ffmpeg -i 图片名称.png -s 图片宽x图片高 -pix_fmt nv12或者nv21 输出名称.yuv)
```

通过上面这个命令行我们就可以将一张图片转换成yuv格式的图片，此时我们可以使用软件 `YUVViewer` 看下你转换的图片对不对，如果本身转换出来的图片就是错的，那么后面的程序就白搭了...

注意：转换图片的宽高最好是2的幂次方，笔者测试了下发现如果宽高不是2的幂次方的话虽然能正常转换，但是查看的时候要么有色差，要么有缺陷，也有可能正常。

又或者你可以极客一点，直接使用ffmpeg代码解码视频的方式获得YUV数据并保存，这个可以参考笔者之前的文章：

《FFmpeg连载3-视频解码》

同时在上面的文章中笔者也介绍了通过 `ffplay` 命令行的方式查看YUV数据的方法。

YUV数据渲染

YUV 渲染步骤：

- 生成 2 个纹理，分别用于承载Y数据和UV数据，编译链接着色器程序；

NV21和NV12格式的YUV数据是只有两个平面的，它们的排列顺序是 `YYYY UVUV` 或者 `YYYY VUVU` 因此我们的片元着色器需要两个纹理采样。

- 确定纹理坐标及对应的顶点坐标；
- 分别加载 NV21 的两个 Plane 数据到 2 个纹理，加载纹理坐标和顶点坐标数据到着色器程序；
- 绘制。

YUV与RGB的转换格式图：

1. **常规转换标准：**

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1.0 & 0.0 & 1.370705 \\ 1.0 & -0.337633 & -0.698001 \\ 1.0 & 1.732446 & 0.0 \end{bmatrix}$$

2. **BT.601 标准：**（SD TV）

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1.0 & 0.0 & 1.13983 \\ 1.0 & -0.39465 & -0.5806 \\ 1.0 & 2.03211 & 0.0 \end{bmatrix}$$

3. **BT.709 标准：**（HD TV）

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1.0 & 0.0 & 1.28033 \\ 1.0 & -0.21482 & -0.38059 \\ 1.0 & 2.12798 & 0.0 \end{bmatrix}$$

思想觉悟

> YUV与RGB的转换公式

在OpenGLES的内置矩阵实际上是一列一列地构建的，比如YUV和RGB的转换矩阵的构建是：

```
// 标准转换，舍弃了部分小数精度
mat3 convertMat = mat3(1.0, 1.0, 1.0,       //第一列
                       0.0, -0.338, 1.732,  //第二列
                       1.371, -0.698,  0.0);//第三列
```

OpenGLES 实现 YUV 渲染需要用到 GL_LUMINANCE 和 GL_LUMINANCE_ALPHA 格式的纹理，其中 GL_LUMINANCE 纹理用来加载 NV21 Y Plane 的数据，GL_LUMINANCE_ALPHA 纹理用来加载 UV Plane 的数据。

废话少说，show me the code

YUVRenderOpengl.h

```
#ifndef NDK_OPENGLES_LEARN_YUVRENDEROPENGL_H
#define NDK_OPENGLES_LEARN_YUVRENDEROPENGL_H
#include "BaseOpengl.h"

class YUVRenderOpengl: public BaseOpengl{

public:
    YUVRenderOpengl();

    virtual ~YUVRenderOpengl();

    virtual void onDraw() override;

    // 设置yuv数据
    virtual void setYUVData(void *y_data,void *uv_data, int width, int
height, int yuvType);

private:
    GLint positionHandle{-1};
    GLint textureHandle{-1};
    GLint y_textureSampler{-1};
    GLint uv_textureSampler{-1};
    GLuint y_textureId{0};
    GLuint uv_textureId{0};
};

#endif //NDK_OPENGLES_LEARN_YUVRENDEROPENGL_H
```

YUVRenderOpengl.cpp

```
#include "YUVRenderOpengl.h"

#include "../utils/Log.h"

// 顶点着色器
```

```
static const char *ver = "#version 300 es\n"
                         "in vec4 aPosition;\n"
                         "in vec2 aTexCoord;\n"
                         "out vec2 TexCoord;\n"
                         "void main() {\n"
                         "  TexCoord = aTexCoord;\n"
                         "  gl_Position = aPosition;\n"
                         "}";

// 片元着色器 nv12
//static const char *fragment = "#version 300 es\n"
//                              "precision mediump float;\n"
//                              "out vec4 FragColor;\n"
//                              "in vec2 TexCoord;\n"
//                              "uniform sampler2D y_texture; \n"
//                              "uniform sampler2D uv_texture;\n"
//                              "void main()\n"
//                              "{\n"
//                              "vec3 yuv;\n"
//                              "yuv.x = texture(y_texture,
TexCoord).r;\n"
//                              "yuv.y = texture(uv_texture,
TexCoord).r-0.5;\n"
//                              "yuv.z = texture(uv_texture,
TexCoord).a-0.5;\n"
//                              "vec3 rgb =mat3( 1.0,1.0,1.0,\n"
//                              "0.0,-0.344,1.770,1.403,-0.714,0.0) *
yuv;\n"
//                              "FragColor = vec4(rgb, 1);\n"
//                              "}";

/**
 *  仅仅是以下两句不同而已
 *  "yuv.y = texture(uv_texture, TexCoord).r-0.5;\n"
 *  "yuv.z = texture(uv_texture, TexCoord).a-0.5;\n"
 */
```

```
// 片元着色器nv21 仅仅是
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
                              "out vec4 FragColor;\n"
                              "in vec2 TexCoord;\n"
                              "uniform sampler2D y_texture; \n"
                              "uniform sampler2D uv_texture;\n"
                              "void main()\n"
                              "{\n"
                              "vec3 yuv;\n"
                              "yuv.x = texture(y_texture,
TexCoord).r;\n"
                              "yuv.y = texture(uv_texture,
TexCoord).a-0.5;\n"
                              "yuv.z = texture(uv_texture,
TexCoord).r-0.5;\n"
                              "vec3 rgb =mat3( 1.0,1.0,1.0,\n"
                              "0.0,-0.344,1.770,1.403,-0.714,0.0) *
yuv;\n"
                              "FragColor = vec4(rgb, 1);\n"
                              "}";

// 使用绘制两个三角形组成一个矩形的形式(三角形带)
// 第一第二第三个点组成一个三角形，第二第三第四个点组成一个三角形
const static GLfloat VERTICES[] = {
        0.5f,-0.5f, // 右下
        0.5f,0.5f, // 右上
        -0.5f,-0.5f, // 左下
        -0.5f,0.5f // 左上
};

// 贴图纹理坐标（参考手机屏幕坐标系统，原点在左上角）
//由于对一个OpenGL纹理来说，它没有内在的方向性，因此我们可以使用不同的坐标把它定向到任何我们喜欢的方向上，然而大多数计算机图像都有一个默认的方向，它们通常被规定为y轴向下，X轴向右
const static GLfloat TEXTURE_COORD[] = {
```

```
        1.0f,1.0f, // 右下
        1.0f,0.0f, // 右上
        0.0f,1.0f, // 左下
        0.0f,0.0f // 左上
};

YUVRenderOpengl::YUVRenderOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
    textureHandle = glGetAttribLocation(program,"aTexCoord");
    y_textureSampler = glGetUniformLocation(program,"y_texture");
    uv_textureSampler = glGetUniformLocation(program,"uv_texture");
    LOGD("program:%d",program);
    LOGD("positionHandle:%d",positionHandle);
    LOGD("textureHandle:%d",textureHandle);
    LOGD("y_textureSampler:%d",y_textureSampler);
    LOGD("uv_textureSampler:%d",uv_textureSampler);
}

YUVRenderOpengl::~YUVRenderOpengl() {

}

void YUVRenderOpengl::setYUVData(void *y_data, void *uv_data, int
width, int height, int yuvType) {

    // 准备y数据纹理
    glGenTextures(1, &y_textureId);
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(y_textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, y_textureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
```

```
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_LUMINANCE, width, height, 0,
GL_LUMINANCE, GL_UNSIGNED_BYTE, y_data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);

    glBindTexture(GL_TEXTURE_2D, y_textureId);
    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);

    // 准备uv数据纹理
    glGenTextures(1, &uv_textureId);
    glActiveTexture(GL_TEXTURE3);
    glUniform1i(uv_textureSampler, 3);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, uv_textureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    // 注意宽高
    // 注意要使用 GL_LUMINANCE_ALPHA
    glTexImage2D(GL_TEXTURE_2D, 0, GL_LUMINANCE_ALPHA, width/2,
height/2, 0, GL_LUMINANCE_ALPHA, GL_UNSIGNED_BYTE, uv_data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);
    glBindTexture(GL_TEXTURE_2D, uv_textureId);
    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}

void YUVRenderOpengl::onDraw() {
```

```
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);

    // 激活纹理
    glActiveTexture(GL_TEXTURE2);
    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, y_textureId);
    glUniform1i(y_textureSampler, 2);

    // 激活纹理
    glActiveTexture(GL_TEXTURE3);
    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, uv_textureId);
    glUniform1i(uv_textureSampler, 3);

    /**
     * size 几个数字表示一个点，显示是两个数字表示一个点
     * normalized 是否需要归一化，不用，这里已经归一化了
     * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
     */
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);

 glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);

    // 纹理坐标
    glEnableVertexAttribArray(textureHandle);

 glVertexAttribPointer(textureHandle,2,GL_FLOAT,GL_FALSE,0,TEXTURE_COO
RD);

    // 4个顶点绘制两个三角形组成矩形
    glDrawArrays(GL_TRIANGLE_STRIP,0,4);
```

```
        glUseProgram(0);

        // 禁用顶点
        glDisableVertexAttribArray(positionHandle);
        if(nullptr != eglHelper){
            eglHelper->swapBuffers();
        }

        glBindTexture(GL_TEXTURE_2D, 0);
    }
```

注意看着色器代码的注释，NV12和NV21的渲染仅仅是着色器代码有细小差别而已。

YUVRenderActivity.java

```
    public class YUVRenderActivity extends BaseGlActivity {

        // 注意改成你自己图片的宽高
        private int yuvWidth = 640;
        private int yuvHeight = 428;

        private String nv21Path;
        private String nv12Path;
        private Handler handler = new Handler(Looper.getMainLooper());

        @Override
        protected void onCreate(@Nullable Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            // 注意申请磁盘写权限
            // 拷贝资源
            nv21Path = getFilesDir().getAbsolutePath() + "/nv21.yuv";
            FileUtils.copyAssertToDest(this,"nv21.yuv",nv21Path);

            nv12Path = getFilesDir().getAbsolutePath() + "/nv12.yuv";
            FileUtils.copyAssertToDest(this,"nv12.yuv",nv12Path);
        }
```

```
    @Override
    public BaseOpengl createOpengl() {
        YUVRenderOpengl yuvRenderOpengl = new YUVRenderOpengl();
        return yuvRenderOpengl;
    }

    @Override
    protected void onResume() {
        super.onResume();
        handler.postDelayed(new Runnable() {
            @Override
            public void run() {
                // 注意nv12和nv21的偏远着色器有点不一样的，需要手动改下调试YUVRenderOpengl.cpp
//                if(!TextUtils.isEmpty(nv12Path)){
//                    loadYuv(nv12Path,BaseOpengl.YUV_DATA_TYPE_NV12);
//                }

                if(!TextUtils.isEmpty(nv21Path)){
                    loadYuv(nv21Path,BaseOpengl.YUV_DATA_TYPE_NV21);
                }
            }
        },200);
    }

    @Override
    protected void onStop() {
        handler.removeCallbacksAndMessages(null);
        super.onStop();
    }

    private void loadYuv(String path,int yuvType){
        try {
            InputStream inputStream = new FileInputStream(new File(path));
```

```
            Log.v("fly_learn_opengl","---length:" +
inputStream.available());
            byte[] yData = new byte[yuvWidth * yuvHeight];
            inputStream.read(yData,0,yData.length);
            byte[] uvData = new byte[yuvWidth * yuvHeight / 2];
            inputStream.read(uvData,0,uvData.length);
            Log.v("fly_learn_opengl","---read:" + (yData.length +
uvData.length) + "available:" + inputStream.available());

 myGLSurfaceView.setYuvData(yData,uvData,yuvWidth,yuvHeight);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

这个主要看懂 `loadYuv` 方法，对于YUV数据的读取即可。

思考

都说YUV的格式较多，本文我们介绍了如何使用Opengl ES渲染YUV420SP数据，那么对于YUV420P数据，使用Opengl ES如何渲染呢？欢迎关注评论解答交流。

Opengl ES之RGB转NV21

前言

在上一篇理论文章中我们介绍了YUV到RGB之间转换的几种公式与一些优化算法，今天我们再来介绍一下RGB到YUV的转换，顺便使用Opengl ES做个实践，将一张RGB的图片通过Shader
的方式转换YUV格式图，然后保存到本地。

可能有的童鞋会问，YUV转RGB是为了渲染显示，那么RGB转YUV的应用场景是什么？在做视频编码的时候我们可以使用MediaCodec搭配Surface就可以完成，貌似也没有用到RGB转YUV的功能啊，
硬编码没有用到，那么软编码呢？一般我们做视频编码的时候都是硬编码优先，软编码兜底的原则，在遇到一些硬编码不可用的情况下可能就需要用到x264库进行软编码了，而此时RGB转YUV可能就派上用场啦。

RGB到YUV的转换公式

在前面 Opengl ES之YUV数据渲染 一文中我们介绍过YUV的几种兼容标准，下面我们看看RGB到YUV的转换公式：

RGB 转 BT.601 YUV

```
Y  =  0.257R + 0.504G + 0.098B + 16
Cb = -0.148R - 0.291G + 0.439B + 128
Cr =  0.439R - 0.368G - 0.071B + 128
```

RGB 转 BT.709 YUV

```
Y  =  0.183R + 0.614G + 0.062B + 16
Cb = -0.101R - 0.339G + 0.439B + 128
Cr =  0.439R - 0.399G - 0.040B + 128
```

或者也可以使用矩阵运算的方式进行转换，更加的便捷：

RGB ——> YUV

1. **常规转换标准：**

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} 0.29882 & 0.58681 & 0.114363 \\ -0.172485 & -0.338718 & 0.511207 \\ 0.51155 & -0.42811 & -0.08343 \end{bmatrix}$$

2. **BT.601 标准：**（SD TV）

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.14713 & -0.28886 & 0.436 \\ 0.615 & -0.51499 & -0.10001 \end{bmatrix}$$

3. **BT.709 标准：**（HD TV）

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} 0.2126 & 0.7152 & 0.0722 \\ -0.09991 & -0.33609 & 0.436 \\ 0.615 & -0.55861 & -0.05639 \end{bmatrix}$$

思想觉悟
https://blog.csdn.net/qq_27524749

> RGB转YUV

RGB转YUV

先说一下RGB转YUV的过程，先将RGB数据按照公式转换为YUV数据，然后将YUV数据按照RGBA进行排布，这一步的目的是为了后续数据读取，最后使用 `glReadPixels` 读取YUV数据。

而对于OpenGL ES来说，目前它输入只认RGBA、lumiance、luminace alpha这几个格式，输出大多数实现只认RGBA格式，因此输出的数据格式虽然是YUV格式，但是在存储时我们仍然要按照RGBA方式去访问texture数据。

以NV21的YUV数据为例，它的内存大小为 `width x height * 3 / 2` 。如果是RGBA的格式存储的话，占用的内存空间大小是 `width x height x 4` （因为 RGBA 一共4个通道）。很显然它们的内存大小是对不上的，
那么该如何调整Opengl buffer的大小让RGBA的输出能对应上YUV的输出呢？我们可以设计输出的宽为 `width / 4` ，高为 `height * 3 / 2` 即可。

为什么是这样的呢？虽然我们的目的是将RGB转换成YUV，但是我们的输入和输出时读取的类型GLenum是依然是RGBA，也就是说：width x height x 4 = (width / 4) x (height * 3 / 2) * 4

而YUV数据在内存中的分布以下这样子的：

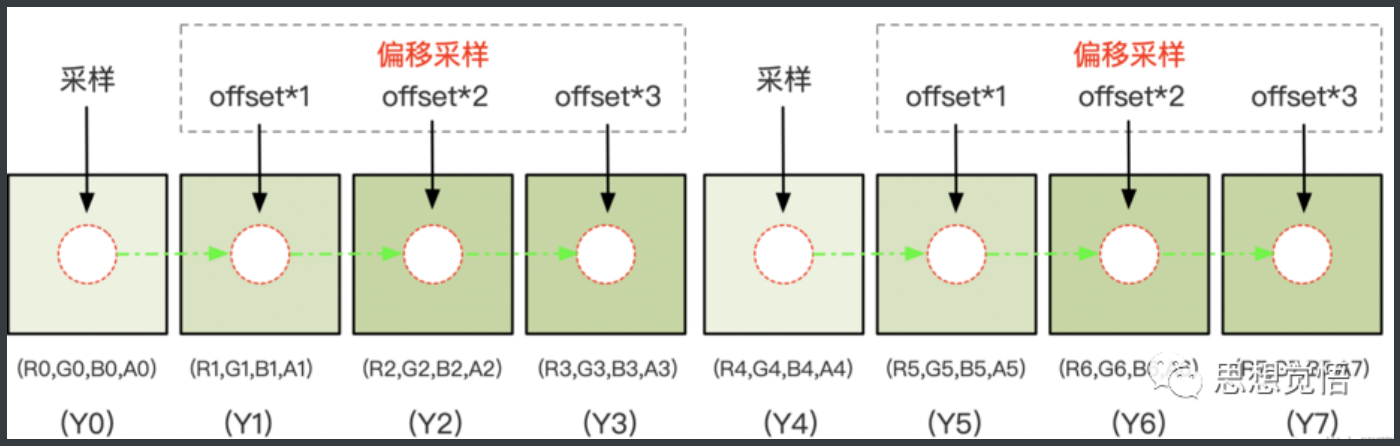
```
   width / 4
   |--------------|
   |              |
   |              | h
   |      Y       |
   |--------------|
   |   U   |  V   |
   |       |      |  h / 2
   |--------------|
```

那么上面的排序如果进行了归一化之后呢，就变成了下面这样子了：

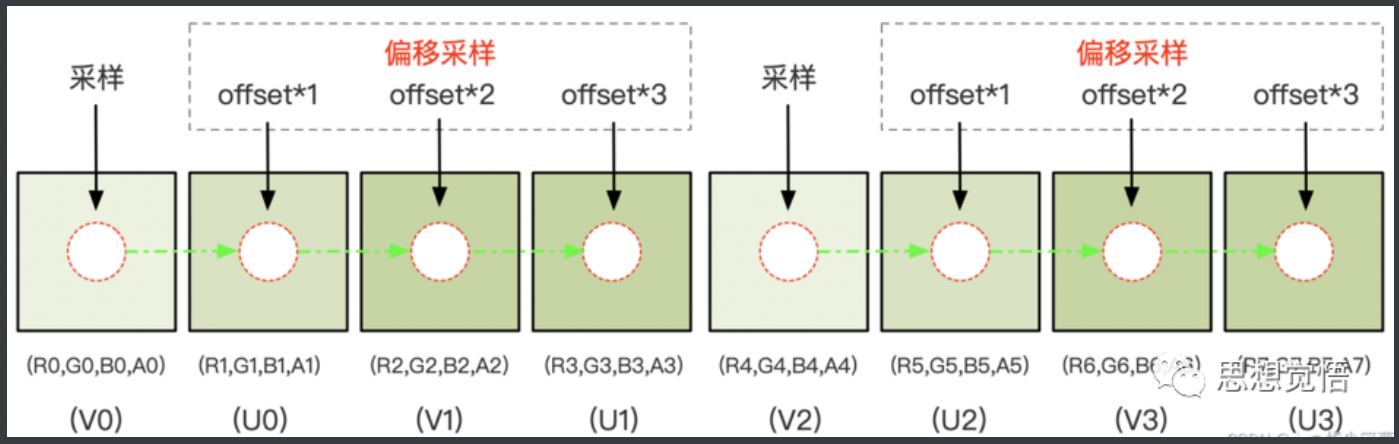
```
(0,0) width / 4  (1,0)
|--------------|
|              |
|              |  h
|      Y       |
|--------------|  (1,2/3)
|   U   |  V   |
|       |      |  h / 2
|--------------|
(0,1)           (1,1)
```

从上面的排布可以看出看出，在纹理坐标 `y < (2/3)` 时，需要完成一次对整个纹理的采样，用于生成Y数据，当纹理坐标 `y > (2/3)` 时，同样需要再进行一次对整个纹理的采样，用于生成UV的数据。同时还需要将我们的视窗设置为 `glViewport(0, 0, width / 4, height * 1.5);`

由于视口宽度设置为原来的 1/4 ，可以简单的认为相对于原来的图像每隔4个像素做一次采样，由于我们生成Y数据是要对每一个像素都进行采样，所以还需要进行3次偏移采样。



同理，生成对于UV数据也需要进行3次额外的偏移采样。



在着色器中offset变量需要设置为一个归一化之后的值：`1.0/width`，按照原理图，在纹理坐标 y < (2/3) 范围，一次采样（加三次偏移采样）4 个 RGBA 像素（R,G,B,A）生成 1 个（Y0,Y1,Y2,Y3），整个范围采样结束时填充好 `width*height` 大小的缓冲区；当纹理坐标 y > (2/3) 范围，一次采样（加三次偏移采样）4 个 RGBA 像素（R,G,B,A）生成 1 个（V0,U0,V0,U1），又因为 UV 缓冲区的高度为 height/2，VU plane 在垂直方向的采样是隔行进行，整个范围采样结束时填充好 `width*height/2` 大小的缓冲区。

主要代码

RGBtoYUVOpengl.cpp

```
#include "../utils/Log.h"
#include "RGBtoYUVOpengl.h"

// 顶点着色器
static const char *ver = "#version 300 es\n"
                         "in vec4 aPosition;\n"
                         "in vec2 aTexCoord;\n"
                         "out vec2 v_texCoord;\n"
                         "void main() {\n"
                         "  v_texCoord = aTexCoord;\n"
                         "  gl_Position = aPosition;\n"
                         "}";

// 片元着色器
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
```

```
                                "in vec2 v_texCoord;\n"
                                "layout(location = 0) out vec4
outColor;\n"
                                "uniform sampler2D s_TextureMap;\n"
                                "uniform float u_Offset;\n"
                                "const vec3 COEF_Y = vec3(0.299, 0.587,
0.114);\n"
                                "const vec3 COEF_U = vec3(-0.147,
-0.289, 0.436);\n"
                                "const vec3 COEF_V = vec3(0.615, -0.515,
-0.100);\n"
                                "const float UV_DIVIDE_LINE = 2.0 /
3.0;\n"
                                "void main(){\n"
                                "    vec2 texelOffset = vec2(u_Offset,
0.0);\n"
                                "    if (v_texCoord.   y <=
UV_DIVIDE_LINE) {\n"
                                "        vec2 texCoord =
vec2(v_texCoord.   x, v_texCoord.   y * 3.0 / 2.0);\n"
                                "        vec4 color0 =
texture(s_TextureMap, texCoord);\n"
                                "        vec4 color1 =
texture(s_TextureMap, texCoord + texelOffset);\n"
                                "        vec4 color2 =
texture(s_TextureMap, texCoord + texelOffset * 2.0);\n"
                                "        vec4 color3 =
texture(s_TextureMap, texCoord + texelOffset * 3.0);\n"
                                "        float y0 = dot(color0.   rgb,
COEF_Y);\n"
                                "        float y1 = dot(color1.   rgb,
COEF_Y);\n"
                                "        float y2 = dot(color2.   rgb,
COEF_Y);\n"
                                "        float y3 = dot(color3.   rgb,
COEF_Y);\n"
```

```
                                "        outColor = vec4(y0, y1, y2,
y3);\n"
                                "    } else {\n"
                                "        vec2 texCoord =
vec2(v_texCoord.x, (v_texCoord.y - UV_DIVIDE_LINE) * 3.0);\n"
                                "        vec4 color0 =
texture(s_TextureMap, texCoord);\n"
                                "        vec4 color1 =
texture(s_TextureMap, texCoord + texelOffset);\n"
                                "        vec4 color2 =
texture(s_TextureMap, texCoord + texelOffset * 2.0);\n"
                                "        vec4 color3 =
texture(s_TextureMap, texCoord + texelOffset * 3.0);\n"
                                "        float v0 = dot(color0.   rgb,
COEF_V) + 0.5;\n"
                                "        float u0 = dot(color1.   rgb,
COEF_U) + 0.5;\n"
                                "        float v1 = dot(color2.   rgb,
COEF_V) + 0.5;\n"
                                "        float u1 = dot(color3.   rgb,
COEF_U) + 0.5;\n"
                                "        outColor = vec4(v0, u0, v1,
u1);\n"
                                "    }\n"
                                "}";


// 使用绘制两个三角形组成一个矩形的形式(三角形带)
// 第一第二第三个点组成一个三角形，第二第三第四个点组成一个三角形
const static GLfloat VERTICES[] = {
        1.0f,-1.0f, // 右下
        1.0f,1.0f, // 右上
        -1.0f,-1.0f, // 左下
        -1.0f,1.0f // 左上
};
```

```
// FBO贴图纹理坐标（参考手机屏幕坐标系统，原点在左下角）
// 注意坐标不要错乱
const static GLfloat TEXTURE_COORD[] = {
        1.0f,0.0f, // 右下
        1.0f,1.0f, // 右上
        0.0f,0.0f, // 左下
        0.0f,1.0f // 左上
};

RGBtoYUVOpengl::RGBtoYUVOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
    textureHandle = glGetAttribLocation(program,"aTexCoord");
    textureSampler = glGetUniformLocation(program,"s_TextureMap");
    u_Offset = glGetUniformLocation(program,"u_Offset");
    LOGD("program:%d",program);
    LOGD("positionHandle:%d",positionHandle);
    LOGD("textureHandle:%d",textureHandle);
    LOGD("textureSample:%d",textureSampler);
    LOGD("u_Offset:%d",u_Offset);
}

RGBtoYUVOpengl::~RGBtoYUVOpengl() noexcept {

}

void RGBtoYUVOpengl::fboPrepare() {
    glGenTextures(1, &fboTextureId);
    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, fboTextureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
```

```
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    glBindTexture(GL_TEXTURE_2D, GL_NONE);

    glGenFramebuffers(1,&fboId);
    glBindFramebuffer(GL_FRAMEBUFFER,fboId);
    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D,fboTextureId);
    glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0,
GL_TEXTURE_2D, fboTextureId, 0);
    // 这个纹理是多大的?
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, imageWidth / 4,
imageHeight * 1.5, 0, GL_RGBA, GL_UNSIGNED_BYTE, nullptr);
    // 检查FBO状态
    if (glCheckFramebufferStatus(GL_FRAMEBUFFER)!=
GL_FRAMEBUFFER_COMPLETE) {
        LOGE("FBOSample::CreateFrameBufferObj glCheckFramebufferStatus
status != GL_FRAMEBUFFER_COMPLETE");
    }
    // 解绑
    glBindTexture(GL_TEXTURE_2D, GL_NONE);
    glBindFramebuffer(GL_FRAMEBUFFER, GL_NONE);
}

// 渲染逻辑
void RGBtoYUVOpengl::onDraw() {

    // 绘制到FBO上去
    // 绑定fbo
    glBindFramebuffer(GL_FRAMEBUFFER, fboId);

    glPixelStorei(GL_UNPACK_ALIGNMENT,1);

    // 设置视口大小
    glViewport(0, 0,imageWidth / 4, imageHeight * 1.5);
    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
```

```
    glUseProgram(program);

    // 激活纹理
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId);

    // 设置偏移
    float texelOffset = (float) (1.f / (float) imageWidth);
    glUniform1f(u_Offset,texelOffset);

    /**
     * size 几个数字表示一个点，显示是两个数字表示一个点
     * normalized 是否需要归一化，不用，这里已经归一化了
     * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
     */
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);

 glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);

    // 纹理坐标
    glEnableVertexAttribArray(textureHandle);

 glVertexAttribPointer(textureHandle,2,GL_FLOAT,GL_FALSE,0,TEXTURE_COO
RD);

    // 4个顶点绘制两个三角形组成矩形
    glDrawArrays(GL_TRIANGLE_STRIP,0,4);

    glUseProgram(0);

    // 禁用顶点
    glDisableVertexAttribArray(positionHandle);
```

```
    if(nullptr != eglHelper){
        eglHelper->swapBuffers();
    }

    glBindTexture(GL_TEXTURE_2D, 0);

    // 解绑fbo
    glBindFramebuffer(GL_FRAMEBUFFER, 0);
}

// 设置RGB图像数据
void RGBtoYUVOpengl::setPixel(void *data, int width, int height, int
length) {
    LOGD("texture setPixel");
    imageWidth = width;
    imageHeight = height;

    // 准备fbo
    fboPrepare();

    glGenTextures(1, &textureId);
    // 激活纹理，注意以下这个两句是搭配的，glActiveTexture激活的是那个纹理，就设
置的sampler2D是那个
    // 默认是0，如果不是0的话，需要在onDraw的时候重新激活一下?
//    glActiveTexture(GL_TEXTURE0);
//    glUniform1i(textureSampler, 0);

// 例如，一样的
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
```

```
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA,
GL_UNSIGNED_BYTE, data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);
    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}

// 读取渲染后的YUV数据
void RGBtoYUVOpengl::readYUV(uint8_t **data, int *width, int *height)
{
    // 从fbo中读取
    // 绑定fbo
    *width = imageWidth;
    *height = imageHeight;
    glBindFramebuffer(GL_FRAMEBUFFER, fboId);
    glBindTexture(GL_TEXTURE_2D, fboTextureId);
    glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0,
            GL_TEXTURE_2D, fboTextureId, 0);
    *data = new uint8_t[imageWidth * imageHeight * 3 / 2];
    glReadPixels(0, 0, imageWidth / 4, imageHeight * 1.5, GL_RGBA,
GL_UNSIGNED_BYTE, *data);
    glBindTexture(GL_TEXTURE_2D, 0);
    // 解绑fbo
    glBindFramebuffer(GL_FRAMEBUFFER, 0);
}
```

下面是Activity的主要代码逻辑：

```
public class RGBToYUVActivity extends AppCompatActivity {

    protected MyGLSurfaceView myGLSurfaceView;
```

```
    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_rgb_to_yuv);
        myGLSurfaceView = findViewById(R.id.my_gl_surface_view);
        myGLSurfaceView.setOpenGlListener(new
MyGLSurfaceView.OnOpenGlListener() {
            @Override
            public BaseOpengl onOpenglCreate() {
                return new RGBtoYUVOpengl();
            }

            @Override
            public Bitmap requestBitmap() {
                BitmapFactory.Options options = new
BitmapFactory.Options();
                options.inScaled = false;
                return
BitmapFactory.decodeResource(getResources(),R.mipmap.ic_smile,options)
;
            }

            @Override
            public void readPixelResult(byte[] bytes) {
                if (null != bytes) {

                }
            }

            // 也就是RGBtoYUVOpengl::readYUV读取到结果数据回调
            @Override
            public void readYUVResult(byte[] bytes) {
                if (null != bytes) {
                    String fileName = System.currentTimeMillis() +
".yuv";
```

```
                    File fileParent = getFilesDir();
                    if (!fileParent.exists()) {
                        fileParent.mkdirs();
                    }
                    FileOutputStream fos = null;
                    try {
                        File file = new File(fileParent, fileName);
                        fos = new FileOutputStream(file);
                        fos.write(bytes,0,bytes.length);
                        fos.flush();
                        fos.close();
                        Toast.makeText(RGBToYUVActivity.this, "YUV图片
保存成功" + file.getAbsolutePath(), Toast.LENGTH_LONG).show();
                    } catch (Exception e) {
                        Log.v("fly_learn_opengl", "图片保存异常: " +
e.getMessage());
                        Toast.makeText(RGBToYUVActivity.this, "YUV图片
保存失败", Toast.LENGTH_LONG).show();
                    }
                }
            }

        });

        Button button = findViewById(R.id.bt_rgb_to_yuv);
        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                myGLSurfaceView.readYuvData();
            }
        });

        ImageView iv_rgb = findViewById(R.id.iv_rgb);
        iv_rgb.setImageResource(R.mipmap.ic_smile);

    }
```

```
}
```

以下是自定义SurfaceView的代码：

```
public class MyGLSurfaceView extends SurfaceView implements
SurfaceHolder.Callback {

    private final static int MSG_CREATE_GL = 101;
    private final static int MSG_CHANGE_GL = 102;
    private final static int MSG_DRAW_GL = 103;
    private final static int MSG_DESTROY_GL = 104;
    private final static int MSG_READ_PIXEL_GL = 105;
    private final static int MSG_UPDATE_BITMAP_GL = 106;
    private final static int MSG_UPDATE_YUV_GL = 107;
    private final static int MSG_READ_YUV_GL = 108;

    public BaseOpengl baseOpengl;
    private OnOpenGlListener onOpenGlListener;
    private HandlerThread handlerThread;
    private Handler renderHandler;

    public int surfaceWidth;
    public int surfaceHeight;

    public MyGLSurfaceView(Context context) {
        this(context,null);
    }

    public MyGLSurfaceView(Context context, AttributeSet attrs) {
        super(context, attrs);
        getHolder().addCallback(this);
        handlerThread = new HandlerThread("RenderHandlerThread");
        handlerThread.start();
        renderHandler = new Handler(handlerThread.getLooper()){
            @Override
            public void handleMessage(@NonNull Message msg) {
```

```
                switch (msg.what){
                    case MSG_CREATE_GL:
                        baseOpengl =
onOpenGlListener.onOpenglCreate();
                        Surface surface = (Surface) msg.obj;
                        if(null != baseOpengl){
                            baseOpengl.surfaceCreated(surface);
                            Bitmap bitmap =
onOpenGlListener.requestBitmap();
                            if(null != bitmap){
                                baseOpengl.setBitmap(bitmap);
                            }
                        }
                        break;

                    case MSG_CHANGE_GL:
                        if(null != baseOpengl){
                            Size size = (Size) msg.obj;

 baseOpengl.surfaceChanged(size.getWidth(),size.getHeight());
                        }
                        break;

                    case MSG_DRAW_GL:
                        if(null != baseOpengl){
                            baseOpengl.onGlDraw();
                        }
                        break;

                    case MSG_READ_PIXEL_GL:
                        if(null != baseOpengl){
                           byte[] bytes = baseOpengl.readPixel();
                           if(null != bytes && null !=
onOpenGlListener){

onOpenGlListener.readPixelResult(bytes);
```

```
                    }
                }
                break;

            case MSG_READ_YUV_GL:
                if(null != baseOpengl){
                    byte[] bytes = baseOpengl.readYUVResult();
                    if(null != bytes && null !=
onOpenGlListener){
                        onOpenGlListener.readYUVResult(bytes);
                    }
                }
                break;


            case MSG_UPDATE_BITMAP_GL:
                if(null != baseOpengl){
                    Bitmap bitmap =
onOpenGlListener.requestBitmap();
                    if(null != bitmap){
                        baseOpengl.setBitmap(bitmap);
                        baseOpengl.onGlDraw();
                    }
                }
                break;

            case MSG_UPDATE_YUV_GL:
                if(null != baseOpengl){
                    YUVBean yuvBean = (YUVBean) msg.obj;
                    if(null != yuvBean){

 baseOpengl.setYuvData(yuvBean.getyData(),yuvBean.getUvData(),yuvBean.
getWidth(),yuvBean.getHeight());
                        baseOpengl.onGlDraw();
                    }
                }
```

```
                        break;

                    case MSG_DESTROY_GL:
                        if(null != baseOpengl){
                            baseOpengl.surfaceDestroyed();
                        }
                        break;
                }
            }
        };
    }

    public void setOpenGlListener(OnOpenGlListener listener) {
        this.onOpenGlListener = listener;
    }

    @Override
    public void surfaceCreated(@NonNull SurfaceHolder surfaceHolder) {
        Message message = Message.obtain();
        message.what = MSG_CREATE_GL;
        message.obj = surfaceHolder.getSurface();
        renderHandler.sendMessage(message);
    }

    @Override
    public void surfaceChanged(@NonNull SurfaceHolder surfaceHolder,
int i, int w, int h) {
        Message message = Message.obtain();
        message.what = MSG_CHANGE_GL;
        message.obj = new Size(w,h);
        renderHandler.sendMessage(message);

        Message message1 = Message.obtain();
        message1.what = MSG_DRAW_GL;
        renderHandler.sendMessage(message1);
```

```
            surfaceWidth = w;
            surfaceHeight = h;

        }

        @Override
        public void surfaceDestroyed(@NonNull SurfaceHolder surfaceHolder)
    {
            Message message = Message.obtain();
            message.what = MSG_DESTROY_GL;
            renderHandler.sendMessage(message);
        }

        public void readGlPixel(){
            Message message = Message.obtain();
            message.what = MSG_READ_PIXEL_GL;
            renderHandler.sendMessage(message);
        }

        public void readYuvData(){
            Message message = Message.obtain();
            message.what = MSG_READ_YUV_GL;
            renderHandler.sendMessage(message);
        }

        public void updateBitmap(){
            Message message = Message.obtain();
            message.what = MSG_UPDATE_BITMAP_GL;
            renderHandler.sendMessage(message);
        }

        public void setYuvData(byte[] yData,byte[] uvData,int width,int
    height){
            Message message = Message.obtain();
            message.what = MSG_UPDATE_YUV_GL;
            message.obj = new YUVBean(yData,uvData,width,height);
```

```
            renderHandler.sendMessage(message);
        }

        public void release(){
            // todo 主要线程同步问题，当心surfaceDestroyed还没有执行到，但是就被
    release了，那就内存泄漏了
            if(null != baseOpengl){
                baseOpengl.release();
            }
        }

        public void requestRender(){
            Message message = Message.obtain();
            message.what = MSG_DRAW_GL;
            renderHandler.sendMessage(message);
        }

        public interface OnOpenGlListener{
            BaseOpengl onOpenglCreate();
            Bitmap requestBitmap();
            void readPixelResult(byte[] bytes);
            void readYUVResult(byte[] bytes);
        }

    }
```

BaseOpengl的java代码：

```
    public class BaseOpengl {

        public static final int YUV_DATA_TYPE_NV12 = 0;
        public static final int YUV_DATA_TYPE_NV21 = 1;

        // 三角形
        public static final int DRAW_TYPE_TRIANGLE = 0;
        // 四边形
```

```
    public static final int DRAW_TYPE_RECT = 1;
    // 纹理贴图
    public static final int DRAW_TYPE_TEXTURE_MAP = 2;
    // 矩阵变换
    public static final int DRAW_TYPE_MATRIX_TRANSFORM = 3;
    // VBO/VAO
    public static final int DRAW_TYPE_VBO_VAO = 4;
    // EBO
    public static final int DRAW_TYPE_EBO_IBO = 5;
    // FBO
    public static final int DRAW_TYPE_FBO = 6;
    // PBO
    public static final int DRAW_TYPE_PBO = 7;
    // YUV  nv12与nv21渲染
    public static final int DRAW_YUV_RENDER = 8;
    // 将rgb图像转换城nv21
    public static final int DRAW_RGB_TO_YUV = 9;

    public long glNativePtr;
    protected EGLHelper eglHelper;
    protected int drawType;

    public BaseOpengl(int drawType) {
        this.drawType = drawType;
        this.eglHelper = new EGLHelper();
    }

    public void surfaceCreated(Surface surface) {
        Log.v("fly_learn_opengl","------------surfaceCreated:" +
surface);
        eglHelper.surfaceCreated(surface);
    }

    public void surfaceChanged(int width, int height) {
        Log.v("fly_learn_opengl","------------surfaceChanged:" +
Thread.currentThread());
```

```
        eglHelper.surfaceChanged(width,height);
    }

    public void surfaceDestroyed() {
        Log.v("fly_learn_opengl","------------surfaceDestroyed:" +
Thread.currentThread());
        eglHelper.surfaceDestroyed();
    }

    public void release(){
        if(glNativePtr != 0){
            n_free(glNativePtr,drawType);
            glNativePtr = 0;
        }
    }

    public void onGlDraw(){
        Log.v("fly_learn_opengl","------------onDraw:" +
Thread.currentThread());
        if(glNativePtr == 0){
            glNativePtr =
n_gl_nativeInit(eglHelper.nativePtr,drawType);
        }
        if(glNativePtr != 0){
            n_onGlDraw(glNativePtr,drawType);
        }
    }

    public void setBitmap(Bitmap bitmap){
        if(glNativePtr == 0){
            glNativePtr =
n_gl_nativeInit(eglHelper.nativePtr,drawType);
        }
        if(glNativePtr != 0){
            n_setBitmap(glNativePtr,bitmap);
        }
```

```
    }

    public void setYuvData(byte[] yData,byte[] uvData,int width,int
height){
        if(glNativePtr != 0){

 n_setYuvData(glNativePtr,yData,uvData,width,height,drawType);
        }
    }

    public void setMvpMatrix(float[] mvp){
        if(glNativePtr == 0){
            glNativePtr =
n_gl_nativeInit(eglHelper.nativePtr,drawType);
        }
        if(glNativePtr != 0){
            n_setMvpMatrix(glNativePtr,mvp);
        }
    }

    public byte[] readPixel(){
        if(glNativePtr != 0){
            return n_readPixel(glNativePtr,drawType);
        }
        return null;
    }

    public byte[] readYUVResult(){
        if(glNativePtr != 0){
            return n_readYUV(glNativePtr,drawType);
        }
        return null;
    }

    // 绘制
    private native void n_onGlDraw(long ptr,int drawType);
```

```
    private native void n_setMvpMatrix(long ptr,float[] mvp);
    private native void n_setBitmap(long ptr,Bitmap bitmap);
    protected native long n_gl_nativeInit(long eglPtr,int drawType);
    private native void n_free(long ptr,int drawType);
    private native byte[] n_readPixel(long ptr,int drawType);
    private native byte[] n_readYUV(long ptr,int drawType);
    private native void n_setYuvData(long ptr,byte[] yData,byte[]
uvData,int width,int height,int drawType);

}
```

将转换后的YUV数据读取保存好后，可以将数据拉取到电脑上使用 `YUVViewer` 这个软件查看是否真正转换成功。

参考

https://juejin.cn/post/7025223104569802789

Opengl ES之踩坑记

前因

最近在尝试使用Opengl ES实现一些LUT滤镜效果，在实现这些滤镜效果的时候遇到一些兼容性的坑，踩过这些坑后我希望把这几个坑分享给读者朋友们，
希望同在学习Opengl ES的朋友们能少走弯路。

关于LUT滤镜相关的介绍，也是这个Opengl ES系列入门教程的一项内容，在后面的文章中会专门介绍，这里暂时不展开讲解，后续大家敬请期待。

踩坑详情

1、着色器编译报错

有以下这个片元着色器，在其他手机上能正常编译，但是在华为鸿蒙系统上却编译报错：

```
#version 300 es
precision mediump float;
in vec2 TexCoord;
uniform sampler2D ourTexture;
uniform sampler3D textureLUT;
out vec4 FragColor;
void main()
{
vec4 color = texture(ourTexture, TexCoord);
FragColor = texture(textureLUT,color.rgb);
}
```

报错内容：

```
no default precision defined for variable 'textureLUT'
```

很明显说是没有声明精度的问题，然而在第二行不是已经使用 `precision mediump float;` 声明了精度了吗？是的，这已经声明了精度，但是这个声明的精度是针对float类型的，针对sampler3D类型的精度并没有声明，因此我们只需要声明以下sampler3D的精度问题即可。下面是经过修改过的正确的着色器代码：

```
#version 300 es
precision mediump float;
in vec2 TexCoord;
uniform sampler2D ourTexture;
// 华为手机上必须加上highp，否则报错no default precision defined for
uniform mediump sampler3D textureLUT;
out vec4 FragColor;
void main()
{
vec4 color = texture(ourTexture, TexCoord);
FragColor = texture(textureLUT,color.rgb);
}
```

2、glTexImage3D报错 `GL_INVALID_OPERATION`

使用以下代码上传3DLut的cube纹理数据时，部分手机能正常工作，但是同样在华为鸿蒙的手机上却报错，使用函数 `glGetError()` 获取到的错误是 `GL_INVALID_OPERATION` 。

```
        GLES30.glTexImage3D(
                GL_TEXTURE_3D,
                0,
                GL_RGB,
                size, size, size,
                0,
                GL_RGB,
                GL_FLOAT,
                lutBuffer
        );
```

后来经过查阅资料发现是使用函数 `glTexImage3D` 时的第三个参数传递错误所致的，将第三个参数 `GL_RGB` 改为 `GLES30.GL_RGB16F` 即可，正确的代码如下：

```
       GLES30.glTexImage3D(
                GL_TEXTURE_3D,
                0,
                // 鸿蒙等要用GLES30.GL_RGB16F，小米等其他手机可用GL_RGB
                GLES30.GL_RGB16F,
                size, size, size,
                0,
                GL_RGB,
                GL_FLOAT,
                lutBuffer
        );
```

GLES30.GL_RGB16F格式是浮点帧缓冲的相关内容，当帧缓冲使用了一个标准化的定点格式(像GL_RGB)为其颜色缓冲的内部格式，OpenGL会在将这些值存入帧缓冲前自动将其约束到0.0到1.0之间。当一个帧缓冲的颜色缓冲的内部格式被设定成了GL_RGB16F,GL_RGBA16F,GL_RGB32F或者GL_RGBA32F时，这些帧缓冲被叫做浮点帧缓冲(Floating Point Framebuffer)，浮点帧缓冲可以存储超过0.0到1.0范围的浮点值。

Opengl ES之矩阵变换(上)

前言

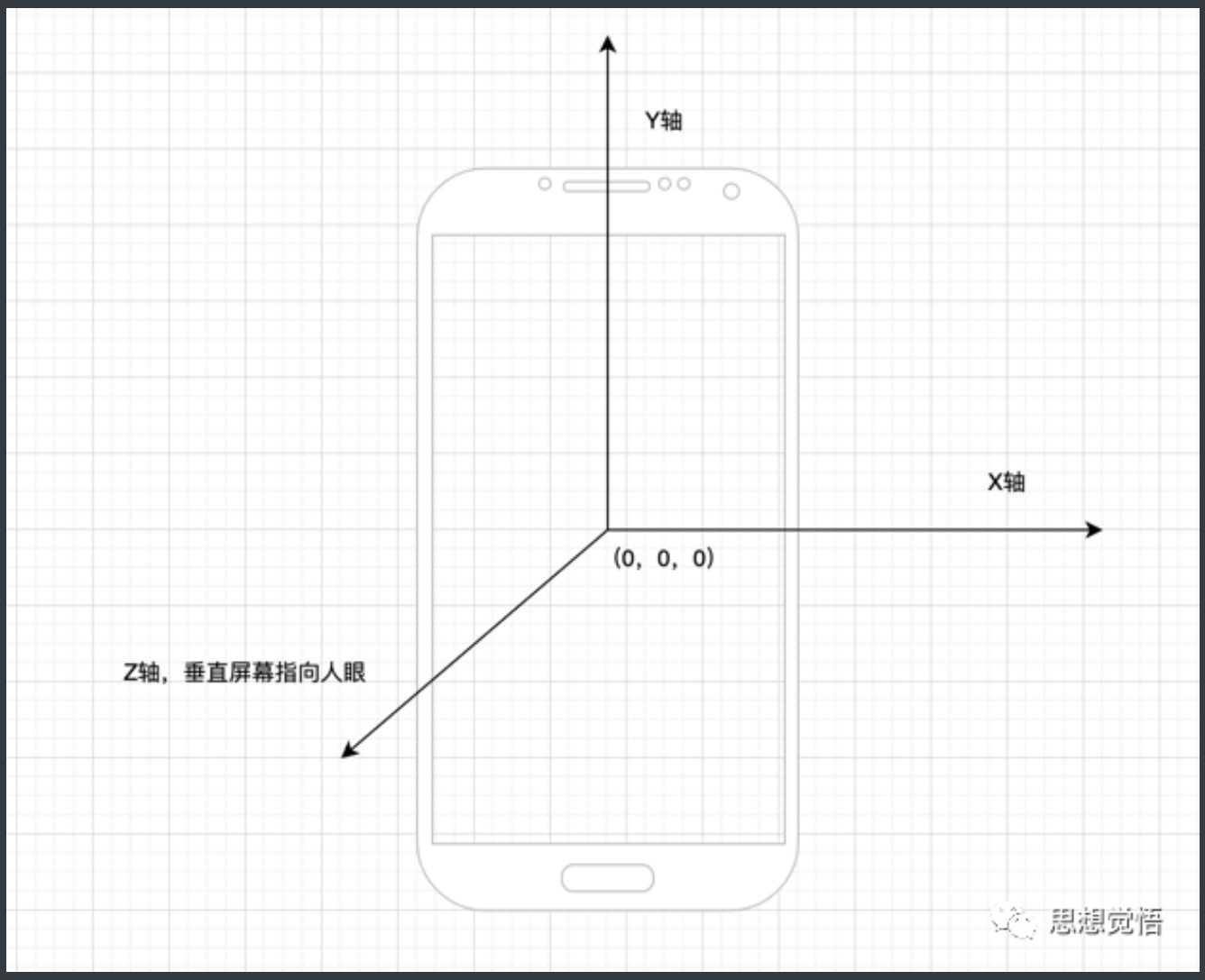
说到矩阵变换，我们第一时间想到的就是大学时代的线性代数这些复杂的东西，突然有了一种令人从入门到放弃的念头，不慌，作为了一个应用层的CV工程师，
在实际应用中线性代数哪些复杂的计算根本不用我们自己去算，绝大部分情境下直接使用 `Matrix` 这个类或者 `glm` 这个库即可。

关于矩阵与向量的相关知识，矩阵的加减乘除等规则，这里就不展开细说，感兴趣的同学自行查阅线性代数即可，不过这些规则忘记了也没关系，反正有API可用。

我们知道在Opengl中有很多中坐标系，在Opengl中矩阵的一大作用就是将坐标从一个坐标系转换到另一个坐标系下，同时还可以通过矩阵实现一些形变的效果，
今天我们就使用矩阵的方式搭配Opengl ES实现平移、缩放、旋转等一些形变变换的效果。

通常来说在Opengl ES中的矩阵都是一个4X4的矩阵，也就是一个包含16个元素的一维数组。

下面以 `Matrix` 这个类介绍一下矩阵变换的一些常用方法。下面介绍的矩阵变换所参考的坐标系统都是一样的，均是下图这个：



单位矩阵

所谓的单位矩阵就是左上角到右下角对角线值均为1的矩阵，又成为单元矩阵。使用 `Matrix.setIdentityM` 方法可以将一个矩阵变为单位矩阵。

矩阵平移

矩阵平移所使用的方法是 `Matrix.translateM` 。

需要注意的是在Opengl在顶点坐标的值是在-1到1之间，因此translateX的范围可以为-2到2。为什么呢？因为-1到1的距离是2，因此往最多可以往左移动2，同理，最多可以往右移动2。

矩阵旋转

矩阵旋转所使用的方法是 `Matrix.rotateM` ，其中第三个参数是表示选旋转的角度，后面的三个参数xyz代表的是绕那个轴旋转，绕那个轴旋转就把那个轴的参数设置成1，其他轴设置成0

即可。

矩阵缩放

矩阵缩放所使用的方法是`Matrix.scaleM`

组合矩阵的写法

假如有以下形变步骤，先绕Z轴旋转90度，再向X轴平移0.5，最后X轴缩放0.9倍，那么最终这个形变矩阵该如何计算呢？是以下这个写法吗？

```
Matrix.rotateM(mvpMatrix, 0, 90, 0, 0, 1);
Matrix.translateM(mvpMatrix, 0, 0.5, 0, 0);
Matrix.scaleM(mvpMatrix, 0, 0.9, 1f, 0f);
```

不是的，组合矩阵的写法有一个规则，这个规则大家一定要记住：

> 在组合矩阵时，先进行缩放操作，然后是旋转，最后才是位移，但是写法需要反正写，也就是先写translateM，然后rotateM，最后scaleM

如果不这样写会发生什么呢？例如顺着写，先写scaleM，然后是rotateM，最后写translateM，测试时就会出现问题，旋转超过180度之后再移动，就会出现移动方向相反的情况。

因此以上例子正确的写法应该是这样子的：

```
Matrix.translateM(mvpMatrix, 0, 0.5, 0, 0);
Matrix.rotateM(mvpMatrix, 0, 90, 0, 0, 1);
Matrix.scaleM(mvpMatrix, 0, 0.9, 1f, 0f);
```

show me code

在Opengl ES中可以使用 `mat4` 来表示一个4X4的矩阵，我们将总的变换矩阵在CPU中计算好之后以 `uniform` 的形式传递到着色器中去。
在顶点着色器中将矩阵与顶点坐标相乘的结果作为新的顶点输出坐标即可完成矩阵变换。

以下是MatrixTransformOpengl.cpp的详细代码：

```
    // 顶点着色器
    static const char *ver = "#version 300 es\n"
                             "in vec4 aPosition;\n"
                             "in vec2 aTexCoord;\n"
                             "out vec2 TexCoord;\n"
                             "uniform mat4 mvpMatrix;\n"
                             "void main() {\n"
                             "  TexCoord = aTexCoord;\n"
                             "  gl_Position = mvpMatrix * aPosition;\n"
                             "}";

    // 片元着色器
    static const char *fragment = "#version 300 es\n"
                                  "precision mediump float;\n"
                                  "out vec4 FragColor;\n"
                                  "in vec2 TexCoord;\n"
                                  "uniform sampler2D ourTexture;\n"
                                  "void main()\n"
                                  "{\n"
                                  "    FragColor = texture(ourTexture,
    TexCoord);\n"
                                  "}";


    // 使用绘制两个三角形组成一个矩形的形式(三角形带)
    // 第一第二第三个点组成一个三角形，第二第三第四个点组成一个三角形
    const static GLfloat VERTICES[] = {
            1.0f,-1.0f, // 右下
            1.0f,1.0f, // 右上
            -1.0f,-1.0f, // 左下
```

```
        -1.0f,1.0f // 左上
};

// 贴图纹理坐标（参考手机屏幕坐标系统，原点在左上角）
//由于对一个OpenGL纹理来说，它没有内在的方向性，因此我们可以使用不同的坐标把它定向到任何我们喜欢的方向上，然而大多数计算机图像都有一个默认的方向，它们通常被规定为y轴向下，X轴向右
const static GLfloat TEXTURE_COORD[] = {
        1.0f,1.0f, // 右下
        1.0f,0.0f, // 右上
        0.0f,1.0f, // 左下
        0.0f,0.0f // 左上
};

MatrixTransformOpengl::MatrixTransformOpengl():BaseOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
    textureHandle = glGetAttribLocation(program,"aTexCoord");
    textureSampler = glGetUniformLocation(program,"ourTexture");
    matrixHandle = glGetUniformLocation(program,"mvpMatrix");
}

MatrixTransformOpengl::~MatrixTransformOpengl() noexcept {
    LOGD("MatrixTransformOpengl析构函数");
}

void MatrixTransformOpengl::setMvpMatrix(float *mvp) {
    for (int i = 0; i < 16; ++i) {
        mvpMatrix[i] = mvp[i];
    }
}

void MatrixTransformOpengl::setPixel(void *data, int width, int height, int length) {
    LOGD("texture setPixel");
    imageWidth = width;
```

```
    imageHeight = height;
    glGenTextures(1, &textureId);

    // 激活纹理，注意以下这个两句是搭配的，glActiveTexture激活的是那个纹理，就设置的sampler2D是那个
    // 默认是0，如果不是0的话，需要在onDraw的时候重新激活一下?
//    glActiveTexture(GL_TEXTURE0);
//    glUniform1i(textureSampler, 0);

// 例如，一样的
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler, 2);
    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA, GL_UNSIGNED_BYTE, data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);

    glBindTexture(GL_TEXTURE_2D, textureId);

    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}

void MatrixTransformOpengl::onDraw() {

//    glViewport(0,0,imageWidth,imageHeight);

    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
```

```
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);

    // 激活纹理
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId);

    // 设置矩阵
    glUniformMatrix4fv(matrixHandle, 1, GL_FALSE,mvpMatrix);

    /**
     * size 几个数字表示一个点，显示是两个数字表示一个点
     * normalized 是否需要归一化，不用，这里已经归一化了
     * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
     */
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);

 glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);

    // 纹理坐标
    glEnableVertexAttribArray(textureHandle);

 glVertexAttribPointer(textureHandle,2,GL_FLOAT,GL_FALSE,0,TEXTURE_COO
RD);

    // 4个顶点绘制两个三角形组成矩形
    glDrawArrays(GL_TRIANGLE_STRIP,0,4);

    glUseProgram(0);

    // 禁用顶点
    glDisableVertexAttribArray(positionHandle);
```

```
    if(nullptr != eglHelper){
        eglHelper->swapBuffers();
    }

    glBindTexture(GL_TEXTURE_2D, 0);
}
```

java层的MatrixActivity.java实例代码如下：

```
public class MatrixActivity extends BaseGlActivity {

    private MatrixTransformOpengl matrixTransformOpengl;
    // 遵守先缩放再旋转最后平移的顺序
    // 首先执行缩放，接着旋转，最后才是平移。这就是矩阵乘法的工作方式。
    private final float[] mvpMatrix = new float[16];
    // 因为在Opengl在顶点坐标的值是在-1到1之间，因此translateX的范围可以为-2到
2。
    private float translateX = 0;
    private float scaleX = 1;
    private float rotationZ = 0;

    @Override
    public int getLayoutId() {
        return R.layout.activity_gl_matrix;
    }

    @Override
    public BaseOpengl createOpengl() {
        matrixTransformOpengl = new MatrixTransformOpengl();
        return matrixTransformOpengl;
    }

    @Override
    public Bitmap requestBitmap() {
        BitmapFactory.Options options = new BitmapFactory.Options();
        // 不缩放
```

```
        options.inScaled = false;
        Bitmap bitmap = BitmapFactory.decodeResource(getResources(),
R.mipmap.ic_boy, options);

        // 设置一下矩阵
        Matrix.setIdentityM(mvpMatrix, 0);
        matrixTransformOpengl.setMvpMatrix(mvpMatrix);

        return bitmap;
    }

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        findViewById(R.id.bt_translate).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if (null != matrixTransformOpengl) {
                    translateX += 0.1;
                    if(translateX >=2 ){
                        translateX = 0f;
                    }
                    updateMatrix();
                }
            }
        });

        findViewById(R.id.bt_scale).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if (null != matrixTransformOpengl) {
                    scaleX += 0.1;
                    updateMatrix();
                }
```

```
            }
        });

        findViewById(R.id.bt_rotate).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if (null != matrixTransformOpengl) {
                    rotationZ += 10;
                    updateMatrix();
                }
            }
        });

        findViewById(R.id.bt_reset).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if (null != matrixTransformOpengl) {
                    translateX = 0;
                    scaleX = 1;
                    rotationZ = 0;
                    updateMatrix();
                }
            }
        });

    }

    private void updateMatrix() {
        Matrix.setIdentityM(mvpMatrix, 0);
        // 重点注释
        // 在组合矩阵时，先进行缩放操作，然后是旋转，最后才是位移，但是写法需要反
正写，也就是先写translateM，然后rotateM，最后scaleM
```

```
        // 如果不这样写会发生什么呢？例如顺这写，先写scaleM，然后是rotateM，最后写translateM，测试时就会出现问题，旋转超过180度之后再移动，就会出现移动方向相反的情况
        Matrix.translateM(mvpMatrix, 0, translateX, 0, 0);
        Matrix.rotateM(mvpMatrix, 0, rotationZ, 0, 0, 1);
        Matrix.scaleM(mvpMatrix, 0, scaleX, 1f, 0f);
        matrixTransformOpengl.setMvpMatrix(mvpMatrix);
        myGLSurfaceView.requestRender();
    }
}
```

Opengl ES之矩阵变换(下)

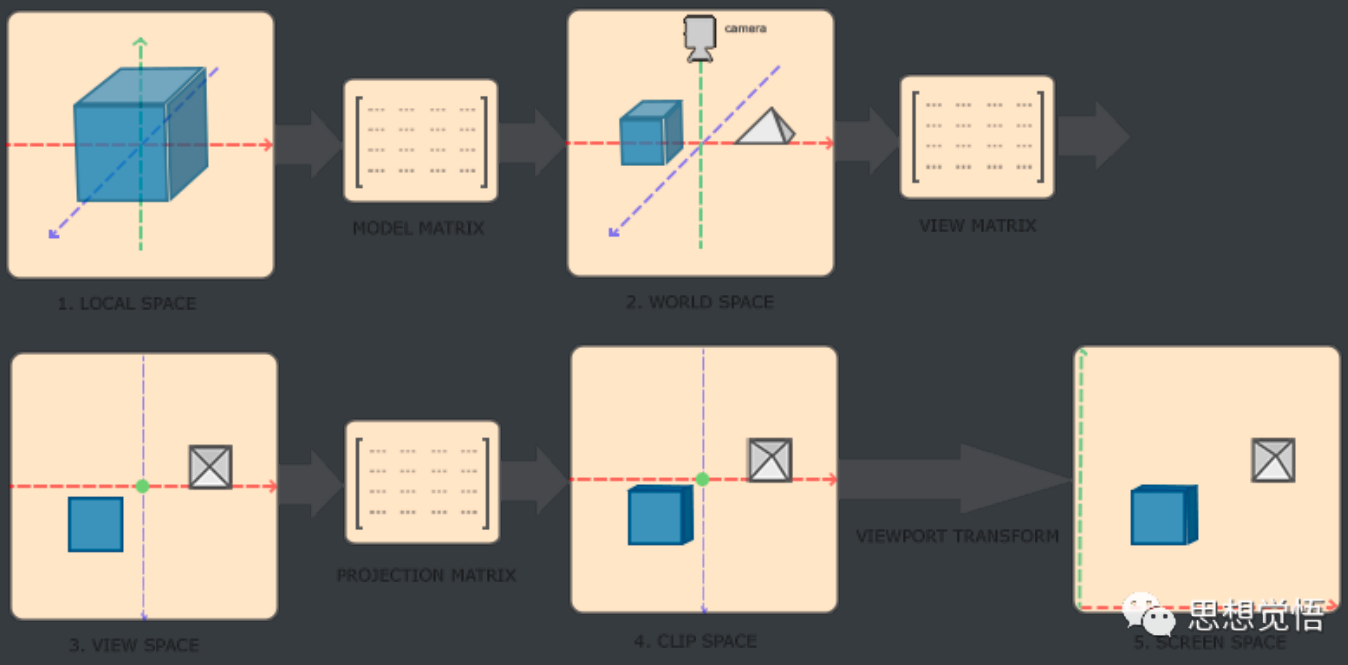
前言

在上一节 《Opengl ES之矩阵变换(上)》 中，我们通过矩阵变换实现一个一些形变的效果。

如果细心的童鞋们可能会发现，我们的运行结果渲染的图片宽高明显是有些变形了，特别是在手机屏幕旋转为横屏之后，变形更加的明显，
那么如果希望无论是横屏还是竖屏都希望渲染的画面可以参照宽高等比拉伸显示该如何处理呢？同时这个需求也是播放器渲染视频画面时的一个基本需求，
通过矩阵的正交投影就能够很好低解决这个问题。

OpenGL中的坐标系及矩阵变换过程

下面这张图展示了OpenGL ES 中的坐标系及矩阵变换过程：



> Opengl坐标系统变换过程图

通过上面这张图可以看到一个物体最终在屏幕上显示出来，需要经过5个坐标系统之间的变换，而这些变换都是依靠矩阵变换完成的。

下面简单介绍以下这五个坐标系统：

- 局部空间(Local Space，或者称为物体空间(Object Space))

局部坐标是对象相对于局部原点的坐标，也是物体起始的坐标。假如我们有一张图，设定这张图的中心为坐标原点(0,0)，左上角的坐标点为（-1，-1）那么这个物体的局部空间坐标系统。

- 世界空间(World Space)

如字面所示，在世界空间中肯定有很多物体模型，那么怎么描述这些物体模型所在的位置呢？这就需要一个统一的参找体系了，这个参照体系就是世界空间。

- 观察空间(View Space，或者称为视觉空间(Eye Space))

观察空间经常被人们称之OpenGL的摄像机，所以有时也称为摄像机空间或视觉空间。观察空间是将世界空间坐标转化为用户视野前方的坐标而产生的结果。因此观察空间就是从摄像机的视角所观察到的空间。

- 裁剪空间(Clip Space)

在一个顶点着色器运行的最后，OpenGL期望所有的坐标都能落在一个特定的范围内，而且任何在这个范围之外的点都应该被裁剪掉，被裁剪掉的坐标就会被忽略不可见，所以剩下的坐标就将变为屏幕上可见的片段，这也就是裁剪空间。

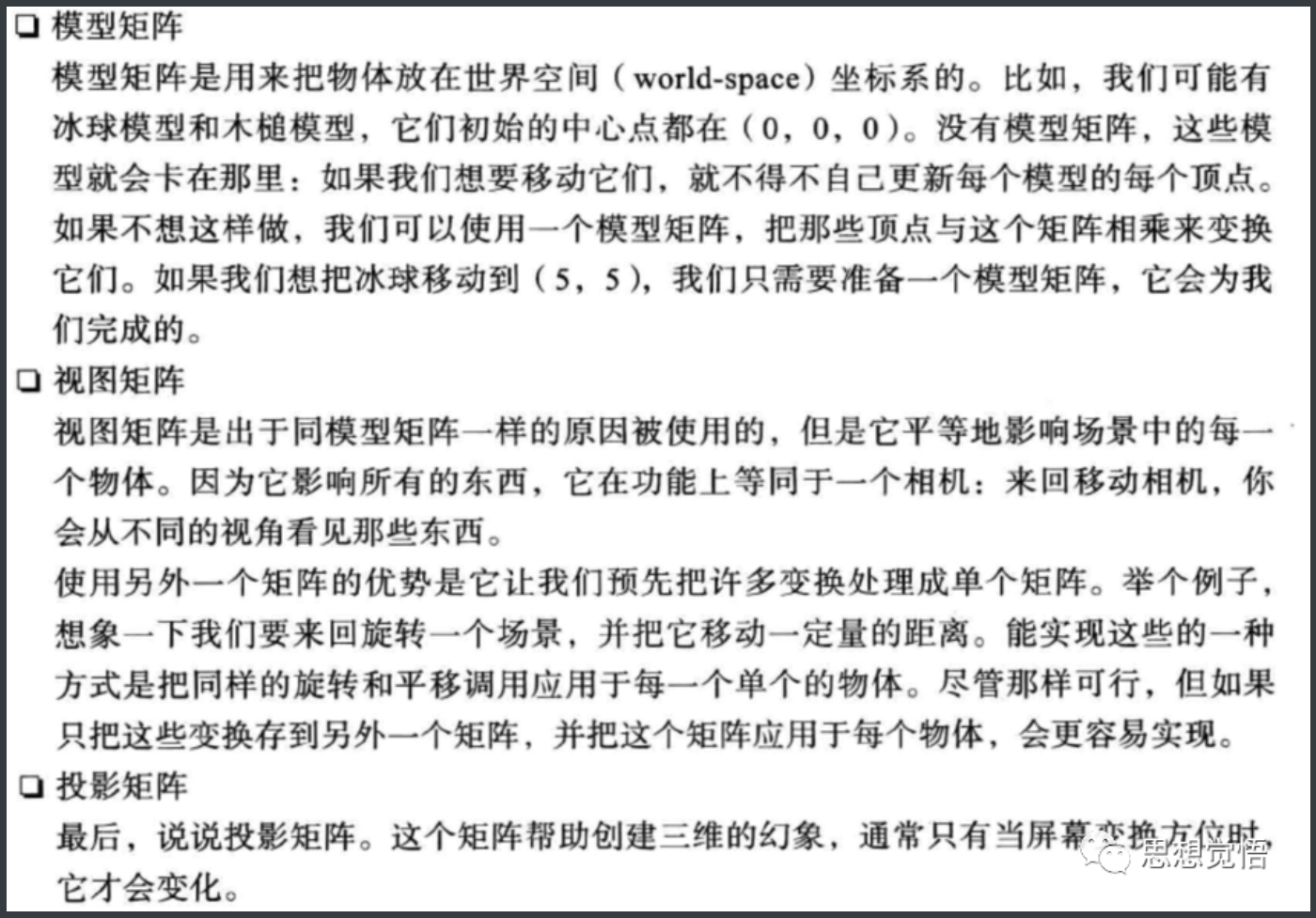
- 屏幕空间(Screen Space)

这个坐标系大家应该非常熟悉了，android中的各种view里用的坐标就是屏幕坐标。以手机的左上角为坐标原点，向右为X轴的正方向，向下为Y轴的正方向，这就是屏幕坐标。

MVP矩阵

为了将坐标从一个坐标系变换到另一个坐标系，我们需要用到几个变换矩阵，最重要的几个分别是模型(Model)、观察(View)、投影(Projection)三个矩阵，这三个矩阵就是我们常说的mvp矩阵。

关于这三个矩阵的定义可以看下面这个图片说明：



❑ 模型矩阵

模型矩阵是用来把物体放在世界空间（world-space）坐标系的。比如，我们可能有冰球模型和木槌模型，它们初始的中心点都在（0，0，0）。没有模型矩阵，这些模型就会卡在那里：如果我们想要移动它们，就不得不自己更新每个模型的每个顶点。如果不想这样做，我们可以使用一个模型矩阵，把那些顶点与这个矩阵相乘来变换它们。如果我们想把冰球移动到（5，5），我们只需要准备一个模型矩阵，它会为我们完成的。

❑ 视图矩阵

视图矩阵是出于同模型矩阵一样的原因被使用的，但是它平等地影响场景中的每一个物体。因为它影响所有的东西，它在功能上等同于一个相机：来回移动相机，你会从不同的视角看见那些东西。

使用另外一个矩阵的优势是它让我们预先把许多变换处理成单个矩阵。举个例子，想象一下我们要来回旋转一个场景，并把它移动一定量的距离。能实现这些的一种方式是把同样的旋转和平移调用应用于每一个单个的物体。尽管那样可行，但如果只把这些变换存到另外一个矩阵，并把这个矩阵应用于每个物体，会更容易实现。

❑ 投影矩阵

最后，说说投影矩阵。这个矩阵帮助创建三维的幻象，通常只有当屏幕变换方位时，它才会变化。

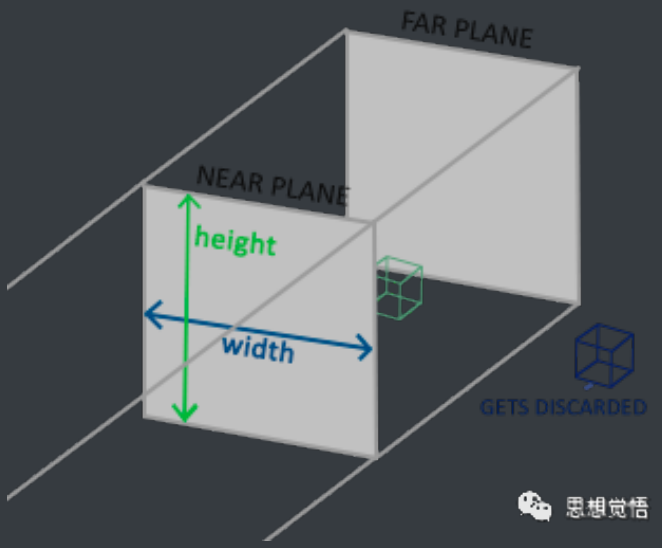
思想觉悟

正交投影

所谓正交投影就是上面三种矩阵当当中的投影矩阵。在Opengl有两种投影矩阵，分别是正交投影和透视投影，正交投影的效果就是无论在远处看还是在近处看，所看到的物体大小都是一样的，

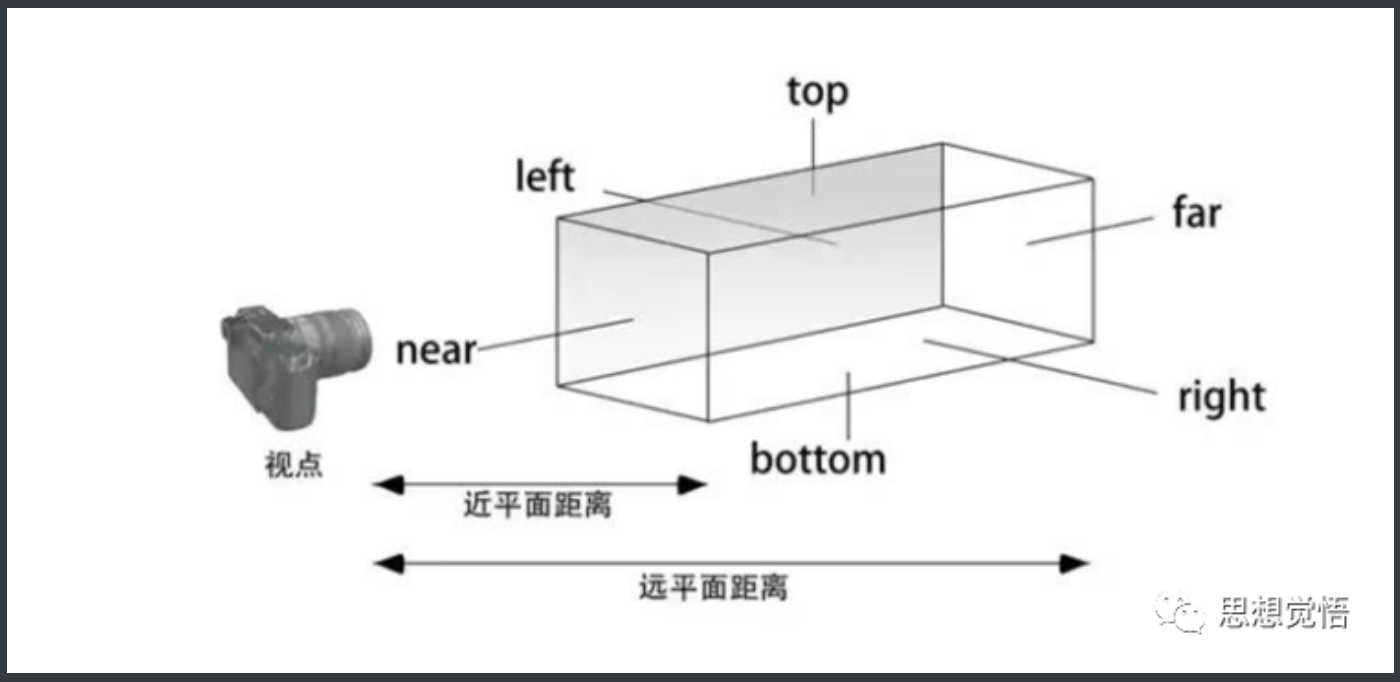
而透视投影则不同，它的投影效果是近大远小，类似我们的眼睛观察一个物体，眼睛离这个物体越近看到的范围越小，感觉这个物体越大，眼睛离这个物体越远则看到的范围越大，感觉这个物体越小。因此透视投影一般在3D场景中常用。

今天我们的场景是2D环境中图片的等比缩放，因此主要关注正交投影，正交投影的效果如下图：



> 正交投影

正交投影所用到的方法是`Matrix.orthoM`，它的具体参数如图：



> 正交投影参数

在Opengl总的变换矩阵公式为`mvpMatrix = projectionMatrix * viewMatrix * modelMatrix;`

这个公式的顺序不能乱，也就是不遵守惩罚交换律，从右往做读取首字母就是所谓的mvp矩阵。而在实际的2D效果中一般会忽略掉试图矩阵`viewMatrix`。

主要代码逻辑

以下是使用正交投影实现渲染图片等比缩放的实例代码，与上一节 《Opengl ES之矩阵变换(上)》 的区别主要是在java层，C++层的渲染逻辑还是MatrixTransformOpengl.cpp不变。

```
public class MvpMatrixActivity extends BaseGlActivity {

    private MatrixTransformOpengl matrixTransformOpengl;
    // 遵守先缩放再旋转最后平移的顺序
    private final float[] mvpMatrix = new float[16];
    private final float[] mProjectMatrix = new float[16];
    private final float[] modelMatrix = new float[16];
    // 因为在Opengl在顶点坐标的值是在-1到1之间，因此translateX的范围可以为-2到
2。
    private float translateX = 0;
    private float scaleX = 1;
    private float rotationZ = 0;

    private int imageWidth;
    private int imageHeight;
    // 是否使用正交投影
    private boolean isUseProjectMatrix = false;

    @Override
    public int getLayoutId() {
        return R.layout.activity_gl_mvp_matrix;
    }

    @Override
    public BaseOpengl createOpengl() {
        matrixTransformOpengl = new MatrixTransformOpengl();
        return matrixTransformOpengl;
    }

    @Override
    public Bitmap requestBitmap() {
```

```
        BitmapFactory.Options options = new BitmapFactory.Options();
        // 不缩放
        options.inScaled = false;
//      Bitmap bitmap = BitmapFactory.decodeResource(getResources(),
R.mipmap.ic_test, options);
//      Bitmap bitmap = BitmapFactory.decodeResource(getResources(),
R.mipmap.ic_big, options);
        Bitmap bitmap = BitmapFactory.decodeResource(getResources(),
R.mipmap.ic_boy, options);
        // 设置一下矩阵
        Matrix.setIdentityM(mvpMatrix, 0);
        matrixTransformOpengl.setMvpMatrix(mvpMatrix);
        imageWidth = bitmap.getWidth();
        imageHeight = bitmap.getHeight();
        return bitmap;
    }

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Matrix.setIdentityM(modelMatrix,0);
//        Matrix.setIdentityM(viewMatrix,0);
        Matrix.setIdentityM(mProjectMatrix,0);

//        Matrix.setLookAtM(viewMatrix, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0);

        findViewById(R.id.bt_translate).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if (null != matrixTransformOpengl) {
                    translateX += 0.1;
                    if (translateX >= 2) {
                        translateX = 0f;
                    }
                    updateMatrix();
```

```
                }
            }
        });

        findViewById(R.id.bt_scale).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if (null != matrixTransformOpengl) {
                    scaleX += 0.1;
                    updateMatrix();
                }
            }
        });

        findViewById(R.id.bt_rotate).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if (null != matrixTransformOpengl) {
                    rotationZ += 10;
                    updateMatrix();
                }
            }
        });

        findViewById(R.id.bt_reset).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if (null != matrixTransformOpengl) {
                    translateX = 0;
                    scaleX = 1;
                    rotationZ = 0;
                    isUseProjectMatrix = false;
                    updateMatrix();
```

```
                }
            }
        });

        findViewById(R.id.bt_mvp).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if (null != matrixTransformOpengl) {
                    isUseProjectMatrix = true;
                    updateMatrix();
                }
            }
        });

    }

    private void calculateProjectMatrix(){
        Matrix.setIdentityM(mProjectMatrix, 0);
        int width = myGLSurfaceView.surfaceWidth;
        int height = myGLSurfaceView.surfaceHeight;
        if(width == 0 || height == 0){
            return;
        }
        // 投影矩阵变换
        // 例如left为-3，right为1，表示整个屏幕是4等分，但是在opengl的顶点坐标
中是-1到1也就是占了屏幕的两分，因此如下这个效果是x轴半屏
//                Matrix.orthoM(mvpMatrix, 0, -3, 1, -1, 1, -1, 1);
        float widthScale = (float) width / imageWidth;
        float heightScale = (float) height / imageHeight;
        // 那个缩放小就以那个为基准
        if (heightScale > widthScale) {
            // 图片高度缩放，也就是以宽度为基准
            float newImageHeight = widthScale * imageHeight;
            float r = (float) height / newImageHeight;
            Matrix.orthoM(mProjectMatrix, 0, -1, 1, -r, r, -1, 1);
```

```
        } else {
            // 图片宽度缩放
            float newImageWidth = heightScale * imageWidth;
            float r = (float) width / newImageWidth;
            Matrix.orthoM(mProjectMatrix, 0, -r, r, -1, 1, -1, 1);
        }
    }

    private void updateMatrix() {

        if(isUseProjectMatrix){
            calculateProjectMatrix();
        }else {
            Matrix.setIdentityM(mProjectMatrix, 0);
        }

        Matrix.setIdentityM(modelMatrix, 0);
        Matrix.setIdentityM(mvpMatrix, 0);
        // 重点注释
        // 在组合矩阵时，先进行缩放操作，然后是旋转，最后才是位移，但是写法需要反正写，也就是先写translateM，然后rotateM，最后scaleM
        // 如果不这样写会发生什么呢？例如顺这写，先写scaleM，然后是rotateM，最后写translateM，测试时就会出现问题，旋转超过180度之后再移动，就会出现移动方向相反的情况
        Matrix.translateM(modelMatrix, 0, translateX, 0, 0);
        Matrix.rotateM(modelMatrix, 0, rotationZ, 0, 0, 1);
        Matrix.scaleM(modelMatrix, 0, scaleX, 1f, 0f);
        // a 表示旋转角度
        // 后面三个参数，那个参数为1则表示绕那个轴旋转
        Matrix.multiplyMM(mvpMatrix,0,mProjectMatrix,0,modelMatrix,0);
        matrixTransformOpengl.setMvpMatrix(mvpMatrix);
        myGLSurfaceView.requestRender();
    }

}
```


18:34
NDK_OpenglES_Learn



> 运行结果

点击 使用正交投影 按钮可以实现渲染纹理图的等比缩放显示。

Opengl ES之水印贴图

前言

水印贴图又称画中画，这种功能在Opengl中是如何实现的呢？我们可以简单地理解成两张纹理的叠加，一个纹理作为背景，另外一个纹理通过调整顶点坐标作为一个小的前景。

说到水印贴图的实现，很多朋友可能会想到通过 `mix` 混合函数实现，但是并不推荐这样做，抛开性能先不说，这种实现方式就不太方便后期拓展。今天我们就通过 `mix` 内建函数和多纹理绘制两种方式实现一个水印贴图的功能。

mix混合贴图

先来看看我们用到的贴图资源，首先我们需要渲染的背景纹理是：



> 背景纹理图

需要贴图的水印是：



片元着色器:

```
#version 300 es
precision mediump float;
out vec4 FragColor;
in vec2 TexCoord;
uniform sampler2D ourTexture;
uniform sampler2D waterTexture;
void main()
{
   vec4 textColor = texture(ourTexture, TexCoord);
   vec4 waterColor = texture(waterTexture, TexCoord);
   FragColor = mix(textColor, waterColor, 0.7);
}
```

这个片元着色器很简单，就是传递两个纹理，一个是背景纹理，一个是水印纹理，然后通过 `mix` 内建混合函数将颜色混合渲染显示。

主要的渲染代码:

```
//    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
//    glClear(GL_COLOR_BUFFER_BIT);
//    glUseProgram(program);
//
//    // 激活纹理
```

```
//    glActiveTexture(GL_TEXTURE2);
//    // 绑定纹理
//    glBindTexture(GL_TEXTURE_2D, textureId);
//    glUniform1i(textureSampler, 2);
//
//
//    // 激活纹理
//    glActiveTexture(GL_TEXTURE3);
//    // 绑定纹理
//    glBindTexture(GL_TEXTURE_2D, waterTextureId);
//    glUniform1i(waterTextureSampler, 3);
//
//    /**
//     * size 几个数字表示一个点，显示是两个数字表示一个点
//     * normalized 是否需要归一化，不用，这里已经归一化了
//     * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
//     */
//    // 启用顶点数据
//    glEnableVertexAttribArray(positionHandle);
//
glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);
//
//    // 纹理坐标
//    glEnableVertexAttribArray(textureHandle);
//
glVertexAttribPointer(textureHandle,2,GL_FLOAT,GL_FALSE,0,TEXTURE_COOR
D);
//
//    // 4个顶点绘制两个三角形组成矩形
//    glDrawArrays(GL_TRIANGLE_STRIP,0,4);
//
//    glUseProgram(0);
//
//    // 禁用顶点
//    glDisableVertexAttribArray(positionHandle);
//    if(nullptr != eglHelper){
```

```
//        eglHelper->swapBuffers();
//    }
//
//    glBindTexture(GL_TEXTURE_2D, 0);
```

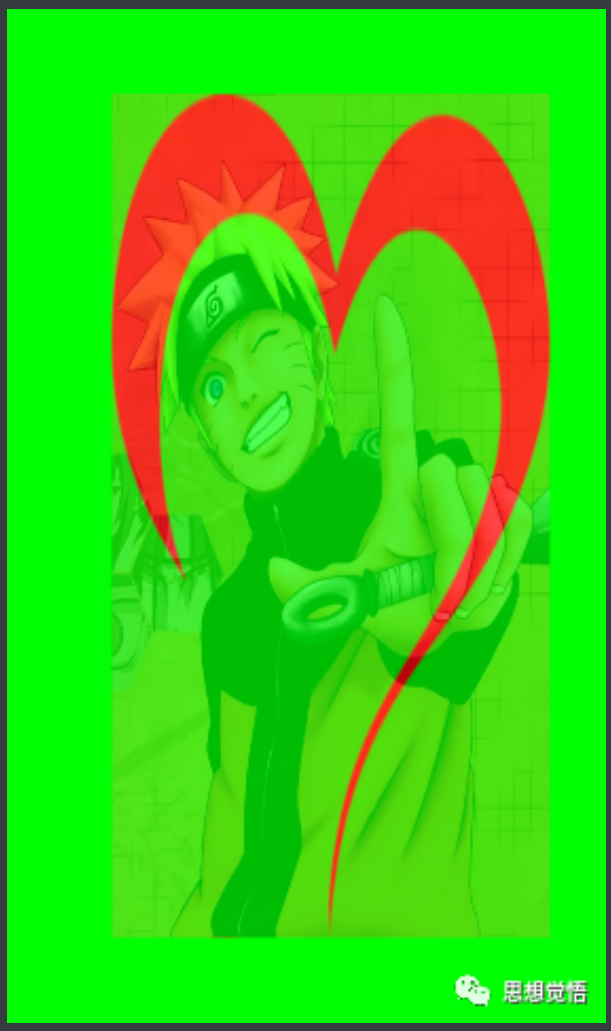
运行起来看到效果是这样的：



貌似不太对？好像被蒙上一层黑色阴影的感觉，这是因为我们没有打开Opengl的混合功能，没有设置混合模式导致的，我们在绘制时打开对应的设置即可：

```
//    // 可以注释掉这两句看看效果是不一样的
////    glEnable(GL_BLEND); //打开混合功能
////    glBlendFunc(GL_ONE, GL_ONE_MINUS_SRC_ALPHA); //指定混合模式
```

再次运行看看效果：



这次对了，看起来不好看是因为水印贴图的大小和背景纹理大小一致了，希望逼真一点的话可以适当修改着色器语言，在特定的顶点区域才采样使用 `mix` 函数混合，其他区域就显示背景纹理的采样即可，同时如果需要调整水印贴图的权重，直接调整着色器中 `mix` 函数的第三个参数即可。

背景纹理和水印贴图单独绘制

上面说到不推荐使用 `mix` 混合贴图的实现方式，那么更加合理的方式是什么呢?

我们编程讲究的是功能粒度拆分要合理，如果能坐到本经纹理和水印纹理单独绘制分离就能更加地方便后续功能的拓展，将绘制分离开来也能更好地搭配FBO离屏渲染。

这里为了贪方便实现单个功能而已，就不采用FBO的方式了，当然如果是实际应用中还是建议使用FBO的方式更加地合理。

废话少扯，直接上代码吧，注意看关键注释：

WaterMarkOpengl.cpp

```
    // 顶点着色器
    static const char *ver = "#version 300 es\n"
                             "in vec4 aPosition;\n"
                             "in vec2 aTexCoord;\n"
                             "out vec2 TexCoord;\n"
                             "void main() {\n"
                             "  TexCoord = aTexCoord;\n"
                             "  gl_Position = aPosition;\n"
                             "}";

    // 片元着色器
    // 多纹理重叠，通过mix混合实现水印
    //static const char *fragment = "#version 300 es\n"
    //                              "precision mediump float;\n"
    //                              "out vec4 FragColor;\n"
    //                              "in vec2 TexCoord;\n"
    //                              "uniform sampler2D ourTexture;\n"
    //                              "uniform sampler2D waterTexture;\n"
    //                              "void main()\n"
    //                              "{\n"
    //                              "    vec4 textColor =
    texture(ourTexture, TexCoord);\n"
    //                              "    vec4 waterColor =
    texture(waterTexture, TexCoord);\n"
    //                              "    FragColor = mix(textColor,
    waterColor, 0.7);\n"
    //                              "}";

    static const char *fragment = "#version 300 es\n"
                                  "precision mediump float;\n"
                                  "out vec4 FragColor;\n"
                                  "in vec2 TexCoord;\n"
                                  "uniform sampler2D ourTexture;\n"
                                  "void main()\n"
```

```
                                "{\n"
                                "    vec4 textColor =
texture(ourTexture, TexCoord);\n"
                                "    FragColor = textColor;\n"
                                "}";


// 使用绘制两个三角形组成一个矩形的形式(三角形带)
// 第一第二第三个点组成一个三角形，第二第三第四个点组成一个三角形
const static GLfloat VERTICES[] = {
        1.0f,-1.0f, // 右下
        1.0f,1.0f, // 右上
        -1.0f,-1.0f, // 左下
        -1.0f,1.0f // 左上
};

// 贴图纹理坐标（参考手机屏幕坐标系统，原点在左上角）
const static GLfloat TEXTURE_COORD[] = {
        1.0f,1.0f, // 右下
        1.0f,0.0f, // 右上
        0.0f,1.0f, // 左下
        0.0f,0.0f // 左上
};

WaterMarkOpengl::WaterMarkOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
    textureHandle = glGetAttribLocation(program,"aTexCoord");
    textureSampler = glGetUniformLocation(program,"ourTexture");
    waterTextureSampler =
glGetUniformLocation(program,"waterTexture");
}

WaterMarkOpengl::~WaterMarkOpengl() noexcept {

}
```

```
void WaterMarkOpengl::onDraw() {

//    // 可以注释掉这两句看看效果是不一样的
////    glEnable(GL_BLEND); //打开混合功能
////    glBlendFunc(GL_ONE, GL_ONE_MINUS_SRC_ALPHA); //指定混合模式
//
//    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
//    glClear(GL_COLOR_BUFFER_BIT);
//    glUseProgram(program);
//
//    // 激活纹理
//    glActiveTexture(GL_TEXTURE2);
//    // 绑定纹理
//    glBindTexture(GL_TEXTURE_2D, textureId);
//    glUniform1i(textureSampler, 2);
//
//
//    // 激活纹理
//    glActiveTexture(GL_TEXTURE3);
//    // 绑定纹理
//    glBindTexture(GL_TEXTURE_2D, waterTextureId);
//    glUniform1i(waterTextureSampler, 3);
//
//    /**
//     * size 几个数字表示一个点，显示是两个数字表示一个点
//     * normalized 是否需要归一化，不用，这里已经归一化了
//     * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
//     */
//    // 启用顶点数据
//    glEnableVertexAttribArray(positionHandle);
//
glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);
//
//    // 纹理坐标
//    glEnableVertexAttribArray(textureHandle);
```

```
//
glVertexAttribPointer(textureHandle,2,GL_FLOAT,GL_FALSE,0,TEXTURE_COOR
D);
//
//    // 4个顶点绘制两个三角形组成矩形
//    glDrawArrays(GL_TRIANGLE_STRIP,0,4);
//
//    glUseProgram(0);
//
//    // 禁用顶点
//    glDisableVertexAttribArray(positionHandle);
//    if(nullptr != eglHelper){
//        eglHelper->swapBuffers();
//    }
//
//    glBindTexture(GL_TEXTURE_2D, 0);

    // 可以注释掉这两句看看效果是不一样的
    glEnable(GL_BLEND); //打开混合功能
    glBlendFunc(GL_ONE, GL_ONE_MINUS_SRC_ALPHA); //指定混合模式

    // 先绘制背景，正常纹理
   // 让纹理图片居中
    glViewport((eglHelper->viewWidth - imageWidth)/2,(eglHelper-
>viewHeight - imageHeight)/2,imageWidth,imageHeight);
    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);

    // 激活纹理
    glActiveTexture(GL_TEXTURE2);
    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId);
    glUniform1i(textureSampler, 2);

    /**
```

```
     * size 几个数字表示一个点，显示是两个数字表示一个点
     * normalized 是否需要归一化，不用，这里已经归一化了
     * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
     */
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);

 glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);

    // 纹理坐标
    glEnableVertexAttribArray(textureHandle);

 glVertexAttribPointer(textureHandle,2,GL_FLOAT,GL_FALSE,0,TEXTURE_COO
RD);

    // 4个顶点绘制两个三角形组成矩形
    glDrawArrays(GL_TRIANGLE_STRIP,0,4);

    // 然后使用统一的着色器，改变纹理数据再绘制一次水印 顶点坐标和纹理坐标和上面使
用的一样，因此不用重新赋值了
    // 调整窗口位置，调整水印的绘制位置，让水印绘制的背景的右下角
    int waterWidth = 200;
    int waterHeight = 300;
    // 以手机屏幕左下角为坐标原点
    glViewport((eglHelper->viewWidth - imageWidth)/2 + imageWidth -
waterWidth,(eglHelper->viewHeight -
imageHeight)/2,waterWidth,waterHeight);
    // 激活纹理
    glActiveTexture(GL_TEXTURE3);
    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, waterTextureId);
    glUniform1i(textureSampler, 3);
    // 4个顶点绘制两个三角形组成矩形
    glDrawArrays(GL_TRIANGLE_STRIP,0,4);

    glUseProgram(0);
```

```
    // 禁用顶点
    glDisableVertexAttribArray(positionHandle);
    if(nullptr != eglHelper){
        eglHelper->swapBuffers();
    }

    glBindTexture(GL_TEXTURE_2D, 0);
}

void WaterMarkOpengl::setPixel(void *data, int width, int height, int
length) {
    LOGD("texture setPixel");
    imageWidth = width;
    imageHeight = height;
    glGenTextures(1, &textureId);
    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA,
GL_UNSIGNED_BYTE, data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);
    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}

void WaterMarkOpengl::setWaterPixel(void *data, int width, int height,
int length) {
    LOGD("texture setWaterPixel");
    glGenTextures(1, &waterTextureId);
```

```
    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, waterTextureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA,
GL_UNSIGNED_BYTE, data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);

    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}
```

主要思想就是先绘制背景纹理，然后通过 `glViewport` 改变视窗的大小再绘制水印纹理，实例中我们将心形水印绘制在背景纹理的右下角。

运行效果：



S

Opengl ES之文字渲染

前言

自此已是我关于Opengl ES系列入门教程的第16篇文章了，虽然写的不咋的，文章产出量也不高，但是这个系列从2022年8月底开始持续到现在也坚持了比较久，
每一篇文章都是经过自己的精心调试通过后再写文总结发布，这个过程耗费了我很多的私人时间，其实一开始想写这个系列就是为了想看看自己不能系统性地把一件事情做好，
现在看来依然比较难，因为自己平常除了工作以外，偶尔也会去峡谷浪一浪，本身自己就是个不靠谱的宝爸，所以经常周末还得往老家跑。

目前这个系列还有滤镜、转场动画等一些内容没有写完，大部分写博文的童鞋都知道，这大概就是一个用爱发电的过程，没有利益，也没有营收，离谱一点写的不好，可能还会被喷...
本来打算是这个系列完善后再放出源码的，但是有不少好学的童鞋加了V信索要，同时也不知道后续这个系列自己还会不会继续完善下去，因此这次就把这个系列的源码放出来啦，仅供参考。

所谓 纸上得来总觉浅，绝知此事要躬行 ，其实在之前的博文中，我都贴上了关键的代码，如果真有时间又想学习的话还是建议自己亲手敲一遍学得牢，理解得透切。

今天的主要内容是使用Opengl ES搭配FreeType进行文字渲染。

FreeType

文字渲染是做音视频开发逃不掉的一个需求，实际上 OpenGL 并没有定义渲染文字的方式，但是我们可以通过纹理贴图的方式实现文字渲染，就是将带有文字的图像上传到纹理，然后进行纹理贴图。
因此在Android平台是使用Opengl ES进行文字渲染就有两个不同的方案，一个是使用Canvas绘制文字生产Bitmap，然后将带文字的Bitmap进行渲染；另外一个是使用跨平台的字体解析库FreeType实现。

FreeType是一个基于 C 语言实现的用于文字渲染的跨平台开源库，它小巧、高效、高度可定制，主要用于加载字体并将其渲染到位图，支持多种字体的相关操作。

FreeType 官网地址：

```
https://www.freetype.org/
```

更多关于FreeType的介绍以及使用文档请自行参考官网教程。

FreeType编译

一贯原则，要将一个跨平台C/C++库移植到Android，首先第一步就是先进行交叉编译，对交叉编译不了解的童鞋们请参考之前的文章：

音视频学习之NDK交叉编译基础

FreeType很贴心地为我们提供了 `configure` 文件，因此我们只需配置一下NDK的相关环境即可编译成功，更多关于 `configure` 可配置参数，
可通过命令行 `configure --help` 查看。

下面是笔者在mac上编译FreeType的一个脚本：

```
#!/bin/bash

# 这里需要配置你的NDK路径
NDK_PATH="/Users/fly/Documents/Android/SDK/ndk/21.4.7075529/"
# 注意我这里是苹果系统，如果是Linux系统是不一样的
HOST_PLATFORM="darwin-x86_64"

COMMON_OPTIONS="
    --with-zlib=no
    --with-bzip2=no
    --with-png=no
    --with-harfbuzz=no
    --with-brotli=no
```

```
    --with-sysroot=${SYSROOT}
    "
TOOLCHAIN_PREFIX="${NDK_PATH}/toolchains/llvm/prebuilt/${HOST_PLATFORM
}/bin"
SYSROOT="${NDK_PATH}/toolchains/llvm/prebuilt/${HOST_PLATFORM}/sysroot
"

make clean

FOLDER=`pwd`/android-libs/arm64-v8a
mkdir -p ${FOLDER}
CC=${TOOLCHAIN_PREFIX}/aarch64-linux-android21-clang \
./configure \
    --prefix=${FOLDER} \
    --libdir=${FOLDER} \
    --host=aarch64-linux-android \
    --enable-debug \
    ${COMMON_OPTIONS}

make -j4 && make install
```

编译成功后即可将静态库引入到Android Studio中使用了，引入的步骤也很简单，主要是配置一下头文件的搜索路径和动态库的查找路径即可，详情可参考之前的博文：

将ffmpeg引入到Android Studio工程中

FreeType的使用

有关FreeType的使用，官网有比较详细的介绍：https://freetype.org/freetype2/docs/tutorial/step1.html

首先引入的头文件是：

```
#include "ft2build.h"
#include <freetype/ftglyph.h>
```

在FreeType中每个字体库加载后就是一个叫做FT_Face的东西，后续通过FT_Face这个结构图可以获取到文字的一系列信息，比如图像宽高等。

首先使用 `FT_Init_FreeType` 进行初始化，然后通过函数 `FT_New_Face` 得到一个FT_Face。

下面这段代码的意思就是将128个ASCII，并将相关字体图像信息缓存起来：

```
void TextRenderOpengl::LoadFacesByASCII() {
    // FreeType
    FT_Library ft;
    FT_Face face;
    do {
        if (FT_Init_FreeType(&ft)) {
            LOGD("FT_Init_FreeType fail");
            break;
        }
        // FONT_FILE_PATH替换成自己的字体文件路径
        if (FT_New_Face(ft, FONT_FILE_PATH, 0, &face)) {
            LOGD("FT_New_Face fail");
            break;
        }

        // 设置文字的大小，此函数设置了字体面的宽度和高度
        // 将宽度值设为0表示我们要从字体面通过给出的高度中动态计算出字形的宽度。
        FT_Set_Pixel_Sizes(face, 0, 90);
        // 字节对齐
        glPixelStorei(GL_UNPACK_ALIGNMENT, 1);

        // Load first 128 characters of ASCII set
        for (unsigned char c = 0; c < 128; c++) {
            // Load character glyph
            if (FT_Load_Char(face, c, FT_LOAD_RENDER)) {
                LOGD("FT_Load_Char: Failed to load Glyph");
                continue;
            }
            // 纹理
```

```
            GLuint texture;
            glGenTextures(1, &texture);
            glBindTexture(GL_TEXTURE_2D, texture);
            glTexImage2D(
                    GL_TEXTURE_2D,
                    0,
                    GL_LUMINANCE,
                    face->glyph->bitmap.width,
                    face->glyph->bitmap.rows,
                    0,
                    GL_LUMINANCE,
                    GL_UNSIGNED_BYTE,
                    face->glyph->bitmap.buffer
            );
            // Set texture options
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);
            // Now store character for later use
            Character character = {
                    texture,
                    glm::ivec2(face->glyph->bitmap.width, face->glyph-
>bitmap.rows),
                    glm::ivec2(face->glyph->bitmap_left, face->glyph-
>bitmap_top),
                    static_cast<GLuint>(face->glyph->advance.x)
            };
            m_Characters.insert(std::pair<GLint, Character>(c,
character));
        }
        break;
```

```
    } while (0);
    glBindTexture(GL_TEXTURE_2D, 0);
    // 字体库释放
    FT_Done_Face(face);
    FT_Done_FreeType(ft);
}
```

使用Opengl ES进行渲染

首先我们的偏远着色器是:

```
#version 300 es
precision mediump float;
out vec4 FragColor;
in vec2 TexCoord;
uniform sampler2D ourTexture;
uniform vec3 u_textColor;
void main()
{
    vec4 color = vec4(1.0, 1.0, 1.0, texture(ourTexture, TexCoord).r);
    FragColor = vec4(u_textColor, 1.0) * color;
}
```

核心逻辑就是获取本身字体纹理的一个单颜色通道，然后通过变量 `u_textColor` 这个变量重新组合改变字体的渲染颜色。

既然前面我们已经获取了每个文字的图像数据，那么将这些文字的图像进行纹理贴图即可渲染出来，因为我们不是渲染一个文字，可能是渲染一行或者多个文字，
因此我们封装一个渲染ASCII的方法:

```
/**
 *
 * @param text 需要渲染的文字
 * @param x 归一化坐标中文字渲染x的起点  [-1,1]
 * @param y 归一化坐标中文字渲染y的起点  [-1,1]
```

```
 * @param scale 字体缩放  这个也可以在顶点坐标中做统一的变换使用
 * @param color 字体颜色
 * @param viewport 视窗大小
 */
void TextRenderOpengl::RenderASCIIText(std::string text, GLfloat x,
GLfloat y, GLfloat scale,
                                       glm::vec3 color, glm::vec2
viewport) {

    // 激活合适的渲染状态
    glUseProgram(program);

    // 纹理坐标都是一样的，先赋值纹理坐标
    // 纹理坐标
    glEnableVertexAttribArray(textureHandle);
    glVertexAttribPointer(textureHandle, 2, GL_FLOAT, GL_FALSE, 0,
TEXTURE_COORD);

    glUniform3f(u_textColorHandle, color.x, color.y, color.z);
    // 对文本中的所有字符迭代
    std::string::const_iterator c;
    x *= viewport.x;
    y *= viewport.y;
    for (c = text.begin(); c != text.end(); c++) {
        Character ch = m_Characters[*c];

        GLfloat xpos = x + ch.bearing.x * scale;
        GLfloat ypos = y - (ch.size.y - ch.bearing.y) * scale;

        // 归一化
        xpos /= viewport.x;
        ypos /= viewport.y;

        // 字体贴图缩放后的宽高
        GLfloat w = ch.size.x * scale;
        GLfloat h = ch.size.y * scale;
```

```
        // 宽高也归一化
        w /= viewport.x;
        h /= viewport.y;

        LOGD("TextRenderSample::RenderText [xpos,ypos,w,h]=[%f, %f,
%f, %f], ch.advance >> 6 = %d",
             xpos, ypos, w, h, ch.advance >> 6);

        // 当前字符的顶点坐标
        GLfloat VERTICES[] = {
                xpos + w, ypos,
                xpos + w, ypos + h,
                xpos, ypos,
                xpos, ypos + h
        };

        /**
         * size 几个数字表示一个点，显示是两个数字表示一个点
         * normalized 是否需要归一化，不用，这里已经归一化了
         * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
         */
        // 启用顶点数据
        glEnableVertexAttribArray(positionHandle);
        glVertexAttribPointer(positionHandle, 2, GL_FLOAT, GL_FALSE,
0, VERTICES);

        // 在方块上绘制字形纹理
        glActiveTexture(GL_TEXTURE0);
        glBindTexture(GL_TEXTURE_2D, ch.textureID);
        glUniform1i(textureSampler, 0);
        // 绘制方块
        glDrawArrays(GL_TRIANGLE_STRIP, 0, 4);
        // 更新位置到下一个字形的原点，注意单位是1/64像素
        x += (ch.advance >> 6) * scale; //(2^6 = 64)
    }
```

```
        glBindVertexArray(0);
        glBindTexture(GL_TEXTURE_2D, 0);

        // 禁用顶点
        glDisableVertexAttribArray(positionHandle);
        glDisableVertexAttribArray(textureHandle);

    }
```

核心思想就是归一化以及动态变更每个文字的顶点坐标和纹理坐标，记住一点就是顶点坐标决定绘制的的位置，纹理坐标决定绘制纹理图的那一部分内容即可。

FreeType渲染中文

自此我们渲染英文以及一些常用的标点字符是没有问题，但是如果我们要渲染中文就不行了，我们尝试使用我们封装的函数 `RenderASCIIText` 渲染中文就会发现是没有效果的。

与 ASCII 码不同的是，中文字符采用 2 字节的 Unicode 编码，所以在加载字体之前，我们首先需要设置编码类型：

```
    FT_Select_Charmap(face, ft_encoding_unicode);
```

另外，还需要注意的一点是中文字符串需要采用宽字符 `wchar_t` 。

下面是加载Unicode编码字体的实例代码：

```
    // 注意你的字体库要支持中文，如果字体库不支持中文的话渲染是没有效果的，拿到的纹理图的宽高都是0
    void TextRenderOpengl::LoadFacesByUnicode(const wchar_t *text, int size) {
        // FreeType
        FT_Library ft;
        FT_Face face;
        glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
        // 因为FT_Library 要释放，所以使用do while
```

```
    do {
        // All functions return a value different than 0 whenever an
error occurred
        if (FT_Init_FreeType(&ft)) {
            // todo 提示
            LOGD("LoadFacesByUnicode -----FT_Init_FreeType fail");
            break;
        }

        if (FT_New_Face(ft, FONT_FILE_PATH, 0, &face)) {
            // todo
            LOGD("LoadFacesByUnicode -----FT_New_Face fail");
            break;
        }

        // Set size to load glyphs as
        FT_Set_Pixel_Sizes(face, 0, 96);
        // 指定使用unicode
        FT_Select_Charmap(face, ft_encoding_unicode);
        for (int i = 0; i < size; ++i) {
            //int index =  FT_Get_Char_Index(face,unicodeArr[i]);
            if (FT_Load_Glyph(face, FT_Get_Char_Index(face, text[i]),
FT_LOAD_DEFAULT)) {
                continue;
            }

            FT_Glyph glyph;
            FT_Get_Glyph(face->glyph, &glyph);

            //Convert the glyph to a bitmap.
            FT_Glyph_To_Bitmap(&glyph, ft_render_mode_normal, 0, 1);
            FT_BitmapGlyph bitmap_glyph = (FT_BitmapGlyph) glyph;

            //This reference will make accessing the bitmap easier
            FT_Bitmap &bitmap = bitmap_glyph->bitmap;
```

```
            LOGD("LoadFacesByUnicode -----text[i]:%d",text[i]);
            LOGD("LoadFacesByUnicode -----
w:%d,h:%d",bitmap.width,bitmap.rows);

            // Generate texture
            GLuint texture;
            glGenTextures(1, &texture);
            glBindTexture(GL_TEXTURE_2D, texture);
            glTexImage2D(
                    GL_TEXTURE_2D,
                    0,
                    GL_LUMINANCE,
                    bitmap.width,
                    bitmap.rows,
                    0,
                    GL_LUMINANCE,
                    GL_UNSIGNED_BYTE,
                    bitmap.buffer
            );
            // Set texture options
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);
            // Now store character for later use
            Character character = {
                    texture,
                    glm::ivec2(face->glyph->bitmap.width, face->glyph-
>bitmap.rows),
                    glm::ivec2(face->glyph->bitmap_left, face->glyph-
>bitmap_top),
```

```
                    static_cast<GLuint>((glyph->advance.x /
MAX_SHORT_VALUE) << 6)
            };
            m_Characters.insert(std::pair<GLint, Character>(text[i],
character));
            LOGD("LoadFacesByUnicode -----success");
        }
        break;
    } while (0);
    glBindTexture(GL_TEXTURE_2D, 0);
    // Destroy FreeType once we're finished
    FT_Done_Face(face);
    FT_Done_FreeType(ft);
}
```

完整的字体渲染代码：

```
/**
 * 本教程首发于公众号"思想觉悟"
 */
#include "EBOOpengl.h"


#include "TextRenderOpengl.h"
#include "ft2build.h"
#include <freetype/ftglyph.h>
#include "utils/Log.h"

// 顶点着色器
static const char *ver = "#version 300 es\n"
                         "in vec4 aPosition;\n"
                         "in vec2 aTexCoord;\n"
                         "out vec2 TexCoord;\n"
                         "void main() {\n"
                         "  TexCoord = aTexCoord;\n"
                         "  gl_Position = aPosition;\n"
```

```
                            "}";

// 片元着色器
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
                              "out vec4 FragColor;\n"
                              "in vec2 TexCoord;\n"
                              "uniform sampler2D ourTexture;\n"
                              "uniform vec3 u_textColor;\n"
                              "void main()\n"
                              "{\n"
                              "    vec4 color = vec4(1.0, 1.0, 1.0,
texture(ourTexture, TexCoord).r);\n"
                              "    FragColor = vec4(u_textColor, 1.0)
* color;\n"
                              "}";


// 贴图纹理坐标（参考手机屏幕坐标系统，原点在左上角）
const static GLfloat TEXTURE_COORD[] = {
        1.0f, 1.0f, // 右下
        1.0f, 0.0f, // 右上
        0.0f, 1.0f, // 左下
        0.0f, 0.0f // 左上
};

//字体路径，写死了
const static char *FONT_FILE_PATH =
"/data/user/0/com.fly.opengles.learn/files/jianti.ttf";

static const int MAX_SHORT_VALUE = 65536;

void TextRenderOpengl::LoadFacesByASCII() {
    // FreeType
    FT_Library ft;
    FT_Face face;
```

```
    do {
        if (FT_Init_FreeType(&ft)) {
            LOGD("FT_Init_FreeType fail");
            break;
        }
        // FONT_FILE_PATH替换成自己的字体文件路径
        if (FT_New_Face(ft, FONT_FILE_PATH, 0, &face)) {
            LOGD("FT_New_Face fail");
            break;
        }

        // 设置文字的大小，此函数设置了字体面的宽度和高度
        // 将宽度值设为0表示我们要从字体面通过给出的高度中动态计算出字形的宽度。
        FT_Set_Pixel_Sizes(face, 0, 90);
        // 字节对齐
        glPixelStorei(GL_UNPACK_ALIGNMENT, 1);

        // Load first 128 characters of ASCII set
        for (unsigned char c = 0; c < 128; c++) {
            // Load character glyph
            if (FT_Load_Char(face, c, FT_LOAD_RENDER)) {
                LOGD("FT_Load_Char: Failed to load Glyph");
                continue;
            }
            // 纹理
            GLuint texture;
            glGenTextures(1, &texture);
            glBindTexture(GL_TEXTURE_2D, texture);
            glTexImage2D(
                    GL_TEXTURE_2D,
                    0,
                    GL_LUMINANCE,
                    face->glyph->bitmap.width,
                    face->glyph->bitmap.rows,
                    0,
                    GL_LUMINANCE,
```

```
                    GL_UNSIGNED_BYTE,
                    face->glyph->bitmap.buffer
            );
            // Set texture options
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);
            // Now store character for later use
            Character character = {
                    texture,
                    glm::ivec2(face->glyph->bitmap.width, face->glyph-
>bitmap.rows),
                    glm::ivec2(face->glyph->bitmap_left, face->glyph-
>bitmap_top),
                    static_cast<GLuint>(face->glyph->advance.x)
            };
            m_Characters.insert(std::pair<GLint, Character>(c,
character));
        }
        break;
    } while (0);
    glBindTexture(GL_TEXTURE_2D, 0);
    // 字体库释放
    FT_Done_Face(face);
    FT_Done_FreeType(ft);
}

// 注意你的字体库要支持中文，如果字体库不支持中文的话渲染是没有效果的，拿到的纹理图
的宽高都是0
void TextRenderOpengl::LoadFacesByUnicode(const wchar_t *text, int
size) {
```

```
    // FreeType
    FT_Library ft;
    FT_Face face;
    glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
    // 因为FT_Library 要释放，所以使用do while
    do {
        // All functions return a value different than 0 whenever an
error occurred
        if (FT_Init_FreeType(&ft)) {
            // todo 提示
            LOGD("LoadFacesByUnicode -----FT_Init_FreeType fail");
            break;
        }

        if (FT_New_Face(ft, FONT_FILE_PATH, 0, &face)) {
            // todo
            LOGD("LoadFacesByUnicode -----FT_New_Face fail");
            break;
        }

        // Set size to load glyphs as
        FT_Set_Pixel_Sizes(face, 0, 96);
        // 指定使用unicode
        FT_Select_Charmap(face, ft_encoding_unicode);
        for (int i = 0; i < size; ++i) {
            //int index =  FT_Get_Char_Index(face,unicodeArr[i]);
            if (FT_Load_Glyph(face, FT_Get_Char_Index(face, text[i]),
FT_LOAD_DEFAULT)) {
                continue;
            }

            FT_Glyph glyph;
            FT_Get_Glyph(face->glyph, &glyph);

            //Convert the glyph to a bitmap.
            FT_Glyph_To_Bitmap(&glyph, ft_render_mode_normal, 0, 1);
```

```
            FT_BitmapGlyph bitmap_glyph = (FT_BitmapGlyph) glyph;

            //This reference will make accessing the bitmap easier
            FT_Bitmap &bitmap = bitmap_glyph->bitmap;

            LOGD("LoadFacesByUnicode -----text[i]:%d",text[i]);
            LOGD("LoadFacesByUnicode -----
w:%d,h:%d",bitmap.width,bitmap.rows);

            // Generate texture
            GLuint texture;
            glGenTextures(1, &texture);
            glBindTexture(GL_TEXTURE_2D, texture);
            glTexImage2D(
                    GL_TEXTURE_2D,
                    0,
                    GL_LUMINANCE,
                    bitmap.width,
                    bitmap.rows,
                    0,
                    GL_LUMINANCE,
                    GL_UNSIGNED_BYTE,
                    bitmap.buffer
            );
            // Set texture options
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);
            // Now store character for later use
            Character character = {
                    texture,
```

```
                    glm::ivec2(face->glyph->bitmap.width, face->glyph->bitmap.rows),
                    glm::ivec2(face->glyph->bitmap_left, face->glyph->bitmap_top),
                    static_cast<GLuint>((glyph->advance.x / MAX_SHORT_VALUE) << 6)
            };
            m_Characters.insert(std::pair<GLint, Character>(text[i], character));
            LOGD("LoadFacesByUnicode -----success");
        }
        break;
    } while (0);
    glBindTexture(GL_TEXTURE_2D, 0);
    // Destroy FreeType once we're finished
    FT_Done_Face(face);
    FT_Done_FreeType(ft);
}

TextRenderOpengl::TextRenderOpengl() {

    initGlProgram(ver, fragment);
    positionHandle = glGetAttribLocation(program, "aPosition");
    textureHandle = glGetAttribLocation(program, "aTexCoord");
    textureSampler = glGetUniformLocation(program, "ourTexture");
    u_textColorHandle = glGetUniformLocation(program, "u_textColor");
    LoadFacesByASCII();
}

TextRenderOpengl::~TextRenderOpengl() noexcept {

}

void TextRenderOpengl::onDraw() {
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    //禁用byte-alignment限制 使用一个字节对齐
```

```
    glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
    // 打开混合设置
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    // 渲染ASCII字符
    RenderASCIIText("Abcdef%h*!@#$", -0.9f, 0.2f, 3, glm::vec3(0.5,
0.5, 0.5),
                    glm::vec2(eglHelper->viewWidth, eglHelper-
>viewHeight));

    const wchar_t text[] = L"我看看中文的渲染";

 RenderUnicodeText(text,sizeof(text)/sizeof(text[0]),-0.9f,-0.4f,2,glm
::vec3(0.5,0.5,0.5),glm::vec2(eglHelper->viewWidth,eglHelper-
>viewHeight));

    if (nullptr != eglHelper) {
        eglHelper->swapBuffers();
    }
}


void TextRenderOpengl::RenderUnicodeText(const wchar_t* text, int
textLen, GLfloat x, GLfloat y, GLfloat scale, glm::vec3 color,
glm::vec2 viewport){

    LoadFacesByUnicode(text,textLen);

    // 以下代码基本和RenderASCIIText一样的，只是字体库加载的方式变了而已

    // 激活合适的渲染状态
    glUseProgram(program);

    // 纹理坐标都是一样的，先赋值纹理坐标
    // 纹理坐标
    glEnableVertexAttribArray(textureHandle);
```

```
    glVertexAttribPointer(textureHandle, 2, GL_FLOAT, GL_FALSE, 0,
TEXTURE_COORD);

    glUniform3f(u_textColorHandle, color.x, color.y, color.z);
    x *= viewport.x;
    y *= viewport.y;
    for (int i = 0; i < textLen; ++i){
        Character ch = m_Characters[text[i]];

        GLfloat xpos = x + ch.bearing.x * scale;
        GLfloat ypos = y - (ch.size.y - ch.bearing.y) * scale;

        // 归一化
        xpos /= viewport.x;
        ypos /= viewport.y;

        // 字体贴图缩放后的宽高
        GLfloat w = ch.size.x * scale;
        GLfloat h = ch.size.y * scale;

        // 宽高也归一化
        w /= viewport.x;
        h /= viewport.y;

        LOGD("TextRenderSample::RenderUnicodeText [xpos,ypos,w,h]=[%f,
%f, %f, %f], ch.advance >> 6 = %d",
             xpos, ypos, w, h, ch.advance >> 6);

        // 当前字符的顶点坐标
        GLfloat VERTICES[] = {
                xpos + w, ypos,
                xpos + w, ypos + h,
                xpos, ypos,
                xpos, ypos + h
        };
```

```
            /**
             * size 几个数字表示一个点，显示是两个数字表示一个点
             * normalized 是否需要归一化，不用，这里已经归一化了
             * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
             */
            // 启用顶点数据
            glEnableVertexAttribArray(positionHandle);
            glVertexAttribPointer(positionHandle, 2, GL_FLOAT, GL_FALSE,
0, VERTICES);

            // 在方块上绘制字形纹理
            glActiveTexture(GL_TEXTURE0);
            glBindTexture(GL_TEXTURE_2D, ch.textureID);
            glUniform1i(textureSampler, 0);
            // 绘制方块
            glDrawArrays(GL_TRIANGLE_STRIP, 0, 4);
            // 更新位置到下一个字形的原点，注意单位是1/64像素
            x += (ch.advance >> 6) * scale; //(2^6 = 64)
        }
        glBindVertexArray(0);
        glBindTexture(GL_TEXTURE_2D, 0);

        // 禁用顶点
        glDisableVertexAttribArray(positionHandle);
        glDisableVertexAttribArray(textureHandle);
    }

    /**
     *
     * @param text 需要渲染的文字
     * @param x 归一化坐标中文字渲染x的起点  [-1,1]
     * @param y 归一化坐标中文字渲染y的起点  [-1,1]
     * @param scale 字体缩放  这个也可以在顶点坐标中做统一的变换使用
     * @param color 字体颜色
     * @param viewport 视窗大小
     */
```

```
void TextRenderOpengl::RenderASCIIText(std::string text, GLfloat x,
GLfloat y, GLfloat scale,
                                       glm::vec3 color, glm::vec2
viewport) {

    // 激活合适的渲染状态
    glUseProgram(program);

    // 纹理坐标都是一样的，先赋值纹理坐标
    // 纹理坐标
    glEnableVertexAttribArray(textureHandle);
    glVertexAttribPointer(textureHandle, 2, GL_FLOAT, GL_FALSE, 0,
TEXTURE_COORD);

    glUniform3f(u_textColorHandle, color.x, color.y, color.z);
    // 对文本中的所有字符迭代
    std::string::const_iterator c;
    x *= viewport.x;
    y *= viewport.y;
    for (c = text.begin(); c != text.end(); c++) {
        Character ch = m_Characters[*c];

        GLfloat xpos = x + ch.bearing.x * scale;
        GLfloat ypos = y - (ch.size.y - ch.bearing.y) * scale;

        // 归一化
        xpos /= viewport.x;
        ypos /= viewport.y;

        // 字体贴图缩放后的宽高
        GLfloat w = ch.size.x * scale;
        GLfloat h = ch.size.y * scale;

        // 宽高也归一化
        w /= viewport.x;
        h /= viewport.y;
```

```
        LOGD("TextRenderSample::RenderText [xpos,ypos,w,h]=[%f, %f, %f, %f], ch.advance >> 6 = %d",
             xpos, ypos, w, h, ch.advance >> 6);

        // 当前字符的顶点坐标
        GLfloat VERTICES[] = {
                xpos + w, ypos,
                xpos + w, ypos + h,
                xpos, ypos,
                xpos, ypos + h
        };

        /**
         * size 几个数字表示一个点，显示是两个数字表示一个点
         * normalized 是否需要归一化，不用，这里已经归一化了
         * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
         */
        // 启用顶点数据
        glEnableVertexAttribArray(positionHandle);
        glVertexAttribPointer(positionHandle, 2, GL_FLOAT, GL_FALSE, 0, VERTICES);

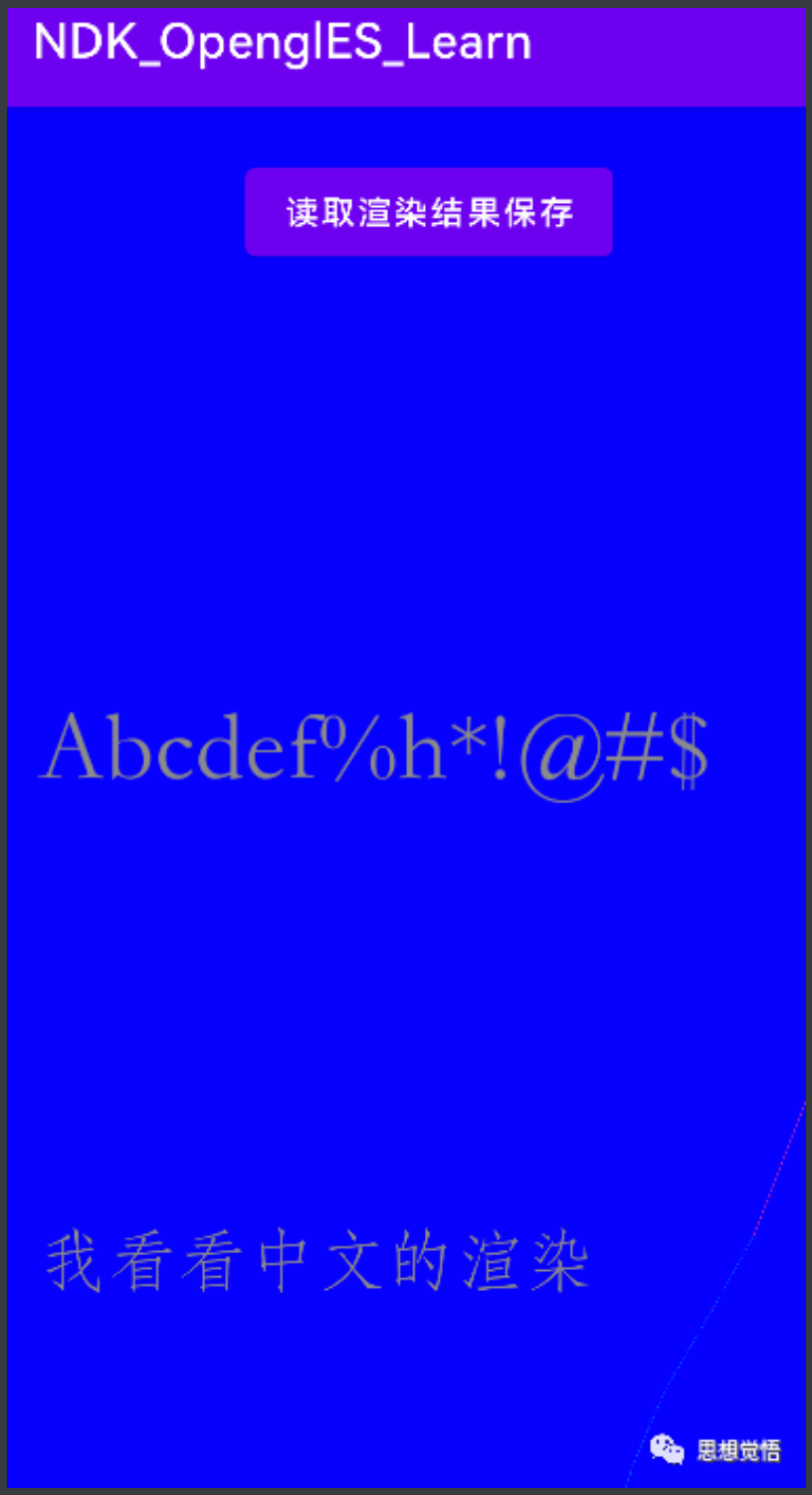
        // 在方块上绘制字形纹理
        glActiveTexture(GL_TEXTURE0);
        glBindTexture(GL_TEXTURE_2D, ch.textureID);
        glUniform1i(textureSampler, 0);
        // 绘制方块
        glDrawArrays(GL_TRIANGLE_STRIP, 0, 4);
        // 更新位置到下一个字形的原点，注意单位是1/64像素
        x += (ch.advance >> 6) * scale; //(2^6 = 64)
    }
    glBindVertexArray(0);
    glBindTexture(GL_TEXTURE_2D, 0);

    // 禁用顶点
```

```
        glDisableVertexAttribArray(positionHandle);
        glDisableVertexAttribArray(textureHandle);

    }
```

运行结果：



遇到的问题

- 测试字体库如何获取

每台个人电脑都自带着很多的字体库，比如搜索 `.ttf` 结尾的文件就能搜索到很多，拷贝到手机中进行加载使用测试即可。

- 使用了Unicode编码依然无法渲染中文

测试的时候发现使用 `RenderUnicodeText` 渲染中文有时候还是可能会没有效果，这时候需要确认 `FT_New_Face` 加载的这个字体库是否支持中文，如果字体库本身就是不支持中文的话，那么自然是无法渲染的。

系列教程源码

https://github.com/feiflyer/NDK_OpenglES_Tutorial

后续demo如果有完善可能会更新。

s

Opengl ES之纹理数组

这是Opengl ES入门系列的第17篇博文啦...



今天的内容比较简单，给大家介绍一下纹理数组，它是OpenGL ES 3.0引入的一个新特性，它能让我们以数组的方式往shader中传递纹理。

2D纹理数组是OpenGL ES 3.0开始支持的纹理类型。通过使用2D纹理数组，在同一个着色器中使用多个2D纹理的情况下可以简化开发。
试想一下，如果没有2D纹理数组技术，当一个着色器中需要使用多个2D纹理时就需要声明多个采样器变量，而如果使用2D纹理数组则同样的功能只需声明一个sampler2DArray类型的变量即可，方便高效，
而且扩展性更强。

如果是单单使用2D纹理的话，一旦着色器程序确定了，那么内部可使用的纹理个数就确定了，而如果使用2D纹理数组的话，即使着色器程序确定了，我们依然可以按照不同的需求使用不同个数的纹理，
这就大大提高了我们编程的灵活性。

2D纹理数组的使用

2D纹理数组的使用和普通的纹理贴图步骤差不多，只是普通纹理使用绑定的是 `GL_TEXTURE_2D`，而2D纹理数组绑定的类型是 `GL_TEXTURE_2D_ARRAY`，同时2D纹理数组分配纹理空间时需要使用函数 `glTexImage3D` 而不是 `glTexImage2D`。

另外在使用2D纹理数组进行采样时，需要提供的纹理坐标有3个分量也就是 `vec3`，前两个与普通2D纹理坐标含义相同，为S、T分量，第三个分量则为2D纹理数组中的索引。

下面是一个简单的使用2D纹理数组的片元着色器：

```
#version 300 es
precision mediump float;
precision mediump sampler2DArray;
out vec4 FragColor;
in vec3 TexCoord;
uniform sampler2DArray ourTexture;
void main()
{
    FragColor = texture(ourTexture, TexCoord);
}
```

可以看到纹理坐标使用的是三个分量变量，而且需要声明 `sampler2DArray` 精度类型。

以下是使用纹理数组绘制两张不同的图片的一个demo源码实例：

TextureArrayOpengl.cpp

```
// 顶点着色器
static const char *ver = "#version 300 es\n"
                         "in vec4 aPosition;\n"
                         "in vec3 aTexCoord;\n"
                         "out vec3 TexCoord;\n"
                         "void main() {\n"
                         "  TexCoord = aTexCoord;\n"
                         "  gl_Position = aPosition;\n"
                         "}";
```

```
// 片元着色器
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
                              "precision mediump sampler2DArray;\n"
                              "out vec4 FragColor;\n"
                              "in vec3 TexCoord;\n"
                              "uniform sampler2DArray ourTexture; \n"
                              "void main()\n"
                              "{\n"
                              "    FragColor = texture(ourTexture,
TexCoord);\n"
                              "}";


// 使用绘制两个三角形组成一个矩形的形式(三角形带)
// 第一第二第三个点组成一个三角形，第二第三第四个点组成一个三角形
const static GLfloat VERTICES[] = {
        0.5f,0.0f, // 右下
        0.5f,0.5f, // 右上
        0.0f,0.0f, // 左下
        0.0f,0.5f, // 左上

        0.0f,-0.5f, // 右下
        0.0f,0.0f, // 右上
        -0.5f,-0.5f, // 左下
        -0.5f,0.0f // 左上
};


// 贴图纹理坐标（参考手机屏幕坐标系统，原点在左上角）
//由于对一个OpenGL纹理来说，它没有内在的方向性，因此我们可以使用不同的坐标把它定向到任何我们喜欢的方向上，然而大多数计算机图像都有一个默认的方向，它们通常被规定为y轴向下，X轴向右
const static GLfloat TEXTURE_COORD[] = {
        1.0f,1.0f,0, // 右下
```

```
        1.0f,0.0f,0, // 右上
        0.0f,1.0f,0, // 左下
        0.0f,0.0f,0, // 左上

        1.0f,1.0f,1, // 右下
        1.0f,0.0f,1, // 右上
        0.0f,1.0f,1, // 左下
        0.0f,0.0f,1 // 左上

};

TextureArrayOpengl::TextureArrayOpengl():BaseOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
    textureHandle = glGetAttribLocation(program,"aTexCoord");
    textureSampler = glGetUniformLocation(program,"ourTexture");
    LOGD("program:%d",program);
    LOGD("positionHandle:%d",positionHandle);
    LOGD("textureHandle:%d",textureHandle);
    LOGD("textureSample:%d",textureSampler);
}

void TextureArrayOpengl::setPixel(void *data, int width, int height,
int length) {
    LOGD("texture setPixel");
    glGenTextures(1, &textureId);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D_ARRAY, textureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D_ARRAY, GL_TEXTURE_WRAP_S,
GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D_ARRAY, GL_TEXTURE_WRAP_T,
GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D_ARRAY, GL_TEXTURE_MIN_FILTER,
GL_LINEAR);
```

```
    glTexParameteri(GL_TEXTURE_2D_ARRAY, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);

    // 使用glTexStorage3D搭配glTexSubImage3D也是可以的
    // 注意这里是GL_RGBA8而不是GL_RGBA
//    glTexStorage3D(GL_TEXTURE_2D_ARRAY, 1, GL_RGBA8, width,
height,2);
    glTexImage3D(GL_TEXTURE_2D_ARRAY, 0, GL_RGBA, width, height,2, 0,
GL_RGBA, GL_UNSIGNED_BYTE,
                 nullptr);

 glTexSubImage3D(GL_TEXTURE_2D_ARRAY,0,0,0,0,width,height,1,GL_RGBA,GL
_UNSIGNED_BYTE,data);
    // 解绑定
    glBindTexture(GL_TEXTURE_2D_ARRAY, 0);
}

// 主要先调用setPixel设置第一张纹理，在setPixel中声明了一个容量为2的纹理数组，先
调用setPixel2的话是不行的哦

void TextureArrayOpengl::setPixel2(void *data, int width, int height,
int length) {

    LOGD("texture setPixel2");

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D_ARRAY, textureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D_ARRAY, GL_TEXTURE_WRAP_S,
GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D_ARRAY, GL_TEXTURE_WRAP_T,
GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D_ARRAY, GL_TEXTURE_MIN_FILTER,
GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D_ARRAY, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);
```

```
 glTexSubImage3D(GL_TEXTURE_2D_ARRAY,0,0,0,1,width,height,1,GL_RGBA,GL
_UNSIGNED_BYTE,data);
    // 解绑定
    glBindTexture(GL_TEXTURE_2D_ARRAY, 0);
}

void TextureArrayOpengl::onDraw() {
    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);

    // 激活纹理
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D_ARRAY, textureId);

    /**
     * size 几个数字表示一个点，显示是两个数字表示一个点
     * normalized 是否需要归一化，不用，这里已经归一化了
     * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
     */
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);

 glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);

    // 纹理坐标
    glEnableVertexAttribArray(textureHandle);

 glVertexAttribPointer(textureHandle,3,GL_FLOAT,GL_FALSE,0,TEXTURE_COO
RD);
```

```
        // 4个顶点绘制两个三角形组成矩形
        glDrawArrays(GL_TRIANGLE_STRIP,0,8);

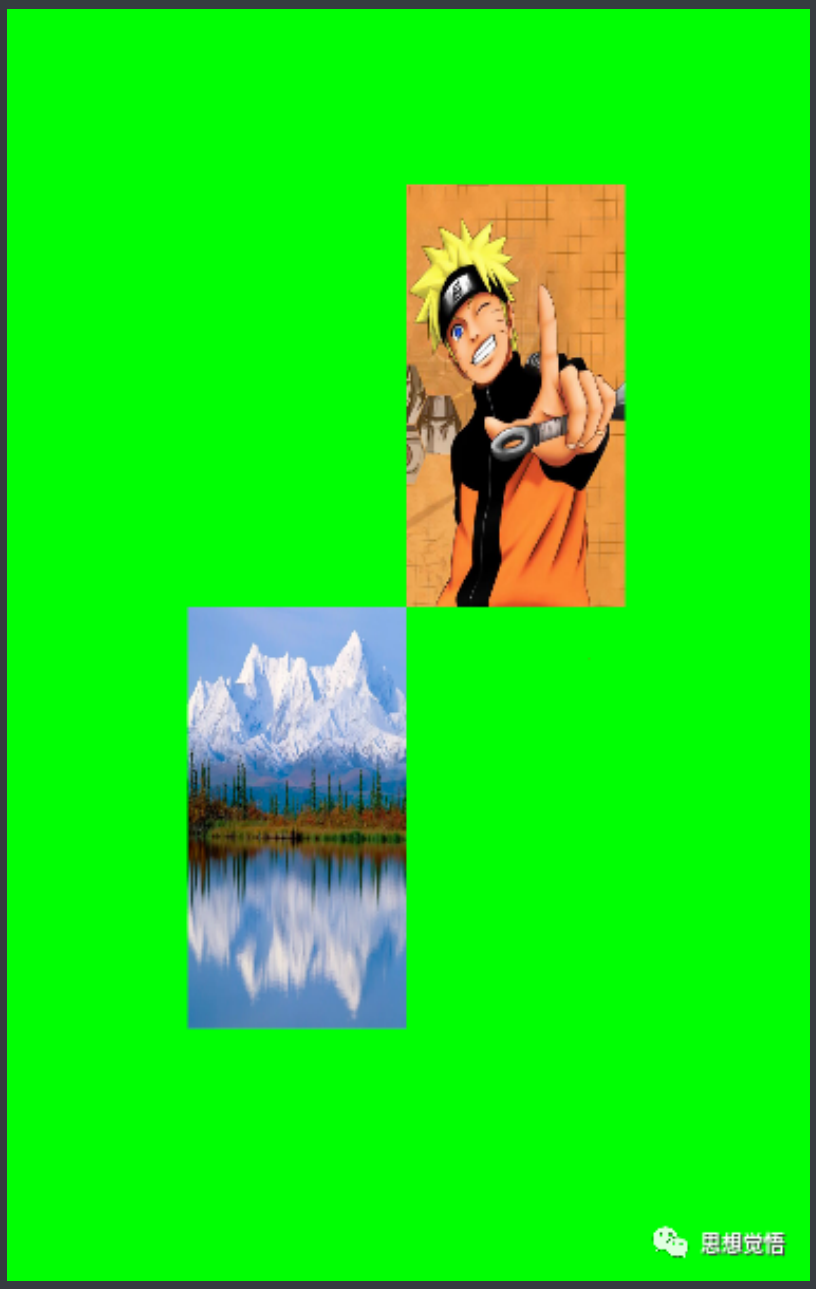
        glUseProgram(0);

        // 禁用顶点
        glDisableVertexAttribArray(positionHandle);
        if(nullptr != eglHelper){
            eglHelper->swapBuffers();
        }

        glBindTexture(GL_TEXTURE_2D_ARRAY, 0);
    }

    TextureArrayOpengl::~TextureArrayOpengl() {
        LOGD("TextureArrayOpengl析构函数");
    }
```

运行结果显示:



注意事项

查阅相关资料解说使用2D纹理数组时，一般会使数组中的纹理保持相同的尺寸。

但是笔者在某些机型上实测发现只要加载的纹理大小不要超过 `glTexImage3D` 中声明的宽高也能正常加载使用，不知道如果尺寸不一只是否有兼容性问题，
实际开发中还是建议大家尽量保存纹理数组的尺寸大小都一致。

系列教程源码

https://github.com/feiflyer/NDK_OpenglES_Tutorial

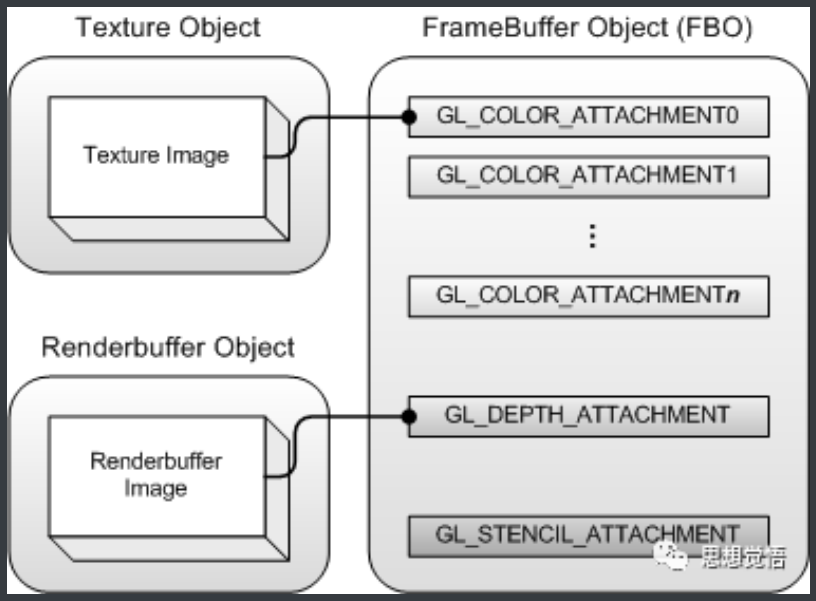
s

OpenGL ES之多目标渲染（MRT）

大家好，我又来啦！今天给大家介绍一个OpenGL ES 3.0中的新特性，多渲染目标（Multiple Render Target）。

所谓的多渲染目标就是指将片元着色器中的输出对应到多个纹理上了，在之前的例子上我们的片元着色器都是只有一个输出，只能对应输出到一个目标上（FBO纹理或者屏幕），
今天我们使用MRT特性，在片元着色器中定义多个输出，并且附着到多个纹理之上。

首先我们来回顾一下之前在 Opengl ES之FBO 一文中提到的挂在点这张图，上面所说的多渲染目标所用到的技术就是
FBO中的多个颜色附着点（GL_COLOR_ATTACHMENT0、GL_COLOR_ATTACHMENT1...）



> FBO提供的挂接点如图所示

在FBO中我们通过 `glFramebufferTexture2D()` 将texture绑定到color attachment上，这时绑定这个frame buffer进行渲染，就会渲染到绑定的texture。

在OpenGL ES 2.0中，FBO只能绑定到0号的color attachment即GL_COLOR_ATTACHMENT0上，而在Opengl ES3.0中则没有了这个限制，这或许就是为什么在OpenGL ES 2.0中片元着色器给我们
内置了一个默认输出变量`gl_FragColor`，而在Opengl ES3.0则需要开发者自定义输出变量的一个原因吧。

下面我们使用MRT特性实现一个小demo，首先使用FBO输出到四个纹理上，分别是原图、R颜色通道分离图、G颜色通道分离图和B颜色通道分离图，然后再将这四个纹理各取四分之一部分组成一张新图渲染到屏幕上。

MRT例子demo

首先我们需要生产一个FBO和四个附着的frame buffer上的纹理，准备代码如下：

```
    // TEXTURE_NUM就是4
    glGenTextures(TEXTURE_NUM, fboTextureIds);
    glGenFramebuffers(1, &fboId);

    glBindFramebuffer(GL_FRAMEBUFFER, fboId);

    for (int i = 0; i < TEXTURE_NUM; ++i) {
        attachments[i] = GL_COLOR_ATTACHMENT0 + i;
        glBindTexture(GL_TEXTURE_2D, fboTextureIds[i]);
        glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE);
        glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);
        // 这个纹理是多大的?
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, imageWidth,
imageHeight, 0, GL_RGBA, GL_UNSIGNED_BYTE,
                     nullptr);
```

```
        glFramebufferTexture2D(GL_DRAW_FRAMEBUFFER, attachments[i],
GL_TEXTURE_2D, fboTextureIds[i], 0);
        // 解绑
        glBindTexture(GL_TEXTURE_2D, GL_NONE);
    }

    //告诉 OpenGL ，我要渲染到 4 个颜色附着上
    glDrawBuffers(TEXTURE_NUM, attachments);

    // 检查FBO状态
    if (glCheckFramebufferStatus(GL_FRAMEBUFFER) !=
GL_FRAMEBUFFER_COMPLETE) {
        LOGE("FBOSample::CreateFrameBufferObj glCheckFramebufferStatus
status != GL_FRAMEBUFFER_COMPLETE");
    }
    // 解绑
    glBindTexture(GL_TEXTURE_2D, GL_NONE);
    glBindFramebuffer(GL_FRAMEBUFFER, GL_NONE);
```

然后在片元着色器中分别定义四个颜色输出，分别对应FBO上的四个颜色附着点：

```
// 片元着色器
// 四个输出，一个是完整的图片数据、另外三个是RGB分离的数据
#version 300 es
precision mediump float;
in vec2 TexCoord;
uniform sampler2D ourTexture;
// 定义多个输出的话layout(location = n)是必须的
layout(location = 0) out vec4 outColor0;
layout(location = 1) out vec4 outColor1;
layout(location = 2) out vec4 outColor2;
layout(location = 3) out vec4 outColor3;
void main()
{
    vec4 outColor = texture(ourTexture, TexCoord);
    // 原图
```

```
        outColor0 = outColor;
        // R分离通道
        outColor1 = vec4(outColor.r, 0.0, 0.0, 1.0);
        // G分离通道
        outColor2 = vec4(0.0, outColor.g, 0.0, 1.0);
        // B分离通道
        outColor3 = vec4(0.0, 0.0, outColor.b, 1.0);
    }
```

都加上注释了，这里不多说，最后的步骤就是进行FBO的渲染，再将FBO渲染后的四个纹理图组合渲染到屏幕上，也就是绘制两次，一次绘制到FBO，一次是将FBO的
内容绘制到屏幕，关于FBO的使用绘制请参考之前的博文 Opengl ES之FBO

这里不同的是两次绘制所使用的片元着色器是不一样的，绘制到屏幕上时所使用到的片元着色器如下：

```
    #version 300 es
    precision mediump float;
    out vec4 FragColor;
    in vec2 TexCoord;
    uniform sampler2D s_Texture0;
    uniform sampler2D s_Texture1;
    uniform sampler2D s_Texture2;
    uniform sampler2D s_Texture3;
    void main()
    {
    if(TexCoord.x < 0.5 && TexCoord.y < 0.5)
        {
            FragColor = texture(s_Texture0, TexCoord);
        }
        else if(TexCoord.x > 0.5 && TexCoord.y < 0.5)
        {
            FragColor = texture(s_Texture1, TexCoord);
        }
        else if(TexCoord.x < 0.5 && TexCoord.y > 0.5)
        {
```

```
            FragColor = texture(s_Texture2, TexCoord);
        }
        else
        {
            FragColor = texture(s_Texture3, TexCoord);
        }
    }
```

也就是将FBO中的四张纹理图各取四分之一组合绘制成一张新图。

完整代码如下：

MRTRenderOpengl.cpp

```
    /**
     * 本教程版权属于公众号"思想觉悟"
     /
    #include "MRTRenderOpengl.h"


    #include "FBOOpengl.h"
    #include "../utils/Log.h"
    #include "../utils/ShaderUtils.h"

    // 顶点着色器
    static const char *ver = "#version 300 es\n"
                             "in vec4 aPosition;\n"
                             "in vec2 aTexCoord;\n"
                             "out vec2 TexCoord;\n"
                             "void main() {\n"
                             "  TexCoord = aTexCoord;\n"
                             "  gl_Position = aPosition;\n"
                             "}";

    // 片元着色器
    // 四个输出，一个是完整的图片数据、另外三个是RGB分离的数据
```

```
// 定义多个输出的话layout(location = n)是必须的
static const char *outs_fragment = "#version 300 es\n"
                                   "precision mediump float;\n"
                                   "in vec2 TexCoord;\n"
                                   "uniform sampler2D ourTexture;\n"
                                   "layout(location = 0) out vec4
outColor0;\n"
                                   "layout(location = 1) out vec4
outColor1;\n"
                                   "layout(location = 2) out vec4
outColor2;\n"
                                   "layout(location = 3) out vec4
outColor3;\n"
                                   "void main()\n"
                                   "{\n"
                                   "    vec4 outColor =
texture(ourTexture, TexCoord);\n"
                                   "    outColor0 = outColor;\n"
                                   "    outColor1 = vec4(outColor.r,
0.0, 0.0, 1.0);\n"
                                   "    outColor2 = vec4(0.0,
outColor.g, 0.0, 1.0);\n"
                                   "    outColor3 = vec4(0.0, 0.0,
outColor.b, 1.0);\n"
                                   "}";

// 片元着色器
static const char *render_fragment = "#version 300 es\n"
                                     "precision mediump float;\n"
                                     "out vec4 FragColor;\n"
                                     "in vec2 TexCoord;\n"
                                     "uniform sampler2D s_Texture0;"
                                     "uniform sampler2D s_Texture1;"
                                     "uniform sampler2D s_Texture2;"
                                     "uniform sampler2D s_Texture3;"
                                     "void main()\n"
```

```
                                        "{\n"
                                        "if(TexCoord.x < 0.5 &&
TexCoord.y < 0.5)\n"
                                        "    {   \n"
                                        "        FragColor =
texture(s_Texture0, TexCoord);\n"
                                        "    }\n"
                                        "    else if(TexCoord.x > 0.5 &&
TexCoord.y < 0.5)\n"
                                        "    {\n"
                                        "        FragColor =
texture(s_Texture1, TexCoord);\n"
                                        "    }\n"
                                        "    else if(TexCoord.x < 0.5 &&
TexCoord.y > 0.5)\n"
                                        "    {\n"
                                        "        FragColor =
texture(s_Texture2, TexCoord);\n"
                                        "    }\n"
                                        "    else\n"
                                        "    {\n"
                                        "        FragColor =
texture(s_Texture3, TexCoord);\n"
                                        "    }"
                                        "}";

const static GLfloat VERTICES_AND_TEXTURE[] = {
        0.5f, -0.5f, // 右下
        // 纹理坐标
        1.0f, 1.0f,
        0.5f, 0.5f, // 右上
        // 纹理坐标
        1.0f, 0.0f,
        -0.5f, -0.5f, // 左下
        // 纹理坐标
        0.0f, 1.0f,
```

```
        -0.5f, 0.5f, // 左上
        // 纹理坐标
        0.0f, 0.0f
};

// 真正的纹理坐标在图片的左下角
const static GLfloat FBO_VERTICES_AND_TEXTURE[] = {
        1.0f, -1.0f, // 右下
        // 纹理坐标
        1.0f, 0.0f,
        1.0f, 1.0f, // 右上
        // 纹理坐标
        1.0f, 1.0f,
        -1.0f, -1.0f, // 左下
        // 纹理坐标
        0.0f, 0.0f,
        -1.0f, 1.0f, // 左上
        // 纹理坐标
        0.0f, 1.0f
};

// 使用byte类型比使用short或者int类型节约内存
const static uint8_t indices[] = {
        // 注意索引从0开始!
        // 此例的索引(0,1,2,3)就是顶点数组vertices的下标,
        // 这样可以由下标代表顶点组合成矩形
        0, 1, 2, // 第一个三角形
        1, 2, 3  // 第二个三角形
};


MRTRenderOpengl::MRTRenderOpengl() {

    mrtProgram = createProgram(ver,outs_fragment);
    mrtPositionHandle = glGetAttribLocation(mrtProgram, "aPosition");
    mrtTextureHandle = glGetAttribLocation(mrtProgram, "aTexCoord");
```

```
    mrtTextureSampler = glGetUniformLocation(mrtProgram,
"ourTexture");

    initGlProgram(ver, render_fragment);
    positionHandle = glGetAttribLocation(program, "aPosition");
    textureHandle = glGetAttribLocation(program, "aTexCoord");
    textureSampler1 = glGetUniformLocation(program, "s_Texture0");
    textureSampler2 = glGetUniformLocation(program, "s_Texture1");
    textureSampler3 = glGetUniformLocation(program, "s_Texture2");
    textureSampler4 = glGetUniformLocation(program, "s_Texture3");
    LOGD("program:%d", program);
    LOGD("positionHandle:%d", positionHandle);
    LOGD("textureHandle:%d", textureHandle);
    LOGD("textureSampler1:%d", textureSampler1);
    LOGD("textureSampler2:%d", textureSampler2);
    LOGD("textureSampler3:%d", textureSampler3);
    LOGD("textureSampler4:%d", textureSampler4);
    // VAO
    glGenVertexArrays(1, &vao);
    glBindVertexArray(vao);

    // vbo
    glGenBuffers(1, &vbo);
    glBindBuffer(GL_ARRAY_BUFFER, vbo);
    glBufferData(GL_ARRAY_BUFFER, sizeof(VERTICES_AND_TEXTURE),
VERTICES_AND_TEXTURE,
                 GL_STATIC_DRAW);

    // stride 步长 每个顶点坐标之间相隔4个数据点，数据类型是float
    glVertexAttribPointer(positionHandle, 2, GL_FLOAT, GL_FALSE, 4 *
sizeof(float), (void *) 0);
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);
    // stride 步长 每个颜色坐标之间相隔4个数据点，数据类型是float，颜色坐标索引
从2开始
```

```
    glVertexAttribPointer(textureHandle, 2, GL_FLOAT, GL_FALSE, 4 *
sizeof(float),
                          (void *) (2 * sizeof(float)));
    // 启用纹理坐标数组
    glEnableVertexAttribArray(textureHandle);

    // EBO
    glGenBuffers(1, &ebo);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, ebo);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices,
GL_STATIC_DRAW);

    // 这个顺序不能乱啊，先解除vao，再解除其他的，不然在绘制的时候可能会不起作用，
需要重新glBindBuffer才生效
    // vao解除
    glBindVertexArray(0);
    // 解除绑定
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    // 解除绑定
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);

    LOGD("program:%d", program);
    LOGD("positionHandle:%d", positionHandle);
    LOGD("colorHandle:%d", textureHandle);
}

MRTRenderOpengl::~MRTRenderOpengl() {
    glDeleteBuffers(1, &ebo);
    glDeleteBuffers(1, &vbo);
    glDeleteVertexArrays(1, &vao);
    // ... 删除其他，例如fbo等
}

void MRTRenderOpengl::onDraw() {

    // 先在FBO离屏渲染
```

```
        onFboDraw();

        //首先获取当前默认帧缓冲区的 id
        GLint defaultFrameBuffer = GL_NONE;
        glGetIntegerv(GL_FRAMEBUFFER_BINDING, &defaultFrameBuffer);

        //绑定默认帧缓冲区对象，绘制到屏幕上
        glBindFramebuffer(GL_DRAW_FRAMEBUFFER, defaultFrameBuffer);

        // 恢复绘制屏幕宽高
        glViewport(0, 0, eglHelper->viewWidth, eglHelper->viewHeight);

        // 绘制到屏幕
        // 清屏
        glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
        glClear(GL_COLOR_BUFFER_BIT);
        glUseProgram(program);

        //指定 4 个纹理作为输入
        for (int i = 0; i < TEXTURE_NUM; ++i)
        {
            glActiveTexture(GL_TEXTURE0 + i);
            glBindTexture(GL_TEXTURE_2D, fboTextureIds[i]);
            LOGD("-----onDraw:%d",fboTextureIds[i]);
            switch (i) {
                case 0:
                    glUniform1i(textureSampler1,  i);
                    break;
                case 1:
                    glUniform1i(textureSampler2,  i);
                    break;
                case 2:
                    glUniform1i(textureSampler3,  i);
                    break;
                case 3:
                    glUniform1i(textureSampler4, i);
```

```
                break;
            }
        }

        // VBO与VAO配合绘制
        // 使用vao
        glBindVertexArray(vao);
        // 使用EBO
// 使用byte类型节省内存
        glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_BYTE, (void *) 0);
        glUseProgram(0);
        // vao解除绑定
        glBindVertexArray(0);

        if (nullptr != eglHelper) {
            eglHelper->swapBuffers();
        }

        glBindTexture(GL_TEXTURE_2D, 0);
        //绑定默认帧缓冲区对象，绘制到屏幕上
        glBindFramebuffer(GL_DRAW_FRAMEBUFFER, GL_NONE);
}


void MRTRenderOpengl::setPixel(void *data, int width, int height, int
length) {
    LOGD("texture setPixel");
    imageWidth = width;
    imageHeight = height;
    glGenTextures(1, &imageTextureId);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, imageTextureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
```

```
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA,
GL_UNSIGNED_BYTE, data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);

    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}

void MRTRenderOpengl::fboPrepare() {
    // VAO
    glGenVertexArrays(1, &fboVao);
    glBindVertexArray(fboVao);

    // vbo
    glGenBuffers(1, &fboVbo);
    glBindBuffer(GL_ARRAY_BUFFER, fboVbo);
    glBufferData(GL_ARRAY_BUFFER, sizeof(FBO_VERTICES_AND_TEXTURE),
FBO_VERTICES_AND_TEXTURE,
                 GL_STATIC_DRAW);

    // stride 步长 每个顶点坐标之间相隔4个数据点，数据类型是float
    glVertexAttribPointer(mrtPositionHandle, 2, GL_FLOAT, GL_FALSE, 4
* sizeof(float), (void *) 0);
    // 启用顶点数据
    glEnableVertexAttribArray(mrtPositionHandle);
    // stride 步长 每个颜色坐标之间相隔4个数据点，数据类型是float，颜色坐标索引
从2开始
    glVertexAttribPointer(mrtTextureHandle, 2, GL_FLOAT, GL_FALSE, 4 *
sizeof(float),
                          (void *) (2 * sizeof(float)));
    // 启用纹理坐标数组
    glEnableVertexAttribArray(mrtTextureHandle);
```

```
    // EBO
    glGenBuffers(1, &ebo);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, ebo);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices,
GL_STATIC_DRAW);

    // 这个顺序不能乱啊，先解除vao，再解除其他的，不然在绘制的时候可能会不起作用，
需要重新glBindBuffer才生效
    // vao解除
    glBindVertexArray(0);
    // 解除绑定
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    // 解除绑定
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);

    glGenTextures(TEXTURE_NUM, fboTextureIds);
    glGenFramebuffers(1, &fboId);

    glBindFramebuffer(GL_FRAMEBUFFER, fboId);

    for (int i = 0; i < TEXTURE_NUM; ++i) {
        attachments[i] = GL_COLOR_ATTACHMENT0 + i;
        glBindTexture(GL_TEXTURE_2D, fboTextureIds[i]);
        glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE);
        glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);
        // 这个纹理是多大的?
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, imageWidth,
imageHeight, 0, GL_RGBA, GL_UNSIGNED_BYTE,
                     nullptr);
```

```
        glFramebufferTexture2D(GL_DRAW_FRAMEBUFFER, attachments[i],
GL_TEXTURE_2D, fboTextureIds[i], 0);
        // 解绑
        glBindTexture(GL_TEXTURE_2D, GL_NONE);
    }

    //告诉 OpenGL ，我要渲染到 4 个颜色附着上
    glDrawBuffers(TEXTURE_NUM, attachments);

    // 检查FBO状态
    if (glCheckFramebufferStatus(GL_FRAMEBUFFER) !=
GL_FRAMEBUFFER_COMPLETE) {
        LOGE("FBOSample::CreateFrameBufferObj glCheckFramebufferStatus
status != GL_FRAMEBUFFER_COMPLETE");
    }
    // 解绑
    glBindTexture(GL_TEXTURE_2D, GL_NONE);
    glBindFramebuffer(GL_FRAMEBUFFER, GL_NONE);
}

void MRTRenderOpengl::onFboDraw() {
    fboPrepare();

    glBindFramebuffer(GL_FRAMEBUFFER, fboId);

    glDrawBuffers(TEXTURE_NUM, attachments);

    // 主要这个的大小要与FBO绑定时的纹理的glTexImage2D 设置的大小一致呀
    glViewport(0, 0, imageWidth, imageHeight);

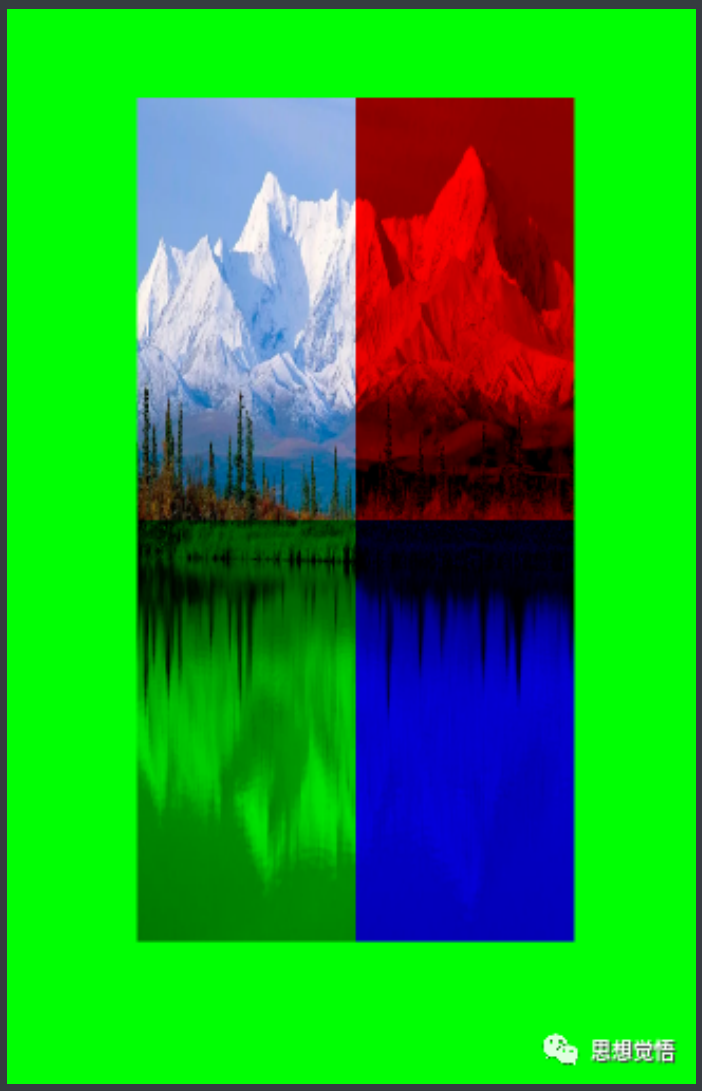
    // FBO绘制
    // 清屏
    glClearColor(0.0f, 0.0f, 1.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(mrtProgram);
```

```
        // 激活纹理
        glActiveTexture(GL_TEXTURE1);
        glUniform1i(mrtTextureSampler, 1);

        // 绑定纹理
        glBindTexture(GL_TEXTURE_2D, imageTextureId);

        // VBO与VAO配合绘制
        // 使用vao
        glBindVertexArray(fboVao);
        // 使用EBO
    // 使用byte类型节省内存
        glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_BYTE, (void *) 0);
        glUseProgram(0);
        // vao解除绑定
        glBindVertexArray(0);
        if (nullptr != eglHelper) {
            eglHelper->swapBuffers();
        }
        glBindTexture(GL_TEXTURE_2D, 0);
        glBindFramebuffer(GL_FRAMEBUFFER, 0);
    }
```

运行结果图：



思考

在一篇博文中我们介绍了纹理数组的使用，那么针对以上这个demo，当绘制到屏幕时，如何使用纹理数组的方式实现呢?

系列教程源码

https://github.com/feiflyer/NDK_OpenglES_Tutorial

Opengl ES之LUT滤镜(上)

什么是LUT滤镜

从今天开始我们开始进入Opengl ES的滤镜专题，提到滤镜就不得不提用得最多的LUT滤镜了。

LUT全称`LookUpTable`，也称为颜色查找表，它代表的是一种映射关系，通过LUT可以将输入的像素数组通过映射关系转换输出成另外的像素数组。比如一个像素的颜色值分别是 R1 G1 B1，经过一次LUT操作后变为R2 G2 B2：

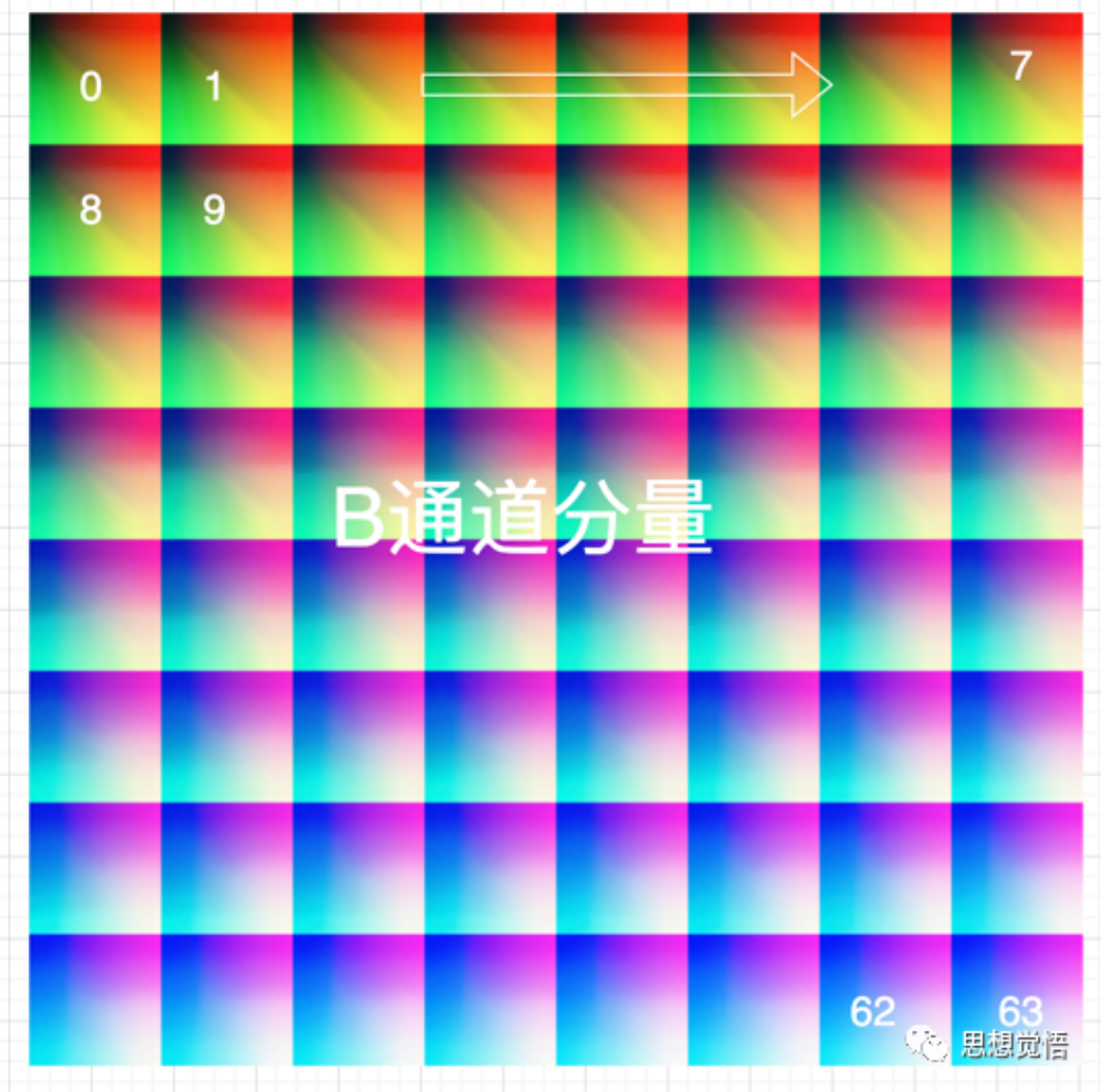
```
R2 = LUT(R1)
G2 = LUT(G1)
B2 = LUT(B1)
```

通过这个映射关系就可以将一个像素的颜色转换为另外一种颜色。

为什么使用LUT滤镜

在正常情况下，8位的RGB颜色模式可以表示的颜色数量为256X256X256种，如果要完全记录这种映射关系，设备需要耗费大量的内存，并且可能在计算时因为计算量大而产生性能问题，为了简化计算量，降低内存占用，可以将相近的n种颜色采用一条映射记录并存储，(n通常为4)这样只需要64X64X64种就可以表示原来256X256X256的颜色数量，我们也将4称为采样步长。

要想熟练使用LUT滤镜，我们首先要了解它是怎么建立颜色映射关系的， 我们看下以下这张图，这张图展示了在LUT中RBG颜色是如何实现映射关系的：



> lut映射关系图

首先这张图的大小是512X512，在横竖方向上这张图都被分成了8个小方格，每个小方格的大小是64X64，也就是一张512X512的图被分割成64个小方格，每个小方格的大小是64X64，这64个小方格就代表了64种B通道的颜色映射，
然后每个B通道的小方格上又是一个64X64像素大小的图像，这个小图像的横坐标代表R分量的64种映射情况，纵坐标代表了G分量的64种映射情况，这样就刚好这就和采样步长是4的映射表对应上了。

在使用上面这张LUT表的时候首选需要找对B分量对应的小格子，然后在找到的小个子上再计算出R分量和G分量的映射结果即可得到完整的RGB映射结果。

如何在Opengl中使用LUT滤镜

如果我们想要对图像进行LUT滤镜处理比较常用的有两种方式，一种是使用OpenCV而另外一种就是使用Opengl了，今天我们就用opengl来实现一个LUT滤镜的小demo。

理论总是枯燥的，下面我们结合一个512X512的LUT滤镜片元着色器讲解以下映射过程：

```
#version 300 es
precision mediump float;
in vec2 TexCoord;
uniform sampler2D ourTexture;
uniform sampler2D textureLUT;
out vec4 FragColor;
vec4 lookupTable(vec4 color){
    float blueColor = color.b * 63.0;
    //取与 B 分量值最接近的 2 个小方格的坐标
    vec2 quad1;
    quad1.y = floor(floor(blueColor) / 8.0);
    quad1.x = floor(blueColor) - (quad1.y * 8.0);
    vec2 quad2;
    quad2.y = floor(ceil(blueColor) / 8.0);
    quad2.x = ceil(blueColor) - (quad2.y * 8.0);
    vec2 texPos1;
    texPos1.x = (quad1.x * 0.125) + 0.5/512.0 + ((0.125 - 1.0/512.0) *
color.r);
    texPos1.y = (quad1.y * 0.125) + 0.5/512.0 + ((0.125 - 1.0/512.0) *
color.g);
    vec2 texPos2;
    texPos2.x = (quad2.x * 0.125) + 0.5/512.0 + ((0.125 - 1.0/512.0) *
color.r);
    texPos2.y = (quad2.y * 0.125) + 0.5/512.0 + ((0.125 - 1.0/512.0) *
color.g);
    //取目标映射对应的像素值
    vec4 newColor1 = texture(textureLUT, texPos1);
    vec4 newColor2 = texture(textureLUT, texPos2);
    // 颜色混合
    vec4 newColor = mix(newColor1, newColor2, fract(blueColor));
    return vec4(newColor.rgb, color.w);
```

```
}

void main()
{
    // 原始图像的RGBA值
    vec4 tmpColor = texture(ourTexture, TexCoord);
    // 通过原始图像LUT映射
    FragColor = lookupTable(tmpColor);
}
```

读懂上面的着色器程序需要了解一些Opengl的内置函数的意思，例如floor(x)是表示向下取整，返回小于等于x的整数，而ceil(x)表示向上取整，返回大于等于x的整数。这部分的内容大家自行搜索学习即可。

注意上面这个着色器的算法仅仅适合与512X512的LUT图，没有适配到各种尺寸的LUT图。

- 获取B分量

首先我们注意看第8行 `float blueColor = color.b * 63.0;` 经过main行数中的 `texture` 采样出来的RGB值都在0到1之间，因此乘以63即可得到B分量所在的小方格，
但是因为会出现浮点误差，为了减少误差，所以取两个B分量，也就是与下一步的B分量值最接近的2个小方格的坐标，最后根据小数点进行插值运算。

其中第11到第12行的核心意思就是计算B分量的小格子在LUT的8X8各种的行列序号，quad1的x和y的值应该在0到7之间。

- 获取RG分量

这部分的核心代码是16到21行，这个计算结果是归一化后的纹理坐标，所以x和y的值应该是在0到1之间，首先解析下 `(quad1.x * 0.125)` ，在512的LUT图中，横竖都被分成了8个小方格，因此在归一化坐标中，
每个小方格所占的比例就是1/8=0.125，因此 `(quad1.x * 0.125)` 的意思就是当前格子的左上角在纹理坐标系中的横坐标的具体坐标点。

因为上面`(quad1.x * 0.125)`得出的是当前格子的左上角在纹理坐标系中的横坐标的具体坐标点，那么再加上`0.5/512.0`就是当前格子的中心点在纹理坐标系中的横坐标的具体坐标点。

最后看下`((0.125 - 1.0/512.0) * color.r)`，其实这是一个经过优化后的计算，原始的计算应该是`((0.125 - 1.0/512.0) * textureColor.r) = （(64-1) * textureColor.r）/512`。

`(64-1)* textureColor.r`意思是首先将当前实际像素的r值映射到0-63的范围内，再除以512是转化为纹理坐标中实际的点，因为我们的LUT纹理图的分辨率为512*512。

同理G通道的计算也是一样的。

下面附上完整核心的demo渲染代码：

Lut2DOpengl.cpp

```
#include "Lut2DOpengl.h"
#include "../utils/Log.h"

// 顶点着色器
static const char *ver = "#version 300 es\n"
                         "in vec4 aPosition;\n"
                         "in vec2 aTexCoord;\n"
                         "out vec2 TexCoord;\n"
                         "void main() {\n"
                         "  TexCoord = aTexCoord;\n"
                         "  gl_Position = aPosition;\n"
                         "}";
// 片元着色器
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
                              "in vec2 TexCoord;\n"
                              "uniform sampler2D ourTexture;\n"
                              "uniform sampler2D textureLUT;\n"
                              "out vec4 FragColor;\n"
```

```
                                "vec4 lookupTable(vec4 color){\n"
                                "    float blueColor = color.b *
63.0;\n"
                                "    vec2 quad1;\n"
                                "    quad1.y = floor(floor(blueColor) /
8.0);\n"
                                "    quad1.x = floor(blueColor) -
(quad1.y * 8.0);\n"
                                "    vec2 quad2;\n"
                                "    quad2.y = floor(ceil(blueColor) /
8.0);\n"
                                "    quad2.x = ceil(blueColor) -
(quad2.y * 8.0);\n"
                                "    vec2 texPos1;\n"
                                "    texPos1.x = (quad1.x * 0.125) +
0.5/512.0 + ((0.125 - 1.0/512.0) * color.r);\n"
                                "    texPos1.y = (quad1.y * 0.125) +
0.5/512.0 + ((0.125 - 1.0/512.0) * color.g);\n"
                                "    vec2 texPos2;\n"
                                "    texPos2.x = (quad2.x * 0.125) +
0.5/512.0 + ((0.125 - 1.0/512.0) * color.r);\n"
                                "    texPos2.y = (quad2.y * 0.125) +
0.5/512.0 + ((0.125 - 1.0/512.0) * color.g);\n"
                                "    vec4 newColor1 =
texture(textureLUT, texPos1);\n"
                                "    vec4 newColor2 =
texture(textureLUT, texPos2);\n"
                                "    vec4 newColor = mix(newColor1,
newColor2, fract(blueColor));\n"
                                "    return vec4(newColor.rgb,
color.w);\n"
                                "}\n"
                                "\n"
                                "void main()\n"
                                "{\n"
```

```
                                "    vec4 tmpColor = texture(ourTexture,
TexCoord);\n"
                                "    FragColor =
lookupTable(tmpColor);\n"
                                "}";

const static GLfloat VERTICES_AND_TEXTURE[] = {
        0.5f, -0.5f, // 右下
        // 纹理坐标
        1.0f,1.0f,
        0.5f, 0.5f, // 右上
        // 纹理坐标
        1.0f,0.0f,
        -0.5f, -0.5f, // 左下
        // 纹理坐标
        0.0f,1.0f,
        -0.5f, 0.5f, // 左上
        // 纹理坐标
        0.0f,0.0f
};

// 真正的纹理坐标在图片的左下角
const static GLfloat FBO_VERTICES_AND_TEXTURE[] = {
        1.0f, -1.0f, // 右下
        // 纹理坐标
        1.0f,0.0f,
        1.0f, 1.0f, // 右上
        // 纹理坐标
        1.0f,1.0f,
        -1.0f, -1.0f, // 左下
        // 纹理坐标
        0.0f,0.0f,
        -1.0f, 1.0f, // 左上
        // 纹理坐标
        0.0f,1.0f
};
```

```
// 使用byte类型比使用short或者int类型节约内存
const static uint8_t indices[] = {
        // 注意索引从0开始!
        // 此例的索引(0,1,2,3)就是顶点数组vertices的下标,
        // 这样可以由下标代表顶点组合成矩形
        0, 1, 2, // 第一个三角形
        1, 2, 3  // 第二个三角形
};

Lut2DOpengl::Lut2DOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
    textureHandle = glGetAttribLocation(program,"aTexCoord");
    textureSampler = glGetUniformLocation(program,"ourTexture");
    lut_textureSampler = glGetUniformLocation(program,"textureLUT");

    LOGD("program:%d",program);
    LOGD("positionHandle:%d",positionHandle);
    LOGD("textureHandle:%d",textureHandle);
    LOGD("textureSample:%d",textureSampler);
    // VAO
    glGenVertexArrays(1, &vao);
    glBindVertexArray(vao);

    // vbo
    glGenBuffers(1, &vbo);
    glBindBuffer(GL_ARRAY_BUFFER, vbo);
    glBufferData(GL_ARRAY_BUFFER, sizeof(VERTICES_AND_TEXTURE),
VERTICES_AND_TEXTURE, GL_STATIC_DRAW);

    // stride 步长 每个顶点坐标之间相隔4个数据点，数据类型是float
    glVertexAttribPointer(positionHandle, 2, GL_FLOAT, GL_FALSE, 4 *
sizeof(float), (void *) 0);
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);
```

```
    // stride 步长 每个颜色坐标之间相隔4个数据点，数据类型是float，颜色坐标索引从2开始
    glVertexAttribPointer(textureHandle, 2, GL_FLOAT, GL_FALSE, 4 * sizeof(float),
                          (void *) (2 * sizeof(float)));
    // 启用纹理坐标数组
    glEnableVertexAttribArray(textureHandle);

    // EBO
    glGenBuffers(1,&ebo);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER,ebo);

 glBufferData(GL_ELEMENT_ARRAY_BUFFER,sizeof(indices),indices,GL_STATIC_DRAW);

    // 这个顺序不能乱啊，先解除vao，再解除其他的，不然在绘制的时候可能会不起作用，需要重新glBindBuffer才生效
    // vao解除
    glBindVertexArray(0);
    // 解除绑定
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    // 解除绑定
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER,0);
}

Lut2DOpengl::~Lut2DOpengl() noexcept {
    glDeleteBuffers(1,&ebo);
    glDeleteBuffers(1,&vbo);
    glDeleteVertexArrays(1,&vao);
    // ... 删除其他，例如fbo等
}

void Lut2DOpengl::setPixel(void *data, int width, int height, int length) {
    imageWidth = width;
    imageHeight = height;
```

```
    glGenTextures(1, &imageTextureId);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, imageTextureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA,
GL_UNSIGNED_BYTE, data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);
    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}

void Lut2DOpengl::setLutPixel(void *data, int width, int height, int
length) {
    glGenTextures(1, &lut_imageTextureId);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, lut_imageTextureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA,
GL_UNSIGNED_BYTE, data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);
    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
```

```
}

void Lut2DOpengl::onDraw() {
    // 恢复绘制屏幕宽高
    glViewport(0,0,eglHelper->viewWidth,eglHelper->viewHeight);

    // 绘制到屏幕
    // 清屏
    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);

    // 激活纹理
    glActiveTexture(GL_TEXTURE1);
    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, imageTextureId);
    glUniform1i(textureSampler, 1);

    // 激活纹理 lut
    glActiveTexture(GL_TEXTURE2);
    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, lut_imageTextureId);
    glUniform1i(lut_textureSampler, 2);

    checkError(program);

    // VBO与VAO配合绘制
    // 使用vao
    glBindVertexArray(vao);
    // 使用EBO
// 使用byte类型节省内存
    glDrawElements(GL_TRIANGLES,6,GL_UNSIGNED_BYTE,(void *)0);
    glUseProgram(0);
    // vao解除绑定
    glBindVertexArray(0);
    // 禁用顶点
```

```
        glDisableVertexAttribArray(positionHandle);
        if (nullptr != eglHelper) {
            eglHelper->swapBuffers();
        }

        glBindTexture(GL_TEXTURE_2D, 0);
    }
```

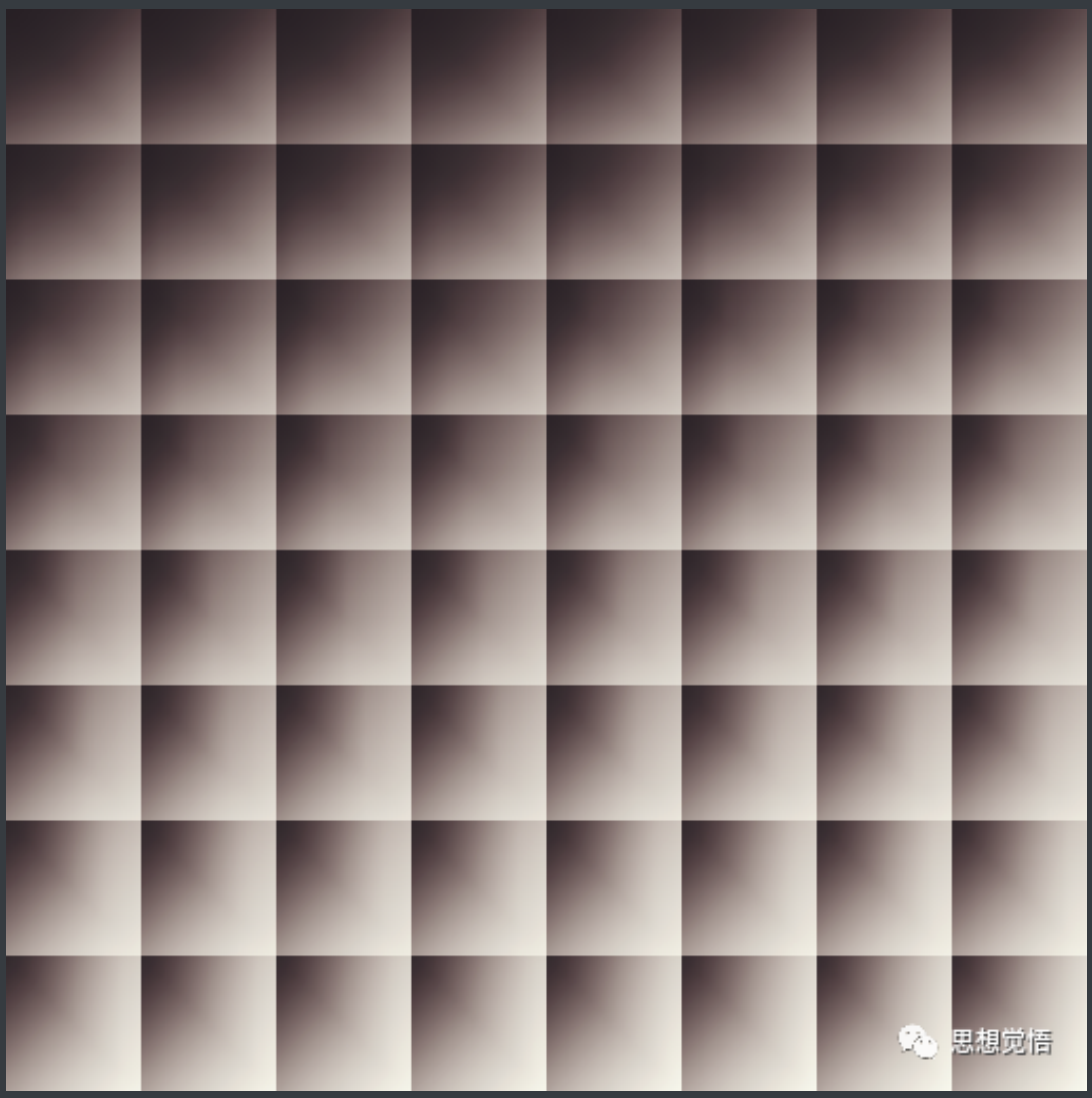
demo运行结果图

以下这张是需要增加滤镜的原图：



> 需要增加滤镜的原图

以下这张是所使用的LUT纹理图：



> 所使用的LUT纹理图

以下这张是运行结果图：



> 运行结果图

思考

如果要在Opengl中给LUT加上滤镜强度调整参数该怎么处理呢?

系列教程源码

https://github.com/feiflyer/NDK_OpenglES_Tutorial

Opengl ES之LUT滤镜(下)-3DLUT

在上一篇文章 Opengl ES之LUT滤镜(上) 中我们详细介绍了基于图片纹理的2D LUT滤镜的使用，
这种方式也是目前大部分SDK在移动端的实现方式，既然有2D LUT滤镜那就有3D LUT，今天我们就来介绍下3D LUT的使用。

博主做这个demo之前查了一些资料，发现针对Opengl ES的3D LUT滤镜的资料比较稀缺，废了不少时间找到一个试了以下在某些机型上能生效，但是在部分手机上是有兼容性问题的，在iOS上是不生效的，
这里笔者也不敢打保票说我的demo就一定能兼容所有机型，但是至少实测起来是比笔者找到的那个例子兼容性好那么一丢丢...

所以这篇应该是需要付费的，也就6块9而已，半杯奶茶钱都不够...

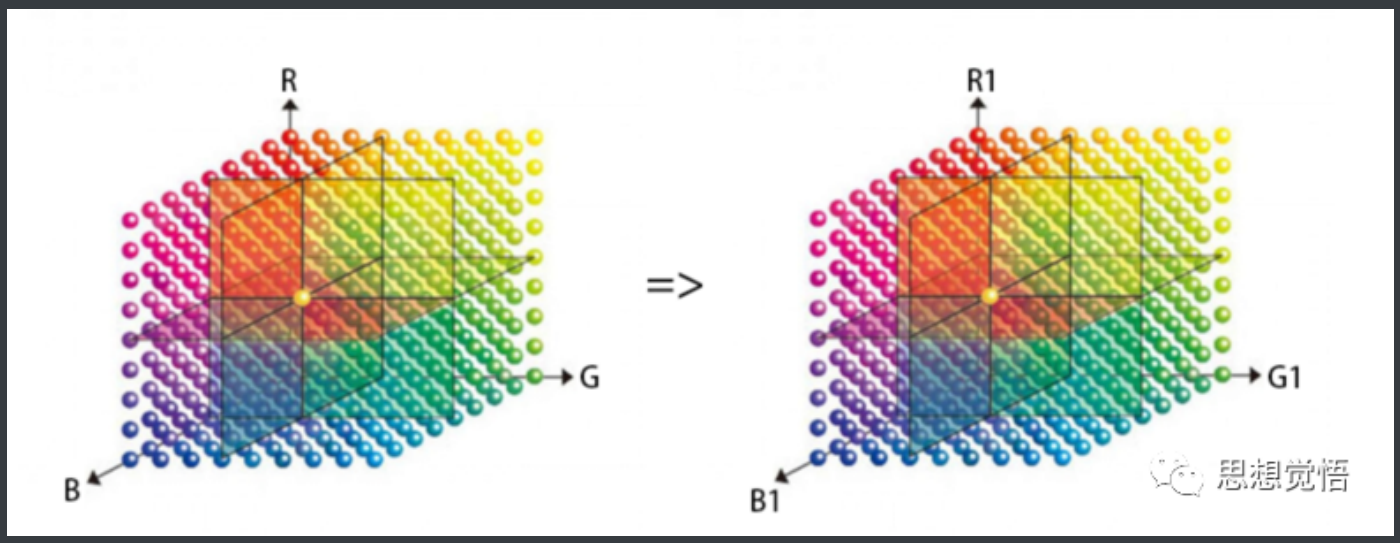
同时在上一篇文章 Opengl ES之LUT滤镜(上) 我们留下一个问题，就是如何给LUT增加强度调节呢？这个问题的答案也会在今天的demo中有所体现。

3D LUT

所谓的3D LUT就是将RGB分别作为与三维的XYZ建立坐标关系，原始图像的RGB颜色通过3D LUT映射后产生新的颜色数据 R1 G1 B1。

```
R1 = LUT(R)
G1 = LUT(G)
B1 = LUT(B)
```

映射关系如图所示：



通过这张图我们可以看出RGB与XYZ是对应的，个人认为其实3D LUT才是真正的LUT，2D的LUT只是为了减少内存使用，提升性能所做的一个简化。因此2D LUT相对与3D LUT来说是会有一些细节点的损失的，如果想要展示更加精准细腻的颜色映射关系，那还是得使用3D LUT，这大概就是为了PC桌面端的视频剪辑软件多使用点cube文件作为3D LUT滤镜的一个原因吧。

总的来说就是3D LUT比2D LUT映射的细节点更多，更细腻，这就有点像HDR与SDR的关系。

OpenglES之分屏滤镜

还记得我们在 Opengl ES之纹理贴图 一文中提到的分屏滤镜吗？当时我们的实现方式是通过GL_REPEAT的方式实现的，
今天我们再来系统地讲下在Opengl ES中实现分屏滤镜的一些细节点，而通过操作片元着色器实现分屏滤镜，这也是业界比较常用的做法。

分屏滤镜的几种实现方式

试想一下如果你拿到一个使用Opengl ES实现分屏滤镜的需求，你能想到几种实现方式呢?

1. 首先最容易想到的也许就是多次绘制吧，比如四分屏就绘制四个相同的纹理，六分屏就绘制六个相同的纹理，以此类推。考虑到纹理相同，我又可以用到纹理数组了，《Opengl ES之纹理数组》
2. 其次可以像上面提到的使用GL_REPEAT取巧的方式也可以实现。
3. 还可以使用glViewport控制绘制区域的方式实现，可能这个比较复杂。
4. 最后也是常用的一种方式就是在片元着色器中根据分屏情况动态改变函数 `texture` 的纹理坐标来实现分屏效果。

笔者在这之前也参考了网上的一些分屏滤镜的Opengl ES教程，发现有些是有缺陷的，虽然分屏效果是有了，但是图片发生了裁剪导致一整张图片显示不完整，如果遇到这样的问题，童鞋们知道该如何修改吗?

分屏滤镜具体实现

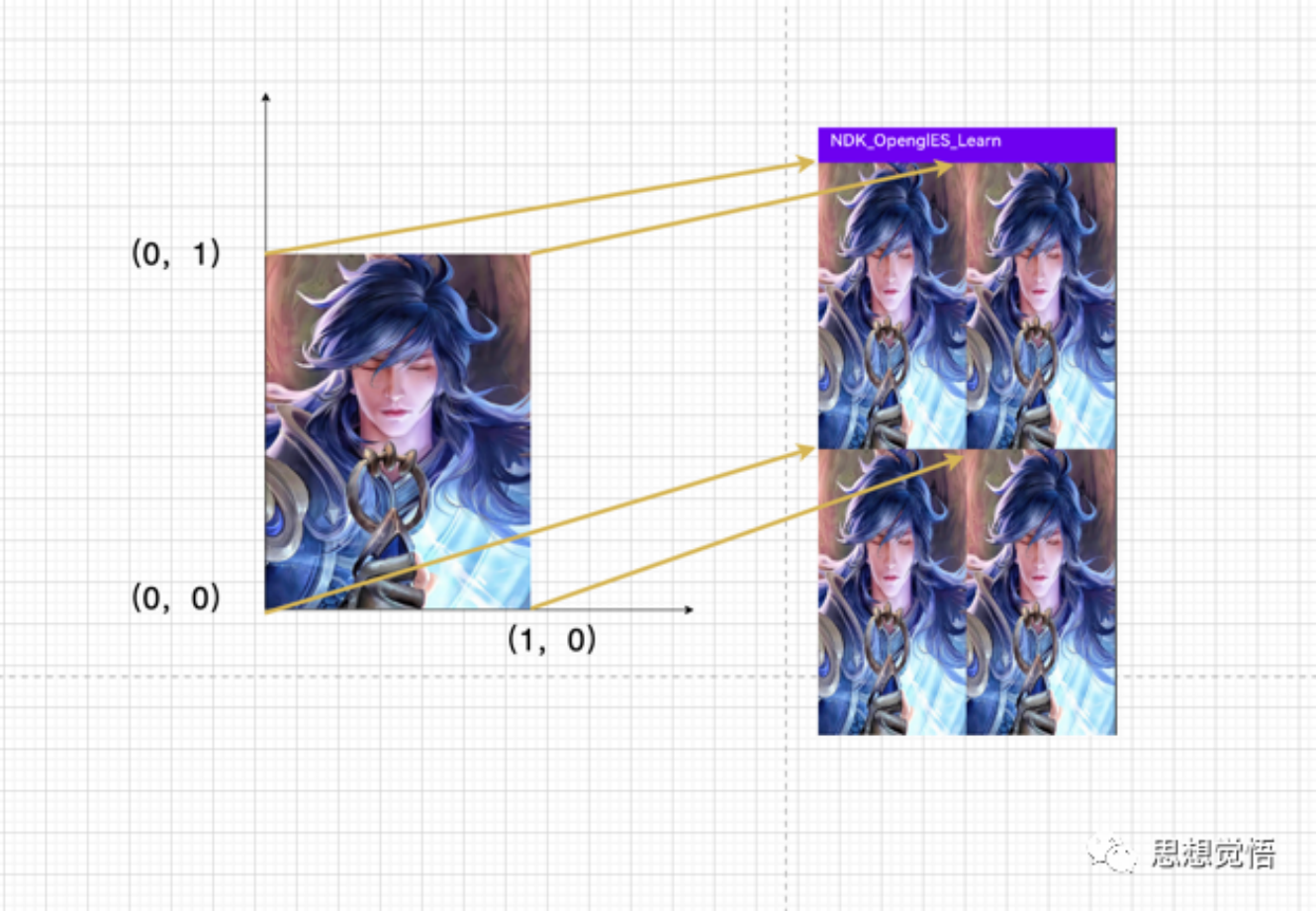
下面我们就以上面的第四中方式实现一下一个四分屏的滤镜效果，代码中也有GL_REPEAT方式实现的分屏滤镜，到时童鞋们可以自行打开相关注释运行查看效果。

首先看一下我们的实现效果：



四分屏，顾名思义就是将屏幕四等分，然后每个区域都完整地显示一张纹理图。

我们看下对应的纹理坐标映射图：



从这个图我们可以得出一个通用结论：

> 在四分屏中滤镜中假如当纹理片段坐标为（x,y），当x小于0.5的时候，则采样纹理图片对应的x值为0到1，
> 同理当y小于0.5时，采样的纹理图片的坐标值也应该时0到1，以此类推当x大于0.5时则采样纹理图片对应的x值也应该是0到1，
> 当y大于0.5时，采样的纹理图片的坐标y值也应该是0到1。

因此我们只要在片元着色器中做好相应的映射，当x小于0时将其坐标映射到0到1之间，同样大于0.5时也将其坐标映射到0到1之间即可。那么我们的片元着色器应该是这样的：

```
#version 300 es
precision mediump float;
out vec4 FragColor;
in vec2 TexCoord;
uniform sampler2D ourTexture;
void main()
{
```

```
    if(TexCoord.y <= 0.5){
        if(TexCoord.x <= 0.5){
            // x的值始终在0到1之间 y也是
            FragColor = texture(ourTexture,vec2(TexCoord.x * 2.0,
(TexCoord.y - 0.5) * 2.0));
        }else{
            // x的值始终在0到1之间 y也是
            FragColor = texture(ourTexture,vec2((TexCoord.x - 0.5) *
2.0,(TexCoord.y - 0.5) * 2.0));
        }
    }else{
        if(TexCoord.x <= 0.5){
            FragColor = texture(ourTexture,vec2(TexCoord.x * 2.0,
(TexCoord.y - 0.5) * 2.0));
        }else{
            FragColor = texture(ourTexture,vec2((TexCoord.x - 0.5) *
2.0,(TexCoord.y - 0.5) * 2.0));
        }
    }
}
```

上面着色器的核心思想就是始终将取样时的x和y值约束在0到1之间，有了着色器，再运用我们之前学过的纹理渲染的知识点，我们就可以完成四分屏滤镜的渲染啦。

下面是核心代码点：

SplitFilterOpengl.cpp

```
#include "SplitFilterOpengl.h"
#include "../utils/Log.h"

// 顶点着色器
static const char *ver = "#version 300 es\n"
                         "in vec4 aPosition;\n"
                         "in vec2 aTexCoord;\n"
                         "out vec2 TexCoord;\n"
```

```
                            "void main() {\n"
                            "  TexCoord = aTexCoord;\n"
                            "  gl_Position = aPosition;\n"
                            "}";

//// 片元着色器
//static const char *fragment = "#version 300 es\n"
//                              "precision mediump float;\n"
//                              "out vec4 FragColor;\n"
//                              "in vec2 TexCoord;\n"
//                              "uniform sampler2D ourTexture;\n"
//                              "void main()\n"
//                              "{\n"
//                              "    FragColor = texture(ourTexture,
TexCoord);\n"
//                              "}";


// 片元着色器 垂直二分屏
// 原理? 搭配GL_REPEAT使用
// 同理y轴上的三分屏就是3.0 * TexCoord.y 以此类推
//static const char *fragment = "#version 300 es\n"
//                              "precision mediump float;\n"
//                              "out vec4 FragColor;\n"
//                              "in vec2 TexCoord;\n"
//                              "uniform sampler2D ourTexture;\n"
//                              "void main()\n"
//                              "{\n"
//                              "    FragColor = texture(ourTexture,
vec2(TexCoord.x, 2.0 * TexCoord.y));\n"
//                              "}";


// 片元着色器 垂直二分屏
// 原理? 将映射到0到1之间, 不必搭配GL_REPEAT
//static const char *fragment = "#version 300 es\n"
```

```
//                                "precision mediump float;\n"
//                                "out vec4 FragColor;\n"
//                                "in vec2 TexCoord;\n"
//                                "uniform sampler2D ourTexture;\n"
//                                "void main()\n"
//                                "{\n"
//                                "\n"
//                                "    if(TexCoord.y <= 0.5){\n"
//                                "        FragColor =
texture(ourTexture,vec2(TexCoord.x,TexCoord.y * 2.0));\n"
//                                "    }else{\n"
//                                "        FragColor =
texture(ourTexture,vec2(TexCoord.x,(TexCoord.y - 0.5) * 2.0));\n"
//                                "    }\n"
//                                "}";

// 三分屏 效果：上下二分，下部分在左右二分
//static const char *fragment = "#version 300 es\n"
//                                "precision mediump float;\n"
//                                "out vec4 FragColor;\n"
//                                "in vec2 TexCoord;\n"
//                                "uniform sampler2D ourTexture;\n"
//                                "void main()\n"
//                                "{\n"
//                                "\n"
//                                "    if(TexCoord.y <= 0.5){\n"
//                                "        FragColor =
texture(ourTexture,vec2(TexCoord.x,TexCoord.y * 2.0));\n"
//                                "    }else{\n"
//                                "        if(TexCoord.x <= 0.5){\n"
//                                "            FragColor =
texture(ourTexture,vec2(TexCoord.x * 2.0,(TexCoord.y - 0.5) *
2.0));\n"
//                                "        }else{\n"
```

```
//                                  "            FragColor =
texture(ourTexture,vec2((TexCoord.x - 0.5) * 2.0,(TexCoord.y - 0.5) *
2.0));\n"
//                                  "        }\n"
//                                  "    }\n"
//                                  "}";

// 四分屏
// 不想写这么复杂的着色器，使用GL_REPEAT如何实现四分屏呢
// 可参考 https://mp.weixin.qq.com/s/jHcR4zzUa4uDw4DV7JlwRQ

static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
                              "out vec4 FragColor;\n"
                              "in vec2 TexCoord;\n"
                              "uniform sampler2D ourTexture;\n"
                              "void main()\n"
                              "{\n"
                              "    if(TexCoord.y <= 0.5){\n"
                              "        if(TexCoord.x <= 0.5){\n"
                              "            FragColor =
texture(ourTexture,vec2(TexCoord.x * 2.0,(TexCoord.y - 0.5) *
2.0));\n"
                              "        }else{\n"
                              "            FragColor =
texture(ourTexture,vec2((TexCoord.x - 0.5) * 2.0,(TexCoord.y - 0.5) *
2.0));\n"
                              "        }\n"
                              "    }else{\n"
                              "        if(TexCoord.x <= 0.5){\n"
                              "            FragColor =
texture(ourTexture,vec2(TexCoord.x * 2.0,(TexCoord.y - 0.5) *
2.0));\n"
                              "        }else{\n"
```

```
                                "            FragColor =
texture(ourTexture,vec2((TexCoord.x - 0.5) * 2.0,(TexCoord.y - 0.5) *
2.0));\n"
                                "        }\n"
                                "    }\n"
                                "}";

// 使用绘制两个三角形组成一个矩形的形式(三角形带)
// 第一第二第三个点组成一个三角形，第二第三第四个点组成一个三角形
const static GLfloat VERTICES[] = {
        1.0f,-1.0f, // 右下
        1.0f,1.0f, // 右上
        -1.0f,-1.0f, // 左下
        -1.0f,1.0f // 左上
};

// 贴图纹理坐标（参考手机屏幕坐标系统，原点在左上角）
//由于对一个OpenGL纹理来说，它没有内在的方向性，因此我们可以使用不同的坐标把它定向到任何我们喜欢的方向上，然而大多数计算机图像都有一个默认的方向，它们通常被规定为y轴向下，X轴向右
const static GLfloat TEXTURE_COORD[] = {
        1.0f,1.0f, // 右下
        1.0f,0.0f, // 右上
        0.0f,1.0f, // 左下
        0.0f,0.0f // 左上
};

SplitFilterOpengl::SplitFilterOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
    textureHandle = glGetAttribLocation(program,"aTexCoord");
    textureSampler = glGetUniformLocation(program,"ourTexture");
    LOGD("program:%d",program);
    LOGD("positionHandle:%d",positionHandle);
    LOGD("textureHandle:%d",textureHandle);
    LOGD("textureSample:%d",textureSampler);
```

```
}

SplitFilterOpengl::~SplitFilterOpengl() noexcept {
    LOGD("SplitFilterOpengl析构函数");
}

void SplitFilterOpengl::setPixel(void *data, int width, int height,
int length) {
    LOGD("texture setPixel");
    glGenTextures(1, &textureId);

    // 激活纹理，注意以下这个两句是搭配的，glActiveTexture激活的是那个纹理，就设
置的sampler2D是那个
    // 默认是0，如果不是0的话，需要在onDraw的时候重新激活一下?
//    glActiveTexture(GL_TEXTURE0);
//    glUniform1i(textureSampler, 0);

// 例如，一样的
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA,
GL_UNSIGNED_BYTE, data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);

    glBindTexture(GL_TEXTURE_2D, textureId);
```

```
    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}

void SplitFilterOpengl::onDraw() {
    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);

    // 激活纹理
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId);

    /**
     * size 几个数字表示一个点，显示是两个数字表示一个点
     * normalized 是否需要归一化，不用，这里已经归一化了
     * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
     */
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);

 glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);

    // 纹理坐标
    glEnableVertexAttribArray(textureHandle);

 glVertexAttribPointer(textureHandle,2,GL_FLOAT,GL_FALSE,0,TEXTURE_COO
RD);

    // 4个顶点绘制两个三角形组成矩形
    glDrawArrays(GL_TRIANGLE_STRIP,0,4);

    glUseProgram(0);
```

```
    // 禁用顶点
    glDisableVertexAttribArray(positionHandle);
    if(nullptr != eglHelper){
        eglHelper->swapBuffers();
    }

    glBindTexture(GL_TEXTURE_2D, 0);
}
```

需要注意的问题

在着色器中需要明确区分整型和浮点型数据，比如如果数据是浮点型的就该写成0.0，而不要写成0，否则着色器编译报错 `Cannot compare 'float' with 'int'` 。

系列教程源码

https://github.com/feiflyer/NDK_OpenglES_Tutorial

Opengl中GL_LUMINANCE被移除的兼容性问题

起因

之前笔者写的Opengl入门教程都是面向Android开发者的Opengl ES方面的，总所周知，Opengl本身就是跨平台的，Opengl ES的程序只需经过稍微改动适配一下即可变成Opengl在桌面端运行起来。

比如之前笔者写的文章 Opengl ES之YUV数据渲染 最近因为工作需要，笔者想把这部分的渲染代码适配到桌面端，
居然惊奇地发现了在Opengl 3.3中，关于颜色格式的内置变量 `GL_LUMINANCE` 和 `GL_LUMINANCE_ALPHA` 居然被删除了。那咋整？此路不通啊...

兼容性修复

在Opengl ES中 `GL_LUMINANCE` 一般表示的是亮度信息，也就是一个颜色通道，在渲染NV21数据时，就表示的是Y的数据，而 `GL_LUMINANCE_ALPHA` 则表示的是两个通道的信息，在渲染NV21数据时表示的就是UV数据。

明白了这个含义之后，其实就是一个通道和两个通道的事情。既然 `GL_LUMINANCE` 和 `GL_LUMINANCE_ALPHA` 被删除了，那我们可以再查询些资料看看Opengl中还有那些变量时可以表示一个数据通道和两个数据通道的，这样问题应该就能够解决了。

在Opengl中表示一个通道和表示两个通道的变量还是不少的，例如 `GL_RED` 、 `GL_RED` 、 `GL_GREEN` 、 `GL_BLUE` 等这些变量可以表示一个颜色通道的数据，而 `GL_RG` 可以表示两个颜色通道的数据，
同理 `GL_RGB` 则可以表示三个颜色通道的数据， `GL_RGBA` 则是带透明度的四个颜色通道的数据。

当然，纹理的数据格式与着色器的渲染是息息相关的，我们改动了纹理数据格式的同时，着色器程序也需要作出相应的修改，否则就是白搭，总不能是我们上传的纹理数据是 `GL_GREEN`，而在着色器中依然使用 `.r` 这样的数据通道吧?

既然原理都清楚了，那下面就是开始适配吧...

首先我们修改下纹理的数据格式，将 `GL_LUMINANCE` 改为 `GL_RED`，将 `GL_LUMINANCE_ALPHA` 改为 `GL_RG`，完整代码如下:

```
void YUVRenderOpengl::setYUVData(void *y_data, void *uv_data, int
width, int height, int yuvType) {

    // 准备y数据纹理
    glGenTextures(1, &y_textureId);
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(y_textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, y_textureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    // 旧的 opengl ES使用
   //    glTexImage2D(GL_TEXTURE_2D, 0, GL_LUMINANCE, width, height,
0, GL_LUMINANCE, GL_UNSIGNED_BYTE, y_data);

   // 新的 opengl 3.3使用
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RED, width, height, 0, GL_RED,
GL_UNSIGNED_BYTE, y_data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);

    glBindTexture(GL_TEXTURE_2D, y_textureId);
```

```
    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);

    // 准备uv数据纹理
    glGenTextures(1, &uv_textureId);
    glActiveTexture(GL_TEXTURE3);
    glUniform1i(uv_textureSampler, 3);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, uv_textureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    // 注意宽高
    // 注意要使用 GL_LUMINANCE_ALPHA
    // 旧的 opengl ES使用
//    glTexImage2D(GL_TEXTURE_2D, 0, GL_LUMINANCE_ALPHA, width/2,
height/2, 0, GL_LUMINANCE_ALPHA, GL_UNSIGNED_BYTE, uv_data);

    // 新的 opengl 3.3使用
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RG, width/2, height/2, 0, GL_RG,
GL_UNSIGNED_BYTE, uv_data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);
    glBindTexture(GL_TEXTURE_2D, uv_textureId);
    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}
```

相关改动部分笔者已增加注释说明。

然后我们修改下着色器部分,以下是原来的着色器代码:

```
static const char *fragment = "#version 300 es\n"
```

```
                                "precision mediump float;\n"
                                "out vec4 FragColor;\n"
                                "in vec2 TexCoord;\n"
                                "uniform sampler2D y_texture; \n"
                                "uniform sampler2D uv_texture;\n"
                                "void main()\n"
                                "{\n"
                                "vec3 yuv;\n"
                                "yuv.x = texture(y_texture,
    TexCoord).r;\n"
                                "yuv.y = texture(uv_texture,
    TexCoord).a-0.5;\n"
                                "yuv.z = texture(uv_texture,
    TexCoord).r-0.5;\n"
                                "vec3 rgb =mat3( 1.0,1.0,1.0,\n"
                                "0.0,-0.344,1.770,1.403,-0.714,0.0) *
    yuv;\n"
                                "FragColor = vec4(rgb, 1);\n"
                                "}";
```

因为我们在纹理上传时修改了Y和UV纹理数据的格式，但是Y数据我们使用的是 `GL_RED` ，因此在着色器中依然使用 `.r` 获取，者无需修改。由于UV纹理我们改为了 `GL_RG` ，因此我们需要将 `yuv.y = texture(uv_texture, TexCoord).a-0.5;\n` 这一行的a通道改为g通道，也就是改为 `yuv.y = texture(uv_texture, TexCoord).g-0.5;\n` 。

同时 `#version 300 es` 是opengl ES着色器的声明，在桌面端使用Opengl 3.3，我们需要将这个声明改为 `#version 330 core` 。

因此完整的着色器代码是：

```
    static const char *fragment = "#version 330 core\n"
                                "precision mediump float;\n"
                                "out vec4 FragColor;\n"
                                "in vec2 TexCoord;\n"
                                "uniform sampler2D y_texture; \n"
```

```
                                "uniform sampler2D uv_texture;\n"
                                "void main()\n"
                                "{\n"
                                "vec3 yuv;\n"
                                "yuv.x = texture(y_texture,
TexCoord).r;\n"
                                "yuv.y = texture(uv_texture,
TexCoord).g-0.5;\n"
                                "yuv.z = texture(uv_texture,
TexCoord).r-0.5;\n"
                                "vec3 rgb =mat3( 1.0,1.0,1.0,\n"
                                "0.0,-0.344,1.770,1.403,-0.714,0.0) *
yuv;\n"
                                "FragColor = vec4(rgb, 1);\n"
                                "}";
```

至此，NV21数据渲染就由Opengl ES向Opengl 3.3兼容适配完成啦...

Opengl ES之转场动画

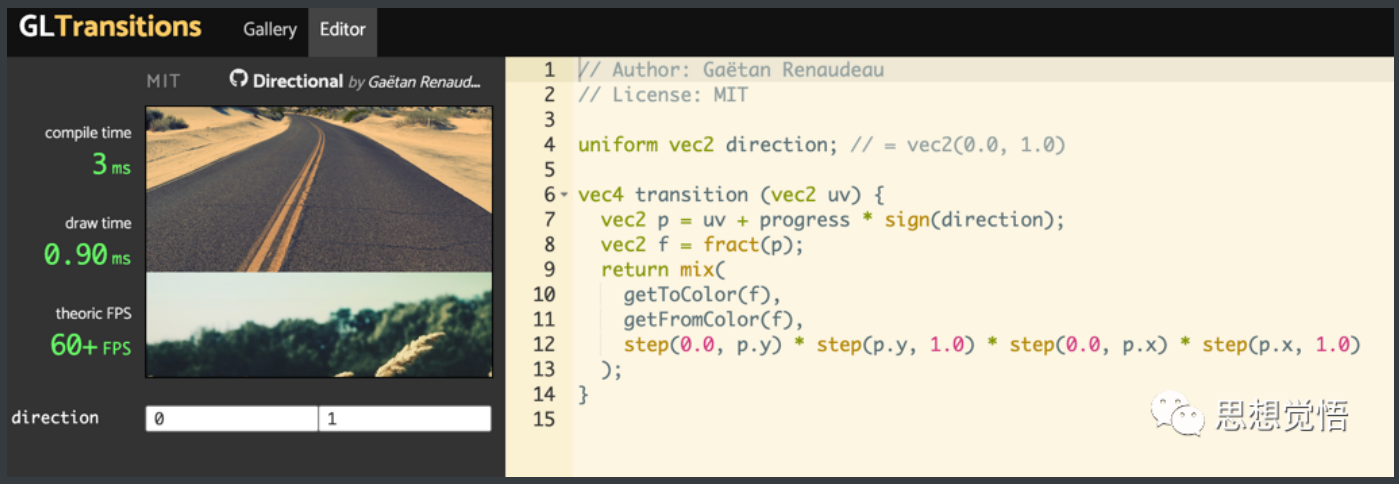
转场

什么是转场效果？一般来说，就是两个视频画面之间的过渡衔接效果。在opengl中，图片的转场，其实就是两个纹理的过渡切换，一般会有两个纹理作为输入，一个是逐渐消失的纹理，一个是逐渐完全可见的纹理。

在这里推荐一个开源项目，该项目主要用来收集各种GL转场特效及其 GLSL 实现代码，开发者可以很方便地移植到自己的项目中，而且该这个项目网站甚至还支持GLSL的在线编译运行，真是学习GLSL的不二之选。
这个项目网站就是GLTransitions:https://gl-transitions.com/gallery

这个项目有一百多张转场特效，这些特效对于想要学习opengl转场的童鞋们来说确是首选推荐。

比如在这里必须按随便选一个从上往下出现的转场特效点进进去，即可查阅看到相关的着色器代码以及参数说明：



上图中的 `transition` 就是封装的转场函数，其中参数 `uv` 表示的是纹理坐标，`getFromColor(uv)` 表示对较要消失的纹理进行采样，`getToColor(uv)` 表示对将要进场的目标纹理进行采样。
然后使用 `mix` 进行纹理融合输出。

我的天，开源就是这么简单...

转场移植

本着将开源包装一下就是自主研发的学习精神，这里我们将这个从上往下的转场效果移植到Opengl ES中去。

首先我们在GLTransitions中找到该转场特效，然后点击选中 `Editor` 菜单，看到它的着色器转场函数如下：

```
// Author: Gaëtan Renaudeau
// License: MIT

uniform vec2 direction; // = vec2(0.0, 1.0)

vec4 transition (vec2 uv) {
  vec2 p = uv + progress * sign(direction);
  vec2 f = fract(p);
  return mix(
    getToColor(f),
    getFromColor(f),
    step(0.0, p.y) * step(p.y, 1.0) * step(0.0, p.x) * step(p.x, 1.0)
  );
}
```

可以看到该特效比较简单，只有两个控制参数，分别是 `direction` 和 `progress` ，文如其义， `direction` 表示方向，通过这个变量可以控制特效的方向，例如从上往下，从左往右，甚至从右上角到左下角等， `progress` 则表示转场的进度，这个值应该在0到1之间，比如该转场持续30帧画面，当第15帧时 `progress` 的值应该是0.5，progress=15/30=0.5。

按照以上意思，那么在Opengl ES中该特效移植后完整的着色器代码如下：

```
#version 300 es
precision mediump float;
uniform vec2 direction;
```

```
uniform float progress;
uniform sampler2D u_texture_01;
uniform sampler2D u_texture_02;
in vec2 TexCoord;
out vec4 FragColor;
vec4 transition (vec2 uv) {
    vec2 p = uv + progress * sign(direction);
    vec2 f = fract(p);
    return mix(
        texture(u_texture_01, f),
        texture(u_texture_02, f),
        step(0.0, p.y) * step(p.y, 1.0) * step(0.0, p.x) * step(p.x,
1.0)
    );
}

void main(){
    FragColor = transition(TexCoord);
}
```



> 放码过来

结合我们之前Opengl ES系列教程封装好的开发环境，完整的转场渲染代码如下：

TransitionOpengl.cpp

```
#include "TransitionOpengl.h"
#include "../utils/Log.h"

// 顶点着色器
static const char *ver = "#version 300 es\n"
                         "in vec4 aPosition;\n"
                         "in vec2 aTexCoord;\n"
                         "out vec2 TexCoord;\n"
                         "void main() {\n"
                         "  TexCoord = aTexCoord;\n"
                         "  gl_Position = aPosition;\n"
                         "}";

// 片元着色器
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
                              "uniform vec2 direction;\n"
                              "uniform float progress;\n"
                              "uniform sampler2D u_texture_01;\n"
                              "uniform sampler2D u_texture_02;\n"
                              "in vec2 TexCoord;\n"
                              "out vec4 FragColor;\n"
                              "vec4 transition (vec2 uv) {\n"
                              "    vec2 p = uv + progress *
sign(direction);\n"
                              "    vec2 f = fract(p);\n"
                              "    return mix(\n"
                              "        texture(u_texture_01, f),\n"
                              "        texture(u_texture_02, f),\n"
                              "        step(0.0, p.y) * step(p.y, 1.0)
* step(0.0, p.x) * step(p.x, 1.0)\n"
                              "    );\n"
                              "}\n"
                              "\n"
```

```
                                "void main(){\n"
                                "    FragColor =
transition(TexCoord);\n"
                                "}";

// 动画总帧数
static const long ALL_FRAME = 60;

// 使用绘制两个三角形组成一个矩形的形式(三角形带)
// 第一第二第三个点组成一个三角形，第二第三第四个点组成一个三角形
const static GLfloat VERTICES[] = {
        0.5f,-0.5f, // 右下
        0.5f,0.5f, // 右上
        -0.5f,-0.5f, // 左下
        -0.5f,0.5f // 左上
};

const static GLfloat TEXTURE_COORD[] = {
        1.0f,1.0f, // 右下
        1.0f,0.0f, // 右上
        0.0f,1.0f, // 左下
        0.0f,0.0f // 左上
};

TransitionOpengl::TransitionOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
    textureHandle = glGetAttribLocation(program,"aTexCoord");
    textureSampler_01 = glGetUniformLocation(program,"u_texture_01");
    textureSampler_02 = glGetUniformLocation(program,"u_texture_02");
    directionHandle = glGetUniformLocation(program,"direction");
    progressHandle = glGetUniformLocation(program,"progress");
    LOGD("program:%d",program);
    LOGD("positionHandle:%d",positionHandle);
    LOGD("textureHandle:%d",textureHandle);
}
```

```
TransitionOpengl::~TransitionOpengl() noexcept {
    LOGD("TransitionOpengl析构函数");
    glDeleteTextures(1,&textureId_01);
    glDeleteTextures(1,&textureId_02);
}

void TransitionOpengl::setPixel(void *data, int width, int height, int
length) {
    LOGD("texture setPixel");
    glGenTextures(1, &textureId_01);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId_01);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA,
GL_UNSIGNED_BYTE, data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);
    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}

void TransitionOpengl::setPixel2(void *data, int width, int height,
int length) {
    LOGD("texture setPixel2");
    LOGD("texture setPixel");
    glGenTextures(1, &textureId_02);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId_02);
```

```
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA,
GL_UNSIGNED_BYTE, data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);
    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}


void TransitionOpengl::onDraw() {
    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);

    // 控制方向 下往上
//    glUniform2f(directionHandle,0 ,1);

    // 控制方向 上往下
    glUniform2f(directionHandle,0 ,-1);

    // 控制进度 0到1之间
    float progress = (currentFrame % ALL_FRAME) * 1.0f / ALL_FRAME;
    currentFrame++;
    glUniform1f(progressHandle,progress);

    // 激活纹理
    glActiveTexture(GL_TEXTURE1);
    glUniform1i(textureSampler_01, 1);

    // 绑定纹理
```

```
    glBindTexture(GL_TEXTURE_2D, textureId_01);

    // 激活纹理
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler_02, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId_02);

    /**
     * size 几个数字表示一个点，显示是两个数字表示一个点
     * normalized 是否需要归一化，不用，这里已经归一化了
     * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
     */
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);

 glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);

    // 纹理坐标
    glEnableVertexAttribArray(textureHandle);

 glVertexAttribPointer(textureHandle,2,GL_FLOAT,GL_FALSE,0,TEXTURE_COO
RD);

    // 4个顶点绘制两个三角形组成矩形
    glDrawArrays(GL_TRIANGLE_STRIP,0,4);

    glUseProgram(0);

    // 禁用顶点
    glDisableVertexAttribArray(positionHandle);
    if(nullptr != eglHelper){
        eglHelper->swapBuffers();
    }
```

```
        glBindTexture(GL_TEXTURE_2D, 0);
    }
```

思考

以上是单个转场的特效效果，如今的短视频编辑软件对于转场的处理都是特别的炫酷的，细细研究分解不难发现其实它们都是多种转场的融合，而不仅仅是使用单一的转场特效，那么针对GLTransitions中的转场特效，作为开发者的你怎么将多个转场特效融合成一个，打造出一个数据自己的炫酷特效转场呢?

OpenglES之裁剪测试

什么是裁剪测试

剪裁测试用于限制绘制区域，在 OpenGL 中启用裁剪测试可以在屏幕或者帧缓冲上指定一个矩形区域，然后在该矩形区域内绘制，只有在该区域内的片元才有机会最终进入帧缓冲，不在该区域内的将会被丢弃，
裁剪测试的效果就相当于在屏幕上开辟一个矩形区域，在该区域内的内容才会处于可见状态。

裁剪测试的一个重要的作用就是提高渲染性能，用于在渲染绘制时只刷新屏幕上发生变化的部分区域。

需要注意的是裁剪测试只是在原来的视口标准的绘制区域内开辟一块矩形区域来控制显示的区域，而不是把内容放到裁剪的区域内来显示。

在Opengl中如何使用裁剪测试

Opengl中关于裁剪测试的几个API，它们分别是

- 启用剪裁测试

```
glEnable(GL_SCISSOR_TEST);
```

- 禁用剪裁测试

```
glDisable(GL_SCISSOR_TEST);
```

- 指定剪裁窗口

```
glScissor(x,y,width,height);
```

x 和 y 坐标所在的坐标系是以视口矩形区域的左下角为原点的，x 轴向右，y 轴向上的坐标系，对应到屏幕的坐标，就是以屏幕的左下角为坐标原点。

实战例子

在之前的系列入门教程中Opengl ES之纹理贴图 我们介绍了纹理贴图的相关知识点，
今天我们就在这个基础上使用裁剪测试绘制一个蓝色的方块。

我们先来看下完整的代码再做相关解析：

```
#include "ScissorTestOpengl.h"

#include "TextureMapOpengl.h"

#include "../utils/Log.h"

// 顶点着色器
static const char *ver = "#version 300 es\n"
                         "in vec4 aPosition;\n"
                         "in vec2 aTexCoord;\n"
                         "out vec2 TexCoord;\n"
                         "void main() {\n"
                         "  TexCoord = aTexCoord;\n"
                         "  gl_Position = aPosition;\n"
                         "}";

// 片元着色器
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
                              "out vec4 FragColor;\n"
                              "in vec2 TexCoord;\n"
                              "uniform sampler2D ourTexture;\n"
                              "void main()\n"
                              "{\n"
                              "    FragColor = texture(ourTexture,
TexCoord);\n"
```

```
                                    "}";


// 使用绘制两个三角形组成一个矩形的形式(三角形带)
// 第一第二第三个点组成一个三角形，第二第三第四个点组成一个三角形
const static GLfloat VERTICES[] = {
        0.5f,-0.5f, // 右下
        0.5f,0.5f, // 右上
        -0.5f,-0.5f, // 左下
        -0.5f,0.5f // 左上
};

// 纹理坐标（原点在左下角，这样贴图看到的会是倒置的
//const static GLfloat TEXTURE_COORD[] = {
//        1.0f,0.0f, // 右下
//        1.0f,1.0f, // 右上
//        0.0f,0.0f, // 左下
//        0.0f,1.0f // 左上
//};

// 贴图纹理坐标（参考手机屏幕坐标系统，原点在左上角）
//由于对一个OpenGL纹理来说，它没有内在的方向性，因此我们可以使用不同的坐标把它定向到任何我们喜欢的方向上，然而大多数计算机图像都有一个默认的方向，它们通常被规定为y轴向下，X轴向右
const static GLfloat TEXTURE_COORD[] = {
        1.0f,1.0f, // 右下
        1.0f,0.0f, // 右上
        0.0f,1.0f, // 左下
        0.0f,0.0f // 左上
};

// 四分屏  GL_REPEAT环绕方式
//const static GLfloat TEXTURE_COORD[] = {
//        2.0f,2.0f, // 右下
//        2.0f,0.0f, // 右上
//        0.0f,2.0f, // 左下
```

```
//        0.0f,0.0f // 左上
//};

// 九分屏 GL_REPEAT环绕方式
//const static GLfloat TEXTURE_COORD[] = {
//        3.0f,3.0f, // 右下
//        3.0f,0.0f, // 右上
//        0.0f,3.0f, // 左下
//        0.0f,0.0f // 左上
//};


ScissorTestOpengl::ScissorTestOpengl():BaseOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
    textureHandle = glGetAttribLocation(program,"aTexCoord");
    textureSampler = glGetUniformLocation(program,"ourTexture");
    LOGD("program:%d",program);
    LOGD("positionHandle:%d",positionHandle);
    LOGD("textureHandle:%d",textureHandle);
    LOGD("textureSample:%d",textureSampler);
}

void ScissorTestOpengl::setPixel(void *data, int width, int height,
int length) {
    LOGD("texture setPixel");
    glGenTextures(1, &textureId);

    // 激活纹理，注意以下这个两句是搭配的，glActiveTexture激活的是那个纹理，就设
置的sampler2D是那个
    // 默认是0，如果不是0的话，需要在onDraw的时候重新激活一下?
//    glActiveTexture(GL_TEXTURE0);
//    glUniform1i(textureSampler, 0);

// 例如，一样的
    glActiveTexture(GL_TEXTURE2);
```

```
    glUniform1i(textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId);
    // 为当前绑定的纹理对象设置环绕、过滤方式
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA,
GL_UNSIGNED_BYTE, data);
    // 生成mip贴图
    glGenerateMipmap(GL_TEXTURE_2D);

    glBindTexture(GL_TEXTURE_2D, textureId);

    // 解绑定
    glBindTexture(GL_TEXTURE_2D, 0);
}

void ScissorTestOpengl::onDraw() {
    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);

    // 激活纹理
    glActiveTexture(GL_TEXTURE2);
    glUniform1i(textureSampler, 2);

    // 绑定纹理
    glBindTexture(GL_TEXTURE_2D, textureId);

    /**
     * size 几个数字表示一个点，显示是两个数字表示一个点
     * normalized 是否需要归一化，不用，这里已经归一化了
```

```
     * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
     */
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);

 glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);

    // 纹理坐标
    glEnableVertexAttribArray(textureHandle);

 glVertexAttribPointer(textureHandle,2,GL_FLOAT,GL_FALSE,0,TEXTURE_COO
RD);

    // 4个顶点绘制两个三角形组成矩形
    glDrawArrays(GL_TRIANGLE_STRIP,0,4);

    glUseProgram(0);

    // 禁用顶点
    glDisableVertexAttribArray(positionHandle);

    // 蓝色
    glClearColor(0.0f, 0.0f, 1.0f, 1.0f);
    // 开启裁剪测试
    glEnable(GL_SCISSOR_TEST);
    // 设置裁剪区域，也就是后面绘制的内容在那个部分可见
    glScissor(eglHelper->viewWidth/2, eglHelper->viewHeight/2,
200,200);
    glClear(GL_COLOR_BUFFER_BIT);

    if(nullptr != eglHelper){
        eglHelper->swapBuffers();
    }

    glBindTexture(GL_TEXTURE_2D, 0);
    // 禁用裁剪测试
```

```
    glDisable(GL_SCISSOR_TEST);
}

ScissorTestOpengl::~ScissorTestOpengl() {
    LOGD("TextureMapOpengl析构函数");
}
```

其实代码都是和Opengl ES之纹理贴图 中的一样的，只是添加了147-153行的相关内容，绘制了一个蓝色的清屏。

倘若将150或152行注释掉，你得到的是一个蓝屏，没有任何纹理，因为纹理图片被蓝屏覆盖掉了。而152行 `glScissor(eglHelper->viewWidth/2, eglHelper->viewHeight/2, 200,200);` 正是设定了
裁剪窗口区域，不在这个区域内的将变得不可见，因此这个区域之外的地步纹理就变得可见了，因为没有被蓝屏覆盖。



> 裁剪测试运行结果

Opengl ES之颜色混合

颜色混合

关于混合，其实我们在之前的文章中也有使用到混合的功能了，例如在 Opengl ES之水印贴图 使用混合实现水印贴图。

什么是颜色混合?

颜色混合就是把两种颜色按某种规则混合起来得到新的颜色。

在使用颜色混合时我们首先需要开启混合功能：

```
glEnable(GL_BLEND);
```

比如我们有两种颜色颜色，一种是纯蓝色 `RGBA(0，0，255，255)`，一种是纯绿色 `RGBA(0，255，0，255)`，混合时分别各取一半，就得到了混合后的颜色 `RGBA(0,127，127，255)`，这就是Opengl ES中颜色混合的工作模式，但是不一定就是上面的这条计算公式，上述的只是笔者举的一个例子。

这个颜色混合的默认公式如下，为什么说是默认呢？因为可以通过其它的函数改变这个公式：

```
C_f = (C_src * S) + (C_dest * D)

C_f ： 最终计算产生的颜色
C_src ： 源颜色
C_dest ： 是目标颜色
S ： 源混合因子
D ： 目标混合因子。
```

而上述的混合因子 `S` 和 `D` 可以使用 `glBlendFunc` 函数进行设置：

```
//  S 和 D 都是枚举值，不是可直接指定的实际值。
glBlendFunc(GLenum s, GLenum D);
```

更多关于这个函数的枚举值可以参考官方介绍：https://registry.khronos.org/OpenGL-Refpages/es2.0/xhtml/glBlendFunc.xml

一般情况下，RGBA都是使用8位来表示，因此在这个官方介绍中的表格数据中的K相关的参数都可以直接看成1即可，也就是简单地将公式中的除数看出1更方便理解。

颜色混合的大致原理

通常OpenGL渲染时会把颜色值放在颜色缓冲区，每个像素的深度值放在深度缓冲区。
当深度测试被关闭，新的颜色值会简单地覆盖颜色缓冲区中已经存在的其他值。当深度测试被打开，则会保留深度值Z更小的。
但如果打开了 混合功能，那么下层的颜色值就不会被清除了，而是按照上面所介绍的那样使用某种公式计算出新的混合颜色。

下面通过一个简单的例子介绍下函数 `glBlendFun` 的使用，例如：

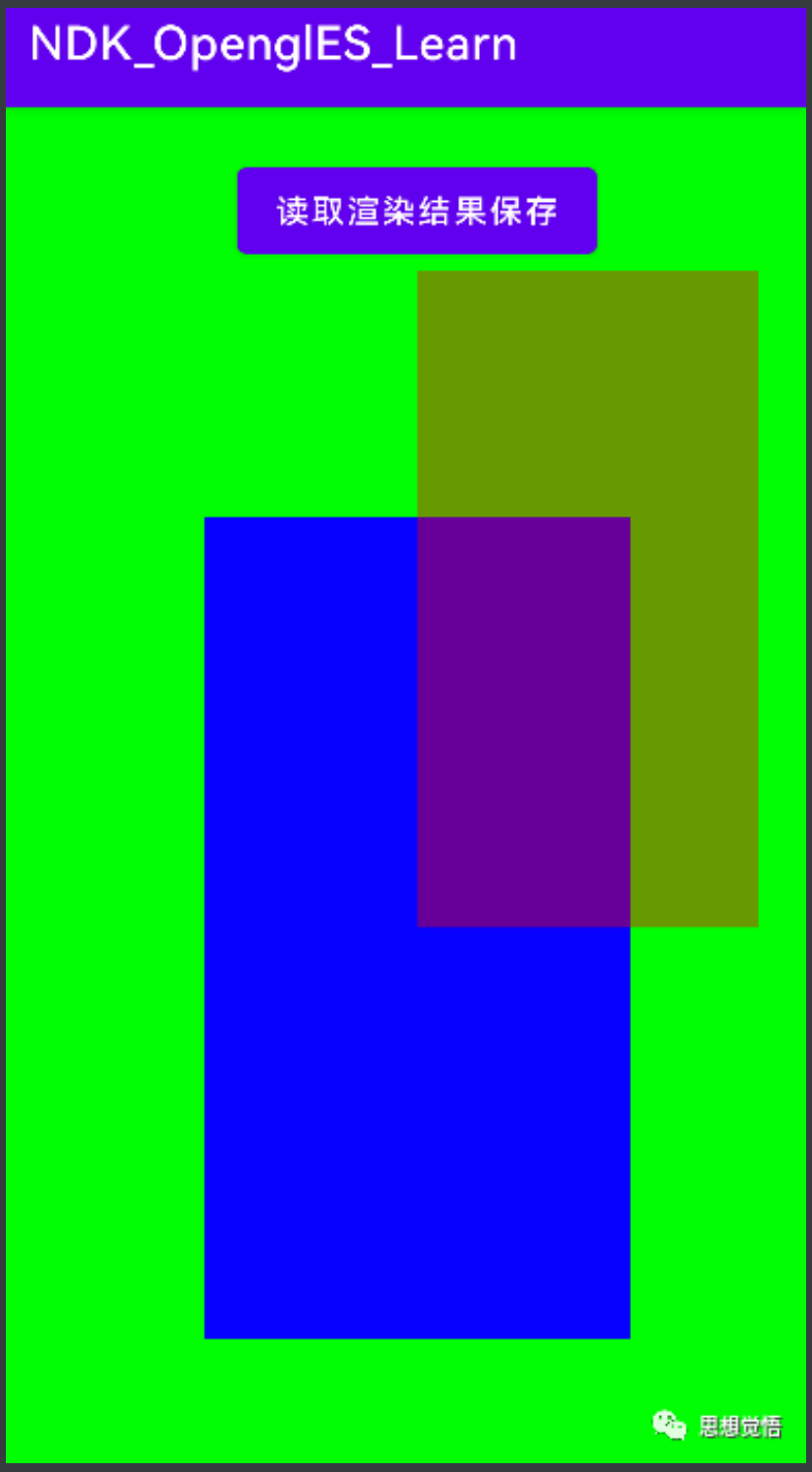
```
glBlendFun(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

如果颜色缓冲区已经有一种蓝色（0.0f, 0.0f, 1.0f, 1.0f），这就是目标颜色，然后在这上面用一种alpha值为 0.4 的红色（1.0f, 0.0f, 0.0f, 0.4f），绘制了一些什么东西（这个后面画上去就是源颜色），那么就最终颜色是如何计算的呢?

```
C_dest = 目标颜色 =  (0.0f, 0.0f, 1.0f, 1.0f)
C_src = 源颜色 =  (1.0f, 0.0f, 0.0f, 0.4f)
//  因为GL_SRC_ALPHA
S = 源 alpha 值 = 0.4
// 因为GL_ONE_MINUS_SRC_ALPHA
D = 1 减去 src alpha 值 = 1.0 - 0.4 = 0.6
默认混合方程式:
C_f = (C_src * S) + (C_dest * D)
也就是
C_f = (C_src * 0.4) + (C_dest * 0.6)
```

因此最终的颜色是原先的蓝色（目标颜色）与后来的红色（源颜色）进行公式混合后的组合。源颜色的 `alpha` 值越高，添加的源颜色成分就越多，目标颜色所保留的成分就越少。

以下时笔者绘制的两个纯色矩形的混合运行结果，一个是蓝色，一个是红色，程序混合效果如图：



完整的opengl渲染代码：

```
#include "ColorBlendOpengl.h"
#include "../utils/Log.h"


// 顶点着色器
static const char *ver = "#version 300 es\n"
                         "in vec4 aColor;\n"
                         "in vec4 aPosition;\n"
                         "out vec4 vColor;\n"
                         "void main() {\n"
                         "    vColor = aColor;\n"
                         "    gl_Position = aPosition;\n"
                         "}";

// 片元着色器
static const char *fragment = "#version 300 es\n"
                              "precision mediump float;\n"
                              "in vec4 vColor;\n"
                              "out vec4 fragColor;\n"
                              "void main() {\n"
                              "    fragColor = vColor;\n"
                              "}";

// 使用绘制两个三角形组成一个矩形的形式(三角形带)
// 第一第二第三个点组成一个三角形，第二第三第四个点组成一个三角形
const static GLfloat VERTICES[] = {
        0.5f,-0.5f, // 右下
        0.5f,0.5f, // 右上
        -0.5f,-0.5f, // 左下
        -0.5f,0.5f // 左上
};

const static GLfloat VERTICES_02[] = {
        0.8f,-0.0f, // 右下
        0.8f,0.8f, // 右上
        -0.0f,-0.0f, // 左下
```

```
        -0.0f,0.8f // 左上
};


// rgba 蓝色
const static GLfloat COLOR_ICES[] = {
        0.0f,0.0f,1.0f,1.0f
};

// rgba 红色
const static GLfloat COLOR_ICES_02[] = {
        1.0f,0.0f,0.0f,0.4f
};

ColorBlendOpengl::ColorBlendOpengl():BaseOpengl() {
    initGlProgram(ver,fragment);
    positionHandle = glGetAttribLocation(program,"aPosition");
    colorHandle = glGetAttribLocation(program,"aColor");
    LOGD("program:%d",program);
    LOGD("positionHandle:%d",positionHandle);
    LOGD("colorHandle:%d",colorHandle);
}

void ColorBlendOpengl::onDraw() {

    // 打开混合
    glEnable(GL_BLEND);
    // 混合模式
    glBlendFunc(GL_SRC_ALPHA,GL_ONE_MINUS_SRC_ALPHA);

    // 清屏
    glClearColor(0.0f, 1.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(program);
    /**
     * size 几个数字表示一个点，显示是两个数字表示一个点
```

```
     * normalized 是否需要归一化，不用，这里已经归一化了
     * stride 步长，连续顶点之间的间隔，如果顶点直接是连续的，也可填0
     */

 glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES);
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);

    // 这个不需要glEnableVertexAttribArray
    glVertexAttrib4fv(colorHandle, COLOR_ICES);

    // 4个顶点绘制两个三角形组成矩形
    glDrawArrays(GL_TRIANGLE_STRIP,0,4);
    // 第二个矩形绘制

 glVertexAttribPointer(positionHandle,2,GL_FLOAT,GL_FALSE,0,VERTICES_0
2);
    // 启用顶点数据
    glEnableVertexAttribArray(positionHandle);

    glVertexAttrib4fv(colorHandle, COLOR_ICES_02);
    // 4个顶点绘制两个三角形组成矩形
    glDrawArrays(GL_TRIANGLE_STRIP,0,4);

    glUseProgram(0);

    // 禁用顶点
    glDisableVertexAttribArray(positionHandle);
    if(nullptr != eglHelper){
        eglHelper->swapBuffers();
    }
}

ColorBlendOpengl::~ColorBlendOpengl() noexcept {

}
```

自定义使用颜色混合

上面说过默认混合公式为：

```
C_f = (C_src * S) + (C_dest * D)
```

既然说是默认，那就有方法改成非默认的，我们可以使用函数 `glBlendEquation` 选择不同的混合公式。

模式	公式
GL_FUNC _ADD	C_f = (C_src * S) + (C_dest * D)
GL_FUNC_SUBTRACT	C_f = (C_src * S) - (C_dest * D)
GL_FUNC_REVERSE_SUBTRACT	C_f = (C_dest * D) - (C_src * S)
GL_MIN	C_f = min(C_src , C_dest)
GL_MAX	C_f = max(C_src , C_dest)

除了函数 `glBlendFunc` 之外，还可以利用下面的函数更加灵活地进行选择:

```
void glBlendFuncSeparate(GLenum srcRGB, GLenum dstRGB, GLenum
srcAlpha, GLenum dstAlpha);
```

函数 `glBlendFuncSeparate` 的参数说明如下：

- strRGB： 源颜色的混合因子
- dstRGB： 目标颜色色的混合因子
- strAlpha： 源颜色的 Alpha 因子
- dstAlpha： 目标颜色的 Alpha 因子

也就是我们可以使用函数 `glBlendFunc` 指定源和目标 RGBA 值的混合函数，再使用函数 `glBlendFuncSeparate` 为 RGB 和 alpha 指定单独的混合因子。

颜色混合与抗锯齿

OpenGL 混合功能的另一个用途就是抗锯齿。大多数情况下一个独立的渲染片段将会映射到计算机屏幕上的一个像素。这些像素是一个微小的矩形，而这个矩形通常可以清除地看到两种颜色的分界，它们常常被称为锯齿，而图像一旦放大往往让人觉得图像分界不自然。
为了消除图元之间的锯齿状边缘，OpenGL使用混合功能来混合片段的颜色，也就是把像素的目标颜色与周围像素的颜色进行混合。

开启抗锯齿功能时，首先需要启动混合功能：

```
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

同时还需要确保把混合方程式设置为 `GL_ADD` 。在启用混合功能并选择正确的混合函数以及混合方程式之后，可以选择调用 glEnable 函数对点、直线和多边形进行抗锯齿处理。

```
// Smooth out points
glEnable(GL_POINT_SMOOTH);
// Smooth out lines
glEnable(GL_LINE_SMOOTH);
// Smooth out polygon edges
glEnable(GL_POLYGON_SMOOTH);
```

好了，关于Opengl中的颜色混合就先介绍到这里，要想真正融会贯通，大家还是得多写代码实践测试。